

C++ Zero to Godhood

The Definitive Guide from C++98 to C++26

Master the Beast.

Community Edition

June 10, 2026

Contents

Preface	xvi
0.1 The Complete C++ Programmer's Guide: From Zero to Godhood (C++98 to C++26)	xvi
0.1.1 About the Book	xvi
0.1.2 About the Author	xvi
0.1.3 How to Use This Book	xvi
I Volume I: C++98/03 - The Foundation	1
1 FOUNDATIONS & COMPILATION MODEL	2
2 ABSOLUTE BASICS (C++98)	3
2.1 Getting Started	3
2.1.1 What is C++?	3
2.1.2 Your First Program (C++98)	3
2.2 Basic Types & Variables	3
2.2.1 Fundamental Types (C++98)	3
2.2.2 Variable Declaration & Initialization	4
2.2.3 Scope & Lifetime (C++98)	4
2.2.4 Deep Dive: The Memory Model of Variables	5
2.3 Operators & Control Flow	5
2.3.1 Arithmetic Operators (C++98)	5
2.3.2 Deep Dive: Bitwise Mastery (Low-Level Optimization)	6
2.3.3 Comparison & Logical Operators (C++98)	6
2.3.4 If-Else Statements (C++98)	7
2.3.5 Loops (C++98)	7
2.3.6 Switch Statement (C++98)	8
2.4 Functions	8
2.4.1 Function Declaration & Definition (C++98)	8
2.4.2 Parameters & Return Values (C++98)	9
2.4.3 Default Parameters (C++98)	9
2.4.4 Function Overloading (C++98)	10
2.5 Arrays & Pointers	10
2.5.1 Arrays (C++98)	10
2.5.2 Pointers (C++98)	11
2.5.3 Dynamic Memory (C++98)	11
3 THE C++ COMPILATION & EXECUTION MODEL	13
3.0.1 1.5.1 The Build Pipeline: From Source to Binary	13
3.0.2 1.5.2 Translation Units (TU) & Linkage	13

3.0.3	1.5.3 The One Definition Rule (ODR)	14
3.0.4	1.5.4 Process Memory Layout	15
3.0.5	1.5.5 Program Startup: Before main()	15
3.0.6	1.5.6 Deep Dive into Data Representation	15
4	Professional Notes: Chapter 1: Getting started with C++	17
5	Professional Notes: Chapter 2: Literals	22
6	Professional Notes: Chapter 4: Floating Point Arithmetic	24
7	MEMORY, TYPES, AND POINTERS	25
8	ADVANCED POINTERS & MEMORY	26
8.1	1.1 Pointer to Const vs Const Pointer	26
8.2	1.2 Void Pointers	26
8.3	1.3 Null Pointer Safety	27
8.4	1.4 Memory Layout & Alignment	27
9	ADVANCED ARRAYS	29
9.1	4.1 Dynamic Arrays	29
9.2	4.2 Variable Length Arrays (Non-standard)	30
9.3	4.3 Array Bounds & Safety	30
10	CONST & VOLATILE	31
10.1	11.1 Const Correctness	31
10.2	11.2 Volatile Keyword	31
11	TYPE CASTING	33
11.1	8.1 C-Style Casting	33
11.2	8.2 Implicit Conversions	33
12	ENUMERATION & UNIONS	35
12.1	10.1 Enumerations	35
12.2	10.2 Unions	35
13	BITWISE OPERATIONS	37
13.1	6.1 Bitwise Operators	37
13.2	6.2 Bit Manipulation Techniques	37
14	Professional Notes: Chapter 8: Arrays	39
15	CONTROL FLOW & PREPROCESSOR	42
16	ADVANCED CONTROL FLOW	43
16.1	9.1 Ternary Operator	43
16.2	9.2 goto Statement (Avoid)	43
16.3	9.3 Label & Goto for Error Handling	44
17	PREPROCESSOR DIRECTIVES	45
17.1	7.1 #define and #include	45
17.2	7.2 Conditional Compilation	45
17.3	7.3 Pragma Directives	46

18 INLINE FUNCTIONS & MACROS	47
18.1 12.1 Macro Functions vs Inline Functions	47
19 Professional Notes: Chapter 3: operator precedence	48
20 Professional Notes: Chapter 5: Bit Operators	50
21 Professional Notes: Chapter 11: Loops	52
22 Professional Notes: Chapter 15: Flow Control	56
23 ADVANCED FUNCTIONS & CALLBACKS	59
24 ADVANCED FUNCTIONS	60
24.1 2.1 Variadic Functions	60
24.2 2.2 Function Recursion	60
24.3 2.3 Inline Functions	61
24.4 2.4 Static Functions	61
25 FUNCTION POINTERS & CALLBACKS	63
25.1 3.1 Function Pointers	63
25.2 3.2 Function Pointer Arrays	63
25.3 3.3 Callbacks	64
26 OBJECT-ORIENTED PROGRAMMING: ENCAPSULATION & DESIGN	65
27 OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS (C++98)	66
27.1 CLASSES & OBJECTS	66
27.1.1 1.1 Basic Class Structure	66
27.1.2 1.2 Access Modifiers & Encapsulation	66
27.1.3 1.3 Constructors & Destructors	67
27.1.4 1.4 The Rule of Three (C++98)	67
27.2 INHERITANCE & POLYMORPHISM	68
27.2.1 2.1 Inheritance Basics	68
27.2.2 2.2 Virtual Functions (Polymorphism)	68
27.2.3 2.3 Pure Virtual Functions (Abstract Classes)	69
27.3 ADVANCED CLASS FEATURES	69
27.3.1 3.1 Static Members	69
27.3.2 3.2 Friend Classes/Functions	69
27.4 PART 2: GENERIC PROGRAMMING (TEMPLATES)	69
27.4.1 4.1 Function Templates	70
27.4.2 4.2 Class Templates	70
27.4.3 4.3 Multiple Template Parameters	70
27.4.4 4.4 Non-Type Template Parameters	70
27.4.5 4.5 Template Specialization	71
27.4.6 4.6 The <code>typename</code> Keyword	71
27.4.7 4.7 Templates vs Macros	72
27.5 13.2 Namespace Aliases	72
28 Professional Notes: Chapter 31: Pointers to members	74
29 Professional Notes: Chapter 34: Classes/Structures	75
30 Professional Notes: Chapter 19: Friend keyword	76

31 Professional Notes: Chapter 31: Pointers to members	78
32 Professional Notes: Chapter 34: Classes/Structures	80
33 POLYMORPHISM & VIRTUALIZATION	85
34 DEEP OBJECT MODEL & VIRTUALIZATION	86
34.0.1 2.5.1 The Cost of Polymorphism (vptr & vtable)	86
34.0.2 2.5.2 Multiple Inheritance & Thunks	86
34.0.3 2.5.3 Virtual Inheritance (The Diamond Problem)	86
34.0.4 2.5.4 Alignment & Padding Rules	87
35 Professional Notes: Chapter 25: Polymorphism	88
36 Professional Notes: Chapter 38: Virtual Member Functions	89
37 Professional Notes: Chapter 40: Special Member Functions	90
38 Professional Notes: Chapter 25: Polymorphism	91
39 Professional Notes: Chapter 38: Virtual Member Functions	93
40 Professional Notes: Chapter 40: Special Member Functions	95
41 THE STANDARD TEMPLATE LIBRARY (STL) CORE	98
42 C++98/03 STANDARD LIBRARY	99
42.1 Standard Template Library (STL)	99
42.2 Introduction to STL	99
42.2.1 Key Advantages	99
42.3 STL Components Overview	99
42.4 CONTAINERS - COMPLETE REFERENCE	100
42.4.1 Container Characteristics (C++98)	100
42.5 1.1 VECTOR - Dynamic Array	100
42.5.1 What is Vector?	100
42.5.2 Declaration & Initialization	100
42.5.3 Accessing Elements	100
42.5.4 Modifying Elements	101
42.5.5 Size & Capacity	101
42.5.6 Iterating Through Vector	101
42.6 1.2 DEQUE - Double Ended Queue	102
42.6.1 What is Deque?	102
42.7 1.3 LIST - Doubly Linked List	102
42.7.1 What is List?	102
42.8 1.4 MAP - Sorted Key-Value Pairs	103
42.8.1 What is Map?	103
42.9 1.5 SET - Sorted Unique Keys	103
42.9.1 What is Set?	103
42.10 1.6 MULTIMAP & MULTISSET	104
42.11 1.7 STACK (Adapter)	104
42.12 1.8 QUEUE (Adapter)	104
42.13 1.9 PRIORITY_QUEUE (Adapter)	105
42.14 ITERATORS	105

42.14.1 Categories	105
42.14.2 Operations	105
42.15 ALGORITHMS	105
42.15.1 1. Non-Modifying	106
42.15.2 2. Modifying	106
42.15.3 3. Sorting	106
42.15.4 4. Binary Search (On Sorted Ranges)	106
42.15.5 5. Set Operations (On Sorted Ranges)	106
42.15.6 6. Numeric ()	106
42.15.7 Example: Sort and Find	106
42.16 STRINGS (std::string)	107
42.17 STREAMS (IOSTREAM)	107
42.17.1 File I/O ()	107
42.17.2 String Streams ()	108
42.18 FUNCTORS (Function Objects)	108
43 ADVANCED STRINGS	109
43.1 5.1 String Manipulation	109
43.2 5.2 String Conversion	110
43.3 5.3 String Tokenization	110
44 FILE I/O ADVANCED	112
44.1 14.1 Binary File I/O	112
44.2 14.2 Stream Positioning	112
45 Professional Notes: Chapter 47: std::string	114
46 Professional Notes: Chapter 49: std::vector	115
47 Professional Notes: Chapter 50: std::map	116
48 Professional Notes: Chapter 62: Standard Library Algorithms	117
49 Professional Notes: Chapter 67: Sorting	118
50 Professional Notes: Chapter 43: C++ Containers	119
51 Professional Notes: Chapter 47: std::string	120
52 Professional Notes: Chapter 49: std::vector	126
53 Professional Notes: Chapter 50: std::map	134
54 Professional Notes: Chapter 62: Standard Library Algorithms	138
55 Professional Notes: Chapter 67: Sorting	142
56 STL INTERNALS DEEP DIVE	145
57 STL INTERNALS DEEP DIVE (C++98)	146
57.0.1 1. The Truth About std::vector	146
57.0.2 2. The std::deque Implementation	146
57.0.3 3. Why std::list is (Almost) Always Wrong	147
57.0.4 4. Associative Containers (Map/Set)	147

57.0.5	5. Iterator Invalidation Cheat Sheet	147
58	ERROR HANDLING & ROBUSTNESS	148
59	ERROR HANDLING & DEBUGGING (C++98)	149
59.1	1. ASSERTIONS	149
59.2	2. EXCEPTIONS	149
59.2.1	2.1 Basic Try-Catch	149
59.2.2	2.2 Catching Multiple Types	150
59.2.3	2.3 Standard Exceptions	150
59.2.4	2.4 Exception Specifications (C++98)	150
59.2.5	2.5 Stack Unwinding & RAII	151
59.3	3. DEBUGGING STRATEGIES	151
59.3.1	3.1 Debug Macros (C++98 Style)	151
60	Professional Notes: Chapter 72: Exceptions	152
61	Professional Notes: Chapter 72: Exceptions	153
II	Volume II: C++11 - The Modern Revolution	157
62	THE MODERN C++11 CORE: SYNTAX & TYPE SYSTEM	158
62.1	1. Type Inference & Safety	158
62.1.1	1.1 <code>auto</code> : Automatic Type Deduction	158
62.1.2	1.2 <code>decltype</code> : Inspecting Types	158
62.1.3	1.3 <code>nullptr</code> : The Null Pointer Literal	158
62.2	2. Initialization Uniformity	159
62.2.1	2.1 Uniform Initialization (Brace Initialization)	159
62.2.2	2.2 <code>std::initializer_list</code>	159
62.3	3. Control Flow & Iteration	159
62.3.1	3.1 Range-Based For Loops	159
62.4	4. Class Features	160
62.4.1	4.1 Explicit Overrides (<code>override</code> , <code>final</code>)	160
62.4.2	4.2 Defaulted and Deleted Functions	160
62.4.3	4.3 Strongly Typed Enums (<code>enum class</code>)	160
62.4.4	4.4 Delegating Constructors	160
62.5	5. <code>constexpr</code> (C++11 Version)	161
62.6	6. Static Assert	161
63	MOVE SEMANTICS & SMART POINTERS	162
63.1	1. Rvalue References & Move Semantics	162
63.1.1	1.1 Lvalues vs Rvalues	162
63.1.2	1.2 Rvalue References (<code>T&&</code>)	162
63.1.3	1.3 Move Constructor & Move Assignment	162
63.1.4	1.4 <code>std::move</code>	163
63.2	2. Smart Pointers (RAII)	163
63.2.1	2.1 <code>std::unique_ptr</code>	163
63.2.2	2.2 <code>std::shared_ptr</code>	163
63.2.3	2.3 <code>std::weak_ptr</code>	164
63.2.4	2.4 <code>std::make_shared</code> vs <code>new</code>	164
63.3	3. Perfect Forwarding	164

64 Professional Notes: Chapter 33: Smart Pointers	165
65 Professional Notes: Chapter 106: Move Semantics	166
66 Professional Notes: Chapter 33: Smart Pointers	167
67 Professional Notes: Chapter 106: Move Semantics	172
68 FUNCTIONAL PROGRAMMING IN C++11	176
68.1 1. Lambda Expressions	176
68.1.1 1.1 Basic Syntax	176
68.1.2 1.2 Capture Lists	176
68.1.3 1.3 Mutable Lambdas	176
68.2 2. <code>std::function</code>	177
68.3 3. <code>std::bind</code>	177
69 Professional Notes: Chapter 52: <code>std::function</code>: To wrap any element that is callable	178
70 Professional Notes: Chapter 73: Lambdas	179
71 Professional Notes: Chapter 52: <code>std::function</code>: To wrap any	180
72 Professional Notes: Chapter 73: Lambdas	183
73 TEMPLATE METAPROGRAMMING & POWER FEATURES	184
73.1 1. Variadic Templates	184
73.1.1 1.1 Parameter Packing (...)	184
73.2 2. Type Traits	184
73.3 3. <code>using</code> Aliases	185
73.4 4. <code>std::tuple</code>	185
74 Professional Notes: Chapter 13: C++ Streams	186
75 Professional Notes: Chapter 16: Metaprogramming	188
76 Professional Notes: Chapter 16: Metaprogramming	189
77 THE C++11 STANDARD LIBRARY EXPANSION	193
77.1 1. Unordered Containers	193
77.2 2. <code>std::array</code>	193
77.3 3. <code>std::regex</code>	193
77.4 4. <code>std::chrono</code>	193
77.5 5. <code>std::random</code>	194
78 CONCURRENCY & MULTITHREADING	195
78.1 1. Threads (<code>std::thread</code>)	195
78.2 2. Mutexes & Locking	195
78.3 3. Condition Variables	195
78.4 4. Futures & Promises	196
78.5 5. Atomics (<code>std::atomic</code>)	196
79 Professional Notes: Chapter 80: Threading	197

80 Professional Notes: Chapter 85: Mutexes	198
81 Professional Notes: Chapter 55: std::atomics	199
82 Professional Notes: Chapter 80: Threading	200
83 Professional Notes: Chapter 85: Mutexes	204
 III Volume III: C++14 - Refinement Generics	 206
84 C++14 CORE LANGUAGE UPGRADES	207
84.1 1. Relaxed constexpr	207
84.1.1 1.1 What is Allowed?	207
84.2 2. Binary Literals	207
84.3 3. Digit Separators	207
84.4 4. [[deprecated]] Attribute	208
 85 C++14 FUNCTIONS & LAMBDA	 209
85.1 1. Generic Lambdas	209
85.2 2. Generalized Lambda Captures (Init-Capture)	209
85.3 3. Automatic Return Type Deduction	209
 86 C++14 TEMPLATES & METAPROGRAMMING	 210
86.1 1. Variable Templates	210
86.2 2. decltype(auto)	210
86.3 3. Standard Library Metafunctions	210
 87 C++14 STANDARD LIBRARY ENHANCEMENTS	 211
87.1 1. std::make_unique	211
87.2 2. Shared Locks (std::shared_timed_mutex)	211
87.3 3. std::exchange	211
87.4 4. std::quoted	211
87.5 5. Tuple Addressing by Type	212
 IV Volume IV: C++17 - Simplification Modernization	 213
88 C++17 CORE LANGUAGE UPGRADES	214
88.1 1. Structured Bindings	214
88.1.1 1.1 Unpacking Tuples and Pairs	214
88.1.2 1.2 Unpacking Structs	214
88.1.3 1.3 Modifiers (const, &)	214
88.2 2. if constexpr	215
88.3 3. Init-Statements for if and switch	215
88.4 4. Inline Variables	215
88.5 5. Nested Namespaces	216
88.6 6. Attributes	216
 89 C++17 TEMPLATE METAPROGRAMMING ENHANCEMENTS	 217
89.1 1. Fold Expressions	217
89.1.1 1.1 Unary Folds	217
89.1.2 1.2 Binary Folds	217

89.1.3	1.3 Operators	217
89.2	2. Class Template Argument Deduction (CTAD)	217
89.2.1	2.1 User-Defined Deduction Guides	218
89.3	3. auto Non-Type Template Parameters	218
89.4	4. std::invoke	218
90	C++17 VOCABULARY TYPES	220
90.1	1. std::string_view	220
90.1.1	1.1 Efficiency	220
90.1.2	1.2 Caveats	220
90.2	2. std::optional	220
90.3	3. std::variant	221
90.4	4. std::any	221
91	Professional Notes: Chapter 51: std::optional	222
92	Professional Notes: Chapter 56: std::variant	223
93	Professional Notes: Chapter 51: std::optional	224
94	Professional Notes: Chapter 56: std::variant	226
95	C++17 FILESYSTEM AND IO	228
95.1	1. std::filesystem	228
95.1.1	1.1 Paths	228
95.1.2	1.2 Iterating Directories	228
95.1.3	1.3 Operations	228
95.2	2. Polymorphic Allocators (std::pmr)	228
96	C++17 PARALLEL ALGORITHMS & CONCURRENCY	230
96.1	1. Parallel Algorithms (std::execution)	230
96.2	2. std::scoped_lock	230
96.3	3. std::shared_mutex	230
97	C++17 STANDARD LIBRARY ADDITIONS	232
97.1	1. std::byte	232
97.2	2. Algorithms	232
97.3	3. Mathematical Special Functions	232
97.4	4. Elementary String Conversions (<charconv>)	232
V	Volume V: C++20 - The Gigantic Leap	233
98	C++20 CONCEPTS	234
98.1	1. The Problem with Templates	234
98.2	2. What are Concepts?	234
98.2.1	2.1 Defining a Concept	234
98.2.2	2.2 Using Concepts (requires clause)	234
98.3	3. The requires Expression	235
98.4	4. Standard Concepts (<concepts>)	235
98.5	5. Constraint Based Overloading	235

99 C++20 MODULES	236
99.1 1. The Death of Headers	236
99.1.1 1.1 Problems with Headers	236
99.2 2. Basic Module Syntax	236
99.2.1 2.1 Interface Unit (<code>.cppm</code> or <code>.ixx</code>)	236
99.2.2 2.2 Importing a Module	236
99.3 3. Module Partitions	236
99.4 4. The Global Module Fragment	237
99.5 5. Build System Implications	237
100C++20 COROUTINES	238
100.11. Asynchronous Programming	238
100.1.1 1.1 Keywords	238
100.22. Generators (The Python Style)	238
100.33. Tasks (<code>co_await</code>)	239
100.44. Under the Hood	239
101C++20 RANGES	240
101.11. The End of Iterator pairs	240
101.22. Views vs Actions	240
101.33. Projections	240
101.44. Dangling Iterators Protection	241
102C++20 CORE LANGUAGE FEATURES	242
102.11. Three-Way Comparison (<code><=></code>)	242
102.22. Designated Initializers	242
102.33. <code>constexpr</code> (Immediate Functions)	242
102.44. <code>constexpr</code>	243
102.55. Non-Type Template Parameters (NTTP) Enhancements	243
103C++20 STANDARD LIBRARY ADDITIONS	244
103.11. <code>std::format</code> (Python-style Formatting)	244
103.22. <code>std::span</code>	244
103.33. <code>std::jthread</code> (Joinable Thread)	244
103.44. Concurrency Primitives (<code><semaphore></code> , <code><barrier></code> , <code><latch></code>)	245
103.55. Mathematical Constants (<code><numbers></code>)	245
103.66. Bit Manipulation (<code><bit></code>)	245
VI Volume VI: C++23/26 - The Future	246
104C++23 CORE LANGUAGE UPGRADES	247
104.11. Deducing <code>this</code> (Explicit Object Parameter)	247
104.1.1 1.1 The Problem (C++20 and older)	247
104.1.2 1.2 The Solution	247
104.1.3 1.3 Recursive Lambdas	247
104.22. <code>if constexpr</code>	248
104.33. Multidimensional Subscript Operator (<code>operator[]</code>)	248
104.44. <code>auto(x)</code> : Decay Copy	248
104.55. Literal Suffixes for <code>size_t</code>	248
104.66. <code>#warning</code>	248

105C++23 STD::PRINT & I/O	249
105.11. <code>std::print</code> and <code>std::println</code>	249
105.1.1 1.1 Basic Usage	249
105.1.2 1.2 Performance	249
105.1.3 1.3 Formatting	249
105.1.4 1.4 Output to Streams	249
105.22. <code>std::spanstream</code>	249
106C++23 MONADIC OPERATIONS & EXPECTED	251
106.11. <code>std::expected</code>	251
106.1.1 1.1 Basics	251
106.1.2 1.2 Monadic Chain	251
106.22. Monadic Operations for <code>std::optional</code>	252
107C++23 CONTAINERS & VIEWS	253
107.11. <code>std::mdspan</code>	253
107.1.1 1.1 Basics	253
107.1.2 1.2 Layouts	253
107.22. <code>std::flat_map</code> and <code>std::flat_set</code>	253
107.33. Ranges Enhancements	254
108C++23 COROUTINES & STACKTRACE	255
108.11. <code>std::generator</code>	255
108.1.1 1.1 Basic Generator	255
108.1.2 1.2 Recursive Generator	255
108.22. <code>std::stacktrace</code>	255
109C++23 LIBRARY UTILITIES	256
109.11. <code>std::unreachable</code>	256
109.22. <code>std::to_underlying</code>	256
109.33. <code>std::byteswap</code>	256
109.44. <code>std::stdatomic.h</code>	256
109.55. <code>std::move_only_function</code>	256
110THE FUTURE - C++26 PREVIEW	258
110.0.1 13.1 Static Reflection (<code>std::meta</code>)	258
110.0.2 13.2 Contracts	258
110.0.3 13.3 Senders & Receivers (<code>std::execution</code>)	259
110.0.4 13.4 Linear Algebra (<code>std::linalg</code>)	259
VII Volume VII: Advanced Topics Systems Architecture	261
111ADVANCED TEMPLATE METAPROGRAMMING	262
111.11. The Evolution of TMP	262
111.22. SFINAE (Substitution Failure Is Not An Error)	262
111.2.1 2.1 <code>std::enable_if</code>	262
111.2.2 2.2 The <code>void_t</code> Trick (C++17)	262
111.33. Curiously Recurring Template Pattern (CRTP)	263
111.44. Policy-Based Design	263
111.55. Modern TMP with Concepts (C++20)	263

112	COMPILE-TIME PROGRAMMING	264
112.11.	<code>constexpr</code> Evolution	264
112.22.	<code>constexpr</code> (C++20)	264
112.33.	Type Traits (<code><type_traits></code>)	264
112.3.13.1	Queries	264
112.3.23.2	Transformations	264
112.44.	<code>std::integral_constant</code>	265
112.55.	Reflection (C++26 Preview)	265
113	THE MEMORY MODEL & ATOMICS	266
113.11.	The C++ Memory Model	266
113.22.	Atomic Operations	266
113.33.	Memory Orderings	266
113.3.13.1	Synchronizes-With	266
113.44.	Fences	267
114	LOCK-FREE PROGRAMMING	268
114.11.	The Concept	268
114.22.	Compare-And-Swap (CAS)	268
114.33.	The ABA Problem	268
114.44.	Lock-Free Data Structures	268
115	ADVANCED CONCURRENCY PATTERNS	269
115.11.	Thread Pools	269
115.22.	The Actor Model	269
115.33.	Disruptor Pattern	269
115.44.	Coroutines for Concurrency	269
115.55.	False Sharing	270
116	CUSTOM MEMORY ALLOCATORS	271
116.11.	Why Custom Allocators?	271
116.22.	Linear / Stack Allocator	271
116.33.	Pool Allocator	271
116.44.	<code>std::pmr</code> (Polymorphic Memory Resources)	271
116.55.	Alignment	272
117	HIGH-PERFORMANCE OPTIMIZATION	273
117.11.	CPU Caching	273
117.22.	Branch Prediction	273
117.33.	SIMD (Single Instruction, Multiple Data)	273
117.44.	Link Time Optimization (LTO)	273
117.55.	Profile-Guided Optimization (PGO)	273
118	Professional Notes: Chapter 143: Optimization in C++	274
119	Professional Notes: Chapter 143: Optimization in C++	275
120	WRITING A C++ COMPILER (BASICS)	278
120.0.1 18.1	Lexical Analysis (Tokenizer)	278
120.0.2 18.2	Parsing (Recursive Descent)	278
120.0.3 18.3	Semantic Analysis (Types & Scopes)	278

121	WRITING A GARBAGE COLLECTOR	280
121.0.1	29.1 Mark-and-Sweep Basics	280
122	THE STANDARD LIBRARY FROM SCRATCH	281
122.0.1	19.1 Implementing my::vector	281
122.0.2	19.2 Implementing my::shared_ptr	281
VIII	Volume VIII: Specialized Domains Expert Mastery	283
123	DISTRIBUTED C++	284
123.0.1	16.1 Serialization (Binary Protocols)	284
123.0.2	16.2 RPC (Remote Procedure Call) Concept	284
123.0.3	16.3 Consensus (Raft Basics)	285
124	NETWORKING FROM SCRATCH	286
124.0.1	28.1 Berkeley Sockets API	286
124.0.2	28.2 Non-Blocking I/O & Epoll (Linux)	286
125	C++ IN THE CLOUD	288
125.0.1	20.1 Microservices with C++	288
125.0.2	20.2 Serverless C++ (AWS Lambda)	288
126	CROSS-PLATFORM DEVELOPMENT	289
126.0.1	21.1 WebAssembly (Wasm) with Emscripten	289
126.0.2	21.2 Mobile C++ (Android NDK & JNI)	289
127	GUI DEVELOPMENT WITH C++	290
127.0.1	22.1 Qt Framework (Retained Mode)	290
127.0.2	22.2 Dear ImGui (Immediate Mode)	290
128	SCIENTIFIC COMPUTING & GPU	291
128.0.1	23.1 Eigen (Linear Algebra)	291
128.0.2	23.2 CUDA (GPU Programming)	291
129	INTEROPERABILITY	292
129.0.1	35.1 Python Bindings (pybind11)	292
129.0.2	35.2 Java Native Interface (JNI)	292
129.0.3	35.3 Stable C ABI	292
130	SECURITY ENGINEERING	293
130.0.1	36.1 Fuzzing (libFuzzer / AFL++)	293
130.0.2	36.2 Cryptography & Timing Attacks	293
130.0.3	36.3 Side-Channel Mitigations (Spectre)	293
131	SPECIALIZED DOMAINS	294
131.0.1	37.1 Game Development (Data-Oriented Design)	294
131.0.2	37.2 Embedded Systems	294
131.0.3	37.3 High-Frequency Trading (HFT)	294
131.0.4	37.4 Automotive (MISRA C++ / AUTOSAR)	294
131.1	FINAL COMPREHENSIVE CHECKLIST	295
131.1.1	C++98/03 Foundation	295
131.1.2	C++11 Major Features	295

131.1.3 C++14 Improvements	295
131.1.4 C++17 Modern Features	295
131.1.5 C++20 Revolutionary	295
131.1.6 C++23 Latest	296
131.1.7 Advanced Concepts	296
131.1.8 Concurrency	296
131.1.9 Performance & Optimization	296
131.1.10 STL Mastery	296
131.1.11 Design Patterns	296
131.1.12 Professional Development	297
131.2 Key Insights for C++ Mastery	297
131.3 Learning Path	297
131.4 Resources	297
131.4.1 Official Documentation	297
131.4.2 Books	298
131.4.3 Practice	298
132 ABA PROBLEM & MEMORY RECLAMATION	299
132.0.1.1. The ABA Problem Explained	299
132.0.2.2. Solution I: Tagged Pointers (Version Counters)	299
132.0.3.3. Solution II: Hazard Pointers (The Gold Standard)	299
132.0.4.4. Solution III: Epoch-Based Reclamation (EBR)	300
133 TEMPLATE METAPROGRAMMING PATTERNS	301
133.0.1.1. Expression Templates (Lazy Evaluation)	301
133.0.2.2. Type Erasure (The <code>std::any</code> Pattern)	301
133.0.3.3. The Detection Idiom (<code>void_t</code>)	301
133.0.4.4. Policy-Based Design	302
134 HIGH-PERFORMANCE DATA STRUCTURES	303
134.0.1.1. The Disruptor (Ring Buffer on Steroids)	303
134.0.2.2. Swiss Table (Open Addressing + Metadata)	303
134.0.3.3. Burst Tries / Judy Arrays	303
134.0.4.4. Slot Map	303
135 REAL-TIME AUDIO & SIGNAL PROCESSING	304
135.0.1.1. Lock-Free IPC (SPSC Queue)	304
135.0.2.2. SIMD for DSP	304
135.0.3.3. Double Buffering	304
136 ROBOTICS & ROS2 DEVELOPMENT	305
136.0.1.1. ROS2 Architecture & DDS	305
136.0.2.2. Zero-Copy Transport (Iceoryx)	305
136.0.3.3. Real-Time Executors	305
136.0.4.4. Custom Allocators (<code>std::pmr</code>)	305
137 MACHINE LEARNING INFRASTRUCTURE	306
137.0.1.1. Tensor Memory Layout	306
137.0.2.2. Broadcasting Implementation	306
137.0.3.3. Automatic Differentiation (Autograd)	306
137.0.4.4. Operator Fusion	306

138	DATABASE INTERNALS (LSM TREES)	307
138.0.1	1. The Write Problem	307
138.0.2	2. LSM Tree (Log-Structured Merge Tree)	307
138.0.3	3. Bloom Filters	307
138.0.4	4. Memory Mapped I/O (mmap)	307
139	THE ULTIMATE ALGORITHM REFERENCE	308
139.0.1	27.1 Non-Modifying Sequence Operations	308
139.0.2	27.2 Modifying Sequence Operations	308
139.0.3	27.3 Partitioning	308
139.0.4	27.4 Sorting	308
139.0.5	27.5 Binary Search (On Sorted Ranges)	308
139.0.6	27.6 Numeric Operations ()	309
140	CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK	310
140.0.1	Project Structure	310
140.0.2	1. Types Module (types.cppm)	310
140.0.3	2. Order Module (order.cppm)	310
140.0.4	3. Order Book Module (book.cppm)	311
140.0.5	4. Main Application (main.cpp)	312
141	APPENDICES	313
A	Appendix A: C++ Keywords & Operators Reference	314
A.0.1	Essential Keywords (Non-Exhaustive)	314
A.0.2	Special Operators	314
B	Appendix B: Common Acronyms	315
C	Appendix C: Recommended Tooling	316
C.0.1	Compilers	316
C.0.2	Build Systems	316
C.0.3	Package Managers	316
C.0.4	Static Analysis & Sanitizers	316
D	Appendix D: Common C++ Traps & Pitfalls	317
D.0.1	I. General & Syntax Traps	317
D.0.2	II. Pointers, References & Memory	317
D.0.3	III. Classes & OOP	318
D.0.4	IV. Concurrency	318
D.0.5	V. Modern C++ & Macros	319
E	Appendix E: C++ Interview Cheat Sheet	320
E.0.1	Core Concepts	320
E.0.2	Modern C++ (C++11/14/17/20)	320
E.0.3	System Design Questions	320
E.0.4	Quick Coding	321
F	Appendix F: The C++ Standard Evolution Matrix	322
F.0.1	1. Versioned Changelog	322
F.0.2	2. Feature Matrix	323
F.0.3	3. Timeline & Release Accuracy	323

G	Appendix G: C++ Standard Library Headers Reference	324
G.0.1	Concepts & Utilities	324
G.0.2	Containers	324
G.0.3	Algorithms & Iterators	324
G.0.4	Concurrency	324
G.0.5	Input/Output	325
G.0.6	Numerics & Math	325
H	Appendix H: Professional C++ Idioms	326
H.0.1	1. RAII (Resource Acquisition Is Initialization)	326
H.0.2	2. Pimpl (Pointer to Implementation)	326
H.0.3	3. Copy-and-Swap	326
H.0.4	4. NVI (Non-Virtual Interface)	326
H.0.5	5. Erase-Remove Idiom	326
H.0.6	6. SFINAE (Substitution Failure Is Not An Error)	326
H.0.7	7. CRTP (Curiously Recurring Template Pattern)	327

Chapter 1

Preface

Chapter 2

Preface

2.1 The Complete C++ Programmer’s Guide: From Zero to Godhood (C++98 to C++26)

Author: Shreejit Verma

2.1.1 About the Book

This book is the culmination of a decade-long journey through the depths of C++. It is written not just as a reference, but as a comprehensive guide for those who wish to transcend the level of a “user” and become a “master” of the language.

The “Zero to Godhood” series was born from a frustration with existing resources. Tutorials often stop at syntax, leaving engineers ill-equipped for the brutal reality of high-frequency trading, kernel development, and large-scale distributed systems. This book bridges that gap.

We cover the entire spectrum: * **The Archaic:** Understanding C++98/03 legacy codebases that still power the world’s infrastructure. * **The Modern:** Mastering C++11 through C++20, where the language found its renaissance. * **The Future:** A forward-looking view into C++23 and C++26, preparing you for the next decade. * **The Metal:** Deep dives into memory models, lock-free concurrency, custom allocators, and compiler intrinsics.

This is a book for those who want to know *how* the machine works, not just how to talk to it.

2.1.2 About the Author

Shreejit Verma is a Senior Software Engineer and Quantitative Developer specializing in low-latency systems and high-performance computing. With extensive experience in building distributed trading platforms, optimizing critical infrastructure, and architecting scalable C++ solutions, Shreejit brings a practical, engineering-first perspective to the language.

His philosophy is simple: **“Performance is not an accident. It is an architectural decision.”**

Shreejit has mentored hundreds of engineers, helping them transition from junior roles to architects by demystifying the complexities of C++ and the hardware it runs on. This book is the crystallized essence of that mentorship.

2.1.3 How to Use This Book

1. **The Foundations (Volume I):** Essential for everyone. Even experts should review the compilation model and object virtualization chapters.
2. **The Modern Era (Volumes II-V):** The core toolkit for the professional developer.

3. **The Expert Domains (Volumes VII-VIII):** For those seeking mastery in specific fields like HFT, Systems, and Graphics.

Prepare yourself. We are about to master the beast.

Chapter 3

VOLUME 01 FOUNDATION C98 03

Part I

Volume I: C++98/03 - The Foundation

Chapter 4

FOUNDATIONS AND COMPILATION

Chapter 5

FOUNDATIONS & COMPILATION MODEL

Chapter 6

ABSOLUTE BASICS (C++98)

6.1 Getting Started

6.1.1 What is C++?

C++ is a statically-typed, compiled programming language that combines low-level memory manipulation with high-level abstractions. It's the language of choice for performance-critical applications.

6.1.2 Your First Program (C++98)

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!" << std::endl;
5     return 0;
6 }
```

6.1.3 Professional Notes: Basics & I/O

1. The Compilation Pipeline (Deep Dive)

Compilation in C++ is a multi-stage process that transforms human-readable source code into machine-executable binaries.

1. **Preprocessing:** The preprocessor (`cpp`) handles directives starting with `\#`. It includes headers (`\#include`) and expands macros (`\#define`).
2. **Compilation:** The compiler (`g++`, `clang++`) translates the preprocessed source into assembly code.
3. **Assembly:** The assembler (`as`) converts assembly into machine-readable object files (`.o` or `.obj`).
4. **Linking:** The linker (`ld`) combines multiple object files and libraries into a single executable, resolving symbols and addresses.

2. Standard Streams and Buffered I/O

C++ provides a robust I/O system based on streams.

- * `std::cout`: Buffered output stream (standard output).
- * `std::cin`: Buffered input stream (standard input).
- * `std::cerr`: Unbuffered output stream (standard error).
- * `std::clog`: Buffered output stream (logging standard error).

Godhood Tip: Use `std::endl` only when you need to flush the buffer (e.g., in real-time logging). For performance, use `'\\n'` to avoid unnecessary flushes.

3. Stream State and Error Handling

Always check the state of a stream after an operation:

```

1 int x;
2 if (!(std::cin >> x)) {
3     std::cin.clear(); // Clear the error flags
4     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); //
    Discard bad input
5     std::cerr << "Invalid input received!" << std::endl;
6 }

```

Stream states include `good()`, `bad()`, `fail()`, and `eof()`.

Breakdown: - `\#include <iostream>` - Include input/output library - `std::cout` - Standard output stream (print to console) - `std::endl` - End line and flush buffer - `main()` - Entry point of program - `return 0` - Exit code (0 = success)

Compile and run:

```

1 g++ -o hello hello.cpp
2 ./hello

```

6.2 Basic Types & Variables

6.2.1 Fundamental Types (C++98)

```

1 #include <iostream>
2 #include <limits>
3
4 int main() {
5     // Integer types
6     int x = 42; // 32-bit integer
7     short y = 10; // 16-bit integer
8     long z = 1000000; // 32 or 64-bit integer
9     long long w = 9999999999; // 64-bit integer
10
11     // Floating-point types
12     float f = 3.14f; // 32-bit (4 bytes)
13     double d = 3.14159265; // 64-bit (8 bytes)
14     long double ld = 3.14159265359L; // 80+ bits
15
16     // Character and boolean types
17     char c = 'A'; // Single byte
18     bool b = true; // true or false
19
20     // Print sizes
21     std::cout << "int: " << sizeof(int) << " bytes\n";
22     std::cout << "double: " << sizeof(double) << " bytes\n";
23
24     // Min/max values
25     std::cout << "int max: " << std::numeric_limits<int>::max() << "\n";
26     std::cout << "int min: " << std::numeric_limits<int>::min() << "\n";
27
28     return 0;
29 }

```

c

1. Integ

3. Core Keywords and Type Qualifiers * `const`: Declares a variable as read-only. * `volatile`: Tells the

6.2.2 Variable Declaration & Initialization

```

1 #include <iostream>
2
3 int main() {
4     // C-style initialization
5     int x = 5;
6     float f = 3.14f;
7
8     // Multiple variables
9     int a, b, c;
10
11     // Uninitialized (dangerous - contains garbage)
12     int uninitialized;
13     std::cout << uninitialized << "\n"; // Undefined behavior!
14
15     // Constants
16     const int MAX_SIZE = 100;
17     // MAX_SIZE = 200; // Error: can't modify const
18
19     return 0;
20 }

```

6.2.3 Scope & Lifetime (C++98)

```

1 #include <iostream>
2
3 int global = 100; // Global scope - lives entire program
4
5 void function() {
6     int local = 5; // Local scope - lives only in function
7     {
8         int nested = 10; // Block scope - lives only in block
9         std::cout << nested << "\n";
10    }
11    // std::cout << nested << "\n"; // Error: nested out of scope
12 }
13
14 int main() {
15     {
16         int x = 5;
17     }
18    // std::cout << x << "\n"; // Error: x out of scope
19
20    return 0;
21 }

```

6.2.4 Deep Dive: The Memory Model of Variables

Understanding *where* your variables live is the first step to Godhood.

1. The Stack (Automatic Storage):

- **What:** Local variables (`int x`).
- **Speed:** Extremely fast (just moving a pointer).
- **Lifetime:** Scope-based (die at `}`).
- **Limit:** Small (typically 1MB-8MB). Recursion depth is limited by this.

2. The Heap (Dynamic Storage):

- **What:** `new int`, `malloc`.
- **Speed:** Slower (allocation requires finding free block).
- **Lifetime:** Manual (until `delete` / `free`).
- **Limit:** RAM size (Gigabytes).

3. Static/Global (Static Storage):

- **What:** Global variables, `static` locals.
- **Speed:** Fast access, but initialization order is tricky.
- **Lifetime:** Program start to program end.

4. Registers:

- **What:** CPU internal storage.
- **Speed:** Instant (0 cycles).
- **Note:** Variables are often optimized into registers, never touching RAM!

6.3 Operators & Control Flow

6.3.1 Arithmetic Operators (C++98)

```

1 #include <iostream>
2
3 int main() {
4     int a = 10, b = 3;
5
6     std::cout << a + b << "\n";    // 13 (addition)
7     std::cout << a - b << "\n";    // 7 (subtraction)
8     std::cout << a * b << "\n";    // 30 (multiplication)
9     std::cout << a / b << "\n";    // 3 (integer division)
10    std::cout << a % b << "\n";    // 1 (modulo)
11
12    // Compound assignment
13    int x = 5;
14    x += 3;    // x = 8
15    x -= 2;    // x = 6
16    x *= 2;    // x = 12
17    x /= 3;    // x = 4
18
19    // Increment/Decrement
20    int y = 5;
21    y++;        // Post-increment: 6
22    ++y;        // Pre-increment: 7

```

```
23     y--;        // Post-decrement: 6
24     --y;        // Pre-decrement: 5
25
26     return 0;
27 }
```

6.3.2 Comparison & Logical Operators (C++98)

```
1  #include <iostream>
2
3  int main() {
4      int a = 10, b = 5;
5
6      // Comparison (return true/false)
7      std::cout << (a > b) << "\n";    // 1 (true)
8      std::cout << (a < b) << "\n";    // 0 (false)
9      std::cout << (a == b) << "\n";   // 0 (false)
10     std::cout << (a != b) << "\n";   // 1 (true)
11     std::cout << (a >= b) << "\n";   // 1 (true)
12     std::cout << (a <= b) << "\n";   // 0 (false)
13
14     // Logical operators
15     bool x = true, y = false;
16     std::cout << (x && y) << "\n";   // 0 (AND)
17     std::cout << (x || y) << "\n";   // 1 (OR)
18     std::cout << (!x) << "\n";      // 0 (NOT)
19
20     return 0;
21 }
```

6.3.3 If-Else Statements (C++98)

```
1  #include <iostream>
2
3  int main() {
4      int score = 85;
5
6      // Basic if-else
7      if (score >= 90) {
8          std::cout << "Grade: A\n";
9      } else if (score >= 80) {
10         std::cout << "Grade: B\n";
11     } else if (score >= 70) {
12         std::cout << "Grade: C\n";
13     } else {
14         std::cout << "Grade: F\n";
15     }
16
17     // Ternary operator
18     std::string grade = (score >= 80) ? "Pass" : "Fail";
19     std::cout << grade << "\n";
20
21     return 0;
22 }
```

6.3.4 Loops (C++98)

```
1 #include <iostream>
2
3 int main() {
4     // While loop
5     int i = 0;
6     while (i < 5) {
7         std::cout << i << " ";
8         i++;
9     }
10    std::cout << "\n";
11
12    // Do-while loop (executes at least once)
13    int j = 0;
14    do {
15        std::cout << j << " ";
16        j++;
17    } while (j < 5);
18    std::cout << "\n";
19
20    // For loop
21    for (int k = 0; k < 5; k++) {
22        std::cout << k << " ";
23    }
24    std::cout << "\n";
25
26    // Break and continue
27    for (int m = 0; m < 10; m++) {
28        if (m == 3) continue; // Skip 3
29        if (m == 7) break;    // Exit at 7
30        std::cout << m << " ";
31    }
32    std::cout << "\n";
33
34    return 0;
35 }
```

6.3.5 Switch Statement (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int day = 3;
5
6     switch (day) {
7         case 1:
8             std::cout << "Monday\n";
9             break;
10        case 2:
11            std::cout << "Tuesday\n";
12            break;
13        case 3:
14            std::cout << "Wednesday\n";
15            break;
16        default:
17            std::cout << "Unknown day\n";
18    }
19
20    return 0;
21 }
```

1. Operator Precedence and Associativity C++ has a strict hierarchy

2. Advanced Loop Patterns * **Range-based for**

3. Flow Control Quirks * **switch Fallthrough**: Without a **break**, execution continues to the next case

6.4 Functions

6.4.1 Function Declaration & Definition (C++98)

```
1 #include <iostream>
2
3 // Function declaration (prototype)
4 int add(int a, int b);
5 void print_hello();
6
7 // Function definition
8 int add(int a, int b) {
9     return a + b;
10 }
11
12 void print_hello() {
13     std::cout << "Hello!\n";
14 }
15
16 int main() {
17     print_hello();
18     std::cout << add(5, 3) << "\n"; // 8
19     return 0;
20 }
```

6.4.2 Parameters & Return Values (C++98)

```
1 #include <iostream>
2
3 // Pass by value (copy)
4 void increment_value(int x) {
5     x++;
6     std::cout << "Inside: " << x << "\n";
7 }
8
9 // Pass by reference (same variable)
10 void increment_ref(int& x) {
11     x++;
12     std::cout << "Inside: " << x << "\n";
13 }
14
15 // Pass by const reference (can't modify)
16 void print_value(const int& x) {
17     std::cout << x << "\n";
18 }
19
20 // Returning by value
21 int get_value() {
22     return 42;
23 }
```



```
23 }
24
25 // Returning by reference (dangerous!)
26 int& get_global() {
27     static int x = 100;
28     return x;
29 }
30
31 int main() {
32     int a = 5;
33
34     increment_value(a);    // Copy passed
35     std::cout << a << "\n"; // Still 5
36
37     increment_ref(a);      // Reference passed
38     std::cout << a << "\n"; // Now 6
39
40     print_value(a);        // Can't modify a
41
42     return 0;
43 }
```

6.4.3 Default Parameters (C++98)

```
1 #include <iostream>
2
3 void greet(const std::string& name = "World") {
4     std::cout << "Hello, " << name << "!\n";
5 }
6
7 int main() {
8     greet();                // Uses default: "World"
9     greet("Alice");         // Uses provided: "Alice"
10    return 0;
11 }
```

6.4.4 Function Overloading (C++98)

```
1 #include <iostream>
2
3 // Same function name, different parameters
4 int add(int a, int b) {
5     return a + b;
6 }
7
8 double add(double a, double b) {
9     return a + b;
10 }
11
12 void print(int x) {
13     std::cout << "Integer: " << x << "\n";
14 }
15
16 void print(double x) {
17     std::cout << "Double: " << x << "\n";
18 }
19
20 void print(const std::string& s) {
21     std::cout << "String: " << s << "\n";
22 }
```

```
23
24 int main() {
25     std::cout << add(5, 3) << "\n";      // 8 (int version)
26     std::cout << add(2.5, 3.7) << "\n";  // 6.2 (double version)
27
28     print(42);          // Integer version
29     print(3.14);        // Double version
30     print("Hello");     // String version
31
32     return 0;
33 }
```

6.5 Arrays & Pointers

6.5.1 Arrays (C++98)

```
1 #include <iostream>
2
3 int main() {
4     // Array declaration and initialization
5     int arr[5] = {1, 2, 3, 4, 5};
6
7     // Access elements (0-indexed)
8     std::cout << arr[0] << "\n"; // 1
9     std::cout << arr[4] << "\n"; // 5
10
11    // Array size (local arrays only)
12    int size = 5;
13    for (int i = 0; i < size; i++) {
14        std::cout << arr[i] << " ";
15    }
16    std::cout << "\n";
17
18    // 2D array
19    int matrix[3][3] = {
20        {1, 2, 3},
21        {4, 5, 6},
22        {7, 8, 9}
23    };
24
25    std::cout << matrix[1][2] << "\n"; // 6
26
27    return 0;
28 }
```

6.5.2 Pointers (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int x = 42;
5
6     // Create a pointer
7     int* ptr = &x; // & = address-of operator
8
9     // Dereference pointer
10    std::cout << *ptr << "\n"; // 42 (dereference with *)

```

```
11
12 // Modify through pointer
13 *ptr = 100;
14 std::cout << x << "\n"; // 100
15
16 // Pointer arithmetic
17 int arr[5] = {10, 20, 30, 40, 50};
18 int* p = arr; // Array decays to pointer
19
20 std::cout << p[0] << "\n"; // 10
21 std::cout << *(p+1) << "\n"; // 20
22 std::cout << p[2] << "\n"; // 30
23
24 // Null pointer
25 int* null_ptr = nullptr; // or NULL in C++98
26
27 // Pointer to pointer
28 int** pp = &ptr;
29 std::cout << **pp << "\n"; // 100
30
31 return 0;
32 }
```

6.5.3 Dynamic Memory (C++98)

```
1 #include <iostream>
2
3 int main() {
4     // Allocate single value on heap
5     int* ptr = new int;
6     *ptr = 42;
7     std::cout << *ptr << "\n";
8     delete ptr; // Must deallocate
9     ptr = nullptr; // Good practice
10
11     // Allocate array on heap
12     int* arr = new int[10];
13     for (int i = 0; i < 10; i++) {
14         arr[i] = i * i;
15     }
16     delete[] arr; // Note the [] for arrays
17
18     // Forgetting to delete = memory leak
19     int* leaked = new int(100);
20     // delete leaked; // Forgot this!
21
22     return 0;
23 }
```

Chapter 7

THE C++ COMPILATION & EXECUTION MODEL

To truly understand C++, you must understand how your code transforms from text to a running process. This section demystifies the “black box” of the compiler.

7.0.1 1.5.1 The Build Pipeline: From Source to Binary

The “compilation” process actually consists of four distinct stages:

1. **Preprocessing:** Text substitution.

- Removes comments.
- Expands macros (`\#define`).
- Includes headers (`\#include`) recursively.
- Handles conditionals (`\#ifdef`).
- *Output:* A single “Translation Unit” (pure C++ source code).

2. **Compilation:** Syntax to Assembly.

- **Lexical Analysis:** Tokenizes source (e.g., `int`, `main`, `{`).
- **Parsing:** Builds Abstract Syntax Tree (AST).
- **Semantic Analysis:** Type checking, overload resolution.
- **Optimization:** Dead code elimination, loop unrolling, inlining (O1, O2, O3).
- **Code Generation:** Generates assembly code for the target architecture (x86_64, ARM64).
- *Output:* Assembly file (`.s` or `.asm`).

3. **Assembly:** Assembly to Machine Code.

- Translates mnemonics (`MOV`, `ADD`) to opcodes (`0x89`, `0x01`).
- *Output:* Object file (`.o` or `.obj`). Contains machine code but with *unresolved symbols*.

4. **Linking:** Combining Object Files.

- Combines multiple `.o` files and static libraries (`.a/.lib`).
- Resolves symbols: Matches function calls in one TU to definitions in another.
- Relocates addresses: Adjusts internal pointers.
- *Output:* Executable (`.exe` or ELF/Mach-O) or Shared Library (`.so/.dll`).

7.0.2 1.5.2 Translation Units (TU) & Linkage

A **Translation Unit** is the input to the compiler after preprocessing. It is the fundamental unit of compilation.

Linkage Types

Linkage determines if a name (variable/function) is visible to other TUs.

1. No Linkage:

- Visible only within the current scope (block).
- Example: Local variables.

2. Internal Linkage:

- Visible only within the current Translation Unit.
- Hidden from the linker.
- Examples: `static` global variables, `static` free functions, `const` globals (by default).
- *Use Case*: Private helper functions.

3. External Linkage:

- Visible to the linker and other TUs.
- Examples: Global variables, non-static functions, `extern` variables.
- *Danger*: Requires strict ODR compliance.

```

1 // TU1.cpp
2 static int internal_var = 10; // Internal Linkage
3 int global_var = 20;         // External Linkage
4
5 // TU2.cpp
6 extern int global_var;        // Declares existence of external symbol
7 // extern int internal_var;   // ERROR: Linker error (symbol not found)

```

7.0.3 1.5.3 The One Definition Rule (ODR)

The ODR is the most important rule in C++ linking.

1. **ODR for Translation Units**: A function/variable can have only **one definition** in a single TU. (Declarations can be repeated).
2. **ODR for Programs**: A non-inline function/variable can have only **one definition** in the entire program.
3. **ODR for Classes/Templates**: Can be defined in multiple TUs (via headers), but definitions must be **token-for-token identical**.

Violation Example:

```
1 // header.h
2 int x = 10; // DEFINITION (allocates memory)
3
4 // a.cpp
5 #include "header.h"
6
7 // b.cpp
8 #include "header.h"
9
10 // Linker Error: multiple definition of 'x'
11 // Fix: Use 'extern int x;' in header, definition in one .cpp
12 // Fix (C++17): Use 'inline int x = 10;'
```

7.0.4 1.5.4 Process Memory Layout

When your C++ program runs, the OS allocates virtual memory segments:

1. Text Segment (.text):

- Contains executable machine instructions.
- Read-Only (prevents self-modifying code exploits).
- Shared between multiple instances of the app.

2. Data Segment (.data):

- Initialized global and static variables.
- `int g = 10;` lives here.

3. BSS Segment (.bss):

- Uninitialized global/static variables.
- `int g;` lives here.
- Automatically zero-initialized by OS before `main()`.

4. Heap:

- Dynamic memory (`new`, `malloc`).
- Grows upwards (typically).
- Managed manually or by smart pointers.

5. Stack:

- Local variables, function parameters, return addresses.
- Grows downwards (typically).
- LIFO (Last-In, First-Out).
- *Stack Overflow*: Exceeding stack size (e.g., deep recursion).

7.0.5 1.5.5 Program Startup: Before main()

main() is NOT the first thing to run.

1. **OS Loader:** Loads EXE into memory.
2. **Entry Point** (_start): Defined by C Runtime (CRT).
3. **Global Initialization:**
 - Constructors of global/static objects run **before** main.
 - *Static Initialization Order Fiasco:* Order between TUs is undefined.
4. **main():** Your code runs.
5. **Global Destruction:**
 - Destructors of global/static objects run **after** main returns.
 - atexit handlers run.

7.0.6 1.5.6 Deep Dive into Data Representation

To understand bugs like integer overflow and floating-point inaccuracy, you must know how data is stored bits-by-bits.

Integer Representation (Two's Complement)

Most modern systems use **Two's Complement** for signed integers. * **Positive Numbers:** Standard binary. 0000 0101 (5). * **Negative Numbers:** Invert all bits and add 1. * 5 = 0000 0101 * Invert: 1111 1010 * Add 1: 1111 1011 (-5) * **Advantage:** Addition/Subtraction works identically for signed/unsigned. * **Range:** $-2^{(N-1)}$ to $2^{(N-1)} - 1$. (One extra negative number).

Floating Point (IEEE 754)

float (32-bit) and double (64-bit) follow IEEE 754. * **Components:** 1. **Sign Bit:** 0 (+) or 1 (-). 2. **Exponent:** Biased integer (allows very large/small numbers). 3. **Mantissa (Significand):** The precision bits (normalized to 1.xxxxx). * **Implication:** * $0.1 + 0.2 \neq 0.3$ because 0.1 cannot be represented exactly in binary (infinite repeating fraction). * **NaN (Not a Number):** Result of 0/0 or sqrt(-1). NaN != NaN is always true. * **Infinity:** Result of 1.0/0.0.

```

1 #include <iostream>
2 #include <cmath>
3 #include <limits>
4
5 int main() {
6     float a = 0.1f;
7     float b = 0.2f;
8     if (a + b == 0.3f) {
9         std::cout << "Math works!\n";
10    } else {
11        std::cout << "Math is broken: " << (a+b) << "\n"; // Prints 0.30000001
12    }
13
14    // Correct comparison
15    if (std::abs((a + b) - 0.3f) < 1e-5) {
16        std::cout << "Close enough!\n";
17    }
18 }

```


Chapter 8

Professional Notes: Chapter 1: Getting started with C++

Version C++98 ISO/IEC 14882:1998 1998-09-01 Standard Release Date C++03 ISO/IEC 14882:2003 2003-10-16 C++11 ISO/IEC 14882:2011 2011-09-01 C++14 ISO/IEC 14882:2014 2014-12-15 C++17 TBD C++20 TBD 2017-01-01 2020-01-01 Section 1.1: Hello World This program prints Hello World! to the standard output stream: `#include <iostream> int main() { std::cout << "Hello World!" << std::endl; }` See it live on Coliru. Analysis Let's examine each part of this code in detail: `#include` is a preprocessor directive that includes the content of the standard C++ header `iostream`. `iostream` is a standard library header file that contains definitions of the standard input and output streams. These definitions are included in the `std` namespace, explained below. The standard input/output (I/O) streams provide ways for programs to get input from and output to an external system – usually the terminal. `int main() { ... }` denotes a new function named `main`. By convention, the `main` function is called upon execution of the program. There must be only one `main` function in a C++ program, and it must always return a number of the `int` type. Here, the `int` is what is called the function's return type. The value returned by the `main` function is an exit code. By convention, a program exit code of 0 or `EXIT_SUCCESS` is interpreted as success by a system that executes the program. Any other return code is associated with an error. If no return statement is present, the `main` function (and thus, the program itself) returns 0 by default. In this example, we don't need to explicitly write `return 0;`. All other functions, except those that return the `void` type, must explicitly return a value according to their return type, or else must not return at all.

`std::cout << "Hello World!" << std::endl;` prints "Hello World!" to the standard output stream: `std` is a namespace, and `::` is the scope resolution operator that allows look-ups for objects by name within a namespace. There are many namespaces. Here, we use `::` to show we want to use `cout` from the `std` namespace. For more information refer to Scope Resolution Operator - Microsoft Documentation. `std::cout` is the standard output stream object, defined in `iostream`, and it prints to the standard output (`stdout`). `<<` is, in this context, the stream insertion operator, so called because it inserts an object into the stream object. The standard library defines the `<<` operator to perform data insertion for certain data types into output streams. `stream << content` inserts `content` into the stream and returns the same, but updated stream. This allows stream insertions to be chained: `std::cout << "Foo" << " Bar";` prints "FooBar" to the console. "Hello World!" is a character string literal, or a "text literal." The stream insertion operator for character string literals is defined in `iostream`. `std::endl` is a special I/O stream manipulator object, also defined in `iostream`. Inserting a manipulator into a stream changes the state of the stream. The stream manipulator `std::endl` does two things: first it inserts the end-of-line character and then it flushes the stream buffer to force the text to show up on the console. This ensures that the data inserted into the stream actually appear on your console. (Stream data is usually stored in a buffer and then "flushed" in batches unless you force a flush immediately.) An alternate

method that avoids the `ush` is: `std::cout << "Hello World!";` where `\n` is the character escape sequence for the newline character. The semicolon (`;`) notifies the compiler that a statement has ended. All C++ statements and class definitions require an ending/terminating semicolon. Section 1.2: Comments A comment is a way to put arbitrary text inside source code without having the C++ compiler interpret it with any functional meaning. Comments are used to give insight into the design or method of a program. There are two types of comments in C++: Single-Line Comments The double forward-slash sequence `//` will mark all text until a newline as a comment: `int main() {`

`// This is a single-line comment. int a; // this also is a single-line comment int i; // this is another single-line comment }` C-Style/Block Comments The sequence `/*` is used to declare the start of the comment block and the sequence `*/` is used to declare the end of comment. All text between the start and end sequences is interpreted as a comment, even if the text is otherwise valid C++ syntax. These are sometimes called “C-style” comments, as this comment syntax is inherited from C++’s predecessor language, C: `int main() { /* This is a block comment. / int a; }` In any block comment, you can write anything you want. When the compiler encounters the symbol `/`, it terminates the block comment: `int main() { /* A block comment with the symbol / Note that the compiler is not affected by the second / however, once the end-block-comment symbol is reached, the comment ends. / int a; }` The above example is valid C++ (and C) code. However, having additional `/` inside a block comment might result in a warning on some compilers. Block comments can also start and end within a single line. For example: `void SomeFunction(/* argument 1 / int a, / argument 2 / int b);` Importance of Comments As with all programming languages, comments provide several benefits: Explicit documentation of code to make it easier to read/maintain Explanation of the purpose and functionality of code Details on the history or reasoning behind the code Placement of copyright/licenses, project notes, special thanks, contributor credits, etc. directly in the source code. However, comments also have their downsides: They must be maintained to reflect any changes in the code Excessive comments tend to make the code less readable The need for comments can be reduced by writing clear, self-documenting code. A simple example is the use of explanatory names for variables, functions, and types. Factoring out logically related tasks into discrete functions goes hand-in-hand with this.

Comment markers used to disable code During development, comments can also be used to quickly disable portions of code without deleting it. This is often useful for testing or debugging purposes, but is not good style for anything other than temporary edits. This is often referred to as commenting out. Similarly, keeping old versions of a piece of code in a comment for reference purposes is frowned upon, as it clutters less while offering little value compared to exploring the code’s history via a versioning system. Section 1.3: The standard C++ compilation process Executable C++ program code is usually produced by a compiler. A compiler is a program that translates code from a programming language into another form which is (more) directly executable for a computer. Using a compiler to translate code is called compilation. C++ inherits the form of its compilation process from its “parent” language, C. Below is a list showing the four major steps of compilation in C++: 1. The C++ preprocessor copies the contents of any included header files into the source code file, generates macro code, and replaces symbolic constants defined using `#define` with their values. 2. The expanded source code file produced by the C++ preprocessor is compiled into assembly language appropriate for the platform. 3. 4. The assembler code generated by the compiler is assembled into appropriate object code for the platform. The object code file generated by the assembler is linked together with the object code files for any library functions used to produce an executable file. Note: some compiled code is linked together, but not to create a final program. Usually, this “linked” code can also be packaged into a format that can be used by other programs. This “bundle of packaged, usable code” is what C++ programmers refer to as a library. Many C++ compilers may also merge or un-merge certain parts of the compilation process for ease or for additional analysis. Many C++ programmers

will use different tools, but all of the tools will generally follow this generalized process when they are involved in the production of a program. The link below extends this discussion and provides a nice graphic to help. [1]: <http://faculty.cs.niu.edu/~mcmahon/CS241/Notes/compile.html>

Section 1.4: Function A function is a unit of code that represents a sequence of statements. Functions can accept arguments or values and return a single value (or not). To use a function, a function call is used on argument values and the use of the function call itself is replaced with its return value. Every function has a type signature – the types of its arguments and the type of its return type. Functions are inspired by the concepts of the procedure and the mathematical function. Note: C++ functions are essentially procedures and do not follow the exact definition or rules of mathematical functions. Functions are often meant to perform a specific task, and can be called from other parts of a program. A function must be declared and defined before it is called elsewhere in a program.

Note: popular function definitions may be hidden in other included files (often for convenience and reuse across many files). This is a common use of header files. Function Declaration A function declaration declares the existence of a function with its name and type signature to the compiler. The syntax is as the following: `int add2(int i);` // The function is of the type `(int) -> (int)` In the example above, the `int add2(int i)` function declares the following to the compiler: The return type is `int`. The name of the function is `add2`. The number of arguments to the function is 1: The first argument is of the type `int`. The first argument will be referred to in the function's contents by the name `i`. The argument name is optional; the declaration for the function could also be the following: `int add2(int);` // Omitting the function arguments' name is also permitted. Per the one-definition rule, a function with a certain type signature can only be declared or defined once in an entire C++ code base visible to the C++ compiler. In other words, functions with a specific type signature cannot be re-defined – they must only be defined once. Thus, the following is not valid C++: `int add2(int i);` // The compiler will note that `add2` is a function `(int) -> int` `int add2(int j);` // As `add2` already has a definition of `(int) -> int`, the compiler // will regard this as an error. If a function returns nothing, its return type is written as `void`. If it takes no parameters, the parameter list should be empty. `void do_something();` // The function takes no parameters, and does not return anything. // Note that it can still affect variables it has access to. **Function Call** A function can be called after it has been declared. For example, the following program calls `add2` with the value of 2 within the function of `main`:

```
#include <iostream>
using namespace std;
int add2(int i); // Declaration of add2 // Note: add2 is still missing a DEFINITION.
// Even though it doesn't appear directly in code, // add2's definition may be LINKED in from
another object file.
int main() { std::cout << add2(2) << " "; // add2(2) will be evaluated at this
point, // and the result is printed. return 0;
} Here, add2(2) is the syntax for a function call.
```

Function Definition A function definition is similar to a declaration, except it also contains the code that is executed when the function is called within its body. An example of a function definition for `add2` might be: `int add2(int i) { // Data that is passed into (int i) will be referred to by the name i { // while in the function's curly brackets or "scope." int j = i + 2; // Definition of a variable j as the value of i+2. return j; // Returning or, in essence, substitution of j for a function call to // add2. }` **Function Overloading** You can create multiple functions with the same name but different parameters. `int add2(int i) { // Code contained in this definition will be evaluated { // when add2() is called with one parameter. int j = i + 2; return j; } int add2(int i, int j) { // However, when add2() is called with two parameters, the { // code from the initial declaration will be overloaded, int k = i + j + 2 ; // and the code in this declaration will be evaluated return k; // instead. }` Both functions are called by the same name `add2`, but the actual function that is called depends directly on the amount and type of the parameters in the call. In most cases, the C++ compiler can compute which function to call. In some cases, the type must be explicitly stated. **Default Parameters** Default values for function parameters can only be specified in function declarations. `int multiply(int a, int b = 7);` // `b` has default value

of 7. `int multiply(int a, int b) { return a * b; // If multiply() is called with one parameter, the // value will be multiplied by the default, 7. In this example, multiply() can be called with one or two parameters. If only one parameter is given, b will have default value of 7. Default arguments must be placed in the latter arguments of the function. For example: int multiply(int a = 10, int b = 20); // This is legal int multiply(int a = 10, int b); // This is illegal since int a is in the former Special Function Calls - Operators` There exist special function calls in C++ which have different syntax than `name_of_function(value1, value2, value3)`. The most common example is that of operators. Certain special character sequences that will be reduced to function calls by the compiler, such as `!`, `+`, `-`, `*`, `%`, and `<<` and many more. These special characters are normally associated with non-programming usage or are used for

aesthetics (e.g. the `+` character is commonly recognized as the addition symbol both within C++ programming as well as in elementary math). C++ handles these character sequences with a special syntax; but, in essence, each occurrence of an operator is reduced to a function call. For example, the following C++ expression: `3+3` is equivalent to the following function call: `operator+(3, 3)` All operator function names start with `operator`. While in C++'s immediate predecessor, C, operator function names cannot be assigned different meanings by providing additional definitions with different type signatures, in C++, this is valid. "Hiding" additional function definitions under one unique function name is referred to as operator overloading in C++, and is a relatively common, but not universal, convention in C++. Section 1.5: Visibility of function prototypes and declarations In C++, code must be declared or defined before usage. For example, the following produces a compile time error: `int main() { foo(2); // error: foo is called, but has not yet been declared } void foo(int x) // this later definition is not known in main { }` There are two ways to resolve this: putting either the definition or declaration of `foo()` before its usage in `main()`. Here is one example: `void foo(int x) {} //Declare the foo function and body first int main() { foo(2); // OK: foo is completely defined beforehand, so it can be called here. } However it is also possible to "forward-declare" the function by putting only a "prototype" declaration before its usage and then defining the function body later: void foo(int); // Prototype declaration of foo, seen by main // Must specify return type, name, and argument list types int main() { foo(2); // OK: foo is known, called even though its body is not yet defined } void foo(int x) //Must match the prototype { // Define body of foo here }`

The prototype must specify the return type (`void`), the name of the function (`foo`), and the argument list variable types (`int`), but the names of the arguments are NOT required. One common way to integrate this into the organization of source files is to make a header file containing all of the prototype declarations: `// foo.h void foo(int); // prototype declaration` and then provide the full definition elsewhere: `// foo.cpp -> foo.o #include "foo.h" // foo's prototype declaration is "hidden" in here void foo(int x) { } // foo's body definition and then, once compiled, link the corresponding object file foo.o into the compiled object file where it is used in the linking phase, main.o: // main.cpp -> main.o #include "foo.h" // foo's prototype declaration is "hidden" in here int main() { foo(2); } // foo is valid to call because its prototype declaration was beforehand. // the prototype and body definitions of foo are linked through the object files An unresolved external symbol error occurs when the function prototype and call exist, but the function body is not defined. These can be trickier to resolve as the compiler won't report the error until the final linking stage, and it doesn't know which line to jump to in the code to show the error. Section 1.6: Preprocessor The preprocessor is an important part of the compiler. It edits the source code, cutting some bits out, changing others, and adding other things. In source files, we can include preprocessor directives. These directives tell the preprocessor to perform specific actions. A directive starts with a # on a new line. Example: #define ZERO 0 The first preprocessor directive you will meet is probably the #include directive. What it does is takes all of something and inserts it in your file where the directive was. The hello world program starts with the line #include This line adds the functions and objects that let you use the standard input and output. The C language, which also uses the preprocessor,`

does not have as many header files as the C++ language, but in C++ you can use all the C header files. The next important directive is probably the

`#define something something_else` directive. This tells the preprocessor that as it goes along the file, it should replace every occurrence of `something` with `something_else`. It can also make things similar to functions, but that probably counts as advanced C++. The `something_else` is not needed, but if you define something as nothing, then outside preprocessor directives, all occurrences of `something` will vanish. This actually is useful, because of the `#if`, `#else` and `#ifdef` directives. The format for these would be the following: `#if something==true //code`
`#else //more code #endif` `#ifdef thing_that_you_want_to_know_if_is_defined //code` `#endif` These directives insert the code that is in the true bit, and deletes the false bits. This can be used to have bits of code that are only included on certain operating systems, without having to rewrite the whole code.

Chapter 9

Professional Notes: Chapter 2: Literals

Traditionally, a literal is an expression denoting a constant whose type and value are evident from its spelling. For example, 42 is a literal, while `x` is not since one must see its declaration to know its type and read previous lines of code to know its value. However, C++11 also added user-defined literals, which are not literals in the traditional sense but can be used as a shorthand for function calls. Section 2.1: *this* Within a member function of a class, the keyword `this` is a pointer to the instance of the class on which the function was called. `this` cannot be used in a static member function. `struct S { int x; S& operator=(const S& other) { x = other.x; // return a reference to the object being assigned to return this; } }; The type of this depends on the cv-qualification of the member function: if X::f is const, then the type of this within f is const X, so this cannot be used to modify non-static data members from within a const member function. Likewise, this inherits volatile qualification from the function it appears in. Version C++11 this can also be used in a brace-or-equal-initializer for a non-static data member. struct S; struct T { T(const S* s); // ... }; struct S { // ... T t{this}; }; this is an rvalue, so it cannot be assigned to. Section 2.2: Integer literal An integer literal is a primary expression of the form decimal-literal It is a non-zero decimal digit (1, 2, 3, 4, 5, 6, 7, 8, 9), followed by zero or more decimal digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) int d = 42; octal-literal It is the digit zero (0) followed by zero or more octal digits (0, 1, 2, 3, 4, 5, 6, 7)`

`int o = 052` hex-literal It is the character sequence `0x` or the character sequence `0X` followed by one or more hexadecimal digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F) `int x = 0x2a;` `int X = 0X2A;` binary-literal (since C++14) It is the character sequence `0b` or the character sequence `0B` followed by one or more binary digits (0, 1) `int b = 0b101010;` // C++14 Integer-suffix, if provided, may contain one or both of the following (if both are provided, they may appear in any order: unsigned-suffix (the character `u` or the character `U`) `unsigned int u_1 = 42u;` long-suffix (the character `l` or the character `L`) or the long-long-suffix (the character sequence `ll` or the character sequence `LL`) (since C++11) The following variables are also initialized to the same value: `unsigned long long l1 = 18446744073709550592ull;` // C++11 `unsigned long long l2 = 18'446'744'073'709'550'592llu;` // C++14 `unsigned long long l3 = 1844'6744'0737'0955'0592uLL;` // C++14 `unsigned long long l4 = 184467'440737'0'95505'92LLU;` // C++14 Notes Letters in the integer literals are case-insensitive: `0xDeAdBaBeU` and `0XdeadBABEU` represent the same number (one exception is the long-long-suffix, which is either `ll` or `LL`, never `lL` or `Ll`) There are no negative integer literals. Expressions such as `-1` apply the unary minus operator to the value represented by the literal, which may involve implicit type conversions. In C prior to C99 (but not in C++), unsixed decimal values that do not fit in `long int` are allowed to have the type `unsigned long int`. When used in a controlling expression of `#if` or `#elif`, all signed integer constants act as if they have type `std::intmax_t` and all unsigned integer constants act as if they have type `std::uintmax_t`.

Section 2.3: `true` A keyword denoting one of the two possible values of type `bool`. `bool ok = true; if (!f()) { ok = false; goto end; }`

Section 2.4: `false` A keyword denoting one of the two possible values of type `bool`. `bool ok = true; if (!f()) { ok = false; goto end; }` Section 2.5: `nullptr` Version C++11 A keyword denoting a null pointer constant. It can be converted to any pointer or pointer-to-member type, yielding a null pointer of the resulting type. `Widget* p = new Widget(); delete p; p = nullptr; // set the pointer to null after deletion` Note that `nullptr` is not itself a pointer. The type of `nullptr` is a fundamental type known as `std::nullptr_t`. `void f(int* p); template void g(T* p); void h(std::nullptr_t p); int main() { f(nullptr); // ok g(nullptr); // error h(nullptr); // ok }`

Chapter 10

Professional Notes: Chapter 4: Floating Point Arithmetic

Section 4.1: Floating Point Numbers are Weird The rst mistake that nearly every single programmer makes is presuming that this code will work as intended: `float total = 0; for(float a = 0; a != 2; a += 0.01f) { total += a; }` The novice programmer assumes that this will sum up every single number in the range 0, 0.01, 0.02, 0.03, . . . , 1.97, 1.98, 1.99, to yield the result 199the mathematically correct answer. Two things happen that make this untrue: 1. 2. The program as written never concludes. `a` never becomes equal to 2, and the loop never terminates. If we rewrite the loop logic to check `a < 2` instead, the loop terminates, but the total ends up being something dierent from 199. On IEEE754-compliant machines, it will often sum up to about 201 instead. The reason that this happens is that Floating Point Numbers represent Approximations of their assigned values. The classical example is the following computation: `double a = 0.1; double b = 0.2; double c = 0.3; if(a + b == c) //This never prints on IEEE754-compliant machines std::cout << "This Computer is Magic!" << std::endl; else std::cout << "This Computer is pretty normal, all things considered." << std::endl;` Though what we the programmer see is three numbers written in base10, what the compiler (and the underlying hardware) see are binary numbers. Because 0.1, 0.2, and 0.3 require perfect division by 10which is quite easy in a base-10 system, but impossible in a base-2 systemthese numbers have to be stored in imprecise formats, similar to how the number 1/3 has to be stored in the imprecise form 0.3333333333333333. . . in base-10. `//64-bit floats have 53 digits of precision, including the whole-number-part. double a = 00111111101110011001100110011001100110011001100110011001100110011010; //imperfect representation of 0.1 double b = 00111111110010011001100110011001100110011001100110011001100110011010; //imperfect representation of 0.2 double c = 001111111101001100110011001100110011001100110011001100110011001100 //imperfect representation of 0.3 double a + b = 001111111101001100110011001100110011001100110011001100110011001100 //Note that this is not quite equal to the "canonical" 0.3! — ### Professional Notes: Build Systems & Automation`

1. The Build Process (Architectural View)

A build system automates the invocation of the compiler, assembler, and linker. * **Makefile**: Uses a dependency graph to determine which files need recompilation. Only rebuilds changed files. * **CMake**: A meta-build system that generates native build files (Makefiles, Ninja, VS Solutions).

2. Linker Symbols and Name Mangling

Use tools like `nm` or `objdump` to inspect object files. Demangle names with `c++filt`.

Chapter 11

MEMORY TYPES AND POINTERS

Chapter 12

MEMORY, TYPES, AND POINTERS

Chapter 13

ADVANCED POINTERS & MEMORY

13.1 1.1 Pointer to Const vs Const Pointer

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 5, y = 10;
6
7     // Pointer to const - can't modify data
8     const int* ptr1 = &x;
9     // *ptr1 = 10; // ERROR
10    ptr1 = &y; // OK - can change pointer
11
12    // Const pointer - can't modify pointer
13    int* const ptr2 = &x;
14    *ptr2 = 10; // OK - can change data
15    // ptr2 = &y; // ERROR
16
17    // Const pointer to const - can't modify either
18    const int* const ptr3 = &x;
19    // *ptr3 = 10; // ERROR
20    // ptr3 = &y; // ERROR
21
22    return 0;
23 }
```

13.2 1.2 Void Pointers

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 42;
6     double y = 3.14;
7
8     // Void pointer can point to any type
9     void* ptr = &x;
10    cout << *(int*)ptr << endl; // 42
11
12    ptr = &y;
13    cout << *(double*)ptr << endl; // 3.14
14 }
```

```

14
15 // Generic function using void*
16 void print_value(void* ptr, char type) {
17     if (type == 'i') {
18         cout << *(int*)ptr << endl;
19     } else if (type == 'd') {
20         cout << *(double*)ptr << endl;
21     }
22 }
23
24 print_value(&x, 'i'); // 42
25 print_value(&y, 'd'); // 3.14
26
27 return 0;
28 }

```

c

1. Arrays as Pointers In C++, an array name decays to a pointer to its first element in most contexts.

2. References vs. Pointers * **References**: Must be initialized upon declaration.

Godhood Tip: Use references for function parameters.

3. Pointers to Members Pointers to members are a specialized feature allowing you to point to a member of a class.

Point p = {1, 2, 3};

4. The this Pointer Inside every non-static member function, **this** is a hidden pointer to the object.

13.2.1 Professional Notes: Language Boundary & Storage

1. C Incompatibilities: The Parent's Shadow

While C++ evolved from C, they are distinct languages. * **void*** **Conversion**: C allows `int* p = malloc(10);`. C++ requires `int* p = static_cast<int*>(malloc(10));`. * **Enumerations**: In C, enums are effectively integers. In C++, they are distinct types. * **Tentative Definitions**: C allows `int x; int x;` at file scope. C++ considers the second one a redefinition (ODR violation).

2. Storage Class Specifiers

- **static**: Internal linkage for globals; persistent lifetime for locals.
- **extern**: External linkage. Tells the compiler the variable is defined elsewhere.
- **thread_local** (C++11): Unique instance per thread.
- **register**: (Deprecated in C++11, removed in C++17) Hint to use a CPU register.
- **auto**: (C++98) Automatic storage. (C++11) Type deduction.

3. Digit Separators and Binary Literals (C++14)

Use `'` to separate digits for readability: `int x = 1'000'000;`. Use `0b` for binary: `int b = 0b1101;`.

13.3 1.3 Null Pointer Safety

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int* ptr = NULL; // Set to null
6
7     // Always check before dereferencing
8     if (ptr != NULL) {
9         cout << *ptr << endl;
10    } else {
11        cout << "Pointer is NULL" << endl;
12    }
13
14    // Safer approach
15    ptr = new int(42);
16    if (ptr) {
17        cout << *ptr << endl;
18        delete ptr;
19        ptr = NULL;
20    }
21
22    return 0;
23 }
```

13.4 1.4 Memory Layout & Alignment

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     struct Data {
6         char a; // 1 byte
7         int b; // 4 bytes
8         double c; // 8 bytes
9     };
10
11    cout << "Size of Data: " << sizeof(Data) << endl;
12    // Likely 16 or 24 (due to alignment padding)
13
14    cout << "Size of char: " << sizeof(char) << endl; // 1
15    cout << "Size of int: " << sizeof(int) << endl; // 4
16    cout << "Size of double: " << sizeof(double) << endl; // 8
17
18    Data data;
19    cout << "Address of a: " << (void*)&data.a << endl;
20    cout << "Address of b: " << (void*)&data.b << endl;
21    cout << "Address of c: " << (void*)&data.c << endl;
22
23    return 0;
24 }
```

Chapter 14

ADVANCED ARRAYS

14.1 4.1 Dynamic Arrays

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // 1D dynamic array
6     int size = 5;
7     int* arr = new int[size];
8
9     for (int i = 0; i < size; i++) {
10         arr[i] = i * 10;
11     }
12
13     for (int i = 0; i < size; i++) {
14         cout << arr[i] << " ";
15     }
16     cout << endl;
17
18     delete[] arr;
19     arr = NULL;
20
21     // 2D dynamic array
22     int rows = 3, cols = 4;
23     int** matrix = new int*[rows];
24     for (int i = 0; i < rows; i++) {
25         matrix[i] = new int[cols];
26     }
27
28     // Fill matrix
29     for (int i = 0; i < rows; i++) {
30         for (int j = 0; j < cols; j++) {
31             matrix[i][j] = i * cols + j;
32         }
33     }
34
35     // Delete matrix
36     for (int i = 0; i < rows; i++) {
37         delete[] matrix[i];
38     }
39     delete[] matrix;
40
41     return 0;
42 }
```

14.2 4.2 Variable Length Arrays (Non-standard)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int size;
6     cout << "Enter size: ";
7     cin >> size;
8
9     // VLA - not standard but supported by many compilers
10    int arr[size]; // GCC extension
11
12    for (int i = 0; i < size; i++) {
13        arr[i] = i;
14    }
15
16    for (int i = 0; i < size; i++) {
17        cout << arr[i] << " ";
18    }
19    cout << endl;
20
21    return 0;
22 }
```

14.3 4.3 Array Bounds & Safety

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int arr[5] = {10, 20, 30, 40, 50};
6
7     // No bounds checking in C++
8     cout << arr[0] << endl; // 10 (OK)
9     cout << arr[10] << endl; // Undefined behavior!
10
11    // Manual bounds checking
12    int index = 5;
13    if (index >= 0 && index < 5) {
14        cout << arr[index] << endl;
15    } else {
16        cout << "Index out of bounds" << endl;
17    }
18
19    return 0;
20 }
```

Chapter 15

CONST & VOLATILE

15.1 11.1 Const Correctness

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10;
6
7     // const variable
8     const int constant = 5;
9     // constant = 10; // ERROR
10
11    // pointer to const
12    const int* ptr1 = &x;
13    // *ptr1 = 20; // ERROR
14    ptr1 = &constant; // OK
15
16    // const pointer
17    int* const ptr2 = &x;
18    *ptr2 = 20; // OK
19    // ptr2 = &constant; // ERROR
20
21    // const reference
22    const int& ref = x;
23    // ref = 20; // ERROR
24
25    cout << x << endl;
26    cout << *ptr1 << endl;
27    cout << *ptr2 << endl;
28    cout << ref << endl;
29
30    return 0;
31 }
```

15.2 11.2 Volatile Keyword

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // volatile - tells compiler value may change unexpectedly
6     volatile int sensor_reading = 0; // From hardware
7
8     // Compiler won't optimize away reads
```

```
9  while (sensor_reading < 100) {
10     // Check actual value each time, not cached
11 }
12
13 // Common use: hardware registers
14 volatile int* hardware_register = (volatile int*)0x1000;
15
16 // Each access reads from actual location
17 int val1 = *hardware_register;
18 int val2 = *hardware_register;
19
20 // Without volatile, compiler might optimize one read
21
22 return 0;
23 }
```

Chapter 16

TYPE CASTING

16.1 8.1 C-Style Casting

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 int main() {
6     double d = 3.14;
7
8     // C-style cast (avoid in modern C++)
9     int i = (int)d; // 3
10    cout << i << endl;
11
12    int x = 65;
13    char c = (char)x; // 'A'
14    cout << c << endl;
15
16    float f = (float)d;
17    cout << f << endl;
18
19    return 0;
20 }
```

16.2 8.2 Implicit Conversions

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // Implicit conversions
6     int x = 5;
7     double d = x; // int to double (automatic)
8     cout << d << endl; // 5.0
9
10    double d2 = 3.9;
11    int y = d2; // double to int (loses precision)
12    cout << y << endl; // 3
13
14    // Char arithmetic
15    char c = 'A';
16    int code = c; // char to int (gets ASCII)
17    cout << code << endl; // 65
18
19    return 0;
20 }
```

20 }

Chapter 17

ENUMERATION & UNIONS

17.1 10.1 Enumerations

```
1 #include <iostream>
2 using namespace std;
3
4 // Enum definition
5 enum Color { RED, GREEN, BLUE };
6
7 enum Direction {
8     NORTH = 0,
9     EAST = 1,
10    SOUTH = 2,
11    WEST = 3
12 };
13
14 int main() {
15     Color c = RED;
16     cout << c << endl;    // 0
17
18     Color colors[3] = {RED, GREEN, BLUE};
19
20     // Switching on enum
21     switch (c) {
22         case RED:
23             cout << "Red" << endl;
24             break;
25         case GREEN:
26             cout << "Green" << endl;
27             break;
28         case BLUE:
29             cout << "Blue" << endl;
30             break;
31     }
32
33     // Iterate through enum values
34     for (int dir = NORTH; dir <= WEST; dir++) {
35         cout << "Direction: " << dir << endl;
36     }
37
38     return 0;
39 }
```

17.2 10.2 Unions

```
1 #include <iostream>
2 using namespace std;
3
4 // Union - all members share same memory
5 union Data {
6     int i;
7     float f;
8     char c;
9 };
10
11 int main() {
12     Data data;
13
14     cout << "Size of Data: " << sizeof(data) << endl; // 4 (size of largest
15 // member)
16
17     data.i = 10;
18     cout << "data.i: " << data.i << endl; // 10
19     cout << "data.f: " << data.f << endl; // Garbage (overwrites data.i)
20
21     data.f = 3.14;
22     cout << "data.i: " << data.i << endl; // Garbage (overwrites by data.f)
23     cout << "data.f: " << data.f << endl; // 3.14
24
25     // Union useful for memory-constrained systems
26     union Variant {
27         int int_val;
28         double double_val;
29         char char_val;
30     };
31
32     cout << "Size of Variant: " << sizeof(Variant) << endl;
33
34     return 0;
35 }
```

Chapter 18

BITWISE OPERATIONS

18.1 6.1 Bitwise Operators

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     unsigned char a = 5;    // 0101
6     unsigned char b = 3;    // 0011
7
8     // AND
9     cout << (a & b) << endl; // 0001 = 1
10
11    // OR
12    cout << (a | b) << endl; // 0111 = 7
13
14    // XOR
15    cout << (a ^ b) << endl; // 0110 = 6
16
17    // NOT (bitwise complement)
18    cout << (~a) << endl;    // 1010 = 250 (for unsigned char)
19
20    // Left shift
21    cout << (a << 1) << endl; // 1010 = 10
22
23    // Right shift
24    cout << (b >> 1) << endl; // 0001 = 1
25
26    return 0;
27 }
```

18.2 6.2 Bit Manipulation Techniques

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     unsigned int num = 5;    // 0101
6
7     // Check if bit is set
8     int bit_pos = 2;
9     bool is_set = (num >> bit_pos) & 1;
10    cout << "Bit " << bit_pos << " is: " << is_set << endl;
11
12    // Set a bit
```

```
13 num |= (1 << 1); // Set bit 1
14 cout << "After setting bit 1: " << num << endl; // 7 (0111)
15
16 // Clear a bit
17 num &= ~(1 << 1); // Clear bit 1
18 cout << "After clearing bit 1: " << num << endl; // 5 (0101)
19
20 // Toggle a bit
21 num ^= (1 << 0); // Toggle bit 0
22 cout << "After toggling bit 0: " << num << endl; // 4 (0100)
23
24 // Count set bits
25 unsigned int count = 0;
26 unsigned int temp = num;
27 while (temp) {
28     count += temp & 1;
29     temp >>= 1;
30 }
31 cout << "Number of set bits: " << count << endl;
32
33 return 0;
34 }
```

Chapter 19

Professional Notes: Chapter 8: Arrays

Arrays are elements of the same type placed in adjoining memory locations. The elements can be individually referenced by a unique identifier with an added index. This allows you to declare multiple variable values of a specific type and access them individually without needing to declare a variable for each value. Section 8.1: Array initialization An array is just a block of sequential memory locations for a specific type of variable. Arrays are allocated the same way as normal variables, but with square brackets appended to its name [] that contain the number of elements that fit into the array memory. The following example of an array uses the type `int`, the variable name `arrayOfInts`, and the number of elements [5] that the array has space for: `int arrayOfInts[5]`; An array can be declared and initialized at the same time like this `int arrayOfInts[5] = {10, 20, 30, 40, 50}`; When initializing an array by listing all of its members, it is not necessary to include the number of elements inside the square brackets. It will be automatically calculated by the compiler. In the following example, it's 5: `int arrayOfInts[] = {10, 20, 30, 40, 50}`; It is also possible to initialize only the first elements while allocating more space. In this case, denoting the length in brackets is mandatory. The following will allocate an array of length 5 with partial initialization, the compiler initializes all remaining elements with the standard value of the element type, in this case zero. `int arrayOfInts[5] = {10, 20}`; // means 10, 20, 0, 0, 0 Arrays of other basic data types may be initialized in the same way. `char arrayOfChars[5]`; // declare the array and allocate the memory, don't initialize `char arrayOfChars[5] = { 'a', 'b', 'c', 'd', 'e' }`; // declare and initialize double `arrayOfDoubles[5] = {1.14159, 2.14159, 3.14159, 4.14159, 5.14159}`; `string arrayOfStrings[5] = { "C++", "is", "super", "duper", "great!" }`; It is also important to take note that when accessing array elements, the array's element index (or position) starts from 0. `int array[5] = { 10/Element no.0/, 20/Element no.1/, 30, 40, 50/Element no.4/ }`; `std::cout << array[4]`; // outputs 50 `std::cout << array[0]`; // outputs 10

Section 8.2: A fixed size raw array matrix (that is, a 2D raw array) // A fixed size raw array matrix (that is, a 2D raw array). `#include <iostream>` `#include <string>` using namespace std; auto main() -> int { `int const n_rows = 3; int const n_cols = 7; int const m[n_rows][n_cols] =` // A raw array matrix. { { 1, 2, 3, 4, 5, 6, 7 }, { 8, 9, 10, 11, 12, 13, 14 }, { 15, 16, 17, 18, 19, 20, 21 } }; `for( int y = 0; y < n_rows; ++y ) { for( int x = 0; x < n_cols; ++x ) { cout << setw( 4 ) << m[y][x];` // Note: do NOT use `m[y,x]!` } `cout << '\n'; } }` Output: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 C++ doesn't support special syntax for indexing a multi-dimensional array. Instead such an array is viewed as an array of arrays (possibly of arrays, and so on), and the ordinary single index notation [i] is used for each level. In the example above `m[y]` refers to row y of m, where y is a zero-based index. Then this row can be indexed in turn, e.g. `m[y][x]`, which refers to the xth item or column of row y. I.e. the last index varies fastest, and in the declaration the range of this index, which here is the number of columns per row, is the last and innermost size specified. Since C++ doesn't provide built-in support for dynamic size arrays, other than

dynamic allocation, a dynamic size matrix is often implemented as a class. Then the raw array matrix indexing notation `m[y][x]` has some cost, either by exposing the implementation (so that e.g. a view of a transposed matrix becomes practically impossible) or by adding some overhead and slight inconvenience when it's done by returning a proxy object from `operator[]`. And so the indexing notation for such an abstraction can and will usually be different, both in look-and-feel and in the order of indices, e.g. `m(x,y)` or `m.at(x,y)` or `m.item(x,y)`. Section 8.3: Dynamically sized raw array // Example of raw dynamic size array. It's generally better to use `std::vector`.
`#include // std::sort #include using namespace std; auto int_from( istream& in ) -> int { int x; in >> x; return x; }`

`auto main() -> int { cout << "Sorting n integers provided by you.\n"; cout << "n?"; int const n = int_from(cin); int* a = new int[n]; // Allocation of array of n items. for(int i = 1; i <= n; ++i) { cout << "The #" << i << " number, please: "; a[i-1] = int_from(cin); } sort(a, a + n); for(int i = 0; i < n; ++i) { cout << a[i] << ' '; } cout << "\n"; delete[] a; }` A program that declares an array `T a[n]`; where `n` is determined a run-time, can compile with certain compilers that support C99 variadic length arrays (VLAs) as a language extension. But VLAs are not supported by standard C++. This example shows how to manually allocate a dynamic size array via a `new[]`-expression, `int* a = new int[n];` // Allocation of array of `n` items. then use it, and finally deallocate it via a `delete[]`-expression: `delete[] a;` The array allocated here has indeterminate values, but it can be zero-initialized by just adding an empty parenthesis `()`, like this: `new int[n]`. More generally, for arbitrary item type, this performs a value-initialization. As part of a function down in a call hierarchy this code would not be exception safe, since an exception before the `delete[]` expression (and after the `new[]`) would cause a memory leak. One way to address that issue is to automate the cleanup via e.g. a `std::unique_ptr` smart pointer. But a generally better way to address it is to just use a `std::vector`: that's what `std::vector` is there for. Section 8.4: Array size: type safe at compile time `#include // size_t, ptrdiff_t //----- Machinery: using Size = ptrdiff_t; template< class Item, size_t n > constexpr auto n_items(Item (&)[n]) noexcept -> Size { return n; } //----- Usage: #include using namespace std; auto main()`

`-> int { int const a[] = {3, 1, 4, 1, 5, 9, 2, 6, 5, 4}; Size const n = n_items(a); int b[n] = {}; // An array of the same size as a. (void) b; cout << } The C idiom for array size, sizeof(a)/sizeof(a[0]), will accept a pointer as argument and will then generally yield an incorrect result. For C++11 using C++11 you can do: std::extent<decltype(MyArray)>::value; Example: char MyArray[] = { 'X','o','c','e' }; const auto n = std::extent<decltype(MyArray)>::value; std::cout << n << "\n"; // Prints 4 Up till C++17 (forthcoming as of this writing) C++ had no built-in core language or standard library utility to obtain the size of an array, but this can be implemented by passing the array by reference to a function template, as shown above. Fine but important point: the template size parameter is a size_t, somewhat inconsistent with the signed Size function result type, in order to accommodate the g++ compiler which sometimes insists on size_t for template matching. With C++17 and later one may instead use std::size, which is specialized for arrays. Section 8.5: Expanding dynamic size array by using std::vector // Example of std::vector as an expanding dynamic size array. #include // std::sort #include #include // std::vector using namespace std; int int_from(std::istream& in) { int x = 0; in >> x; return x; } int main() { cout << "Sorting integers provided by you."; cout << "You can indicate EOF via F6 in Windows or Ctrl+D in Unix-land."; vector a; // Zero size by default. while(cin) { cout << "One number, please, or indicate EOF: "; int const x = int_from(cin); if(!cin.fail()) { a.push_back(x); } // Expands as necessary. } sort(a.begin(), a.end());`

`int const n = a.size(); for( int i = 0; i < n; ++i ) { cout << a[i] << ' '; } cout << "; }` `std::vector` is a standard library class template that provides the notion of a variable size array. It takes care of all the memory management, and the buffer is contiguous so a pointer to the buffer (e.g. `&v[0]` or `v.data()`) can be passed to API functions requiring a raw array. A vector can even be expanded at run time, via e.g. the `push_back` member function that appends an item. The complexity of

the sequence of n push_back operations, including the copying or moving involved in the vector expansions, is amortized $O(n)$. Amortized: on average. Internally this is usually achieved by the vector doubling its buer size, its capacity, when a larger buer is needed. E.g. for a buer starting out as size 1, and being repeatedly doubled as needed for $n=17$ push_back calls, this involves $1 + 2 + 4 + 8 + 16 = 31$ copy operations, which is less than $2n = 34$. And more generally the sum of this sequence can't exceed $2n$. Compared to the dynamic size raw array example, this vector-based code does not require the user to supply (and know) the number of items up front. Instead the vector is just expanded as necessary, for each new item value specied by the user. Section 8.6: A dynamic size matrix using std::vector for storage Unfortunately as of C++14 there's no dynamic size matrix class in the C++ standard library. Matrix classes that support dynamic size are however available from a number of 3rd party libraries, including the Boost Matrix library (a sub-library within the Boost library). If you don't want a dependency on Boost or some other library, then one poor man's dynamic size matrix in C++ is just like `vector<vector> m( 3, vector( 7 ) );` where vector is `std::vector`. The matrix is here created by copying a row vector n times where n is the number of rows, here 3. It has the advantage of providing the same `m[y][x]` indexing notation as for a xed size raw array matrix, but it's a bit inecient because it involves a dynamic allocation for each row, and it's a bit unsafe because it's possible to inadvertently resize a row. A more safe and ecient approach is to use a single vector as storage for the matrix, and map the client code's (x, y) to a corresponding index in that vector: // A dynamic size matrix using std::vector for storage. //

----- Machinery: #include // std::copy #include // assert #include // std::initializer_list #include // std::vector #include // ptrdiff_t namespace my { using Size = ptrdiff_t; using std::initializer_list; using std::vector;

```
template< class Item > class Matrix { private: vector items_; Size n_cols_; auto index_for(
Size const x, Size const y ) const -> Size { return yn_cols_ + x; } public: auto n_rows() const
-> Size { return items_.size()/n_cols_; } auto n_cols() const -> Size { return n_cols_; }
auto item( Size const x, Size const y ) -> Item& { return items_[index_for(x, y)]; } auto item(
Size const x, Size const y ) const -> Item const& { return items_[index_for(x, y)]; } Matrix(
n_cols_( 0 ) {} Matrix( Size const n_cols, Size const n_rows ) : items_( n_colsn_rows ) ,
n_cols_( n_cols ) {} Matrix( initializer_list< initializer_list > const& values ) : items_(
n_cols_( values.size() == 0? 0 : values.begin()->size() ) { for( auto const& row : values ) {
assert( Size( row.size() ) == n_cols_ ); items_.insert( items_.end(), row.begin(), row.end() );
} } }; } // namespace my //----- Usage: using my::Matrix; auto
some_matrix() -> Matrix { return { { 1, 2, 3, 4, 5, 6, 7 }, { 8, 9, 10, 11, 12, 13, 14 }, { 15, 16,
17, 18, 19, 20, 21 } }; } #include #include using namespace std; auto main() -> int { Matrix
const m = some_matrix(); assert( m.n_cols() == 7 );
```

```
assert( m.n_rows() == 3 ); for( int y = 0, y_end = m.n_rows(); y < y_end; ++y ) { for(
int x = 0, x_end = m.n_cols(); x < x_end; ++x ) { cout < Note: not m[y][x]! } cout < } }
Output: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 The above code is not industrial
grade: it's designed to show the basic principles, and serve the needs of students learning C++.
For example, one may dene operator() overloads to simplify the indexing notation.
```

Chapter 20

CONTROL FLOW AND PREPROCESSOR

Chapter 21

CONTROL FLOW & PREPROCESSOR

Chapter 22

ADVANCED CONTROL FLOW

22.1 9.1 Ternary Operator

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10, y = 5;
6
7     // condition ? true_value : false_value
8     int max = (x > y) ? x : y;
9     cout << "Max: " << max << endl; // 10
10
11    // Nested ternary (use with caution)
12    int age = 20;
13    string status = (age < 18) ? "Minor" : (age < 65) ? "Adult" : "Senior";
14    cout << status << endl;
15
16    // String ternary
17    cout << (x % 2 == 0 ? "Even" : "Odd") << endl;
18
19    return 0;
20 }
```

22.2 9.2 goto Statement (Avoid)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // goto is generally discouraged
6     int x = 0;
7
8 loop:
9     cout << x << " ";
10    x++;
11
12    if (x < 5) {
13        goto loop;
14    }
15    cout << endl;
16
17    // Better alternative: use loops
18    for (int i = 0; i < 5; i++) {
19        cout << i << " ";
20    }
```

```
20     }
21     cout << endl;
22
23     return 0;
24 }
```

22.3 9.3 Label & Goto for Error Handling

```
1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;
4
5  int main() {
6      FILE* file = NULL;
7      char* buffer = NULL;
8
9      // Using goto for cleanup (rare acceptable use)
10     file = fopen("test.txt", "r");
11     if (!file) {
12         cout << "Failed to open file" << endl;
13         goto cleanup;
14     }
15
16     buffer = new char[100];
17     if (!buffer) {
18         cout << "Memory allocation failed" << endl;
19         goto cleanup;
20     }
21
22     // Do work...
23
24 cleanup:
25     if (buffer) delete[] buffer;
26     if (file) fclose(file);
27
28     return 0;
29 }
```

Chapter 23

PREPROCESSOR DIRECTIVES

23.1 7.1 #define and #include

```
1 // Define constants
2 #define PI 3.14159
3 #define MAX_SIZE 100
4 #define SQUARE(x) ((x) * (x))
5
6 // Conditional compilation
7 #define DEBUG
8
9 #ifdef DEBUG
10     #define LOG(msg) cout << msg << endl
11 #else
12     #define LOG(msg) // Do nothing in release
13 #endif
14
15 #include <iostream>
16 using namespace std;
17
18 int main() {
19     cout << "PI = " << PI << endl;
20
21     int arr[MAX_SIZE];
22     cout << "Array size: " << sizeof(arr) << endl;
23
24     cout << "Square of 5: " << SQUARE(5) << endl;
25
26     LOG("Debug message");
27
28     return 0;
29 }
```

23.2 7.2 Conditional Compilation

```
1 #include <iostream>
2 using namespace std;
3
4 // Platform-specific code
5 #ifdef _WIN32
6     #define OS "Windows"
7 #elif __APPLE__
8     #define OS "macOS"
9 #elif __linux__
10    #define OS "Linux"
```



```
11 #else
12     #define OS "Unknown"
13 #endif
14
15 int main() {
16     cout << "Running on: " << OS << endl;
17
18     #if defined(DEBUG)
19         cout << "Debug mode" << endl;
20     #else
21         cout << "Release mode" << endl;
22     #endif
23
24     return 0;
25 }
```

23.3 7.3 Pragma Directives

```
1 #include <iostream>
2 using namespace std;
3
4 // Disable specific warnings
5 #pragma warning(disable: 4996) // MSVC
6
7 // Pack structure
8 #pragma pack(1)
9 struct PackedData {
10     char a;
11     int b;
12     double c;
13 };
14 #pragma pack()
15
16 int main() {
17     cout << "Size of PackedData: " << sizeof(PackedData) << endl;
18     // Without pragma pack: 24 (aligned)
19     // With pragma pack: 13 (packed)
20
21     return 0;
22 }
```

Chapter 24

INLINE FUNCTIONS & MACROS

24.1 12.1 Macro Functions vs Inline Functions

```
1 #include <iostream>
2 using namespace std;
3
4 // Macro function - preprocessor substitution
5 #define ADD_MACRO(a, b) ((a) + (b))
6
7 // Inline function - type-safe
8 inline int add_inline(int a, int b) {
9     return a + b;
10 }
11
12 int main() {
13     cout << ADD_MACRO(5, 3) << endl;           // 8
14     cout << add_inline(5, 3) << endl;           // 8
15
16     // Macro danger: side effects
17     int x = 5, y = 3;
18     cout << ADD_MACRO(x++, y++) << endl;        // Evaluates as: ((x++) + (y++))
19     cout << "x = " << x << ", y = " << y << endl; // x = 6, y = 4
20
21     // Inline function is safer
22     x = 5, y = 3;
23     cout << add_inline(x++, y++) << endl;        // 8
24     cout << "x = " << x << ", y = " << y << endl; // x = 6, y = 4 (correct)
25
26     return 0;
27 }
```

Chapter 25

Professional Notes: Chapter 3: operator precedence

Section 3.1: Logical `&&` and `||` operators: short-circuit `&&` has precedence over `||`, this means that parentheses are placed to evaluate what would be evaluated together. `c++` uses short-circuit evaluation in `&&` and `||` to not do unnecessary executions. If the left hand side of `||` returns true the right hand side does not need to be evaluated anymore. `#include <iostream>` using namespace std; bool True(string id){ cout << "True" << id << endl; return true; } bool False(string id){ cout << "False" << id << endl; return false; } int main(){ bool result; //let's evaluate 3 booleans with `||` and `&&` to illustrate operator precedence //precedence does not mean that `&&` will be evaluated first but rather where

```
//parentheses would be added //example 1 result = False("A") || False("B") && False("C");
// eq. False("A") || (False("B") && False("C")) //FalseA //FalseB //"Short-circuit evaluation
skip of C" //A is false so we have to evaluate the right of ||, //B being false we do not have to
evaluate C to know that the result is false result = True("A") || False("B") && False("C"); // eq.
True("A") || (False("B") && False("C")) cout << result << " :===== "
<< endl; //TrueA //"Short-circuit evaluation skip of B" //"Short-circuit evaluation skip of C"
//A is true so we do not have to evaluate // the right of || to know that the result is true //If
|| had precedence over && the equivalent evaluation would be: // (True("A") || False("B"))
&& False("C") //What would print //TrueA //"Short-circuit evaluation skip of B" //FalseC
//Because the parentheses are placed differently //the parts that get evaluated are differently
//which makes that the end result in this case would be False because C is false
```

} Section 3.2: Unary Operators Unary operators act on the object upon which they are called and have high precedence. (See Remarks) When used postfix, the action occurs only after the entire operation is evaluated, leading to some interesting arithmetics: `int a = 1; ++a; // result: 2 a--; // result: 1 int minusa=-a; // result: -1 bool b = true; !b; // result: false a=4; int c = a++/2; // equal to: (a==4) 4 / 2 result: 2 ('a' incremented postfix) cout << a << endl; // prints 5! int d = ++a/2; // equal to: (a+1) == 6 / 2 result: 3 int arr[4] = {1,2,3,4}; int ptr1 = &arr[0]; // points to arr[0] which is 1 int ptr2 = ptr1++; // ptr2 points to arr[0] which is still 1; ptr1 incremented std::cout << ptr1++ << std::endl; // prints 2 int e = arr[0]++; // receives the value of arr[0] before it is incremented std::cout << e << std::endl; // prints 1 std::cout << ptr2 << std::endl; // prints arr[0] which is now 2`

Section 3.3: Arithmetic operators Arithmetic operators in `C++` have the same precedence as they do in mathematics: Multiplication and division have left associativity (meaning that they will be evaluated from left to right) and they have higher precedence than addition and subtraction, which also have left associativity. We can also force the precedence of expression using parentheses `( )`. Just the same way as you would do that in normal mathematics. // volume of a spherical shell = $4\pi R^3 - 4\pi r^3$ double vol = $4.0\pi RRR/3.0 - 4.0\pi rrr/3.0$; //Addition: `int a = 2+4/2; // equal to: 2+(4/2) result: 4 int b = (3+3)/2; // equal to: (3+3)/2 result: 3 //With Multiplication int c = 3+4/26; // equal`

to: $3 + ((4/2)6)$ result: 15 `int d = 3(3+6)/9; // equal to: (3(3+6))/9` result: 3 //Division and Modulo `int g = 3-3%1; // equal to: 3 % 1 = 0 3 - 0 = 3` `int h = 3-(3%1); // equal to: 3 % 1 = 0 3 - 0 = 3`

`int i = 3-3/1%3; // equal to: 3 / 1 = 3 3 % 3 = 0 3 - 0 = 3` `int l = 3-(3/1)%3; // equal to: 3 / 1 = 3 3 % 3 = 0 3 - 0 = 3` `int m = 3-(3/(1%3)); // equal to: 1 % 3 = 1 3 / 1 = 3 3 - 3 = 0` Section 3.4: Logical AND and OR operators These operators have the usual precedence in C++: AND before OR. // You can drive with a foreign license for up to 60 days `bool can_drive = has_domestic_license || has_foreign_license && num_days <= 60;` This code is equivalent to the following: // You can drive with a foreign license for up to 60 days `bool can_drive = has_domestic_license || (has_foreign_license && num_days <= 60);` Adding the parenthesis does not change the behavior, though, it does make it easier to read. By adding these parentheses, no confusion exist about the intent of the writer.

Chapter 26

Professional Notes: Chapter 5: Bit Operators

Section 5.1: `|` - bitwise OR `int a = 5; // 0101b (0x05) int b = 12; // 1100b (0x0C) int c = a | b; // 1101b (0x0D) std::cout << "a =" << a << ", b =" << b << ", c =" << c << std::endl; Output a = 5, b = 12, c = 13` Why A bit wise OR operates on the bit level and uses the following Boolean truth table: true OR true = true true OR false = true false OR false = false When the binary value for a (0101) and the binary value for b (1100) are OR'ed together we get the binary value of 1101: `int a = 0 1 0 1 int b = 1 1 0 0 | ——— int c = 1 1 0 1` The bit wise OR does not change the value of the original values unless specically assigned to using the bit wise assignment compound operator `|=`: `int a = 5; // 0101b (0x05) a |= 12; // a = 0101b | 1101b` Section 5.2: `^` - bitwise XOR (exclusive OR) `int a = 5; // 0101b (0x05) int b = 9; // 1001b (0x09) int c = a ^ b; // 1100b (0x0C) std::cout << "a =" << a << ", b =" << b << ", c =" << c << std::endl; Output a = 5, b = 9, c = 12` Why A bit wise XOR (exclusive or) operates on the bit level and uses the following Boolean truth table: true OR true = false true OR false = true false OR false = false

Notice that with an XOR operation true OR true = false where as with operations true AND/OR true = true, hence the exclusive nature of the XOR operation. Using this, when the binary value for a (0101) and the binary value for b (1001) are XOR'ed together we get the binary value of 1100: `int a = 0 1 0 1 int b = 1 0 0 1 ^ ——— int c = 1 1 0 0` The bit wise XOR does not change the value of the original values unless specically assigned to using the bit wise assignment compound operator `^=`: `int a = 5; // 0101b (0x05) a ^= 9; // a = 0101b ^ 1001b` The bit wise XOR can be utilized in many ways and is often utilized in bit mask operations for encryption and compression. Note: The following example is often shown as an example of a nice trick. But should not be used in production code (there are better ways `std::swap()` to achieve the same result). You can also utilize an XOR operation to swap two variables without a temporary: `int a = 42; int b = 64; // XOR swap a ^= b; b ^= a; a ^= b; std::cout << "a =" << a << ", b =" << b << ";` To productionalize this you need to add a check to make sure it can be used. `void doXORSwap(int& a, int& b) { // Need to add a check to make sure you are not swapping the same // variable with itself. Otherwise it will zero the value. if (&a != &b) { // XOR swap a ^= b; b ^= a; a ^= b; } }` So though it looks like a nice trick in isolation it is not useful in real code. xor is not a base logical operation, but a combination of others: `a^c=~(a&c)&(a|c)` also in 2015+ compilers variables may be assigned as binary: `int cn=0b0111;`

Section 5.3: `&` - bitwise AND `int a = 6; // 0110b (0x06) int b = 10; // 1010b (0x0A) int c = a & b; // 0010b (0x02) std::cout << "a =" << a << ", b =" << b << ", c =" << c << std::endl; Output a = 6, b = 10, c = 2` Why A bit wise AND operates on the bit level and uses the following Boolean truth table: TRUE AND TRUE = TRUE TRUE AND FALSE = FALSE FALSE AND FALSE = FALSE When the binary value for a (0110) and the binary value for b (1010) are AND'ed together we get the binary value of 0010: `int a = 0 1 1 0 int b = 1 0 1 0 & ——— int c = 0 0 1 0`

0 The bit wise AND does not change the value of the original values unless specifically assigned to using the bit wise assignment compound operator `&=`: `int a = 5; // 0101b (0x05) a &= 10; // a = 0101b & 1010b` Section 5.4: `« - left shift int a = 1; // 0001b int b = a « 1; // 0010b std::cout « "a =" « a « ", b =" « b « std::endl;` Output `a = 1, b = 2` Why The left bit wise shift will shift the bits of the left hand value (a) the number specified on the right (1), essentially padding the least significant bits with 0's, so shifting the value of 5 (binary 0000 0101) to the left 4 times (e.g. `5 « 4`) will yield the value of 80 (binary 0101 0000). You might note that shifting a value to the left 1 time is also the same as multiplying the value by 2, example: `int a = 7; while (a < 200) { std::cout « "a =" « a « std::endl; a «= 1;`

`} a = 7; while (a < 200) { std::cout « "a =" « a « std::endl; a *= 2; }` But it should be noted that the left shift operation will shift all bits to the left, including the sign bit, example: `int a = 2147483647; // 0111 1111 1111 1111 1111 1111 1111 1111 int b = a « 1; // 1111 1111 1111 1111 1111 1111 1111 1110 std::cout « "a =" « a « ", b =" « b « std::endl;` Possible output: `a = 2147483647, b = -2` While some compilers will yield results that seem expected, it should be noted that if you left shift a signed number so that the sign bit is affected, the result is undefined. It is also undefined if the number of bits you wish to shift by is a negative number or is larger than the number of bits the type on the left can hold, example: `int a = 1; int b = a « -1; // undefined behavior char c = a « 20; // undefined behavior` The bit wise left shift does not change the value of the original values unless specifically assigned to using the bit wise assignment compound operator `«=`: `int a = 5; // 0101b a «= 1; // a = a « 1;` Section 5.5: `» - right shift int a = 2; // 0010b int b = a » 1; // 0001b std::cout « "a =" « a « ", b =" « b « std::endl;` Output `a = 2, b = 1` Why The right bit wise shift will shift the bits of the left hand value (a) the number specified on the right (1); it should be noted that while the operation of a right shift is standard, what happens to the bits of a right shift on a signed negative number is implementation defined and thus cannot be guaranteed to be portable, example: `int a = -2; int b = a » 1; // the value of b will be depend on the compiler` It is also undefined if the number of bits you wish to shift by is a negative number, example: `int a = 1; int b = a » -1; // undefined behavior`

The bit wise right shift does not change the value of the original values unless specifically assigned to using the bit wise assignment compound operator `»=`: `int a = 2; // 0010b a »= 1; // a = a » 1;`

Chapter 27

Professional Notes: Chapter 11: Loops

A loop statement executes a group of statements repeatedly until a condition is met. There are 3 types of primitive loops in C++: for, while, and do...while. Section 11.1: Range-Based For Version C++11 for loops can be used to iterate over the elements of a iterator-based range, without using a numeric index or directly accessing the iterators: `vector v = {0.4f, 12.5f, 16.234f}; for(auto val: v) { std::cout << val << " "; } std::cout << std::endl;` This will iterate over every element in v, with val getting the value of the current element. The following statement: `for (for-range-declaration : for-range-initializer )` statement is equivalent to: `{ auto&& __range = for-range-initializer; auto __begin = begin-expr, __end = end-expr; for (; __begin != __end; ++__begin) { for-range-declaration = *__begin; statement } }` Version C++17 `{ auto&& __range = for-range-initializer; auto __begin = begin-expr; auto __end = end-expr; // end` is allowed to be a different type than begin in C++17 `for (; __begin != __end; ++__begin) { for-range-declaration = *__begin; statement } }` This change was introduced for the planned support of Ranges TS in C++20. In this case, our loop is equivalent to: `{ auto&& __range = v; auto __begin = v.begin(), __end = v.end(); for (; __begin != __end; ++__begin) { auto val = *__begin; std::cout << val << " ";`

`} }` Note that `auto val` declares a value type, which will be a copy of a value stored in the range (we are copy-initializing it from the iterator as we go). If the values stored in the range are expensive to copy, you may want to use `const auto &val`. You are also not required to use `auto`; you can use an appropriate typename, so long as it is implicitly convertible from the range's value type. If you need access to the iterator, range-based for cannot help you (not without some effort, at least). If you wish to reference it, you may do so: `vector v = {0.4f, 12.5f, 16.234f}; for(float &val: v) { std::cout << val << " "; }` You could iterate on const reference if you have const container: `const vector v = {0.4f, 12.5f, 16.234f}; for(const float &val: v) { std::cout << val << " "; }` One would use forwarding references when the sequence iterator returns a proxy object and you need to operate on that object in a non-const way. Note: it will most likely confuse readers of your code. `vector v(10); for(auto&& val: v) { val = true; }` The "range" type provided to range-based for can be one of the following: Language arrays: `float arr[] = {0.4f, 12.5f, 16.234f}; for(auto val: arr) { std::cout << val << " "; }` Note that allocating a dynamic array does not count: `float arr = new float[3]{0.4f, 12.5f, 16.234f}; for(auto val: arr) //Compile error. { std::cout << val << " "; }`

Any type which has member functions `begin()` and `end()`, which return iterators to the elements of the type. The standard library containers qualify, but user-defined types can be used as well: `struct Rng { float arr[3]; // pointers are iterators const float begin() const {return &arr[0];} const float end() const {return &arr[3];} float* begin() {return &arr[0];} float* end() {return &arr[3];} }; int main() { Rng rng = {{0.4f, 12.5f, 16.234f}}; for(auto val: rng) { std::cout << val << " "; } }` Any type which has non-member `begin(type)` and `end(type)` functions which*

can found via argument dependent lookup, based on type. This is useful for creating a range type without having to modify class type itself: namespace Mine { struct Rng {float arr[3];}; // pointers are iterators const float* begin(const Rng &rng) {return &rng.arr[0];} const float* end(const Rng &rng) {return &rng.arr[3];} float* begin(Rng &rng) {return &rng.arr[0];} float* end(Rng &rng) {return &rng.arr[3];} } int main() { Mine::Rng rng = {{0.4f, 12.5f, 16.234f}}; for(auto val: rng) { std::cout << val << " "; } } Section 11.2: For loop A for loop executes statements in the loop body, while the loop condition is true. Before the loop initialization statement is executed exactly once. After each cycle, the iteration execution part is executed. A for loop is dened as follows: *for (/initialization statement/; /condition/; /iteration execution/)*

{ // body of the loop } Explanation of the placeholder statements: initialization statement: This statement gets executed only once, at the beginning of the for loop. You can enter a declaration of multiple variables of one type, such as int i = 0, a = 2, b = 3. These variables are only valid in the scope of the loop. Variables dened before the loop with the same name are hidden during execution of the loop. condition: This statement gets evaluated ahead of each loop body execution, and aborts the loop if it evaluates to false. iteration execution: This statement gets executed after the loop body, ahead of the next condition evaluation, unless the for loop is aborted in the body (by break, goto, return or an exception being thrown). You can enter multiple statements in the iteration execution part, such as a++, b+=10, c=b+a. The rough equivalent of a for loop, rewritten as a while loop is: */initialization/ while (/condition/)* { // body of the loop; using 'continue' will skip to increment part below */iteration execution/* } The most common case for using a for loop is to execute statements a specic number of times. For example, consider the following: for(int i = 0; i < 10; i++) { std::cout << i << std::endl; } A valid loop is also: for(int a = 0, b = 10, c = 20; (a+b+c < 100); c--, b++, a+=c) { std::cout << a << " " << b << " " << c << std::endl; } An example of hiding declared variables before a loop is: int i = 99; //i = 99 for(int i = 0; i < 10; i++) { //we declare a new variable i //some operations, the value of i ranges from 0 to 9 during loop execution } //after the loop is executed, we can access i with value of 99 But if you want to use the already declared variable and not hide it, then omit the declaration part: int i = 99; //i = 99 for(i = 0; i < 10; i++) { //we are using already declared variable i //some operations, the value of i ranges from 0 to 9 during loop execution } //after the loop is executed, we can access i with value of 10 Notes:

The initialization and increment statements can perform operations unrelated to the condition statement, or nothing at all - if you wish to do so. But for readability reasons, it is best practice to only perform operations directly relevant to the loop. A variable declared in the initialization statement is visible only inside the scope of the for loop and is released upon termination of the loop. Don't forget that the variable which was declared in the initialization statement can be modied during the loop, as well as the variable checked in the condition. Example of a loop which counts from 0 to 10: for (int counter = 0; counter <= 10; ++counter) { std::cout << counter << " "; } // counter is not accessible here (had value 11 at the end) Explanation of the code fragments: int counter = 0 initializes the variable counter to 0. (This variable can only be used inside of the for loop.) counter <= 10 is a Boolean condition that checks whether counter is less than or equal to 10. If it is true, the loop executes. If it is false, the loop ends. ++counter is an increment operation that increments the value of counter by 1 ahead of the next condition check. By leaving all statements empty, you can create an innite loop: // infinite loop for (;;) std::cout << "Never ending!"; The while loop equivalent of the above is: // infinite loop while (true) std::cout << "Never ending!"; However, an innite loop can still be left by using the statements break, goto, or return or by throwing an exception. The next common example of iterating over all elements from an STL collection (e.g., a vector) without using the header is: std::vector names = {"Albert Einstein", "Stephen Hawking", "Michael Ellis"}; for(std::vector::iterator it = names.begin(); it != names.end(); ++it) { std::cout << *it << std::endl; } Section 11.3: While loop A while loop executes statements repeatedly until the given condition evaluates to false. This control statement is used when it is not known, in advance, how many

times a block of code is to be executed. For example, to print all the numbers from 0 up to 9, the following code can be used: `int i = 0;`

`while (i < 10) { std::cout << i << " "; ++i; // Increment counter } std::cout << std::endl; // End of line;`0 1 2 3 4 5 6 7 8 9" is printed to the console Version C++17 Note that since C++17, the rst 2 statements can be combined `while (int i = 0; i < 10) //...` The rest is the same To create an innite loop, the following construct can be used: `while (true) { // Do something forever (however, you can exit the loop by calling 'break' } There is another variant of while loops, namely the do...while construct. See the do-while loop example for more information. Section 11.4: Do-while loop A do-while loop is very similar to a while loop, except that the condition is checked at the end of each cycle, not at the start. The loop is therefore guaranteed to execute at least once. The following code will print 0, as the condition will evaluate to false at the end of the rst iteration: int i = 0; do { std::cout << i; ++i; // Increment counter } while (i < 0); std::cout << std::endl; // End of line; 0 is printed to the console Note: Do not forget the semicolon at the end of while(condition);, which is needed in the do-while construct. In contrast to the do-while loop, the following will not print anything, because the condition evaluates to false at the beginning of the rst iteration: int i = 0; while (i < 0) { std::cout << i; ++i; // Increment counter } std::cout << std::endl; // End of line; nothing is printed to the console Note: A while loop can be exited without the condition becoming false by using a break, goto, or return statement. int i = 0; do {`

`std::cout << i; ++i; // Increment counter if (i > 5) { break; } } while (true); std::cout << std::endl; // End of line; 0 1 2 3 4 5 is printed to the console A trivial do-while loop is also occasionally used to write macros that require their own scope (in which case the trailing semicolon is omitted from the macro denition and required to be provided by the user): #define BAD_MACRO(x) f1(x); f2(x); f3(x); // Only the call to f1 is protected by the condition here if (cond) BAD_MACRO(var); #define GOOD_MACRO(x) do { f1(x); f2(x); f3(x); } while(0) // All calls are protected here if (cond) GOOD_MACRO(var); Section 11.5: Loop Control statements : Break and Continue Loop control statements are used to change the ow of execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. The break and continue are loop control statements. The break statement terminates a loop without any further consideration. for (int i = 0; i < 10; i++) { if (i == 4) break; // this will immediately exit our loop std::cout << i << " "; } The above code will print out: The continue statement does not immediately exit the loop, but rather skips the rest of the loop body and goes to the top of the loop (including checking the condition). for (int i = 0; i < 6; i++) { if (i % 2 == 0) // evaluates to true if i is even continue; // this will immediately go back to the start of the loop / the next line will only be reached if the above "continue" statement does not execute */ std::cout << i << " is an odd number"; } The above code will print out:`

1 is an odd number 3 is an odd number 5 is an odd number Because such control ow changes are sometimes dicult for humans to easily understand, break and continue are used sparingly. More straightforward implementation are usually easier to read and understand. For example, the rst for loop with the break above might be rewritten as: `for (int i = 0; i < 4; i++) { std::cout << i << " "; } The second example with continue might be rewritten as: for (int i = 0; i < 6; i++) { if (i % 2 != 0) { std::cout << i << " is an odd number"; } } Section 11.6: Declaration of variables in conditions In the condition of the for and while loops, it's also permitted to declare an object. This object will be considered to be in scope until the end of the loop, and will persist through each iteration of the loop: for (int i = 0; i < 5; ++i) { do_something(i); } // i is no longer in scope. for (auto& a : some_container) { a.do_something(); } // a is no longer in scope. while(std::shared_ptr p = get_object()) { p->do_something(); } // p is no longer in scope. However, it is not permitted to do the same with a do...while loop; instead, declare the variable before the loop, and (optionally) enclose both the variable and the loop within a local`

scope if you want the variable to go out of scope after the loop ends: `//This doesn't compile`
`do { s = do_something(); } while (short s > 0); // Good short s; do { s = do_something(); }`
`while (s > 0);`

This is because the statement portion of a `do...while` loop (the loop's body) is evaluated before the expression portion (the `while`) is reached, and thus, any declaration in the expression will not be visible during the first iteration of the loop. Section 11.7: Range-for over a sub-range

Using range-based loops, you can loop over a sub-part of a given container or other range by generating a proxy object that qualifies for range-based for loops. `template<class Iterator, class Sentinel=Iterator> struct range_t { Iterator b; Sentinel e; Iterator begin() const { return b; } Sentinel end() const { return e; } bool empty() const { return begin()==end(); } range_t without_front(std::size_t count=1) const { if (std::is_same< std::random_access_iterator_tag, typename std::iterator_traits::iterator_category >{}) { count = (std::min)(std::size_t(std::distance(b,e)), count); } return {std::next(b, count), e}; } range_t without_back(std::size_t count=1) const { if (std::is_same< std::random_access_iterator_tag, typename std::iterator_traits::iterator_category >{}) { count = (std::min)(std::size_t(std::distance(b,e)), count); } return {b, std::prev(e, count)}; } }; template<class Iterator, class Sentinel> range_t<Iterator, Sentinel> range(Iterator b, Sentinel e) { return {b,e}; } template auto range(Iterable& r) { using std::begin; using std::end; return range(begin(r),end(r)); } template auto except_first(C& c) { auto r = range(c); if (r.empty()) return r; return r.without_front(); } now we can do: std::vector v = {1,2,3,4}; for (auto i : except_first(v)) std::cout << i << " "; and print out Be aware that intermediate objects generated in the for(range_expression) part of the for loop will have expired by the time the for loop starts.`

Chapter 28

Professional Notes: Chapter 15: Flow Control

Section 15.1: `case` Introduces a case label of a switch statement. The operand must be a constant expression and match the switch condition in type. When the switch statement is executed, it will jump to the case label with operand equal to the condition, if any. `char c = getchar(); bool confirmed; switch (c) { case 'y': confirmed = true; break; case 'n': confirmed = false; break; default: std::cout << "invalid response!"; abort(); }` Section 15.2: `switch` According to the C++ standard, The switch statement causes control to be transferred to one of several statements depending on the value of a condition. The keyword `switch` is followed by a parenthesized condition and a block, which may contain case labels and an optional default label. When the switch statement is executed, control will be transferred either to a case label with a value matching that of the condition, if any, or to the default label, if any. The condition must be an expression or a declaration, which has either integer or enumeration type, or a class type with a conversion function to integer or enumeration type. `char c = getchar(); bool confirmed; switch (c) { case 'y': confirmed = true; break; case 'n': confirmed = false; break; default: std::cout << "invalid response!"; abort(); }` Section 15.3: `catch` The `catch` keyword introduces an exception handler, that is, a block into which control will be transferred when an exception of compatible type is thrown. The `catch` keyword is followed by a parenthesized exception declaration,

which is similar in form to a function parameter declaration: the parameter name may be omitted, and the ellipsis `...` is allowed, which matches any type. The exception handler will only handle the exception if its declaration is compatible with the type of the exception. For more details, see catching exceptions. `try { std::vector v(N); // do something } catch (const std::bad_alloc&) { std::cout << "failed to allocate memory for vector!" << std::endl; } catch (const std::runtime_error& e) { std::cout << "runtime error:" << e.what() << std::endl; } catch (...) { std::cout << "unexpected exception!" << std::endl; throw; }` Section 15.4: `throw` 1. When `throw` occurs in an expression with an operand, its effect is to throw an exception, which is a copy of the operand. `void print_asterisks(int count) { if (count < 0) { throw std::invalid_argument("count cannot be negative!"); } while (count--) { putchar('*'); } }` 2. When `throw` occurs in an expression without an operand, its effect is to rethrow the current exception. If there is no current exception, `std::terminate` is called. `try { // something risky } catch (const std::bad_alloc&) { std::cerr << "out of memory" << std::endl; } catch (...) { std::cerr << "unexpected exception" << std::endl; // hope the caller knows how to handle this exception throw; }` 3. When `throw` occurs in a function declarator, it introduces a dynamic exception specification, which lists the types of exceptions that the function is allowed to propagate. *// this function might propagate a `std::runtime_error`, // but not, say, a `std::logic_error`* `void risky() throw(std::runtime_error); // this function can't propagate any exceptions` `void safe() throw();` Dynamic exception specifications are deprecated as of C++11. Note that the first two uses of `throw` listed above constitute expressions rather than statements. (The type of a `throw` expression is `void`.) This makes it possible to nest them within

expressions, like so:

`unsigned int predecessor(unsigned int x) { return (x > 0) ? (x - 1) : (throw std::invalid_argument("0 has no predecessor"));` } Section 15.5: default In a switch statement, introduces a label that will be jumped to if the condition's value is not equal to any of the case labels' values. `char c = getchar(); bool confirmed; switch (c) { case 'y': confirmed = true; break; case 'n': confirmed = false; break; default: std::cout << "invalid response!"; abort(); }` Version C++11 Denes a default constructor, copy constructor, move constructor, destructor, copy assignment operator, or move assignment operator to have its default behaviour. `class Base { // ... // we want to be able to delete derived classes through Base, // but have the usual behaviour for Base's destructor. virtual ~Base() = default; };` Section 15.6: try The keyword try is followed by a block, or by a constructor initializer list and then a block (see here). The try block is followed by one or more catch blocks. If an exception propagates out of the try block, each of the corresponding catch blocks after the try block has the opportunity to handle the exception, if the types match. `std::vector v(N); // if an exception is thrown here, // it will not be caught by the following catch block try { std::vector v(N); // if an exception is thrown here, // it will be caught by the following catch block // do something with v } catch (const std::bad_alloc&) { // handle bad_alloc exceptions from the try block }`

Section 15.7: if Introduces an if statement. The keyword if must be followed by a parenthesized condition, which can be either an expression or a declaration. If the condition is truthy, the substatement after the condition will be executed. `int x;`

`std::cout << "Please enter a positive number." << std::endl; std::cin >> x; if (x <= 0) { std::cout << "You didn't enter a positive number!" << std::endl; abort(); }` Section 15.8: else The rst substatement of an if statement may be followed by the keyword else. The substatement after the else keyword will be executed when the condition is falsey (that is, when the rst substatement is not executed). `int x; std::cin >> x; if (x%2 == 0) { std::cout << "The number is even"; } else { std::cout << "The number is odd"; }` Section 15.9: Conditional Structures: if, if..else if and else: it used to check whether the given expression returns true or false and acts as such: if (condition) statement the condition can be any valid C++ expression that returns something that be checked against truth/falsehood for example: `if (true) { /* code here / } // evaluate that true is true and execute the code in the brackets if (false) { / code here */ } // always skip the code since false is always false the condition can be anything, a function, a variable, or a comparison for example if(istrue()) { } // evaluate the function, if it returns true, the if will execute the code if(isTrue(var)) { } //evaluate the return of the function after passing the argument var if(a == b) { } // this will evaluate the return of the experssion (a==b) which will be true if equal and false if unequal if(a) { } //if a is a boolean type, it will evaluate for its value, if it's an integer, any non zero value will be true, if we want to check for a multiple expressions we can do it in two ways : using binary operators : if (a && b) { } // will be true only if both a and b are true (binary operators are outside the scope here if (a || b) { } //true if a or b is true using if/iffelse/else: for a simple switch either if or else if (a== "test") {`

`//will execute if a is a string "test" } else { // only if the first failed, will execute }` for multiple choices : `if (a=='a') { // if a is a char valued 'a' } else if (a=='b') { // if a is a char valued 'b' } else if (a=='c') { // if a is a char valued 'c' } else { //if a is none of the above }` however it must be noted that you should use 'switch' instead if your code checks for the same variable's value Section 15.10: goto Jumps to a labelled statement, which must be located in the current function. `bool f(int arg) { bool result = false; hWidget widget = get_widget(arg); if (!g()) { // we can't continue, but must do cleanup still goto end; } // ... result = true; end: release_widget(widget); return result; }` Section 15.11: Jump statements : break, continue, goto, exit The break instruction: Using break we can leave a loop even if the condition for its end is not fulfilled. It can be used to end an innite loop, or to force it to end before its natural end The syntax is break; Example: we often use break in switch cases,ie once a case i switch is satised then the code block of that condition is executed

```
. switch(conditon){ case 1: block1; case 2: block2; case 3: block3; default: blockdefault; }
```

in this case if case 1 is satised then block 1 is executed , what we really want is only the block1 to be processed but instead once the block1 is processed remaining blocks,block2,block3 and blockdefault are also processed even though only case 1 was satied.To avoid this we use break at the end of each block like : `switch(condition){ case 1: block1; break; case 2: block2; break; case 3: block3; break; default: blockdefault; break; }` so only one block is processed and the control moves out of the switch loop. break can also be used in other conditional and non conditional loops like if,while,for etc; example: `if(condition1){ if(condition2){ break; } }` The continue instruction: The continue instruction causes the program to skip the rest of the loop in the present iteration as if the end of the statement block would have been reached, causing it to jump to the following iteration. The syntax is `continue;` Example consider the following : `for(int i=0;i<10;i++){ if(i%2==0) continue; cout<<"@"<<i; }` which produces the output: @1 @3 @5 @7 @9 i this code whenever the condition `i%2==0` is satised continue is processed,this causes the compiler to skip all the remaining code(printing @ and i) and increment/decrement statement of the loop gets executed.

Chapter 29

ADVANCED FUNCTIONS AND CALLBACKS

Chapter 30

ADVANCED FUNCTIONS & CALLBACKS

Chapter 31

ADVANCED FUNCTIONS

31.1 2.1 Variadic Functions

```
1 #include <iostream>
2 #include <cstdarg>
3 using namespace std;
4
5 // Function with variable number of arguments
6 int sum(int count, ...) {
7     va_list args;
8     va_start(args, count);
9
10    int total = 0;
11    for (int i = 0; i < count; i++) {
12        total += va_arg(args, int);
13    }
14
15    va_end(args);
16    return total;
17 }
18
19 int main() {
20     cout << sum(3, 10, 20, 30) << endl;    // 60
21     cout << sum(5, 1, 2, 3, 4, 5) << endl; // 15
22
23     return 0;
24 }
```

31.2 2.2 Function Recursion

```
1 #include <iostream>
2 using namespace std;
3
4 // Factorial using recursion
5 int factorial(int n) {
6     if (n <= 1) {
7         return 1; // Base case
8     }
9     return n * factorial(n - 1); // Recursive case
10 }
11
12 // Fibonacci using recursion (inefficient)
13 int fibonacci(int n) {
14     if (n <= 1) return n;
15     return fibonacci(n - 1) + fibonacci(n - 2);
16 }
```



```

16 }
17
18 // Fibonacci with memoization (efficient)
19 int fib_memo(int n, int memo[]) {
20     if (n <= 1) return n;
21     if (memo[n] != -1) return memo[n];
22
23     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo);
24     return memo[n];
25 }
26
27 int main() {
28     cout << factorial(5) << endl; // 120
29     cout << fibonacci(10) << endl; // 55
30
31     int memo[11];
32     for (int i = 0; i < 11; i++) memo[i] = -1;
33     cout << fib_memo(10, memo) << endl; // 55
34
35     return 0;
36 }

```

31.3 2.3 Inline Functions

```

1 #include <iostream>
2 using namespace std;
3
4 // Inline function - compiler may expand code
5 inline int square(int x) {
6     return x * x;
7 }
8
9 // Inline with condition
10 inline double max_value(double a, double b) {
11     return (a > b) ? a : b;
12 }
13
14 int main() {
15     cout << square(5) << endl; // 25
16     cout << max_value(3.5, 2.1) << endl; // 3.5
17
18     return 0;
19 }

```

31.3.1 Professional Notes: Functional Depth

1. Recursion and Tail-Call Optimization (TCO)

Recursion is a technique where a function calls itself. * **Base Case**: The condition where recursion stops. * **Stack Overflow**: Deep recursion can exhaust the stack. * **Tail-Call Optimization**: If the recursive call is the *last* action in the function, some compilers can transform it into a loop, saving stack space.

```

1 // Tail-recursive factorial
2 int factorial_tail(int n, int acc = 1) {
3     if (n <= 1) return acc;
4     return factorial_tail(n - 1, n * acc);

```

```
5 }
```

2. Callable Objects (Functors)

In C++, anything that can be invoked with `()` is a callable. * **Function Pointers**: `void (*ptr)(int)`. * **Functors**: Classes that overload `operator()`. * **Lambdas (C++11)**: Anonymous functions. * `std::function` (C++11): A polymorphic wrapper for any callable.

3. Argument Dependent Lookup (ADL) - Recap

Functions are found in the namespaces of their arguments. This is why you can call `std::cout << obj` without qualifying the operator if it's defined in the same namespace as `obj`.

31.4 2.4 Static Functions

```
1 #include <iostream>
2 using namespace std;
3
4 // File scope - only visible in this file
5 static void internal_function() {
6     cout << "Internal function" << endl;
7 }
8
9 // Static with counter
10 int get_call_count() {
11     static int count = 0; // Persists between calls
12     return ++count;
13 }
14
15 int main() {
16     cout << get_call_count() << endl; // 1
17     cout << get_call_count() << endl; // 2
18     cout << get_call_count() << endl; // 3
19
20     internal_function(); // OK in same file
21
22     return 0;
23 }
```

Chapter 32

FUNCTION POINTERS & CALLBACKS

32.1 3.1 Function Pointers

```
1 #include <iostream>
2 using namespace std;
3
4 // Function pointer declaration: return_type (*name)(parameters)
5 int add(int a, int b) {
6     return a + b;
7 }
8
9 int subtract(int a, int b) {
10    return a - b;
11 }
12
13 int multiply(int a, int b) {
14    return a * b;
15 }
16
17 int main() {
18     // Declare function pointer
19     int (*operation)(int, int);
20
21     // Assign function to pointer
22     operation = add;
23     cout << operation(5, 3) << endl;    // 8
24
25     operation = subtract;
26     cout << operation(5, 3) << endl;    // 2
27
28     operation = multiply;
29     cout << operation(5, 3) << endl;    // 15
30
31     return 0;
32 }
```

32.2 3.2 Function Pointer Arrays

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b) { return a + b; }
```

```
5 int subtract(int a, int b) { return a - b; }
6 int multiply(int a, int b) { return a * b; }
7 int divide(int a, int b) { return a / b; }
8
9 int main() {
10     // Array of function pointers
11     int (*operations[4])(int, int) = {
12         add, subtract, multiply, divide
13     };
14
15     int a = 20, b = 4;
16
17     for (int i = 0; i < 4; i++) {
18         cout << "Result: " << operations[i](a, b) << endl;
19     }
20     // Output: 24, 16, 80, 5
21
22     return 0;
23 }
```

32.3 3.3 Callbacks

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Callback function type
6 typedef void (*Callback)(const string&);
7
8 class Button {
9 private:
10     Callback on_click;
11
12 public:
13     void set_click_handler(Callback callback) {
14         on_click = callback;
15     }
16
17     void click() {
18         if (on_click) {
19             on_click("Button clicked!");
20         }
21     }
22 };
23
24 void handle_click(const string& message) {
25     cout << message << endl;
26 }
27
28 int main() {
29     Button button;
30     button.set_click_handler(handle_click);
31     button.click(); // Output: Button clicked!
32
33     return 0;
34 }
```

Chapter 33

Chapter 4: Floating Point & Bit Manipulation

Understanding how data is stored at the bit level and how floating-point numbers approximate real values is essential for high-performance C++ engineering.

33.1 4.1 Floating Point Arithmetic

Floating point numbers represent approximations of their assigned values. This is due to the binary representation of base-10 decimals.

33.1.1 1. The Imprecision Pitfall

A common mistake is assuming floating point equality will work as expected:

```
1 double a = 0.1;
2 double b = 0.2;
3 double c = 0.3;
4 if (a + b == c) {
5     std::cout << "Exact match!" << std::endl;
6 } else {
7     std::cout << "Imprecise!" << std::endl; // Usually prints this
8 }
```

In IEEE 754 standard (used by most C++ compilers), 0.1 and 0.2 cannot be represented exactly in binary, leading to a small rounding error when added.

33.1.2 2. Comparison Strategy

Never use == with floats. Use an “epsilon” (a small tolerance):

```
1 #include <cmath>
2 #include <limits>
3
4 bool nearly_equal(double a, double b) {
5     return std::abs(a - b) < std::numeric_limits<double>::epsilon();
6 }
```

33.1.3 3. Special Values: NaN and Infinity

- `std::numeric_limits<double>::quiet_NaN()`: Not a Number (e.g., 0/0).
- `std::numeric_limits<double>::infinity()`: Infinity (e.g., 1/0).

33.2 4.2 Bitwise Mastery (Low-Level Optimization)

Bitwise operators manipulate individual bits. Essential for embedded systems, graphics, and cryptography.

33.2.1 The Operators

- `&` (AND): Both bits must be 1.
- `|` (OR): At least one bit must be 1.
- `^` (XOR): Bits must be different.
- `~` (NOT): Flip all bits.
- `<<` (Left Shift): Multiply by 2^N .
- `>>` (Right Shift): Divide by 2^N .

33.2.2 God-Tier Tricks

1. **Check Odd/Even:** `(x & 1) == 0` (Even). Faster than `% 2`.
2. **Multiply by 2:** `x << 1`.
3. **Divide by 2:** `x >> 1`.
4. **Clear Last Set Bit:** `x &= (x - 1)`. Used to count set bits (Kernighan's Algorithm).
5. **Check Power of 2:** `(x > 0) && ((x & (x - 1)) == 0)`.
6. **Toggle Bit N:** `x ^= (1 << N)`.
7. **Set Bit N:** `x |= (1 << N)`.
8. **Clear Bit N:** `x &= ~(1 << N)`.

```
1 // Fast Power of 2 check
2 bool isPowerOf2(int x) {
3     return x && !(x & (x - 1));
4 }
```

33.3 4.3 Bit Fields

Bit fields allow you to specify the number of bits each member of a struct or class should occupy. This is crucial for matching hardware protocols or saving space.

```
1 struct HardwareRegister {
2     unsigned int enable : 1; // 1 bit
3     unsigned int mode   : 3; // 3 bits
4     unsigned int value  : 4; // 4 bits
5 };
```

Professional Note: The exact layout of bit fields is implementation-defined and depends on the platform's endianness and alignment rules.

1. Bit Manipulation Hacks * **Swapping without temp:** $a \wedge= b; b \wedge= a; a \wedge= b;$ (
2. Floating Point Models * **fast-math:** A compiler flag (e.g., `-ffast-math` in GCC) that allows the comp

Chapter 34

OOP AND ENCAPSULATION

Chapter 35

OBJECT-ORIENTED PROGRAMMING: ENCAPSULATION & DESIGN

Chapter 36

OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS (C++98)

36.1 CLASSES & OBJECTS

36.1.1 1.1 Basic Class Structure

A class is a blueprint for objects. It encapsulates data and behavior.

```
1 #include <iostream>
2 #include <string>
3
4 class Person {
5 public:      // Access modifier: Public
6     std::string name;
7     int age;
8
9     // Member function (Method)
10    void introduce() {
11        std::cout << "I am " << name << ", age " << age << "\n";
12    }
13 };
14
15 int main() {
16     Person p;
17     p.name = "Alice";
18     p.age = 30;
19     p.introduce();
20     return 0;
21 }
```

c

1. Function Overloading and Name Mangling Function overloading allows multiple functions with the

2. Operator Overloading: Giving Syntax

cpp class Complex { double

Overloading + as member Complex operator+(const Complex& &ot;

Overloading << for output friend std::ostream& operator<<(s

c.i << "i)"; return os; \} \};

3

36.1.2 1.2 Access Modifiers & Encapsulation

Encapsulation hides internal state to prevent invalid access.

- **public:** Accessible from anywhere.
- **private:** Accessible only from within the class.
- **protected:** Accessible from the class and its derived classes.

```
1 class BankAccount {
2 private:
3     double balance; // Hidden data
4
5 public:
6     void deposit(double amount) {
7         if (amount > 0) balance += amount;
8     }
9
10    double get_balance() const { // Read-only access
11        return balance;
12    }
13};
```

36.1.3 1.3 Constructors & Destructors

Constructors initialize objects. Destructors clean up resources.

```
1 class Car {
2     std::string brand;
3     int* buffer;
4
5 public:
6     // Default Constructor
7     Car() : brand("Unknown"), buffer(new int[10]) {
8         std::cout << "Default Constructor\n";
9     }
10
11    // Parameterized Constructor
12    Car(const std::string& b) : brand(b), buffer(new int[10]) {
13        std::cout << "Param Constructor\n";
14    }
15
16    // Copy Constructor (Deep Copy)
17    Car(const Car& other) : brand(other.brand) {
18        buffer = new int[10];
19        // memcpy or loop to copy buffer content
20        std::cout << "Copy Constructor\n";
21    }
22
23    // Destructor
24    ~Car() {
25        delete[] buffer; // Cleanup
26        std::cout << "Destructor\n";
27    }
28};
```

36.1.4 1.4 The Rule of Three (C++98)

If you explicitly define one of the following, you likely need all three to manage resources correctly: 1. **Destructor**: To free resources (e.g., delete memory). 2. **Copy Constructor**: To perform deep copies. 3. **Copy Assignment Operator**: To handle assignment (`a = b`).

```

1 class Buffer {
2     int* ptr;
3 public:
4     Buffer() { ptr = new int[10]; }
5     ~Buffer() { delete[] ptr; }
6
7     // Copy Constructor
8     Buffer(const Buffer& other) {
9         ptr = new int[10];
10        // copy data...
11    }
12
13    // Assignment Operator
14    Buffer& operator=(const Buffer& other) {
15        if (this != &other) { // Self-assignment check
16            delete[] ptr;     // Free old
17            ptr = new int[10]; // Allocate new
18            // copy data...
19        }
20        return *this;
21    }
22 };

```

36.2 INHERITANCE & POLYMORPHISM

36.2.1 2.1 Inheritance Basics

Inheritance allows a class to derive features from another.

```

1 class Animal {
2 public:
3     void eat() { std::cout << "Eating...\n"; }
4 };
5
6 class Dog : public Animal { // Dog inherits from Animal
7 public:
8     void bark() { std::cout << "Woof!\n"; }
9 };

```

36.2.2 2.2 Virtual Functions (Polymorphism)

Polymorphism allows objects to be treated as instances of their base class, but behave like their actual derived class.

```

1 class Shape {
2 public:
3     // Virtual function: Can be overridden
4     virtual void draw() { std::cout << "Drawing Shape\n"; }
5
6     // Virtual Destructor (Crucial for inheritance)
7     virtual ~Shape() {}
8 };

```

```

9
10 class Circle : public Shape {
11 public:
12     void draw() { std::cout << "Drawing Circle\n"; } // Override
13 };
14
15 int main() {
16     Shape* s = new Circle();
17     s->draw(); // Prints "Drawing Circle" (Dynamic Dispatch)
18     delete s; // Properly calls ~Circle then ~Shape
19     return 0;
20 }

```

36.2.3 2.3 Pure Virtual Functions (Abstract Classes)

A pure virtual function (= 0) makes a class **Abstract**. It cannot be instantiated and must be subclassed.

```

1 class Interface {
2 public:
3     virtual void execute() = 0; // Pure virtual
4     virtual ~Interface() {}
5 };
6
7 class Concrete : public Interface {
8 public:
9     void execute() { std::cout << "Executed.\n"; }
10 };

```

36.3 ADVANCED CLASS FEATURES

36.3.1 3.1 Static Members

Shared by all instances of the class.

```

1 class User {
2 public:
3     static int userCount; // Declaration
4     User() { userCount++; }
5 };
6
7 // Definition (Must be outside class)
8 int User::userCount = 0;

```

36.3.2 3.2 Friend Classes/Functions

Friends can access private members.

```

1 class Box {
2     int width;
3 public:
4     Box(int w) : width(w) {}
5     friend void printWidth(Box& b);
6 };
7
8 void printWidth(Box& b) {
9     // Can access private 'width'

```

```

10     std::cout << b.width << "\n";
11 }

```

36.4 PART 2: GENERIC PROGRAMMING (TEMPLATES)

Templates are the foundation of Generic Programming in C++. They allow writing code that works with any data type. This is how the STL is implemented.

36.4.1 4.1 Function Templates

A function template defines a family of functions.

```

1 // Template declaration
2 template <typename T>
3 T myMax(T a, T b) {
4     return (a > b) ? a : b;
5 }
6
7 int main() {
8     std::cout << myMax(10, 20) << "\n";           // T is int
9     std::cout << myMax(3.14, 2.71) << "\n";       // T is double
10    // std::cout << myMax(10, 3.14) << "\n";      // Error: Mismatched types
11    return 0;
12 }

```

36.4.2 4.2 Class Templates

Classes can also be templated to hold data of any type.

```

1 template <typename T>
2 class Box {
3 private:
4     T content;
5 public:
6     Box(T val) : content(val) {}
7
8     T getContent() const { return content; }
9     void setContent(T val) { content = val; }
10 };
11
12 int main() {
13     Box<int> intBox(123);
14     Box<std::string> strBox("Hello");
15
16     std::cout << intBox.getContent() << "\n";
17     return 0;
18 }

```

36.4.3 4.3 Multiple Template Parameters

Templates can accept multiple types.

```

1 template <typename T1, typename T2>
2 struct Pair {
3     T1 first;
4     T2 second;

```

```

5      Pair(T1 a, T2 b) : first(a), second(b) {}
6  };
7
8
9  int main() {
10     Pair<std::string, int> p("Alice", 30);
11     return 0;
12 }

```

36.4.4 4.4 Non-Type Template Parameters

Templates can take values (integers, pointers) as parameters, not just types.

```

1  // N is a compile-time constant
2  template <typename T, int N>
3  class Array {
4      T data[N];
5  public:
6      int size() const { return N; }
7  };
8
9  int main() {
10     Array<int, 5> arr; // Fixed size array of 5 ints
11     // Array<int, 10> arr2 = arr; // Error: Different types
12     return 0;
13 }

```

36.4.5 4.5 Template Specialization

You can define specific implementations for specific types.

Full Specialization

```

1  template <typename T>
2  class Formatter {
3  public:
4      void format(T val) { std::cout << "General: " << val << "\n"; }
5  };
6
7  // Specialized for bool
8  template <>
9  class Formatter<bool> {
10 public:
11     void format(bool val) {
12         std::cout << "Boolean: " << (val ? "true" : "false") << "\n";
13     }
14 };

```

Partial Specialization (Class Templates Only)

```

1  template <typename T1, typename T2>
2  class MyMap { /* ... */ };
3
4  // Partial specialization for when both types are the same
5  template <typename T>
6  class MyMap<T, T> { /* Optimized implementation */ };

```

36.4.6 4.6 The `typename` Keyword

When a type depends on a template parameter (dependent type), you must use `typename`.

```

1 template <typename T>
2 void func() {
3     // T::iterator *iter; // Ambiguous: Is iterator a type or a static member?
4
5     typename T::iterator *iter; // Correct: Tells compiler iterator is a type
6 }

```

36.4.7 4.7 Templates vs Macros

Templates are type-safe and processed by the compiler. Macros are text substitution processed by the preprocessor. Always prefer templates.

Summary: - **Templates** allow code reuse for different types. - **Function Templates** deduce types automatically. - **Class Templates** require explicit type arguments. - **Specialization** allows handling specific types differently.

```

1 <!-- Merged content from Chapter_14_NAMESPACES.md -->
2
3 # NAMESPACES
4
5
6 ## 13.1 Namespace Basics
7
8 '''cpp
9 #include <iostream>
10 using namespace std;
11
12 // Define namespace
13 namespace Math {
14     double PI = 3.14159;
15
16     double circle_area(double radius) {
17         return PI * radius * radius;
18     }
19 }
20
21 namespace Graphics {
22     double PI = 3.14; // Different PI
23
24     void draw_circle(double radius) {
25         cout << "Drawing circle with radius: " << radius << endl;
26     }
27 }
28
29 int main() {
30     // Access with namespace::name
31     cout << Math::PI << endl;
32     cout << Graphics::PI << endl;
33
34     cout << Math::circle_area(5) << endl;
35     Graphics::draw_circle(5);
36
37     return 0;
38 }

```

36.5 13.2 Namespace Aliases


```
1 #include <iostream>
2 using namespace std;
3
4 namespace Very {
5     namespace Long {
6         namespace Namespace {
7             void function() {
8                 cout << "Long namespace function" << endl;
9             }
10        }
11    }
12}
13
14int main() {
15    // Use alias to shorten
16    namespace VLN = Very::Long::Namespace;
17
18    VLN::function();
19
20    // or use using
21    using namespace Very::Long::Namespace;
22    function();
23
24    return 0;
25}
```

Chapter 37

Professional Notes: Chapter 31: Pointers to members

Section 31.1: Pointers to static member functions Section 31.2: Pointers to member functions
Section 31.3: Pointers to member variables Section 31.4: Pointers to static member variables

Chapter 38

Professional Notes: Chapter 34: Classes/Structures

Section 34.1: Class basics Section 34.2: Final classes and structs Section 34.3: Access specifiers
Section 34.4: Inheritance Section 34.5: Friendship Section 34.6: Virtual Inheritance Section
34.7: Private inheritance: restricting base class interface Section 34.8: Accessing class mem-
bers Section 34.9: Member Types and Aliases Section 34.10: Nested Classes/Structures Section
34.11: Unnamed struct/class Section 34.12: Static class members Section 34.13: Multiple In-
heritance Section 34.14: Non-static member functions

Chapter 39

Professional Notes: Chapter 19: Friend keyword

Well-designed classes encapsulate their functionality, hiding their implementation while providing a clean, documented interface. This allows redesign or change so long as the interface is unchanged. In a more complex scenario, multiple classes that rely on each others' implementation details may be required. Friend classes and functions allow these peers access to each others' details, without compromising the encapsulation and information hiding of the documented interface. Section 19.1: Friend function A class or a structure may declare any function it's friend. If a function is a friend of a class, it may access all it's protected and private members: `// Forward declaration of functions. void friend_function(); void non_friend_function(); class PrivateHolder { public: PrivateHolder(int val) : private_value(val) {} private: int private_value; // Declare one of the function as a friend. friend void friend_function(); }; void non_friend_function() { PrivateHolder ph(10); // Compilation error: private_value is private. std::cout << ph.private_value << std::endl; } void friend_function() { // OK: friends may access private values. PrivateHolder ph(10); std::cout << ph.private_value << std::endl; }` Access modifiers do not alter friend semantics. Public, protected and private declarations of a friend are equivalent. Friend declarations are not inherited. For example, if we subclass PrivateHolder: `class PrivateHolderDerived : public PrivateHolder { public: PrivateHolderDerived(int val) : PrivateHolder(val) {} private: int derived_private_value = 0; };` and try to access it's members, we'll get the following: `void friend_function() { PrivateHolderDerived pd(20); // OK. std::cout << pd.private_value << std::endl; // Compilation error: derived_private_value is private.`

`std::cout << pd.derived_private_value << std::endl; }` Note that PrivateHolderDerived member function cannot access PrivateHolder::private_value, while friend function can do it. Section 19.2: Friend method Methods may declared as friends as well as functions: `class Accesser { public: void private_accesser(); }; class PrivateHolder { public: PrivateHolder(int val) : private_value(val) {} friend void Accesser::private_accesser(); private: int private_value; }; void Accesser::private_accesser() { PrivateHolder ph(10); // OK: this method is declares as friend. std::cout << ph.private_value << std::endl; }` Section 19.3: Friend class A whole class may be declared as friend. Friend class declaration means that any member of the friend may access private and protected members of the declaring class: `class Accesser { public: void private_accesser1(); void private_accesser2(); }; class PrivateHolder { public: PrivateHolder(int val) : private_value(val) {} friend class Accesser; private: int private_value; }; void Accesser::private_accesser1() { PrivateHolder ph(10); // OK. std::cout << ph.private_value << std::endl; } void Accesser::private_accesser2() { PrivateHolder ph(10); // OK. std::cout << ph.private_value + 1 << std::endl;`

`}` Friend class declaration is not reexive. If classes need private access in both directions, both of them need friend declarations. `class Accesser { public: void private_accesser1(); void private_accesser2(); private: int private_value = 0; }; class PrivateHolder { public: Private-`

```
Holder(int val) : private_value(val) {} // Accesser is a friend of PrivateHolder friend class
Accesser; void reverse_accesse() { // but PrivateHolder cannot access Accesser's members.
Accesser a; std::cout « a.private_value; } private: int private_value; };
```

Chapter 40

Professional Notes: Chapter 31: Pointers to members

Section 31.1: Pointers to static member functions A static member function is just like an ordinary C/C++ function, except with scope: It is inside a class, so it needs its name decorated with the class name; It has accessibility, with public, protected or private. So, if you have access to the static member function and decorate it correctly, then you can point to the function like any normal function outside a class: `typedef int Fn(int); // Fn is a type-of function that accepts an int and returns an int // Note that MyFn() is of type 'Fn' int MyFn(int i) { return 2i; }` `class Class { public: // Note that Static() is of type 'Fn' static int Static(int i) { return 3i; } }; // Class int main() { Fn fn; // fn is a pointer to a type-of Fn fn = &MyFn; // Point to one function fn(3); // Call it fn = &Class::Static; // Point to the other function fn(4); // Call it } // main() Section 31.2: Pointers to member functions To access a member function of a class, you need to have a “handle” to the particular instance, as either the instance itself, or a pointer or reference to it. Given a class instance, you can point to various of its members with a pointer-to-member, IF you get the syntax correct! Of course, the pointer has to be declared to be of the same type as what you are pointing to... typedef int Fn(int); // Fn is a type-of function that accepts an int and returns an int class Class { public: // Note that A() is of type 'Fn' int A(int a) { return 2a; } // Note that B() is of type 'Fn' int B(int b) { return 3b; } }; // Class int main() { Class c; // Need a Class instance to play with Class p = &c; // Need a Class pointer to play with Fn Class::fn; // fn is a pointer to a type-of Fn within Class fn = &Class::A; // fn now points to A within any Class (c.fn)(5); // Pass 5 to c's function A (via fn)`

`fn = &Class::B; // fn now points to B within any Class (p->fn)(6); // Pass 6 to c's (via p) function B (via fn) } // main() Unlike pointers to member variables (in the previous example), the association between the class instance and the member pointer need to be bound tightly together with parentheses, which looks a little strange (as though the . and ->* aren't strange enough!) Section 31.3: Pointers to member variables To access a member of a class, you need to have a “handle” to the particular instance, as either the instance itself, or a pointer or reference to it. Given a class instance, you can point to various of its members with a pointer-to-member, IF you get the syntax correct! Of course, the pointer has to be declared to be of the same type as what you are pointing to... class Class { public: int x, y, z; char m, n, o; }; // Class int x; // Global variable int main() { Class c; // Need a Class instance to play with Class p = &c; // Need a Class pointer to play with int p_i; // Pointer to an int p_i = &x; // Now pointing to x p_i = &c.x; // Now pointing to c's x int Class::p_C_i; // Pointer to an int within Class p_C_i = &Class::x; // Point to x within any Class int i = c.p_C_i; // Use p_C_i to fetch x from c's instance p_C_i = &Class::y; // Point to y within any Class i = c.p_C_i; // Use p_C_i to fetch y from c's instance p_C_i = &Class::m; // ERROR! m is a char, not an int! char Class::p_C_c = &Class::m; // That's better... } // main() The syntax`

of pointer-to-member requires some extra syntactic elements: To denote the type of the pointer, you need to mention the base type, as well as the fact that it is inside a class: `int Class::ptr`. If you have a class or reference and want to use it with a pointer-to-member, you need to use the `.` operator (akin to the `.` operator). If you have a pointer to a class and want to use it with a pointer-to-member, you need to use the `->` operator (akin to the `->` operator). *Section 31.4: Pointers to static member variables* A static member variable is just like an ordinary C/C++ variable, except with scope:

It is inside a class, so it needs its name decorated with the class name; It has accessibility, with public, protected or private. So, if you have access to the static member variable and decorate it correctly, then you can point to the variable like any normal variable outside a class:

```
class Class { public: static int i; }; // Class
int Class::i = 1; // Define the value of i (and where it's stored!)
int j = 2; // Just another global variable
int main() { int k = 3; // Local variable
int p; p = &k; // Point to k
p = 2; // Modify it
p = &j; // Point to j
p = 3; // Modify it
p = &Class::i; // Point to Class::i
*p = 4; // Modify it } // main()
```

Chapter 41

Professional Notes: Chapter 34: Classes/Structures

Section 34.1: Class basics A class is a user-defined type. A class is introduced with the `class`, `struct` or `union` keyword. In colloquial usage, the term “class” usually refers only to non-union classes. A class is a collection of class members, which can be: member variables (also called “elds”), member functions (also called “methods”), member types or typedefs (e.g. “nested classes”), member templates (of any kind: variable, function, class or alias template) The `class` and `struct` keywords, called class keys, are largely interchangeable, except that the default access specifier for members and bases is “private” for a class declared with the `class` key and “public” for a class declared with the `struct` or `union` key (cf. Access modifiers). For example, the following code snippets are identical: `struct Vector { int x; int y; int z; };` // are equivalent to `class Vector { public: int x; int y; int z; };` By declaring a

class a new type is added to your program, and it is possible to instantiate objects of that class by `Vector my_vector;` Members of a class are accessed using dot-syntax. `my_vector.x = 10; my_vector.y = my_vector.x + 1; // my_vector.y = 11; my_vector.z = my_vector.y - 4; // my_vector.z = 7;` Section 34.2: Final classes and structs Version C++11 Deriving a class may be forbidden with final specifier. Let’s declare a final class: `class A final { }`; Now any attempt to subclass it will cause a compilation error:

```
//
Compilation error: cannot derive from final class: class B : public A { };
Final class may appear anywhere in class hierarchy: class A { }; // OK. class B final : public A { }; //
Compilation error: cannot derive from final class B. class C : public B { };
Section 34.3: Access specifiers There are three keywords that act as access specifiers. These limit the access to class members following the specifier, until another specifier changes the access level again:
Keyword public Everyone has access Description protected Only the class itself, derived classes and friends have access private Only the class itself and friends have access
When the type is defined using the class keyword, the default access specifier is private, but if the type is defined using the struct keyword, the default access specifier is public:
struct MyStruct { int x; };
class MyClass { int x; };
MyStruct s; s.x = 9; // well formed, because x is public
MyClass c; c.x = 9; // ill-formed, because x is private
Access specifiers are mostly used to limit access to internal elds and methods, and force the programmer to use a specific interface, for example to force use of getters and setters instead of referencing a variable directly:
class MyClass { public: /* Methods: */ int x() const noexcept { return m_x; } void setX(int const x) noexcept { m_x = x; } private: /* Fields: */ int m_x; };
Using protected is useful for allowing certain functionality of the type to be only accessible to the derived classes, for example, in the following code, the method calculateValue() is only accessible to classes deriving from the
```

```
base
class Plus2Base, such as FortyTwo: struct Plus2Base { int value() noexcept { return calculateValue() + 2; } protected: /* Methods: */ virtual int calculateValue() noexcept = 0; };
struct FortyTwo: Plus2Base { protected: /* Methods: */ int calculateValue() noexcept final override { return 40; } };
Note that the friend keyword can be used to add access exceptions to functions or types for accessing protected and private members. The public, protected, and private keywords can also be used to grant or limit access to base class
```


subobjects. See the Inheritance example. Section 34.4: Inheritance Classes/structs can have inheritance relations. If a class/struct B inherits from a class/struct A, this means that B has as a parent A. We say that B is a derived class/struct from A, and A is the base class/struct. struct A { public: int p1; protected: int p2; private: int p3; }; //Make B inherit publicly (default) from A struct B : A {}; There are 3 forms of inheritance for a class/struct: public private protected Note that the default inheritance is the same as the default visibility of members: public if you use the struct keyword, and private for the class keyword. It's even possible to have a class derive from a struct (or vice versa). In this case, the default inheritance is controlled by the child, so a struct that derives from a class will default to public inheritance, and a class that derives from a struct will have private inheritance by default. public inheritance: struct B : public A // or just struct B : A {

```
void foo() { p1 = 0; //well formed, p1 is public in B p2 = 0; //well
formed, p2 is protected in B p3 = 0; //ill formed, p3 is private in A } }; B b; b
.p1 = 1; //well formed, p1 is public b.p2 = 1; //ill formed, p2 is protected b.p3 = 1; //ill
formed, p3 is inaccessible private inheritance: struct B : private A { void foo() {
p1 = 0; //well formed, p1 is private in B p2 = 0; //well formed, p2 is private
in B p3 = 0; //ill formed, p3 is private in A } }; B b; b.p1 = 1; //ill formed,
p1 is private b.p2 = 1; //ill formed, p2 is private b.p3 = 1; //ill formed, p3 is inaccessible
protected inheritance: struct B : protected A { void foo() { p1 = 0; //well
formed, p1 is protected in B p2 = 0; //well formed, p2 is protected in B p3 =
0; //ill formed, p3 is private in A } }; B b; b.p1 = 1; //ill formed, p1 is protected b.
p2 = 1; //ill formed, p2 is protected b.p3 = 1; //ill formed, p3 is inaccessible Note that
although protected inheritance is allowed, the actual use of it is rare. One instance of how
protected inheritance is used in application is in partial base class specialization (usually
referred to as "controlled polymorphism"). When OOP was relatively new, (public) inheritance
was frequently said to model an "IS-A" relationship. That is, public inheritance is correct
only if an instance of the derived class is also an instance of the base class. This was later
renamed into the Liskov Substitution Principle: public inheritance should only be used when/if an
instance of the derived class can be substituted for an instance of the base class under any
possible circumstance (and still make sense). Private inheritance is typically said to embody a
completely different relationship: "is implemented in terms of"
```

(sometimes called a "HAS-A" relationship). For example, a Stack class could inherit privately from a Vector class. Private inheritance bears a much greater similarity to aggregation than to public inheritance. Protected inheritance is almost never used, and there's no general agreement on what sort of relationship it embodies. Section 34.5: Friendship The friend keyword is used to give other classes and functions access to private and protected members of the class, even though they are denied outside the class scope. class Animal { private: double weight; double height; public: friend void printWeight(Animal animal); friend class AnimalPrinter; // A common use for a friend function is to overload the operator« for streaming. friend std::ostream& operator«(std::ostream& os, Animal animal); }; void printWeight(Animal animal) { std::cout « animal.weight « " "; } class AnimalPrinter { public: void print(const Animal& animal) { // Because of the friend class AnimalPrinter;" declaration , we are // allowed to access private members here. std::cout << animal.weight << " " << animal.height << std::endl; } }; std::ostream& operator«(std::ostream& os, Animal animal) { os << "Animal height: " << animal.height << "\n"; return os; } int main() { Animal animal = {10, 5}; printWeight(animal); AnimalPrinter aPrinter; aPrinter.print(animal); std::cout << animal; } 10, 5 Animal height: 5

Section

34.6: Virtual Inheritance When using inheritance, you can specify the virtual keyword: struct A { }; struct B: public virtual A { }; When class B has virtual base A it means that A will reside in most derived class of inheritance tree, and thus that most derived class is also responsible for initializing that virtual base: struct A { int member; A(int param) { member = param; } }; struct B: virtual A { B(): A(5) { } }; struct C: B { C(): /*A(88)*/ { } }; void f() { C object; //error since C is not initializing it's indirect virtual baseA' } If we un-comment /A(88)/ we won't get any error since C is now initializing

it's indirect virtual base A. Also note that when we're creating variable object, most derived class is C, so C is responsible for creating (calling constructor of) A and thus value of A::member is 88, not 5 (as it would be if we were creating object of type B). It is useful when solving the diamond problem: A A A / || B C B C / / D D virtual inheritance normal inheritance B and C both inherit from A, and D inherits from B and C, so there are 2 instances of A in D! This results in ambiguity when you're accessing member of A through D, as the compiler has no way of knowing from which class do you want to access that member (the one which B inherits, or the one that is inherited by C?). Virtual inheritance solves this problem: Since virtual base resides only in most derived object, there will be only one instance of A in D.

```
struct A { void foo() {}
```

```
}; struct B : public /virtual/ A {}; struct C : public /virtual/ A {}; struct D : public B, public C { void bar() { foo(); //Error, which foo? B::foo() or C::foo()? - Ambiguous } }; Removing the comments resolves the ambiguity. Section 34.7: Private inheritance: restricting base class interface Private inheritance is useful when it is required to restrict the public interface of the class: class A { public: int move(); int turn(); }; class B : private A { public: using A::turn; }; B b; b.move(); // compile error b.turn(); // OK This approach efficiently prevents an access to the A public methods by casting to the A pointer or reference: B b; A& a = static_cast<A&>(b); // compile error In the case of public inheritance such casting will provide access to all the A public methods despite on alternative ways to prevent this in derived B, like hiding: class B : public A { private: int move(); }; or private using: class B : public A { private: using A::move; }; then for both cases it is possible: B b;
```

```
A& a = static_cast<A&>(b); // OK for public inheritance a.move(); // OK Section 34.8: Accessing class members To access member variables and member functions of an object of a class, the . operator is used: struct SomeStruct { int a; int b; void foo() {} }; SomeStruct var; // Accessing member variable a in var. std::cout << var.a << std::endl; // Assigning member variable b in var. var.b = 1; // Calling a member function. var.foo(); When accessing the members of a class via a pointer, the -> operator is commonly used. Alternatively, the instance can be dereferenced and the . operator used, although this is less common: struct SomeStruct { int a; int b; void foo() {} }; SomeStruct var; SomeStruct p = &var; // Accessing member variable a in var via pointer. std::cout << p->a << std::endl; std::cout << (p).a << std::endl; // Assigning member variable b in var via pointer. p->b = 1; (p).b = 1; // Calling a member function via a pointer. p->foo(); (p).foo(); When accessing static class members, the :: operator is used, but on the name of the class instead of an instance of it. Alternatively, the static member can be accessed from an instance or a pointer to an instance using the . or -> operator, respectively, with the same syntax as accessing non-static members. struct SomeStruct { int a; int b; void foo() {} static int c; static void bar() {} }; int SomeStruct::c; SomeStruct var; SomeStruct* p = &var; // Assigning static member variable c in struct SomeStruct.
```

```
SomeStruct::c = 5; // Accessing static member variable c in struct SomeStruct, through var and p. var.a = var.c; var.b = p->c; // Calling a static member function. SomeStruct::bar(); var.bar(); p->bar(); Background The -> operator is needed because the member access operator . has precedence over the dereferencing operator . One would expect that p.a would dereference p (resulting in a reference to the object p is pointing to) and then accessing its member a. But in fact, it tries to access the member a of p and then dereference it. I.e. p.a is equivalent to (p.a). In the example above, this would result in a compiler error because of two facts: First, p is a pointer and does not have a member a. Second, a is an integer and, thus, can't be dereferenced. The uncommonly used solution to this problem would be to explicitly control the precedence: (p).a Instead, the -> operator is almost always used. It is a short-hand for rst dereferencing the pointer and then accessing it. I.e. (p).a is exactly the same as p->a. The :: operator is the scope operator, used in the same manner as accessing a member of a namespace. This is because a static class member is considered to be in that class' scope, but isn't considered a member of
```

instances of that class. The use of normal `.` and `->` is also allowed for static members, despite them not being instance members, for historical reasons; this is of use for writing generic code in templates, as the caller doesn't need to be concerned with whether a given member function is static or non-static. Section 34.9: Member Types and Aliases A class or struct can also define member type aliases, which are type aliases contained within, and treated as members of, the class itself. `struct IHaveATypedef { typedef int MyTypedef; }; struct IHaveATemplateTypedef { template using MyTemplateTypedef = std::vector; }; Like static members, these typedefs are accessed using the scope operator, ::. IHaveATypedef::MyTypedef i = 5; // i is an int. IHaveATemplateTypedef::MyTemplateTypedef v; // v is a std::vector. As with normal type aliases, each member type alias is allowed to refer to any type defined or aliased before, but not after, its definition. Likewise, a typedef outside the class definition can refer to any accessible typedefs within the class definition, provided it comes after the class definition. template struct Helper {`

```

    T get() const { return static_cast(42); } }; struct IHaveTypedefs { // typedef MyTypedef
NonLinearTypedef; // Error if uncommented. typedef int MyTypedef; typedef Helper MyType-
defHelper; }; IHaveTypedefs::MyTypedef i; // x_i is an int. IHaveTypedefs::MyTypedefHelper
hi; // x_hi is a Helper. typedef IHaveTypedefs::MyTypedef TypedefBeFree; TypedefBeFree ii;
// ii is an int. Member type aliases can be declared with any access level, and will respect the
appropriate access modifier. class TypedefAccessLevels { typedef int PrvInt; protected: type-
def int ProInt; public: typedef int PubInt; }; TypedefAccessLevels::PrvInt prv_i; // Error:
TypedefAccessLevels::PrvInt is private. TypedefAccessLevels::ProInt pro_i; // Error: Typede-
fAccessLevels::ProInt is protected. TypedefAccessLevels::PubInt pub_i; // Good. class Derived
: public TypedefAccessLevels { PrvInt prv_i; // Error: TypedefAccessLevels::PrvInt is private.
ProInt pro_i; // Good. PubInt pub_i; // Good. }; This can be used to provide a level of ab-
straction, allowing a class' designer to change its internal workings without breaking code that
relies on it. class Something { friend class SomeComplexType; short s; // ... public: type-
def SomeComplexType MyHelper; MyHelper get_helper() const { return MyHelper(8, s, 19.5,
"shoe", false); } // ... }; // ... Something s; Something::MyHelper hlp = s.get_helper(); In
this situation, if the helper class is changed from SomeComplexType to some other type, only
the typedef and the

```

friend declaration would need to be modified; as long as the helper class provides the same functionality, any code that uses it as `Something::MyHelper` instead of specifying it by name will usually still work without any modifications. In this manner, we minimise the amount of code that needs to be modified when the underlying implementation is changed, such that the type name only needs to be changed in one location. This can also be combined with `decltype`, if one so desires. `class SomethingElse { AnotherComplexType<bool, int, SomeThirdClass> helper; public: typedef decltype(helper) MyHelper; private: InternalVariable ivh; // ... public: MyHelper& get_helper() const { return helper; } // ... }; In this situation, changing the implementation of SomethingElse::helper will automatically change the typedef for us, due to decltype. This minimises the number of modifications necessary when we want to change helper, which minimises the risk of human error. As with everything, however, this can be taken too far. If the typename is only used once or twice internally and zero times externally, for example, there's no need to provide an alias for it. If it's used hundreds or thousands of times throughout a project, or if it has a long enough name, then it can be useful to provide it as a typedef instead of always using it in absolute terms. One must balance forwards compatibility and convenience with the amount of unnecessary noise created. This can also be used with template classes, to provide access to the template parameters from outside the class. template class SomeClass { // ... public: typedef T MyParam; MyParam getParam() { return static_cast(42); } }; template typename T::MyParam some_func(T& t) { return t.getParam(); } SomeClass si; int i = some_func(si); This is commonly used with containers, which will usually provide their element type, and other helper types, as member type aliases. Most of the containers in the`

C++ standard library, for example, provide the following 12 helper types, along with any other special types they might need. `template`

```
class SomeContainer { // ... public: // Let's provide the same helper types as most
standard containers. typedef T value_type; typedef std::allocator allocator_type; typedef
value_type& reference; typedef const value_type& const_reference; typedef value_type* pointer;
typedef const value_type* const_pointer; typedef MyIterator iterator; typedef MyConstItera-
tor const_iterator; typedef std::reverse_iterator reverse_iterator; typedef std::reverse_iterator
const_reverse_iterator; typedef size_t size_type; typedef ptrdiff_t difference_type; }; Prior
to C++11, it was also commonly used to provide a “template typedef” of sorts, as the feature
wasn’t yet available; these have become a bit less common with the introduction of alias tem-
plates, but are still useful in some situations (and are combined with alias templates in other
situations, which can be very useful for obtaining individual components of a complex type
such as a function pointer). They commonly use the name type for their type alias. template
struct TemplateTypedef { typedef T type; } TemplateTypedef::type i; // i is an int. This
was often used with types with multiple template parameters, to provide an alias that denes
one or more of the parameters. template<typename T, size_t SZ, size_t D> class Array {
/* ... */ }; template<typename T, size_t SZ> struct OneDArray { typedef Array<T, SZ,
1> type; }; template<typename T, size_t SZ> struct TwoDArray { typedef Array<T, SZ, 2>
type; }; template struct MonoDisplayLine { typedef Array<T, 80, 1> type; }; OneDArray<int,
3>::type arr1i; // arr1i is an Array<int, 3, 1>. TwoDArray<short, 5>::type arr2s; // arr2s is
an Array<short, 5, 2>. MonoDisplayLine::type arr3c; // arr3c is an Array<char, 80, 1>.
```

Section 34.10: Nested Classes/Structures A class or struct can also contain another class/struct denition inside itself, which is called a “nested class”; in this situation, the containing class is referred to as the “enclosing class”. The nested class denition is considered to be a member of the enclosing class, but is otherwise separate. `struct Outer { struct Inner { }; };` From outside of the enclosing class, nested classes are accessed using the scope operator. From inside the enclosing class, however, nested classes can be used without qualifiers: `struct Outer { struct Inner { }; Inner in; }; // ... Outer o; Outer::Inner i = o.in;` As with a non-nested class/struct, member functions and static variables can be dened either within a nested class, or in the enclosing namespace. However, they cannot be dened within the enclosing class, due to it being considered to be a different class than the nested class. `// Bad. struct Outer { struct Inner { void do_something(); }; void Inner::do_something() {} }; // Good. struct Outer { struct Inner { void do_something(); }; }; void Outer::Inner::do_something() {}` As with non-nested classes, nested classes can be forward declared and dened later, provided they are dened before being used directly. `class Outer { class Inner1; class Inner2; class Inner1 { }; Inner1 in1;`

Chapter 42

POLYMORPHISM AND VIRTUALIZATION

Chapter 43

POLYMORPHISM & VIRTUALIZATION

Chapter 44

DEEP OBJECT MODEL & VIRTUALIZATION

Understanding the “C++ Object Model” distinguishes a user from a master. This section explains what the compiler generates for your classes.

44.0.1 2.5.1 The Cost of Polymorphism (vptr & vtable)

Every class with *at least one* virtual function has a hidden overhead.

1. **vptr (Virtual Pointer)**: A hidden member added to the *layout* of the object.
2. **vtable (Virtual Table)**: A static table of function pointers for that class.

```
1 class Base {
2     int data;
3     virtual void func() {}
4 };
5 // sizeof(Base) = sizeof(int) + sizeof(void*) + padding
6 // On 64-bit: 4 bytes (int) + 4 bytes (padding) + 8 bytes (ptr) = 16 bytes
```

44.0.2 Professional Notes: Polymorphism & Virtual Mechanics

1. Polymorphism and Virtual Destructors

Always declare the destructor as `virtual` in a polymorphic base class. * **The Danger**: If you delete a derived object via a base class pointer without a virtual destructor, only the base part is destroyed, leading to **Memory Leaks**.

```
1 class Base {
2 public:
3     virtual ~Base() {} // Essential!
4 };
```

2. Pure Virtual Functions and Abstract Classes

A function with `= 0` is pure virtual. It forces derived classes to provide an implementation. * **Abstract Class**: A class with at least one pure virtual function cannot be instantiated. * **Interface**: In C++, interfaces are typically classes with only public pure virtual functions and a virtual destructor.

3. Virtual Functions in Constructors/Destructors

Godhood Warning: Never call virtual functions in constructors or destructors. * **Reason:** During the base class constructor, the derived class members haven't been initialized yet. The vtable still points to the base class implementation. This is a common source of bugs.

4. The `override` and `final` Keywords (C++11)

- **`override`:** Ensures the function actually overrides a base class virtual function. Catches signature mismatches at compile time.
- **`final`:** Prevents further overriding or inheritance.

44.0.3 2.5.2 Multiple Inheritance & Thunks

When inheriting from multiple classes, pointer arithmetic gets tricky.

```

1 class A { int a; virtual void f() {} };
2 class B { int b; virtual void g() {} };
3 class C : public A, public B { int c; };
4
5 C obj;
6 A* pa = &obj; // Points to start of obj
7 B* pb = &obj; // Points to obj + sizeof(A) !!

```

- **Thunk:** A small piece of assembly code generated by the compiler to adjust the `this` pointer when calling a virtual function from a base class pointer that isn't at offset 0.

44.0.4 2.5.3 Virtual Inheritance (The Diamond Problem)

```

1 class Top { int t; };
2 class Left : virtual public Top { int l; };
3 class Right : virtual public Top { int r; };
4 class Bottom : public Left, public Right { int b; };

```

To solve the Diamond Problem (Top appearing twice), virtual inheritance ensures Top is shared. * **Cost:** Left and Right now contain a **vbptr** (Virtual Base Pointer) pointing to the shared Top instance. Accessing members of Top becomes slower (indirection).

44.0.5 2.5.4 Alignment & Padding Rules

CPU reads are efficient at specific addresses (multiples of 4 or 8). Compilers insert "padding bytes".

Rule: A member of size N must sit at an offset divisible by N .

```

1 struct Mixed {
2     char a;           // 1 byte
3                       // 3 bytes PADDING
4     int b;            // 4 bytes
5     short c;          // 2 bytes
6                       // 6 bytes PADDING (to align structure size to 8)
7 };
8 // sizeof(Mixed) = 16 (on 64-bit)

```

Optimization: Sort members by size (Largest to Smallest) to minimize padding.

Chapter 45

Professional Notes: Chapter 25: Polymorphism

Section 25.1: Dene polymorphic classes Section 25.2: Safe downcasting Section 25.3: Polymorphism & Destructors

Chapter 46

Professional Notes: Chapter 38: Virtual Member Functions

Section 38.1: Final virtual functions Section 38.2: Using override with virtual in C++11 and later Section 38.3: Virtual vs non-virtual member functions Section 38.4: Behaviour of virtual functions in constructors and destructors Section 38.5: Pure virtual functions

Chapter 47

Professional Notes: Chapter 40: Special Member Functions

Section 40.1: Default Constructor Section 40.2: Destructor Section 40.3: Copy and swap Section 40.4: Implicit Move and Copy

Chapter 48

Professional Notes: Chapter 25: Polymorphism

Section 25.1: Dene polymorphic classes The typical example is an abstract shape class, that can then be derived into squares, circles, and other concrete shapes. The parent class: Let's start with the polymorphic class: `class Shape { public: virtual ~Shape() = default; virtual double get_surface() const = 0; virtual void describe_object() const { std::cout << "this is a shape" << std::endl; }`

`double get_doubled_surface() const { return 2 * get_surface(); } };` How to read this denition ? You can dene polymorphic behavior by introduced member functions with the keyword `virtual`. Here `get_surface()` and `describe_object()` will obviously be implemented dierently for a square than for a circle. When the function is invoked on an object, function corresponding to the real class of the object will be determined at runtime. It makes no sense to dene `get_surface()` for an abstract shape. This is why the function is followed by `= 0`. This means that the function is pure virtual function. A polymorphic class should always dene a virtual destructor. You may dene non virtual member functions. When these function will be invoked for an object, the function will be chosen depending on the class used at compile-time. Here `get_double_surface()` is dened in this way. A class that contains at least one pure virtual function is an abstract class. Abstract classes cannot be instantiated. You may only have pointers or references of an abstract class type. Derived classes Once a polymorphic base class is dened you can derive it. For example: `class Square : public Shape { Point top_left; double side_length; public: Square (const Point& top_left, double side) : top_left(top_left), side_length(side_length) {} double get_surface() override { return side_length * side_length; } void describe_object() override { std::cout << "this is a square starting at" << top_left.x << ", " << top_left.y << " with a length of " << side_length << std::endl; } };`

Some explanations: You can dene or override any of the virtual functions of the parent class. The fact that a function was virtual in the parent class makes it virtual in the derived class. No need to tell the compiler the keyword `virtual` again. But it's recommended to add the keyword `override` at the end of the function declaration, in order to prevent subtle bugs caused by unnoticed variations in the function signature. If all the pure virtual functions of the parent class are dened you can instantiate objects for this class, else it will also become an abstract class. You are not obliged to override all the virtual functions. You can keep the version of the parent if it suits your need. Example of instantiation in `main()` `{ Square square(Point(10.0, 0.0), 6); // we know it's a square, the compiler also square.describe_object(); std::cout << "Surface:" << square.get_surface() << std::endl; Circle circle(Point(0.0, 0.0), 5); Shape ps = nullptr; // we don't know yet the real type of the object ps = &circle; // it's a circle, but it could as well be a square ps->describe_object(); std::cout << "Surface:" << ps->get_surface() << std::endl; }` Section 25.2: Safe downcasting Suppose that you have a pointer to an object of a polymorphic class:

Shape ps; // see example on defining a polymorphic class ps = get_a_new_random_shape();
 // if you don't have such a function yet, you // could just write ps = new Square(0.0,0.0, 5);
 a downcast would be to cast from a general polymorphic Shape down to one of its derived and more specific shape like Square or Circle. Why to downcast ? Most of the time, you would not need to know which is the real type of the object, as the virtual functions allow you to manipulate your object independently of its type: std::cout << "Surface:" << ps->get_surface() << std::endl; If you don't need any downcast, your design would be perfect. However, you may need sometimes to downcast. A typical example is when you want to invoke a non virtual function that exist only for the child class. Consider for example circles. Only circles have a diameter. So the class would be defined as : class Circle: public Shape { // for Shape, see example on defining a polymorphic class Point center; double radius; public:

```
Circle(const Point& center, double radius) : center(center), radius(radius) {} double get_surface()
const override { return r * r * M_PI; }
// this is only for circles. Makes no sense for other shapes double get_diameter() const { return
2 * r; } }; The get_diameter() member function only exist for circles. It was not defined for a
Shape object: Shape* ps = get_any_shape(); ps->get_diameter(); // OUCH !!! Compilation
error How to downcast ? If you'd know for sure that ps points to a circle you could opt for a
static_cast: std::cout << "Diameter:" << static_cast<Circle>(ps)->get_diameter() << std::endl;
This will do the trick. But it's very risky: if ps appears to be anything else than a Circle the
behavior of your code will be undefined. So rather than playing Russian roulette, you should
safely use a dynamic_cast. This is specifically for polymorphic classes : int main() { Circle
circle(Point(0.0, 0.0), 10); Shape &shape = circle; std::cout << "The shape has a surface of" <<
shape.get_surface() << std::endl; //shape.get_diameter(); // OUCH !!! Compilation error Circle
pc = dynamic_cast<Circle>(&shape); // will be nullptr if ps wasn't a circle if (pc) std::cout
<< "The shape is a circle of diameter" << pc->get_diameter() << std::endl; else std::cout << "The
shape isn't a circle !" << std::endl; }
```

Note that *dynamic_cast* is not possible on a class that is not polymorphic. You'd need at least one virtual function in the class or its parents to be able to use it. Section 25.3: Polymorphism & Destructors If a class is intended to be used polymorphically, with derived instances being stored as base pointers/references, its base class' destructor should be either virtual or protected. In the former case, this will cause object destruction to check the vtable, automatically calling the correct destructor based on the dynamic type. In the latter case, destroying the object through a base class pointer/reference is disabled, and the object can only be deleted when explicitly treated as its actual type. struct VirtualDestructor { virtual ~VirtualDestructor() = default; }; struct VirtualDerived : VirtualDestructor {};

```
struct ProtectedDestructor { protected: ~ProtectedDestructor() = default; }; struct ProtectedDerived : ProtectedDestructor { ~ProtectedDerived() = default; }; // ... VirtualDestructor
vd = new VirtualDerived; delete vd; // Looks up VirtualDestructor::~~VirtualDestructor()
in vtable, sees it's // VirtualDerived::~~VirtualDerived(), calls that. ProtectedDestructor* pd
= new ProtectedDerived; delete pd; // Error: ProtectedDestructor::~~ProtectedDestructor()
is protected. delete static_cast<ProtectedDerived*>(pd); // Good. Both of these practices
guarantee that the derived class' destructor will always be called on derived class instances,
preventing memory leaks.
```

Chapter 49

Professional Notes: Chapter 38: Virtual Member Functions

Section 38.1: Final virtual functions C++11 introduced final specifier which forbids method overriding if appeared in method signature: `class Base { public: virtual void foo() { std::cout << "Base::foo"; } }; class Derived1 : public Base { public: // Overriding Base::foo void foo() final { std::cout << "Derived1::foo"; } }; class Derived2 : public Derived1 { public: // Compilation error: cannot override final method virtual void foo() { std::cout << "Derived2::foo"; } };` The specifier final can only be used with 'virtual' member function and can't be applied to non-virtual member functions Like final, there is also a specifier caller 'override' which prevent overriding of virtual functions in the derived class. The specifiers override and final may be combined together to have desired effect: `class Derived1 : public Base { public: void foo() final override { std::cout << "Derived1::foo"; } };` Section 38.2: Using override with virtual in C++11 and later The specifier override has a special meaning in C++11 onwards, if appended at the end of function signature. This signifies that a function is Overriding the function present in base class & The Base class function is virtual There is no run time significance of this specifier as is mainly meant as an indication for compilers The example below will demonstrate the change in behaviour with or without using override. Without override:

```
#include <iostream>
struct X { virtual void f() { std::cout << "X::f()"; } };
struct Y : X { // Y::f() will not override X::f() because it has a different signature, // but the compiler will accept the code (and silently ignore Y::f()).
    virtual void f(int a) { std::cout << a << " "; } };
With override:
#include <iostream>
struct X { virtual void f() { std::cout << "X::f()"; } };
struct Y : X { // The compiler will alert you to the fact that Y::f() does not // actually override anything.
    virtual void f(int a) override { std::cout << a << " "; } };
Note that override is not a keyword, but a special identifier which only may appear in function signatures. In all other contexts override still may be used as an identifier:
void foo() { int override = 1; // OK.
int virtual = 2; // Compilation error: keywords can't be used as identifiers. }
Section 38.3: Virtual vs non-virtual member functions
With virtual member functions:
#include <iostream>
struct X { virtual void f() { std::cout << "X::f()"; } };
struct Y : X { // Specifying virtual again here is optional // because it can be inferred from X::f().
    virtual void f() { std::cout << "Y::f()"; } };
void call(X& a) { a.f(); }
int main() { X x; Y y; call(x); // outputs "X::f()"
call(y); // outputs "Y::f()"
```

```
}
Without virtual member functions:
#include <iostream>
struct X { void f() { std::cout << "X::f()"; } };
struct Y : X { void f() { std::cout << "Y::f()"; } };
void call(X& a) { a.f(); }
int main() { X x; Y y; call(x); // outputs "X::f()"
call(y); // outputs "X::f()" }
Section 38.4: Behaviour of virtual functions in constructors and destructors
The behaviour of virtual functions in constructors and destructors is often confusing when first encountered.
#include <iostream>
using namespace std;
class base { public: base() { f("base constructor"); }
~base() { f("base destructor"); }
virtual const char* v() { return "base::v()"; }
void f(const char* caller) { cout << "When called from" << caller << ", " << v() << " gets called."; } }
```

```
}; class derived : public base { public: derived() { f("derived constructor"); } ~derived() { f("derived destructor"); } const char* v() override { return "derived::v()"; } }; int main() { derived d; } Output:
```

When called from base constructor, `base::v()` gets called. When called from derived constructor, `derived::v()` gets called. When called from derived destructor, `derived::v()` gets called. When called from base destructor, `base::v()` gets called. The reasoning behind this is that the derived class may define additional members which are not yet initialized (in the constructor case) or already destroyed (in the destructor case), and calling its member functions would be unsafe. Therefore during construction and destruction of C++ objects, the dynamic type of *this* is considered to be the constructor's or destructor's class and not a more-derived class.

Example: `#include #include using namespace std; class base { public: base() { std::cout << "foo is" << foo() << std::endl; } virtual int foo() { return 42; } }; class derived : public base { unique_ptr ptr_; public: derived(int i) : ptr_(new int(i)) { } // The following cannot be called before derived::derived due to how C++ behaves, // if it was possible... Kaboom! int foo() override { return ptr_; } }; int main() { derived d(4); } Section 38.5: Pure virtual functions`

We can also specify that a virtual function is pure virtual (abstract), by appending `= 0` to the declaration. Classes with one or more pure virtual functions are considered to be abstract, and cannot be instantiated; only derived classes which define, or inherit definitions for, all pure virtual functions can be instantiated. `struct Abstract { virtual void f() = 0; }; struct Concrete { void f() override { } }; Abstract a; // Error. Concrete c; // Good. Even if a function is specified as pure virtual, it can be given a default implementation. Despite this, the function will still be considered abstract, and derived classes will have to define it before they can be instantiated. In this case,`

the derived class' version of the function is even allowed to call the base class' version.

```
struct DefaultAbstract { virtual void f() = 0; }; void DefaultAbstract::f() { } struct WhyWouldWeDoThis : DefaultAbstract { void f() override { DefaultAbstract::f(); } }; There are a couple of reasons why we might want to do this: If we want to create a class that can't itself be instantiated, but doesn't prevent its derived classes from being instantiated, we can declare the destructor as pure virtual. Being the destructor, it must be defined anyways, if we want to be able to deallocate the instance. And as the destructor is most likely already virtual to prevent memory leaks during polymorphic use, we won't incur an unnecessary performance hit from declaring another function virtual. This can be useful when making interfaces. struct Interface { virtual ~Interface() = 0; }; Interface::~~Interface() = default; struct Implementation : Interface { }; // ~Implementation() is automatically defined by the compiler if not explicitly // specified, meeting the "must be defined before instantiation" requirement. If most or all implementations of the pure virtual function will contain duplicate code, that code can instead be moved to the base class version, making the code easier to maintain. class SharedBase { State my_state; std::unique_ptr my_helper; // ... public: virtual void config(const Context& cont) = 0; // ... }; / virtual / void SharedBase::config(const Context& cont) { my_helper = new Helper(my_state, cont.relevant_field); do_this(); and_that(); } class OneImplementation : public SharedBase { int i; // ... public: void config(const Context& cont) override; // ... }; void OneImplementation::config(const Context& cont) / override */ { my_state = { cont.some_field, cont.another_field, i }; SharedBase::config(cont); my_unique_setup(); }; // And so on, for other classes derived from SharedBase.
```

Chapter 50

Professional Notes: Chapter 40: Special Member Functions

Section 40.1: Default Constructor A default constructor is a type of constructor that requires no parameters when called. It is named after the type it constructs and is a member function of it (as all constructors are). `class C{ int i; public: // the default constructor definition C() : i(0){ // member initializer list – initialize i to 0 // constructor function body – can do more complex things here } }; C c1; // calls default constructor of C to create object c1 C c2 = C(); // calls default constructor explicitly C c3(); // ERROR: this intuitive version is not possible due to “most vexing parse” C c4{}; // but in C++11 {} CAN be used in a similar way C c5[2]; // calls default constructor for both array elements C* c6 = new C[2]; // calls default constructor for both array elements Another way to satisfy the “no parameters” requirement is for the developer to provide default values for all parameters: class D{ int i; int j; public: // also a default constructor (can be called with no parameters) D(int i = 0, int j = 42) : i(i), j(j){ } }; D d; // calls constructor of D with the provided default values for the parameters Under some circumstances (i.e., the developer provides no constructors and there are no other disqualifying conditions), the compiler implicitly provides an empty default constructor: class C{ std::string s; // note: members need to be default constructible themselves }; C c1; // will succeed – C has an implicitly defined default constructor Having some other type of constructor is one of the disqualifying conditions mentioned earlier: class C{ int i; public: C(int i) : i(i){ } };`

`C c1; // Compile ERROR: C has no (implicitly defined) default constructor Version < c++11 To prevent implicit default constructor creation, a common technique is to declare it as private (with no definition). The intention is to cause a compile error when someone tries to use the constructor (this either results in an Access to private error or a linker error, depending on the compiler). To be sure a default constructor (functionally similar to the implicit one) is denied, a developer could write an empty one explicitly. Version c++11 In C++11, a developer can also use the delete keyword to prevent the compiler from providing a default constructor. class C{ int i; public: // default constructor is explicitly deleted C() = delete; }; C c1; // Compile ERROR: C has its default constructor deleted Furthermore, a developer may also be explicit about wanting the compiler to provide a default constructor. class C{ int i; public: // does have automatically generated default constructor (same as implicit one) C() = default; C(int i) : i(i){ } }; C c1; // default constructed C c2(1); // constructed with the int taking constructor Version c++14 You can determine whether a type has a default constructor (or is a primitive type) using std::is_default_constructible from <type_traits>: class C1{ }; class C2{ public: C2(){ } }; class C3{ public: C3(int){ } }; using std::cout; using std::boolalpha; using std::endl; using std::is_default_constructible; cout << boolalpha << is_default_constructible() << endl; // prints true cout << boolalpha << is_default_constructible() << endl; // prints true cout << boolalpha << is_default_constructible() << endl; // prints true cout << boolalpha << is_default_constructible() << endl; // prints true`


```
« endl; // prints false
Version = c++11
In C++11, it is still possible to use the non-functor
version of std::is_default_constructible:
cout « boolalpha « is_default_constructible::value «
endl; // prints true
```

Section 40.2: Destructor A destructor is a function without arguments that is called when a user-defined object is about to be destroyed. It is named after the type it destructs with a ~ prex. class C{ int* is; string s; public: C() : is(new int[10]){ } ~C(){ // destructor definition delete[] is; } }; class C_child : public C{ string s_ch; public: C_child(){ } ~C_child(){ } // child destructor }; void f(){ C c1; // calls default constructor C c2[2]; // calls default constructor for both elements C* c3 = new C[2]; // calls default constructor for both array elements C_child c_ch; // when destructed calls destructor of s_ch and of C base (and in turn s) delete[] c3; // calls destructors on c3[0] and c3[1] } // automatic variables are destroyed here – i.e. c1, c2 and c_ch Under most circumstances (i.e., a user provides no destructor, and there are no other disqualifying conditions), the compiler provides a default destructor implicitly: class C{ int i; string s; }; void f(){ C* c1 = new C; delete c1; // C has a destructor } class C{ int m; private: ~C(){ } // not public destructor! }; class C_container{ C c; }; void f(){

```
C_container* c_cont = new C_container; delete c_cont; // Compile ERROR: C has no
accessible destructor }
Version > c++11
In C++11, a developer can override this behavior by
preventing the compiler from providing a default destructor.
class C{ int m; public: ~C() =
delete; // does NOT have implicit destructor }; void f{ C c1; } // Compile ERROR: C has no
destructor
Furthermore, a developer may also be explicit about wanting the compiler to provide
a default destructor.
class C{ int m; public: ~C() = default; // saying explicitly it does have
implicit/empty destructor }; void f(){ C c1; } // C has a destructor – c1 properly destroyed
Version > c++11
You can determine whether a type has a destructor (or is a primitive type)
using std::is_destructible from :
class C1{ }; class C2{ public: ~C2() = delete }; class C3 :
public C2{ }; using std::cout; using std::boolalpha; using std::endl; using std::is_destructible;
cout « boolalpha « is_destructible() « endl; // prints true
cout « boolalpha « is_destructible() « endl; // prints true
cout « boolalpha « is_destructible() « endl; // prints false
cout « boolalpha « is_destructible() « endl; // prints false
Section 40.3: Copy and swap
If you're writing a class that manages resources, you need to implement all the special member functions (see Rule of Three/Five/Zero). The most direct approach to writing the copy constructor and assignment operator would be:
person(const person &other) : name(new char[std::strlen(other.name) + 1]),
age(other.age) { std::strcpy(name, other.name); }
person& operator=(person const& rhs) { if
(this != &other) { delete [] name;
```

```
name = new char[std::strlen(other.name) + 1]; std::strcpy(name, other.name); age = other.age;
} return this; }
But this approach has some problems. It fails the strong exception guarantee - if new[] throws, we've already cleared the resources owned by this and cannot recover. We're duplicating a lot of the logic of copy construction in copy assignment. And we have to remember the self-assignment check, which usually just adds overhead to the copy operation, but is still critical. To satisfy the strong exception guarantee and avoid code duplication (double so with the subsequent move assignment operator), we can use the copy-and-swap idiom:
class person {
char name; int age; public: /* all the other functions ... / friend void swap(person& lhs, person& rhs) { using std::swap; // enable ADL swap(lhs.name, rhs.name); swap(lhs.age, rhs.age); }
person& operator=(person rhs) { swap(this, rhs); return this; } };
Why does this work? Consider what happens when we have person p1 = ...; person p2 = ...; p1 = p2; First, we copy-construct rhs from p2 (which we didn't have to duplicate here). If that operation throws, we don't do anything in operator= and p1 remains untouched. Next, we swap the members between this and rhs, and then rhs goes out of scope. When operator=, that implicitly cleans the original resources of this (via the destructor, which we didn't have to duplicate). Self-assignment works too - it's less efficient with copy-and-swap (involves an extra allocation and deallocation), but if that's the unlikely scenario, we don't slow down the typical use case to account for it.
Version C++11
The above formulation works as-is already for move assignment.
p1 = std::move(p2);
```

Here, we move-construct rhs from p2, and all the rest is just as valid. If a class is movable but not copyable, there is no need to delete the copy-assignment, since this assignment operator will simply be ill-formed due to the deleted copy constructor.

Section 40.4: Implicit Move and Copy Bear in mind that declaring a destructor inhibits the compiler from generating implicit move constructors and move assignment operators. If you declare a destructor, remember to also add appropriate definitions for the move operations. Furthermore, declaring move operations will suppress the generation of copy operations, so these should also be added (if the objects of this class are required to have copy semantics).

```
class Movable { public: virtual ~Movable() noexcept = default; // compiler won't generate these unless we tell it to // because we declared a destructor Movable(Movable&&) noexcept = default; Movable& operator=(Movable&&) noexcept = default; // declaring move operations will suppress generation // of copy operations unless we explicitly re-enable them Movable(const Movable&) = default; Movable& operator=(const Movable&) = default; };
```

Chapter 51

STANDARD TEMPLATE LIBRARY CORE

Chapter 52

THE STANDARD TEMPLATE LIBRARY (STL) CORE

Chapter 53

C++98/03 STANDARD LIBRARY

53.1 Standard Template Library (STL)

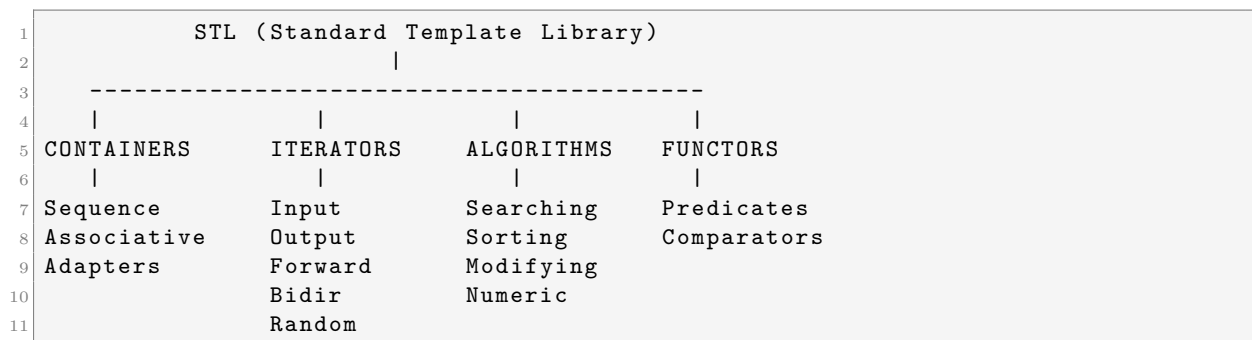
53.2 Introduction to STL

The Standard Template Library (STL) is a collection of template classes and functions that provide: - **Containers** - Data structures to hold objects (e.g., vectors, lists, maps). - **Iterators** - Objects to traverse containers (generalization of pointers). - **Algorithms** - Functions to manipulate data (e.g., sorting, searching). - **Function Objects (Functors)** - Objects that act like functions.

53.2.1 Key Advantages

- **Generic Programming:** Write code that works with any data type.
 - **Performance:** Heavily optimized implementations.
 - **Reusability:** Standardized components prevent reinventing the wheel.
 - **Type Safety:** Templates ensure type correctness at compile time.
-

53.3 STL Components Overview



53.3.1 Professional Notes: STL Core Depth

1. Iterators: The Bridge between Algorithms and Containers

Iterators provide a uniform interface for traversing data. * **Input/Output**: Single pass, read or write once. * **Forward**: Multiple passes, read/write, move forward only (e.g., `std::forward_iterator`). * **Bidirectional**: Move forward and backward (e.g., `std::list`, `std::map`). * **Random Access**: Jump to any element in constant time (e.g., `std::vector`, `std::deque`).

Godhood Tip: Prefer prefix increment (`++it`) over postfix increment (`it++`) for iterators. Postfix creates a temporary copy of the iterator, which can be costly for complex types.

2. Container Choice and Memory Layout

- `std::vector`: The gold standard. Contiguous memory ensures high **Cache Locality**. Always `reserve()` if you know the final size to avoid reallocations.
- `std::deque`: A “Double-Ended Queue.” Implemented as a sequence of fixed-size memory blocks. Offers $O(1)$ at both ends but is slower for random access than vector.
- `std::list`: Doubly linked list. $O(1)$ insertions anywhere if you have the iterator, but terrible cache locality and high memory overhead per element (2 pointers).

3. Associative Containers and Custom Comparators

`std::map` and `std::set` are typically implemented as **Red-Black Trees**. * **Complexity**: $O(\log N)$ for all major operations. * **Comparators**: You can provide a custom function or functor to define the ordering.

```

1 struct CaseInsensitiveCompare {
2     bool operator()(const std::string& a, const std::string& b) const {
3         return strcasecmp(a.c_str(), b.c_str()) < 0;
4     }
5 };
6 std::set<std::string, CaseInsensitiveCompare> my_set;
```

c

1. Binary Search Trees (`std::map`, `std::set`) Typically implemented as **Red-Black Trees**. *
 ##### 2. Hash Tables (`std::unordered_map`) Typically implemented as an array of buckets (linked lists). *
 ##### 3. Heap (`std::priority_queue`) Implemented using `std::vector`.

53.4 CONTAINERS - COMPLETE REFERENCE

53.4.1 Container Characteristics (C++98)

Container	Type	Insert	Delete	Search	Random Access	Memory	
<code>vector</code>	Sequence	$O(n)$	$O(n)$	$O(n)$	$O(1)$		Contiguous
<code>list</code>	Sequence	$O(1)$	$O(1)$	$O(n)$	-		Scattered
<code>deque</code>	Sequence	$O(n)$	$O(n)$	$O(n)$	$O(1)$		Chunks
<code>map</code>	Associative	$O(\log n)$	$O(\log n)$	$O(\log n)$	-		Tree
<code>set</code>	Associative	$O(\log n)$	$O(\log n)$	$O(\log n)$	-		Tree

Container	Type	Insert	Delete	Search	Random Access	Memory	
<code>multimap</code>	Associative	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	-	Tree
<code>multiset</code>	Associative	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	-	Tree
<code>priority_queue</code>	Adapter	$O(\log n)$	$O(\log n)$	-	-	-	Heap
<code>queue</code>	Adapter	$O(1)$	$O(1)$	-	-	-	-
<code>stack</code>	Adapter	$O(1)$	$O(1)$	-	-	-	-

53.5 1.1 VECTOR - Dynamic Array

53.5.1 What is Vector?

A dynamic array that grows automatically. Use this as your default container.

53.5.2 Declaration & Initialization

```

1 #include <vector>
2 using namespace std;
3
4 // Empty vector
5 vector<int> v1;
6
7 // Vector with initial size (10 elements initialized to 0)
8 vector<int> v2(10);
9
10 // Vector with initial size and value (5 elements, all 10)
11 vector<int> v3(5, 10);
12
13 // Copy constructor
14 vector<int> v4(v3);
15
16 // From array (using pointers)
17 int arr[] = {1, 2, 3, 4, 5};
18 vector<int> v5(arr, arr + 5);

```

53.5.3 Accessing Elements

```

1 vector<int> v;
2 v.push_back(10);
3 v.push_back(20);
4
5 // 1. Using operator[] (No bounds check)
6 cout << v[0] << "\n"; // 10
7
8 // 2. Using at() (With bounds check, throws std::out_of_range)
9 cout << v.at(1) << "\n"; // 20
10
11 // 3. Front and back
12 cout << v.front() << "\n"; // 10
13 cout << v.back() << "\n"; // 20

```

53.5.4 Modifying Elements

```
1 vector<int> v;
2
3 // Adding elements
4 v.push_back(10);
5 v.push_back(20);
6 v.push_back(30); // {10, 20, 30}
7
8 // Inserting elements (Iterators required)
9 v.insert(v.begin() + 1, 15); // {10, 15, 20, 30}
10
11 // Removing elements
12 v.pop_back(); // Removes 30
13 v.erase(v.begin() + 1); // Removes 15
14
15 // Clearing
16 v.clear(); // Empty
```

53.5.5 Size & Capacity

```
1 vector<int> v(10);
2
3 cout << v.size() << "\n"; // 10
4 cout << v.capacity() << "\n"; // >= 10
5 cout << v.empty() << "\n"; // 0 (false)
6
7 // Reserve space (Optimization to avoid reallocations)
8 v.reserve(100);
9
10 // Resize (Changes size, fills new elements with default or value)
11 v.resize(20); // Size becomes 20
12 v.resize(5); // Size becomes 5, extra elements destroyed
```

53.5.6 Iterating Through Vector

```
1 vector<int> v;
2 v.push_back(10); v.push_back(20); v.push_back(30);
3
4 // 1. Index loop
5 for (size_t i = 0; i < v.size(); ++i) {
6     cout << v[i] << " ";
7 }
8
9 // 2. Iterator loop (Recommended)
10 for (vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
11     cout << *it << " ";
12 }
13
14 // 3. Reverse Iterator
15 for (vector<int>::reverse_iterator rit = v.rbegin(); rit != v.rend(); ++rit) {
16     cout << *rit << " ";
17 }
```


53.6 1.2 DEQUE - Double Ended Queue

53.6.1 What is Deque?

Fast insertion/deletion at both ends. Unlike vector, storage is not guaranteed to be contiguous.

```
1 #include <deque>
2 using namespace std;
3
4 deque<int> dq;
5
6 dq.push_back(10);
7 dq.push_front(5); // {5, 10}
8
9 dq.pop_back();    // {5}
10 dq.pop_front();  // {}
11
12 // Supports random access
13 dq.push_back(100);
14 cout << dq[0] << "\n";
```

53.7 1.3 LIST - Doubly Linked List

53.7.1 What is List?

Efficient insertion/deletion anywhere ($O(1)$ if iterator is known). No random access.

```
1 #include <list>
2 using namespace std;
3
4 list<int> lst;
5 lst.push_back(10);
6 lst.push_front(5);
7
8 // Insert at position
9 list<int>::iterator it = lst.begin();
10 ++it; // Move to second position
11 lst.insert(it, 7); // {5, 7, 10}
12
13 // Remove
14 lst.remove(7); // Remove all elements with value 7
15
16 // Unique (Removes consecutive duplicates)
17 lst.push_back(10);
18 lst.unique(); // {5, 10} (second 10 removed)
19
20 // Sort
21 lst.sort(); // Internal sort (std::sort doesn't work on lists)
```

53.8 1.4 MAP - Sorted Key-Value Pairs

53.8.1 What is Map?

Associative container storing key-value pairs sorted by key. Implemented as a Balanced BST (usually Red-Black Tree).

```
1 #include <map>
2 #include <string>
3 using namespace std;
4
5 map<string, int> ages;
6
7 // Insertion
8 ages["Alice"] = 30;
9 ages["Bob"] = 25;
10 ages.insert(make_pair("Charlie", 28));
11
12 // Access / Search
13 cout << ages["Alice"] << "\n"; // 30
14
15 map<string, int>::iterator it = ages.find("Bob");
16 if (it != ages.end()) {
17     cout << "Bob is " << it->second << " years old.\n";
18 }
19
20 // Iteration (Sorted by key)
21 for (it = ages.begin(); it != ages.end(); ++it) {
22     cout << it->first << ": " << it->second << "\n";
23 }
```

53.9 1.5 SET - Sorted Unique Keys

53.9.1 What is Set?

Stores unique elements sorted by value.

```
1 #include <set>
2 using namespace std;
3
4 set<int> s;
5 s.insert(10);
6 s.insert(5);
7 s.insert(10); // Duplicate ignored
8
9 // {5, 10}
10
11 if (s.find(5) != s.end()) {
12     cout << "5 is present.\n";
13 }
14
15 // Range erase
16 s.erase(s.begin()); // Removes 5
```

53.10 1.6 MULTIMAP & MULTISSET

Allow duplicate keys (multimap) or duplicate values (multiset).

```
1 #include <map>
2 using namespace std;
3
4 multimap<string, int> scores;
```

```

5 scores.insert(make_pair("Alice", 90));
6 scores.insert(make_pair("Alice", 85)); // Allowed
7
8 // Finding all values for a key
9 pair<multimap<string, int>::iterator, multimap<string, int>::iterator> range;
10 range = scores.equal_range("Alice");
11
12 for (multimap<string, int>::iterator it = range.first; it != range.second; ++it
13     ) {
14     cout << it->second << " ";
15 }
16 // Output: 90 85

```

53.11 1.7 STACK (Adapter)

LIFO (Last In, First Out). Wraps deque (default), vector, or list.

```

1 #include <stack>
2 using namespace std;
3
4 stack<int> st;
5 st.push(10);
6 st.push(20);
7
8 cout << st.top() << "\n"; // 20
9 st.pop();                // Removes 20

```

53.12 1.8 QUEUE (Adapter)

FIFO (First In, First Out). Wraps deque (default) or list.

```

1 #include <queue>
2 using namespace std;
3
4 queue<int> q;
5 q.push(10);
6 q.push(20);
7
8 cout << q.front() << "\n"; // 10
9 q.pop();                  // Removes 10

```

53.13 1.9 PRIORITY_QUEUE (Adapter)

Sorted queue (Heap). Max element is always at top().

```

1 #include <queue>
2 #include <vector>
3 #include <functional> // for greater<int>
4 using namespace std;
5
6 // Max Heap (Default)

```

```
7 priority_queue<int> pq;
8 pq.push(10);
9 pq.push(30);
10 pq.push(20);
11
12 cout << pq.top() << "\n"; // 30
13
14 // Min Heap
15 priority_queue<int, vector<int>, greater<int> > min_pq;
16 min_pq.push(10);
17 min_pq.push(30);
18
19 cout << min_pq.top() << "\n"; // 10
```

53.14 ITERATORS

Iterators are the glue between Containers and Algorithms.

53.14.1 Categories

1. **Input:** Read-only, single pass (e.g., `istream_iterator`).
2. **Output:** Write-only, single pass (e.g., `ostream_iterator`).
3. **Forward:** Read/Write, multi-pass (e.g., `slist` - non-std).
4. **Bidirectional:** Forward + Backward (e.g., `list`, `map`, `set`).
5. **Random Access:** O(1) jump (e.g., `vector`, `deque`, arrays).

53.14.2 Operations

- `*it`: Dereference.
- `++it`: Advance.
- `--it`: Retreat (Bidirectional+).
- `it + n`: Jump (Random Access only).
- `it1 - it2`: Distance (Random Access only).

```
1 vector<int> v(5, 1);
2 vector<int>::iterator it = v.begin();
3 advance(it, 2); // Move 2 steps
4 cout << distance(v.begin(), it); // 2
```

53.15 ALGORITHMS

Found in `<algorithm>` and `<numeric>`. They work on iterator ranges `[first, last)`.

53.15.1 1. Non-Modifying

- `find(begin, end, val)`
 - `count(begin, end, val)`
 - `equal(b1, e1, b2)`
 - `search(b1, e1, b2, e2)`
-

53.15.2 Professional Notes: Algorithm Mastery

1. Range Integrity and End Iterators

All STL algorithms operate on half-open ranges `[first, last)`. * **The “Last” Iterator:** Points *beyond* the last element. Dereferencing it is **Undefined Behavior**. * **Empty Range:** If `first == last`, the range is empty. Algorithms correctly handle this (e.g., `find` returns `last`).

2. Sorting and Stability

- `std::sort`: Usually implemented as **Introsort** (Hybrid of Quicksort, Heapsort, and Insertion Sort). Average complexity $O(N \log N)$.
- `std::stable_sort`: Maintains the relative order of equal elements. Requires extra memory for its work buffer.
- `std::partial_sort`: Find the top K elements without sorting the whole range.

3. The Lambda Evolution (C++11)

Algorithms are most powerful when combined with lambdas:

```
1 // Find first even number
2 auto it = std::find_if(vec.begin(), vec.end(), [](int x){ return x % 2 == 0; })
    ;
```

4. Binary Search and Sorted Ranges

Functions like `binary_search`, `lower_bound`, and `upper_bound` require the range to be **sorted** or at least partitioned by the search value. Using them on unsorted ranges is UB.

53.15.3 2. Modifying

- `copy(b1, e1, b2)`
- `transform(b1, e1, out, op)`
- `replace(b1, e1, old, new)`
- `fill(b, e, val)`
- `swap(a, b)`
- `reverse(b, e)`
- `rotate(b, mid, e)`
- `random_shuffle(b, e)`

53.15.4 3. Sorting

- `sort(b, e)`: $O(N \log N)$.
- `stable\_sort(b, e)`: Preserves order of equal elements.
- `partial\_sort(b, mid, e)`: Top K elements.

53.15.5 4. Binary Search (On Sorted Ranges)

- `binary\_search(b, e, val)`: Returns bool.
- `lower\_bound(b, e, val)`: First element $\geq$ val.
- `upper\_bound(b, e, val)`: First element $>$ val.

53.15.6 5. Set Operations (On Sorted Ranges)

- `set\_union`, `set\_intersection`, `set\_difference`.

53.15.7 6. Numeric ()

- `accumulate(b, e, init)`: Sum.
- `inner\_product`: Dot product.
- `adjacent\_difference`.

53.15.8 Example: Sort and Find

```
1 #include <algorithm>
2 #include <vector>
3 #include <iostream>
4 using namespace std;
5
6 bool descending(int a, int b) {
7     return a > b;
8 }
9
10 int main() {
11     int arr[] = {5, 2, 9, 1, 5, 6};
12     vector<int> v(arr, arr + 6);
13
14     // Sort ascending
15     sort(v.begin(), v.end());
16
17     // Binary Search
18     if (binary_search(v.begin(), v.end(), 9)) {
19         cout << "Found 9\n";
20     }
21
22     // Sort descending with predicate
23     sort(v.begin(), v.end(), descending);
24
25     return 0;
26 }
```

53.16 STRINGS (std::string)

A specialization of `basic_string<char>`. Replaces C-style `char*` strings.

```
1 #include <string>
2 #include <iostream>
3 using namespace std;
4
5 string s = "Hello";
6 s += " World"; // Concatenation
7
8 // Substring
9 string sub = s.substr(0, 5); // "Hello"
10
11 // Find
12 size_t pos = s.find("World");
13 if (pos != string::npos) {
14     cout << "Found at " << pos << "\n";
15 }
16
17 // C-string compatibility
18 const char* cstr = s.c_str();
```

53.17 STREAMS (IOSTREAM)

53.17.1 File I/O ()

```
1 #include <fstream>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // Write
7     ofstream out("test.txt");
8     if (out) {
9         out << "Line 1" << endl;
10        out << 123 << endl;
11    }
12    out.close();
13
14    // Read
15    ifstream in("test.txt");
16    string line;
17    int num;
18
19    if (in >> line >> num) { // Reads "Line" then fails on "1" vs int? No.
20        // "Line" goes to line. "1" goes to num?
21        // Need careful parsing.
22    }
23    // Better: getline
24    getline(in, line);
25
26    return 0;
27 }
```

53.17.2 String Streams ()

Useful for parsing strings or formatting.

```
1 #include <sstream>
2 using namespace std;
3
4 // Int to String
5 int x = 42;
6 stringstream ss;
7 ss << x;
8 string s = ss.str();
9
10 // String to Int
11 string s2 = "100";
12 stringstream ss2(s2);
13 int y;
14 ss2 >> y;
```

53.18 FUNCTORS (Function Objects)

Classes that overload `operator()`. Used by algorithms.

```
1 struct AddX {
2     int x;
3     AddX(int val) : x(val) {}
4
5     int operator()(int y) const {
6         return x + y;
7     }
8 };
9
10 // Usage with transform
11 vector<int> v(5, 1); // {1, 1, 1, 1, 1}
12 transform(v.begin(), v.end(), v.begin(), AddX(10));
13 // v is now {11, 11, 11, 11, 11}
```

This covers the foundational C++98 Standard Library components necessary for mastery.

Chapter 54

ADVANCED STRINGS

54.1 5.1 String Manipulation

```
1 #include <iostream>
2 #include <string>
3 #include <cstring>
4 using namespace std;
5
6 int main() {
7     string s = "Hello World";
8
9     // Length and capacity
10    cout << "Length: " << s.length() << endl;
11    cout << "Capacity: " << s.capacity() << endl;
12
13    // Access characters
14    cout << "First char: " << s[0] << endl;
15    cout << "Last char: " << s[s.length() - 1] << endl;
16
17    // Finding substrings
18    size_t pos = s.find("World");
19    if (pos != string::npos) {
20        cout << "Found at position: " << pos << endl;
21    }
22
23    // Replace
24    s.replace(6, 5, "C++");
25    cout << s << endl; // Hello C++
26
27    // Insert
28    s.insert(5, " there");
29    cout << s << endl; // Hello there C++
30
31    // Erase
32    s.erase(5, 6);
33    cout << s << endl; // Hello C++
34
35    // Substring
36    cout << s.substr(0, 5) << endl; // Hello
37
38    // Reverse
39    reverse(s.begin(), s.end());
40    cout << s << endl; // ++C olleH
41
42    return 0;
43 }
```

54.2 5.2 String Conversion

```
1 #include <iostream>
2 #include <string>
3 #include <sstream>
4 #include <cstdlib>
5 using namespace std;
6
7 int main() {
8     // String to number (C style)
9     string s1 = "42";
10    int num = atoi(s1.c_str());
11    cout << num << endl;
12
13    string s2 = "3.14";
14    double dbl = atof(s2.c_str());
15    cout << dbl << endl;
16
17    // Number to string (using stringstream)
18    stringstream ss;
19    ss << 42 << " " << 3.14 << " " << true;
20    string result = ss.str();
21    cout << result << endl; // 42 3.14 1
22
23    // Reverse conversion
24    stringstream ss2("100 200 300");
25    int a, b, c;
26    ss2 >> a >> b >> c;
27    cout << a << " " << b << " " << c << endl; // 100 200 300
28
29    return 0;
30 }
```

54.3 5.3 String Tokenization

```
1 #include <iostream>
2 #include <string>
3 #include <cstring>
4 using namespace std;
5
6 int main() {
7     string line = "apple,banana,orange,grape";
8
9     // Using stringstream and getline
10    stringstream ss(line);
11    string token;
12
13    while (getline(ss, token, ',')) {
14        cout << token << endl;
15    }
16    // Output: apple, banana, orange, grape
17
18    // Using strtok (C style)
19    char str[] = "hello world how are you";
20    char* ptr = strtok(str, " ");
21
22    while (ptr != NULL) {
23        cout << ptr << endl;
24        ptr = strtok(NULL, " ");
25    }
```

```
26 // Output: hello, world, how, are, you
27
28 return 0;
29 }
```

Chapter 55

FILE I/O ADVANCED

55.1 14.1 Binary File I/O

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 int main() {
6     // Write binary data
7     ofstream outfile("data.bin", ios::binary);
8
9     int numbers[] = {10, 20, 30, 40, 50};
10    outfile.write((char*)numbers, sizeof(numbers));
11    outfile.close();
12
13    // Read binary data
14    ifstream infile("data.bin", ios::binary);
15
16    int buffer[5];
17    infile.read((char*)buffer, sizeof(buffer));
18
19    for (int i = 0; i < 5; i++) {
20        cout << buffer[i] << " ";
21    }
22    cout << endl;
23
24    infile.close();
25
26    return 0;
27 }
```

55.2 14.2 Stream Positioning

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 int main() {
6     // Write to file
7     ofstream outfile("test.txt");
8     outfile << "0123456789";
9     outfile.close();
10
11    // Read with positioning
12    ifstream infile("test.txt");
```

```
13
14 // Tell position
15 cout << "Current position: " << infile.tellg() << endl;
16
17 // Seek to position
18 infile.seekg(5);
19 char c;
20 infile.get(c);
21 cout << "Character at position 5: " << c << endl; // '5'
22
23 // Seek from end
24 infile.seekg(-3, ios::end);
25 infile.get(c);
26 cout << "Third from end: " << c << endl; // '7'
27
28 infile.close();
29
30 return 0;
31 }
```

Chapter 56

Professional Notes: Chapter 47: `std::string`

Section 47.1: Tokenize Section 47.2: Conversion to (const) `char*` Section 47.3: Using the `std::string_view` class Section 47.4: Conversion to `std::wstring` Section 47.5: Lexicographical comparison Section 47.6: Trimming characters at start/end Section 47.7: String replacement Section 47.8: Converting to `std::string` Section 47.9: Splitting Section 47.10: Accessing a character Section 47.11: Checking if a string is a prex of another Section 47.12: Looping through each character Section 47.13: Conversion to integers/loating point types Section 47.14: Concatenation

Section 47.15: Converting between character encodings Section 47.16: Finding character(s) in a string

Chapter 57

Professional Notes: Chapter 49: `std::vector`

Section 49.1: Accessing Elements Section 49.2: Initializing a `std::vector` Section 49.3: Deleting Elements Section 49.4: Iterating Over `std::vector` Section 49.5: `vector`: The Exception To So Many, So Many Rules Section 49.6: Inserting Elements Section 49.7: Using `std::vector` as a C array Section 49.8: Finding an Element in `std::vector` Section 49.9: Concatenating Vectors Section 49.10: Matrices Using Vectors Section 49.11: Using a Sorted Vector for Fast Element Lookup Section 49.12: Reducing the Capacity of a Vector Section 49.13: Vector size and capacity Section 49.14: Iterator/Pointer Invalidation Section 49.15: Find max and min Element and Respective Index in a Vector Section 49.16: Converting an array to `std::vector` Section 49.17: Functions Returning Large Vectors

Chapter 58

Professional Notes: Chapter 50: std::map

Section 50.1: Accessing elements Section 50.2: Inserting elements Section 50.3: Searching in std::map or in std::multimap Section 50.4: Initializing a std::map or std::multimap Section 50.5: Checking number of elements Section 50.6: Types of Maps Section 50.7: Deleting elements Section 50.8: Iterating over std::map or std::multimap Section 50.9: Creating std::map with user-defined types as key

Chapter 59

Professional Notes: Chapter 62: Standard Library Algorithms

Section 62.1: `std::next_permutation` Section 62.2: `std::for_each` Section 62.3: `std::accumulate`
Section 62.4: `std::nd` Section 62.5: `std::min_element` Section 62.6: `std::nd_if` Section 62.7:
Using `std::nth_element` To Find The Median (Or Other Quantiles) Section 62.8: `std::count`
Section 62.9: `std::count_if`

Chapter 60

Professional Notes: Chapter 67: Sorting

Section 67.1: Sorting and sequence containers Section 67.2: sorting with `std::map` (ascending and descending) Section 67.3: Sorting sequence containers by overloaded less operator Section 67.4: Sorting sequence containers using compare function Section 67.5: Sorting sequence containers using lambda expressions (C++11) Section 67.6: Sorting built-in arrays Section 67.7: Sorting sequence containers with specified ordering

Chapter 61

Professional Notes: Chapter 43: C++ Containers

C++ containers store a collection of elements. Containers include vectors, lists, maps, etc. Using Templates, C++ containers contain collections of primitives (e.g. ints) or custom classes (e.g. MyClass). Section 43.1: C++ Containers Flowchart Choosing which C++ Container to use can be tricky, so here's a simple owchart to help decide which Container is right for the job. This owchart was based on Mikael Persson's post. This little graphic in the owchart is from Megan Hopkins

Chapter 62

Professional Notes: Chapter 47: std::string

Strings are objects that represent sequences of characters. The standard string class provides a simple, safe and versatile alternative to using explicit arrays of chars when dealing with text and other sequences of characters. The C++ string class is part of the std namespace and was standardized in 1998. Section 47.1: Tokenize Listed from least expensive to most expensive at run-time: 1. std::strtok is the cheapest standard provided tokenization method, it also allows the delimiter to be modified between tokens, but it incurs 3 difficulties with modern C++: std::strtok cannot be used on multiple strings at the same time (though some implementations do extend to support this, such as: strtok_s) For the same reason std::strtok cannot be used on multiple threads simultaneously (this may however be implementation denied, for example: Visual Studio's implementation is thread safe) Calling std::strtok modifies the std::string it is operating on, so it cannot be used on const strings, const chars, or literal strings, to tokenize any of these with std::strtok or to operate on a std::string whose contents need to be preserved, the input would have to be copied, then the copy could be operated on Generally any of these options cost will be hidden in the allocation cost of the tokens, but if the cheapest algorithm is required and std::strtok's difficulties are not overcomable consider a hand-spun solution. // String to tokenize std::string str{ "The quick brown fox" }; // Vector to store tokens vector tokens; for (auto i = strtok(&str[0], " "); i != NULL; i = strtok(NULL, " ")) tokens.push_back(i); Live Example 2. The std::istream_iterator uses the stream's extraction operator iteratively. If the input std::string is white-space delimited this is able to expand on the std::strtok option by eliminating its difficulties, allowing inline tokenization thereby supporting the generation of a const vector, and by adding support for multiple delimiting white-space character: // String to tokenize const std::string str("The quick "); std::istream is(str); // Vector to store tokens const std::vector tokens = std::vector(std::istream_iterator(is), std::istream_iterator()); Live Example 3. The std::regex_token_iterator uses a std::regex to iteratively tokenize. It provides for a more flexible delimiter definition. For example, non-delimited commas and white-space:

Version C++11 // String to tokenize const std::string str{"The ,qu\,ick ,, fox" }; const std::regex re{ "\\s((?:[^\\"\\\\,]|\\\\".?)\\s(?:,|\$))" }; // Vector to store tokens const std::vector tokens{ std::sregex_token_iterator(str.begin(), str.end(), re, 1), std::sregex_token_iterator() }; Live Example See the regex_token_iterator Example for more details. Section 47.2: Conversion to (const) char In order to get const char* access to the data of a std::string you can use the string's c_str() member function. Keep in mind that the pointer is only valid as long as the std::string object is within scope and remains unchanged, that means that only const methods may be called on the object. Version C++17 The data() member function can be used to obtain a modifiable char, which can be used to manipulate the std::string object's data. Version C++11 A modifiable char can also be obtained by taking the address of the first character: &str[0]. Within C++11, this is guaranteed to yield a well-formed, null-terminated string. Note that &str[0] is

well-formed even if *s* is empty, whereas *Es.front()* is undened if *s* is empty. Version C++11

```
std::string str("This is a string."); const char* cstr = str.c_str(); // cstr points to: "This is
a string.\0" const char data = str.data(); // data points to: "This is a string.\0" std::string
str("This is a string."); // Copy the contents of str to untie lifetime from the std::string ob-
ject std::unique_ptr<char []> cstr = std::make_unique<char []>(str.size() + 1); // Alterna-
tive to the line above (no exception safety): // char* cstr_unsafe = new char[str.size() + 1];
std::copy(str.data(), str.data() + str.size(), cstr); cstr[str.size()] = '\0'; // A null-terminator
needs to be added // delete[] cstr_unsafe; std::cout << cstr.get();
```

Section 47.3: Using the `std::string_view` class Version C++17 C++17 introduces `std::string_view`, which is simply a non-owning range of const chars, implementable as either a pair of pointers or a pointer and a length. It is a superior parameter type for functions that requires non-modifiable string data. Before C++17, there were three options for this: `void foo(std::string const& s);` // pre-C++17, single argument, could incur

```
// allocation if caller's data was not in a string // (e.g. string literal or vector ) void
foo(const char* s, size_t len); // pre-C++17, two arguments, have to pass them // both
everywhere void foo(const char* s); // pre-C++17, single argument, but need to call //
strlen() template void foo(StringT const& s); // pre-C++17, caller can pass arbitrary char
data // provider, but now foo() has to live in a header All of these can be replaced with:
void foo(std::string_view s); // post-C++17, single argument, tighter coupling // zero copies
regardless of how caller is storing // the data Note that std::string_view cannot modify its
underlying data. string_view is useful when you want to avoid unnecessary copies. It oers
a useful subset of the functionality that std::string does, although some of the functions be-
have dierently: std::string str = "llllooonnnngggg ssssttrrrriinnnggg"; //A really long string
//Bad way - 'string::substr' returns a new string (expensive if the string is long) std::cout <<
str.substr(15, 10) << "; //Good way - No copies are created! std::string_view view = str; //
string_view::substr returns a new string_view std::cout << view.substr(15, 10) << ";
```

Section 47.4: Conversion to `std::wstring` In C++, sequences of characters are represented by specializing the `std::basic_string` class with a native character type. The two major collections dened by the standard library are `std::string` and `std::wstring`: `std::string` is built with elements of type `char` `std::wstring` is built with elements of type `wchar_t` To convert between the two types, use `wstring_convert`: `#include #include #include` `std::string input_str = "this is a -string-, which is a sequence based on the -char- type."; std::wstring input_wstr = L"this is a -wide-string, which is based on the -wchar_t- type."; // conversion std::wstring str_turned_to_wstr = std::wstring_convert<std::codecvt_utf8>().from_bytes(input_str);`

```
std::string wstr_turned_to_str = std::wstring_convert<std::codecvt_utf8>().to_bytes(input_wstr);
```

In order to improve usability and/or readability, you can dene functions to perform the conversion: `#include #include #include` `using convert_t = std::codecvt_utf8; std::wstring_convert<convert_t, wchar_t> strconverter; std::string to_string(std::wstring wstr) { return strconverter.to_bytes(wstr); } std::wstring to_wstring(std::string str) { return strconverter.from_bytes(str); }` Sample usage: `std::wstring a_wide_string = to_wstring("Hello World!");` That's certainly more readable than `std::wstring_convert<std::codecvt_utf8>().from_bytes("Hello World!");`. Please note that `char` and `wchar_t` do not imply encoding, and gives no indication of size in bytes. For instance, `wchar_t` is commonly implemented as a 2-bytes data type and typically contains UTF-16 encoded data under Windows (or UCS-2 in versions prior to Windows 2000) and as a 4-bytes data type encoded using UTF-32 under Linux. This is in contrast with the newer types `char16_t` and `char32_t`, which were introduced in C++11 and are guaranteed to be large enough to hold any UTF16 or UTF32 "character" (or more precisely, code point) respectively. Section 47.5: Lexicographical comparison Two `std::string`s can be compared lexicographically using the operators `==`, `!=`, `<`, `<=`, `>`, and `>=`: `std::string str1 = "Foo"; std::string str2 = "Bar"; assert(!(str1 < str2)); assert(str > str2); assert(!(str1 <= str2)); assert(str1 >= str2); assert(!(str1 == str2)); assert(str1 != str2);` All these functions use the underlying `std::string::compare()` method to

perform the comparison, and return for convenience boolean values. The operation of these functions may be interpreted as follows, regardless of the actual implementation: `operator==`: If `str1.length() == str2.length()` and each character pair matches, then returns true, otherwise returns

false. `operator!=`: If `str1.length() != str2.length()` or one character pair doesn't match, returns true, otherwise it returns false. `operator<` or `operator>`: Finds the first different character pair, compares them then returns the boolean result. `operator<=` or `operator>=`: Finds the first different character pair, compares them then returns the boolean result. Note: The term character pair means the corresponding characters in both strings of the same positions. For better understanding, if two example strings are `str1` and `str2`, and their lengths are `n` and `m` respectively, then character pairs of both strings means each `str1[i]` and `str2[i]` pairs where `i = 0, 1, 2, ..., max(n,m)`. If for any `i` where the corresponding character does not exist, that is, when `i` is greater than or equal to `n` or `m`, it would be considered as the lowest value. Here is an example of using `<`: `std::string str1 = "Barr"; std::string str2 = "Bar"; assert(str2 < str1);` The steps are as follows: 1. 2. 3. 4. Compare the first characters, 'B' == 'B' - move on. Compare the second characters, 'a' == 'a' - move on. Compare the third characters, 'r' == 'r' - move on. The `str2` range is now exhausted, while the `str1` range still has characters. Thus, `str2 < str1`. Section 47.6: Trimming characters at start/end This example requires the headers `<string>`, `<string_view>`, and `<string_view_traits>`. Version C++11 To trim a sequence or string means to remove all leading and trailing elements (or characters) matching a certain predicate. We first trim the trailing elements, because it doesn't involve moving any elements, and then trim the leading elements. Note that the generalizations below work for all types of `std::basic_string` (e.g. `std::string` and `std::wstring`), and accidentally also for sequence containers (e.g. `std::vector` and `std::list`). `template <typename Sequence, // any basic_string, vector, list etc. typename Pred> // a predicate on the element (character) type Sequence& trim(Sequence& seq, Pred pred) { return trim_end(trim_start(seq, pred), pred); }`

Trimming the trailing elements involves finding the last element not matching the predicate, and erasing from there on: `template <typename Sequence, typename Pred> Sequence& trim_end(Sequence& seq, Pred pred) { auto last = std::find_if_not(seq.rbegin(), seq.rend(), pred); seq.erase(last.base(), seq.end()); return seq; }` Trimming the leading elements involves finding the first element not matching the predicate and erasing up to there: `template <typename Sequence, typename Pred> Sequence& trim_start(Sequence& seq, Pred pred) { auto first = std::find_if_not(seq.begin(), seq.end(), pred); seq.erase(seq.begin(), first); return seq; }` To specialize the above for trimming whitespace in a `std::string` we can use the `std::isspace()` function as a predicate: `std::string& trim(std::string& str, const std::locale& loc = std::locale()) { return trim(str, [&loc] { return std::isspace(c, loc); }); } std::string& trim_start(std::string& str, const std::locale& loc = std::locale()) { return trim_start(str, [&loc] { return std::isspace(c, loc); }); } std::string& trim_end(std::string& str, const std::locale& loc = std::locale()) { return trim_end(str, [&loc] { return std::isspace(c, loc); }); }` Similarly, we can use the `std::iswspace()` function for `std::wstring` etc. If you wish to create a new sequence that is a trimmed copy, then you can use a separate function: `template <typename Sequence, typename Pred> Sequence trim_copy(Sequence seq, Pred pred) { // NOTE: passing seq by value trim(seq, pred); return seq; }` Section 47.7: String replacement Replace by position To replace a portion of a `std::string` you can use the method `replace` from `std::string`. `replace` has a lot of useful overloads:

```
//Define string std::string str = "Hello foo, bar and world!"; std::string alternate = "Hello
foobar"; //1) str.replace(6, 3, "bar"); //"Hello bar, bar and world!" //2) str.replace(str.begin()
+ 6, str.end(), "nobody!"); //"Hello nobody!" //3) str.replace(19, 5, alternate, 6, 6); //"Hello
foo, bar and foobar!" Version C++14 //4) str.replace(19, 5, alternate, 6); //"Hello foo,
bar and foobar!" //5) str.replace(str.begin(), str.begin() + 5, str.begin() + 6, str.begin() +
9); //"foo foo, bar and world!" //6) str.replace(0, 5, 3, 'z'); //"zzz foo, bar and world!"
//7) str.replace(str.begin() + 6, str.begin() + 9, 3, 'x'); //"Hello xxx, bar and world!" Ver-
sion C++11 //8) str.replace(str.begin(), str.begin() + 5, { 'x', 'y', 'z' }); //"xyz foo, bar
```

and world!” Replace occurrences of a string with another string Replace only the rst occurrence of replace with with in str: `std::string replaceString(std::string str, const std::string& replace, const std::string& with){ std::size_t pos = str.find(replace); if (pos != std::string::npos) str.replace(pos, replace.length(), with); return str; }` Replace all occurrence of replace with with in str: `std::string replaceStringAll(std::string str, const std::string& replace, const std::string& with) { if(!replace.empty()) { std::size_t pos = 0; while ((pos = str.find(replace, pos)) != std::string::npos) { str.replace(pos, replace.length(), with); pos += with.length(); } } return str; }` Section 47.8: Converting to `std::string` `std::ostringstream` can be used to convert any streamable type to a string representation, by inserting the object

into a `std::ostringstream` object (with the stream insertion operator `«`) and then converting the whole `std::ostringstream` to a `std::string`. For instance: `#include <iostream> int main() { int val = 4; std::ostringstream str; str « val; std::string converted = str.str(); return 0; }` Writing your own conversion function, the simple: `template<typename T> std::string toString(const T& x) { std::ostringstream ss; ss « x; return ss.str(); }` works but isn’t suitable for performance critical code. User-defined classes may implement the stream insertion operator if desired: `std::ostream operator«(const A& a, std::ostream& out) { // write a string representation of a to out return out; }` Version C++11 Aside from streams, since C++11 you can also use the `std::to_string` (and `std::to_wstring`) function which is overloaded for all fundamental types and returns the string representation of its parameter. `std::string s = to_string(0x12f3);` // after this the string `s` contains “4851” Section 47.9: Splitting Use `std::string::substr` to split a string. There are two variants of this member function. The rst takes a starting position from which the returned substring should begin. The starting position must be valid in the range `(0, str.length())`: `std::string str = “Hello foo, bar and world!”; std::string newstr = str.substr(11);` // “bar and world!” The second takes a starting position and a total length of the new substring. Regardless of the length, the substring will never go past the end of the source string: `std::string str = “Hello foo, bar and world!”;`

`std::string newstr = str.substr(15, 3);` // “and” Note that you can also call `substr` with no arguments, in this case an exact copy of the string is returned `std::string str = “Hello foo, bar and world!”; std::string newstr = str.substr();` // “Hello foo, bar and world!” Section 47.10: Accessing a character There are several ways to extract characters from a `std::string` and each is subtly different. `std::string str(“Hello world!”);` operator `[]` Returns a reference to the character at index `n`. `std::string::operator[]` is not bounds-checked and does not throw an exception. The caller is responsible for asserting that the index is within the range of the string: `char c = str[6];` // ‘w’ at(6) Returns a reference to the character at index `n`. `std::string::at` is bounds checked, and will throw `std::out_of_range` if the index is not within the range of the string: `char c = str.at(7);` // ‘o’ Version C++11 Note: Both of these examples will result in undefined behavior if the string is empty. `front()` Returns a reference to the rst character: `char c = str.front();` // ‘H’ `back()` Returns a reference to the last character: `char c = str.back();` // ‘!’ Section 47.11: Checking if a string is a prefix of another Version C++14 In C++14, this is easily done by `std::mismatch` which returns the rst mismatching pair from two ranges: `std::string prefix = “foo”;`

`std::string string = “foobar”;` `bool isPrefix = std::mismatch(prefix.begin(), prefix.end(), string.begin(), string.end()).first == prefix.end();` Note that a range-and-a-half version of `mismatch()` existed prior to C++14, but this is unsafe in the case that the second string is the shorter of the two. Version < C++14 We can still use the range-and-a-half version of `std::mismatch()`, but we need to rst check that the rst string is at most as big as the second: `bool isPrefix = prefix.size() <= string.size() && std::mismatch(prefix.begin(), prefix.end(), string.begin(), string.end()).first == prefix.end();` Version C++17 With `std::string_view`, we can write the direct comparison we want without having to worry about allocation overhead or making copies: `bool isPrefix(std::string_view prefix, std::string_view full) { return prefix == full.substr(0, prefix.size()); }` Section 47.12: Looping through each character Version C++11

std::string supports iterators, and so you can use a ranged based loop to iterate through each character: `std::string str = "Hello World!"; for (auto c : str) std::cout << c;` You can use a "traditional" for loop to loop through every character: `std::string str = "Hello World!"; for (std::size_t i = 0; i < str.length(); ++i) std::cout << str[i];` Section 47.13: Conversion to integers/loating point types A std::string containing a number can be converted into an integer type, or a oating point type, using conversion functions. Note that all of these functions stop parsing the input string as soon as they encounter a non-numeric character, so "123abc" will be converted into 123. The std::ato* family of functions converts C-style strings (character arrays) to integer or oating-point types: `std::string ten = "10"; double num1 = std::atof(ten.c_str());`

`int num2 = std::atoi(ten.c_str()); long num3 = std::atol(ten.c_str());` Version C++11 long `long num4 = std::atoll(ten.c_str());` However, use of these functions is discouraged because they return 0 if they fail to parse the string. This is bad because 0 could also be a valid result, if for example the input string was "0", so it is impossible to determine if the conversion actually failed. The newer std::sto* family of functions convert std::strings to integer or oating-point types, and throw exceptions if they could not parse their input. You should use these functions if possible: Version C++11 `std::string ten = "10"; int num1 = std::stoi(ten); long num2 = std::stol(ten); long long num3 = std::stoll(ten); float num4 = std::stof(ten); double num5 = std::stod(ten); long double num6 = std::stold(ten);` Furthermore, these functions also handle octal and hex strings unlike the std::ato* family. The second parameter is a pointer to the rst unconverted character in the input string (not illustrated here), and the third parameter is the base to use. 0 is automatic detection of octal (starting with 0) and hex (starting with 0x or 0X), and any other value is the base to use `std::string ten = "10"; std::string ten_octal = "12"; std::string ten_hex = "0xA"; int num1 = std::stoi(ten, 0, 2); // Returns 2 int num2 = std::stoi(ten_octal, 0, 8); // Returns 10 long num3 = std::stol(ten_hex, 0, 16); // Returns 10 long num4 = std::stol(ten_hex); // Returns 0 long num5 = std::stol(ten_hex, 0, 0); // Returns 10 as it detects the leading 0x` Section 47.14: Concatenation You can concatenate std::strings using the overloaded + and += operators. Using the + operator: `std::string hello = "Hello"; std::string world = "world"; std::string helloworld = hello + world; // "Helloworld"` Using the += operator: `std::string hello = "Hello"; std::string world = "world"; hello += world; // "Helloworld"` You can also append C strings, including string literals: `std::string hello = "Hello"; std::string world = "world";`

`const char *comma = ","; std::string newhelloworld = hello + comma + world + "!"; // "Hello, world!"` You can also use `push_back()` to push back individual chars: `std::string s = "a, b,"; s.push_back('c'); // "a, b, c"` There is also `append()`, which is pretty much like +=: `std::string app = "test and"; app.append("test"); // "test and test"` Section 47.15: Converting between character encodings Converting between encodings is easy with C++11 and most compilers are able to deal with it in a cross-platform manner through and headers. `#include #include #include using namespace std; int main() { // converts between wstring and utf8 string wstring_convert<codecvt_utf8_utf16> wchar_to_utf8; // converts between u16string and utf8 string wstring_convert<codecvt_utf8_utf16, char16_t> utf16_to_utf8; wstring wstr = L"foobar"; string utf8str = wchar_to_utf8.to_bytes(wstr); wstring wstr2 = wchar_to_utf8.from_bytes(utf8str); wcout << wstr << endl; cout << utf8str << endl; wcout << wstr2 << endl; u16string u16str = u"foobar"; string utf8str2 = utf16_to_utf8.to_bytes(u16str); u16string u16str2 = utf16_to_utf8.from_bytes(utf8str2); return 0; }` Mind that Visual Studio 2015 provides supports for these conversion but a bug in their library implementation requires to use a dierent template for `wstring_convert` when dealing with `char16_t`: `using utf16_char = unsigned short; wstring_convert<codecvt_utf8_utf16, utf16_char> conv_utf8_utf16; void strings::utf16_to_utf8(const std::u16string& utf16, std::string& utf8) { std::basic_string tmp; tmp.resize(utf16.length()); std::copy(utf16.begin(), utf16.end(), tmp.begin()); utf8 = conv_utf8_utf16.to_byte`

`} void strings::utf8_to_utf16(const std::string& utf8, std::u16string& utf16) { std::basic_string`

```
tmp = conv_utf8_utf16.from_bytes(utf8); utf16.clear(); utf16.resize(tmp.length()); std::copy(tmp.begin(),
tmp.end(), utf16.begin()); }
```

Section 47.16: Finding character(s) in a string To find a character or another string, you can use `std::string::find`. It returns the position of the first character of the first match. If no matches were found, the function returns `std::string::npos`. `std::string str = "Curiosity killed the cat"; auto it = str.find("cat"); if (it != std::string::npos) std::cout << "Found at position:" << it << '\n'; else std::cout << "Not found!\n";` Found at position: 21

The search opportunities are further expanded by the following functions: `find_first_of` // Find first occurrence of characters `find_first_not_of` // Find first absence of characters `find_last_of` // Find last occurrence of characters `find_last_not_of` // Find last absence of characters. These functions can allow you to search for characters from the end of the string, as well as find the negative case (ie. characters that are not in the string). Here is an example: `std::string str = "dog dog cat cat"; std::cout << "Found at position:" << str.find_last_of("gzx") << '\n';` Found at position: 6

Note: Be aware that the above functions do not search for substrings, but rather for characters contained in the search string. In this case, the last occurrence of 'g' was found at position 6 (the other characters weren't found).

Chapter 63

Professional Notes: Chapter 49: `std::vector`

A vector is a dynamic array with automatically handled storage. The elements in a vector can be accessed just as efficiently as those in an array with the advantage being that vectors can dynamically change in size. In terms of storage the vector data is (usually) placed in dynamically allocated memory thus requiring some minor overhead; conversely C-arrays and `std::array` use automatic storage relative to the declared location and thus do not have any overhead.

Section 49.1: Accessing Elements There are two primary ways of accessing elements in a `std::vector` index-based access iterators Index-based access: This can be done either with the subscript operator `[]`, or the member function `at()`. Both return a reference to the element at the respective position in the `std::vector` (unless it's a const vector), so that it can be read as well as modified (if the vector is not const). `[]` and `at()` differ in that `[]` is not guaranteed to perform any bounds checking, while `at()` does. Accessing elements where `index < 0` or `index >= size` is undefined behavior for `[]`, while `at()` throws a `std::out_of_range` exception. Note: The examples below use C++11-style initialization for clarity, but the operators can be used with all versions (unless marked C++11). Version C++11 `std::vector v{ 1, 2, 3 }; // using [] int a = v[1]; // a is 2 v[1] = 4; // v now contains { 1, 4, 3 } // using at() int b = v.at(2); // b is 3 v.at(2) = 5; // v now contains { 1, 4, 5 } int c = v.at(3); // throws std::out_of_range exception` Because the `at()` method performs bounds checking and can throw exceptions, it is slower than `[]`. This makes `[]` preferred code where the semantics of the operation guarantee that the index is in bounds. In any case, accesses to elements of vectors are done in constant time. That means accessing to the first element of the vector has the same cost (in time) of accessing the second element, the third element and so on. For example, consider this loop for `(std::size_t i = 0; i < v.size(); ++i) { v[i] = 1; }` Here we know that the index variable `i` is always in bounds, so it would be a waste of CPU cycles to check that `i` is in bounds for every call to operator `[]`.

The `front()` and `back()` member functions allow easy reference access to the first and last element of the vector, respectively. These positions are frequently used, and the special accessors can be more readable than their alternatives using `[]`: `std::vector v{ 4, 5, 6 }; // In pre-C++11 this is more verbose int a = v.front(); // a is 4, v.front() is equivalent to v[0] v.front() = 3; // v now contains {3, 5, 6} int b = v.back(); // b is 6, v.back() is equivalent to v[v.size() - 1] v.back() = 7; // v now contains {3, 5, 7} Note: It is undefined behavior to invoke front() or back() on an empty vector. You need to check that the container is not empty using the empty() member function (which checks if the container is empty) before calling front() or back(). A simple example of the use of 'empty()' to test for an empty vector follows: int main () { std::vector v; int sum (0); for (int i=1;i<=10;i++) v.push_back(i); //create and initialize the vector while (!v.empty()) //loop through until the vector tests to be empty { sum += v.back(); //keep a running total v.pop_back(); //pop out the element which removes it from the vector } std::cout << "total:" << sum << "; //output the total to the user return 0; } The example above creates`

a vector with a sequence of numbers from 1 to 10. Then it pops the elements of the vector out until the vector is empty (using ‘empty()’) to prevent undened behavior. Then the sum of the numbers in the vector is calculated and displayed to the user. Version C++11 The data() method returns a pointer to the raw memory used by the std::vector to internally store its elements. This is most often used when passing the vector data to legacy code that expects a C-style array. std::vector v{ 1, 2, 3, 4 }; // v contains {1, 2, 3, 4} int* p = v.data(); // p points to 1 p = 4; // v now contains {4, 2, 3, 4} ++p; // p points to 2 p = 3; // v now contains {4, 3, 3, 4} p[1] = 2; // v now contains {4, 3, 2, 4} (p + 2) = 1; // v now contains {4, 3, 2, 1} Version < C++11 Before C++11, the data() method can be simulated by calling front() and taking the address of the returned value: std::vector v(4); int ptr = &(v.front()); // or &v[0] This works because vectors are always guaranteed to store their elements in contiguous memory locations,

assuming the contents of the vector doesn’t override unary operator&. If it does, you’ll have to re-implement std::addressof in pre-C++11. It also assumes that the vector isn’t empty. Iterators: Iterators are explained in more detail in the example “Iterating over std::vector” and the article Iterators. In short, they act similarly to pointers to the elements of the vector: Version C++11 std::vector v{ 4, 5, 6 }; auto it = v.begin(); int i = it; // i is 4 ++it; i = it; // i is 5 it = 6; // v contains { 4, 6, 6 } auto e = v.end(); // e points to the element after the end of v. It can be // used to check whether an iterator reached the end of the vector: ++it; it == v.end(); // false, it points to the element at position 2 (with value 6) ++it; it == v.end(); // true It is consistent with the standard that a std::vector’s iterators actually be Ts, but most standard libraries do not do this. Not doing this both improves error messages, catches non-portable code, and can be used to instrument the iterators with debugging checks in non-release builds. Then, in release builds, the class wrapping around the underlying pointer is optimized away. You can persist a reference or a pointer to an element of a vector for indirect access. These references or pointers to elements in the vector remain stable and access remains dened unless you add/remove elements at or before the element in the vector, or you cause the vector capacity to change. This is the same as the rule for invalidating iterators. Version C++11 std::vector v{ 1, 2, 3 }; int* p = v.data() + 1; // p points to 2 v.insert(v.begin(), 0); // p is now invalid, accessing p is a undefined behavior. p = v.data() + 1; // p points to 1 v.reserve(10); // p is now invalid, accessing p is a undefined behavior. p = v.data() + 1; // p points to 1 v.erase(v.begin()); // p is now invalid, accessing *p is a undefined behavior. Section 49.2: Initializing a std::vector A std::vector can be initialized in several ways while declaring it: Version C++11 std::vector v{ 1, 2, 3 }; // v becomes {1, 2, 3} // Different from std::vector v(3, 6) std::vector v{ 3, 6 }; // v becomes {3, 6} // Different from std::vector v{3, 6} in C++11 std::vector v(3, 6); // v becomes {6, 6, 6} std::vector v(4); // v becomes {0, 0, 0, 0} A vector can be initialized from another container in several ways:

Copy construction (from another vector only), which copies data from v2: std::vector v(v2); std::vector v = v2; Version C++11 Move construction (from another vector only), which moves data from v2: std::vector v(std::move(v2)); std::vector v = std::move(v2); Iterator (range) copy-construction, which copies elements into v: // from another vector std::vector v(v2.begin(), v2.begin() + 3); // v becomes {v2[0], v2[1], v2[2]} // from an array int z[] = { 1, 2, 3, 4 }; std::vector v(z, z + 3); // v becomes {1, 2, 3} // from a list std::list list1{ 1, 2, 3 }; std::vector v(list1.begin(), list1.end()); // v becomes {1, 2, 3} Version C++11 Iterator move-construction, using std::make_move_iterator, which moves elements into v: // from another vector std::vector v(std::make_move_iterator(v2.begin()), std::make_move_iterator(v2.end())); // from a list std::list list1{ 1, 2, 3 }; std::vector v(std::make_move_iterator(list1.begin()), std::make_move_iterator(list1.end())); With the help of the assign() member function, a std::vector can be reinitialized after its construction: v.assign(4, 100); // v becomes {100, 100, 100, 100} v.assign(v2.begin(), v2.begin() + 3); // v becomes {v2[0], v2[1], v2[2]} int z[] = { 1, 2, 3, 4 }; v.assign(z + 1, z + 4); // v becomes {2, 3, 4} Section 49.3: Deleting Elements Deleting the

last element: `std::vector v{ 1, 2, 3 }; v.pop_back();` // v becomes {1, 2} Deleting all elements: `std::vector v{ 1, 2, 3 }; v.clear();` // v becomes an empty vector Deleting element by index: `std::vector v{ 1, 2, 3, 4, 5, 6 }; v.erase(v.begin() + 3);` // v becomes {1, 2, 3, 5, 6}

Note: For a vector deleting an element which is not the last element, all elements beyond the deleted element have to be copied or moved to fill the gap, see the note below and `std::list`. Deleting all elements in a range: `std::vector v{ 1, 2, 3, 4, 5, 6 }; v.erase(v.begin() + 1, v.begin() + 5);` // v becomes {1, 6} Note: The above methods do not change the capacity of the vector, only the size. See Vector Size and Capacity. The erase method, which removes a range of elements, is often used as a part of the erase-remove idiom. That is, first `std::remove` moves some elements to the end of the vector, and then erase chops them off. This is a relatively inefficient operation for any indices less than the last index of the vector because all elements after the erased segments must be relocated to new positions. For speed critical applications that require efficient removal of arbitrary elements in a container, see `std::list`. Deleting elements by value: `std::vector v{ 1, 1, 2, 2, 3, 3 }; int value_to_remove = 2; v.erase(std::remove(v.begin(), v.end(), value_to_remove), v.end());` // v becomes {1, 1, 3, 3} Deleting elements by condition: // `std::remove_if` needs a function, that takes a vector element as argument and returns true, // if the element shall be removed `bool __predicate(const int& element) { return (element > 3); }` // This will cause all elements to be deleted that are larger than 3 } `std::vector v{ 1, 2, 3, 4, 5, 6 }; v.erase(std::remove_if(v.begin(), v.end(), __predicate), v.end());` // v becomes {1, 2, 3} Deleting elements by lambda, without creating additional predicate function Version C++11 `std::vector v{ 1, 2, 3, 4, 5, 6 }; v.erase(std::remove_if(v.begin(), v.end(), {return element > 3;} ), v.end() );` Deleting elements by condition from a loop: `std::vector v{ 1, 2, 3, 4, 5, 6 }; std::vector::iterator it = v.begin(); while (it != v.end()) { if (condition) it = v.erase(it);` // after erasing, 'it' will be set to the next element in v else ++it; // manually set 'it' to the next element in v } While it is important not to increment it in case of a deletion, you should consider using a different method when then erasing repeatedly in a loop. Consider `remove_if` for a more efficient way. Deleting elements by condition from a reverse loop: `std::vector v{ -1, 0, 1, 2, 3, 4, 5, 6 }; typedef std::vector::reverse_iterator rev_itr; rev_itr it = v.rbegin(); while (it != v.rend()) { // after the loop only '0' will be in v int value = *it; if (value) { ++it;`

`// See explanation below for the following line. it = rev_itr(v.erase(it.base())); } else ++it;` } Note some points for the preceding loop: Given a reverse iterator it pointing to some element, the method `base` gives the regular (non-reverse) iterator pointing to the same element. `vector::erase(iterator)` erases the element pointed to by an iterator, and returns an iterator to the element that followed the given element. `reverse_iterator::reverse_iterator(iterator)` constructs a reverse iterator from an iterator. Put altogether, the line `it = rev_itr(v.erase(it.base()))` says: take the reverse iterator it, have v erase the element pointed by its regular iterator; take the resulting iterator, construct a reverse iterator from it, and assign it to the reverse iterator it. Deleting all elements using `v.clear()` does not free up memory (`capacity()` of the vector remains unchanged). To reclaim space, use: `std::vector().swap(v);` Version C++11 `shrink_to_fit()` frees up unused vector capacity: `v.shrink_to_fit();` The `shrink_to_fit` does not guarantee to really reclaim space, but most current implementations do. Section 49.4: Iterating Over `std::vector` You can iterate over a `std::vector` in several ways. For each of the following sections, v is dened as follows: `std::vector v;` Iterating in the Forward Direction Version C++11 // Range based for `for(const auto& value: v) { std::cout << value << " "; }` // Using a for loop with iterator `for(auto it = std::begin(v); it != std::end(v); ++it) { std::cout << *it << " "; }` // Using `for_each` algorithm, using a function or functor: `void fun(int const& value) { std::cout << value << " "; }` `std::for_each(std::begin(v), std::end(v), fun);`

// Using `for_each` algorithm. Using a lambda: `std::for_each(std::begin(v), std::end(v), { std::cout << *it << " "; });` Version < C++11 // Using a for loop with iterator `for(std::vector::iterator it = std::begin(v); it != std::end(v); ++it) { std::cout << *it << " "; }` // Using a for loop with index `for(std::size_t i = 0; i < v.size(); ++i) { std::cout << v[i] << " "; }` Iterating in the

Reverse Direction Version C++14 // There is no standard way to use range based for for this. // See below for alternatives. // Using `for_each` algorithm // Note: Using a lambda for clarity. But a function or functor will work `std::for_each(std::rbegin(v), std::rend(v), { std::cout << value << " "; });` // Using a for loop with iterator `for(auto rit = std::rbegin(v); rit != std::rend(v); ++rit) { std::cout << rit << " "; }` // Using a for loop with index `for(std::size_t i = 0; i < v.size(); ++i) { std::cout << v[v.size() - 1 - i] << " "; }` Though there is no built-in way to use the range based for to reverse iterate; it is relatively simple to x this. The range based for uses `begin()` and `end()` to get iterators and thus simulating this with a wrapper object can achieve the results we require. Version C++14 template struct `ReverseRange` { C c; // could be a reference or a copy, if the original was a temporary `ReverseRange(C&& cin): c(std::forward(cin)) {} ReverseRange(ReverseRange&&)=default; ReverseRange& operator=(ReverseRange&&)=delete; auto begin() const {return std::rbegin(c);} auto end() const {return std::rend(c);} }; // C is meant to be deduced, and perfect forwarded into template ReverseRange make_ReverseRange(C&& c) {return {std::forward(c)}; int main() { std::vector v { 1,2,3,4}; for(auto const& value: make_ReverseRange(v)) { std::cout << value << " "; } } Enforcing const elements`

Since C++11 the `cbegin()` and `cend()` methods allow you to obtain a constant iterator for a vector, even if the vector is non-const. A constant iterator allows you to read but not modify the contents of the vector which is useful to enforce const correctness: Version C++11 // forward iteration `for (auto pos = v.cbegin(); pos != v.cend(); ++pos) { // type of pos is vector::const_iterator // pos = 5; // Compile error - can't write via const iterator }` // reverse iteration `for (auto pos = v.crbegin(); pos != v.crend(); ++pos) { // type of pos is vector::const_iterator // pos = 5; // Compile error - can't write via const iterator }` // expects `Functor::operand()(T&)` `for_each(v.begin(), v.end(), Functor());` // expects `Functor::operand()(const T&)` `for_each(v.cbegin(), v.cend(), Functor())` Version C++17 `as_const` extends this to range iteration: `for (auto const& e : std::as_const(v)) { std::cout << e << " "; }` This is easy to implement in earlier versions of C++. Version C++14 template `constexpr std::add_const_t& as_const(T& t) noexcept { return t; }` A Note on Efficiency Since the class `std::vector` is basically a class that manages a dynamically allocated contiguous array, the same principle explained here applies to C++ vectors. Accessing the vector's content by index is much more efficient when following the row-major order principle. Of course, each access to the vector also puts its management content into the cache as well, but as has been debated many times (notably here and here), the difference in performance for iterating over a `std::vector` compared to a raw array is negligible. So the same principle of efficiency for raw arrays in C also applies for C++'s `std::vector`. Section 49.5: vector: The Exception To So Many, So Many Rules The standard (section 23.3.7) specifies that a specialization of vector is provided, which optimizes space by packing the bool values, so that each takes up only one bit. Since bits aren't addressable in C++, this means that several requirements on vector are not placed on vector: The data stored is not required to be contiguous, so a vector can't be passed to a C API which expects a bool array. `at()`, `operator []`, and dereferencing of iterators do not return a reference to bool. Rather they return a

proxy object that (imperfectly) simulates a reference to a bool by overloading its assignment operators. As an example, the following code may not be valid for `std::vector`, because dereferencing an iterator does not return a reference: Version C++11 `std::vector v = {true, false}; for (auto &b: v) { } // error` Similarly, functions expecting a `bool&` argument cannot be used with the result of `operator []` or `at()` applied to vector, or with the result of dereferencing its iterator: `void f(bool& b); f(v[0]); // error f(v.begin()); // error` The implementation of `std::vector` is dependent on both the compiler and architecture. The specialisation is implemented by packing *n* Booleans into the lowest addressable section of memory. Here, *n* is the size in bits of the lowest addressable memory. In most modern systems this is 1 byte or 8 bits. This means that one byte can store 8 Boolean values. This is an improvement over the traditional implemen-

tation where 1 Boolean value is stored in 1 byte of memory. Note: The below example shows possible bitwise values of individual bytes in a traditional vs. optimized vector. This will not always hold true in all architectures. It is, however, a good way of visualising the optimization. In the below examples a byte is represented as $[x, x, x, x, x, x, x, x]$. Traditional `std::vector` storing 8 Boolean values: Version C++11 `std::vector trad_vect = {true, false, false, false, true, false, true, true}`; Bitwise representation: $[0,0,0,0,0,0,0,1]$, $[0,0,0,0,0,0,0,0]$, $[0,0,0,0,0,0,0,0]$, $[0,0,0,0,0,0,0,0]$, $[0,0,0,0,0,0,0,1]$, $[0,0,0,0,0,0,0,0]$, $[0,0,0,0,0,0,0,1]$, $[0,0,0,0,0,0,0,1]$ Specialized `std::vector` storing 8 Boolean values: Version C++11 `std::vector optimized_vect = {true, false, false, false, true, false, true, true}`; Bitwise representation: $[1,0,0,0,1,0,1,1]$ Notice in the above example, that in the traditional version of `std::vector`, 8 Boolean values take up 8 bytes of memory, whereas in the optimized version of `std::vector`, they only use 1 byte of memory. This is a significant improvement on memory usage. If you need to pass a vector to a C-style API, you may need to copy the values to an array, or find a better way to use the API, if memory and performance are at risk. Section 49.6: Inserting Elements Appending an element at the end of a vector (by copying/moving): `struct Point { double x, y;`

`Point(double x, double y) : x(x), y(y) {}`; `std::vector v; Point p(10.0, 2.0); v.push_back(p);` // `p` is copied into the vector. Version C++11 Appending an element at the end of a vector by constructing the element in place: `std::vector v; v.emplace_back(10.0, 2.0);` // The arguments are passed to the constructor of the // given type (here `Point`). The object is constructed // in the vector, avoiding a copy. Note that `std::vector` does not have a `push_front()` member function due to performance reasons. Adding an element at the beginning causes all existing elements in the vector to be moved. If you want to frequently insert elements at the beginning of your container, then you might want to use `std::list` or `std::deque` instead. Inserting an element at any position of a vector: `std::vector v{ 1, 2, 3 }; v.insert(v.begin(), 9);` // `v` now contains $\{9, 1, 2, 3\}$ Version C++11 Inserting an element at any position of a vector by constructing the element in place: `std::vector v{ 1, 2, 3 }; v.emplace(v.begin()+1, 9);` // `v` now contains $\{1, 9, 2, 3\}$ Inserting another vector at any position of the vector: `std::vector v(4);` // contains: 0, 0, 0, 0 `std::vector v2(2, 10);` // contains: 10, 10 `v.insert(v.begin()+2, v2.begin(), v2.end());` // contains: 0, 0, 10, 10, 0, 0 Inserting an array at any position of a vector: `std::vector v(4);` // contains: 0, 0, 0, 0 `int a[] = {1, 2, 3};` // contains: 1, 2, 3 `v.insert(v.begin()+1, a, a+sizeof(a)/sizeof(a[0]));` // contains: 0, 1, 2, 3, 0, 0, 0 Use `reserve()` before inserting multiple elements if resulting vector size is known beforehand to avoid multiple reallocations (see vector size and capacity): `std::vector v; v.reserve(100); for(int i = 0; i < 100; ++i) v.emplace_back(i);` Be sure to not make the mistake of calling `resize()` in this case, or you will inadvertently create a vector with 200 elements where only the latter one hundred will have the value you intended. Section 49.7: Using `std::vector` as a C array There are several ways to use a `std::vector` as a C array (for example, for compatibility with C libraries). This is possible because the elements in a vector are stored contiguously.

Version C++11 `std::vector v{ 1, 2, 3 }; int p = v.data();` In contrast to solutions based on previous C++ standards (see below), the member function `.data()` may also be applied to empty vectors, because it doesn't cause undefined behavior in this case. Before C++11, you would take the address of the vector's first element to get an equivalent pointer, if the vector isn't empty, these both methods are interchangeable: `int* p = &v[0];` // combine subscript operator and 0 literal `int* p = &v.front();` // explicitly reference the first element Note: If the vector is empty, `v[0]` and `v.front()` are undefined and cannot be used. When storing the base address of the vector's data, note that many operations (such as `push_back`, `resize`, etc.) can change the data memory location of the vector, thus invalidating previous data pointers. For example: `std::vector v; int* p = v.data(); v.resize(42);` // internal memory location changed; value of `p` is now invalid Section 49.8: Finding an Element in `std::vector` The function `std::find`, defined in the header, can be used to find an element in a `std::vector`. `std::find` uses the operator `==` to compare elements for equality. It returns an iterator to the first element in the range that compares equal to the value.

If the element in question is not found, `std::find` returns `std::vector::end` (or `std::vector::cend` if the vector is const). Version < C++11 `static const int arr[] = {5, 4, 3, 2, 1}; std::vector v (arr, arr + sizeof(arr) / sizeof(arr[0])); std::vector::iterator it = std::find(v.begin(), v.end(), 4); std::vector::difference_type index = std::distance(v.begin(), it); // it points to the second element of the vector, index is 1 std::vector::iterator missing = std::find(v.begin(), v.end(), 10); std::vector::difference_type index_missing = std::distance(v.begin(), missing); // missing is v.end(), index_missing is 5 (ie. size of the vector)` Version C++11 `std::vector v { 5, 4, 3, 2, 1 }; auto it = std::find(v.begin(), v.end(), 4); auto index = std::distance(v.begin(), it); // it points to the second element of the vector, index is 1 auto missing = std::find(v.begin(), v.end(), 10); auto index_missing = std::distance(v.begin(), missing); // missing is v.end(), index_missing is 5 (ie. size of the vector)` If you need to perform many searches in a large vector, then you may want to consider sorting the vector first,

before using the `binary_search` algorithm. To find the first element in a vector that satisfies a condition, `std::find_if` can be used. In addition to the two parameters given to `std::find`, `std::find_if` accepts a third argument which is a function object or function pointer to a predicate function. The predicate should accept an element from the container as an argument and return a value convertible to `bool`, without modifying the container: Version < C++11 `bool isEven(int val) { return (val % 2 == 0); } struct moreThan { moreThan(int limit) : _limit(limit) {} bool operator()(int val) { return val > _limit; } int _limit; }; static const int arr[] = {1, 3, 7, 8}; std::vector v (arr, arr + sizeof(arr) / sizeof(arr[0])); std::vector::iterator it = std::find_if(v.begin(), v.end(), isEven); // it points to 2, the first even element std::vector::iterator missing = std::find_if(v.begin(), v.end(), moreThan(10)); // missing is v.end(), as no element is greater than 10` Version C++11 `// find the first value that is even std::vector v = {1, 3, 7, 8}; auto it = std::find_if(v.begin(), v.end(), {return val % 2 == 0;}); // it points to 2, the first even element auto missing = std::find_if(v.begin(), v.end(), {return val > 10;}); // missing is v.end(), as no element is greater than 10` Section 49.9: Concatenating Vectors One `std::vector` can be appended to another by using the member function `insert()`: `std::vector a = {0, 1, 2, 3, 4}; std::vector b = {5, 6, 7, 8, 9}; a.insert(a.end(), b.begin(), b.end());` However, this solution fails if you try to append a vector to itself, because the standard specifies that iterators given to `insert()` must not be from the same range as the receiver object's elements. Version c++11 Instead of using the vector's member functions, the functions `std::begin()` and `std::end()` can be used: `a.insert(std::end(a), std::begin(b), std::end(b));` This is a more general solution, for example, because `b` can also be an array. However, also this solution doesn't

allow you to append a vector to itself. If the order of the elements in the receiving vector doesn't matter, considering the number of elements in each vector can avoid unnecessary copy operations: `if (b.size() < a.size()) a.insert(a.end(), b.begin(), b.end()); else b.insert(b.end(), a.begin(), a.end());` Section 49.10: Matrices Using Vectors Vectors can be used as a 2D matrix by denoting them as a vector of vectors. A matrix with 3 rows and 4 columns with each cell initialised as 0 can be denoted as: `std::vector<std::vector> matrix(3, std::vector(4));` Version C++11 The syntax for initializing them using initializer lists or otherwise are similar to that of a normal vector. `std::vector<std::vector> matrix = { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} };` Values in such a vector can be accessed similar to a 2D array `int var = matrix[0][2];` Iterating over the entire matrix is similar to that of a normal vector but with an extra dimension. `for(int i = 0; i < 3; ++i) { for(int j = 0; j < 4; ++j) { std::cout << matrix[i][j] << std::endl; } }` Version C++11 `for(auto& row: matrix) { for(auto& col: row) { std::cout << col << std::endl; } }` A vector of vectors is a convenient way to represent a matrix but it's not the most efficient: individual vectors are scattered around memory and the data structure isn't cache friendly. Also, in a proper matrix, the length of every row must be the same (this isn't the case for a vector of vectors). The additional flexibility can be a source of errors.

Section 49.11: Using a Sorted Vector for Fast Element Lookup The header provides a number

of useful functions for working with sorted vectors. An important prerequisite for working with sorted vectors is that the stored values are comparable with `<`. An unsorted vector can be sorted by using the function `std::sort()`: `std::vector v; // add some code here to fill v with some elements std::sort(v.begin(), v.end());` Sorted vectors allow efficient element lookup using the function `std::lower_bound()`. Unlike `std::find()`, this performs an efficient binary search on the vector. The downside is that it only gives valid results for sorted input ranges: `// search the vector for the first element with value 42 std::vector::iterator it = std::lower_bound(v.begin(), v.end(), 42); if (it != v.end() && *it == 42) { // we found the element! }` Note: If the requested value is not part of the vector, `std::lower_bound()` will return an iterator to the first element that is greater than the requested value. This behavior allows us to insert a new element at its right place in an already sorted vector: `int const new_element = 33; v.insert(std::lower_bound(v.begin(), v.end(), new_element), new_element);` If you need to insert a lot of elements at once, it might be more efficient to call `push_back()` for all them first and then call `std::sort()` once all elements have been inserted. In this case, the increased cost of the sorting can pay off against the reduced cost of inserting new elements at the end of the vector and not in the middle. If your vector contains multiple elements of the same value, `std::lower_bound()` will try to return an iterator to the first element of the searched value. However, if you need to insert a new element after the last element of the searched value, you should use the function `std::upper_bound()` as this will cause less shifting around of elements: `v.insert(std::upper_bound(v.begin(), v.end(), new_element), new_element);` If you need both the upper bound and the lower bound iterators, you can use the function `std::equal_range()` to retrieve both of them efficiently with one call: `std::pair<std::vector::iterator, std::vector::iterator> rg = std::equal_range(v.begin(), v.end(), 42); std::vector::iterator lower_bound = rg.first; std::vector::iterator upper_bound = rg.second;` In order to test whether an element exists in a sorted vector (although not specific to vectors), you can use the function `std::binary_search()`: `bool exists = std::binary_search(v.begin(), v.end(), value_to_find);`

Section 49.12: Reducing the Capacity of a Vector A `std::vector` automatically increases its capacity upon insertion as needed, but it never reduces its capacity after element removal. `// Initialize a vector with 100 elements std::vector v(100); // The vector's capacity is always at least as large as its size auto const old_capacity = v.capacity(); // old_capacity >= 100 // Remove half of the elements v.erase(v.begin() + 50, v.end()); // Reduces the size from 100 to 50 (v.size() == 50), // but not the capacity (v.capacity() == old_capacity)` To reduce its capacity, we can copy the contents of a vector to a new temporary vector. The new vector will have the minimum capacity that is needed to store all elements of the original vector. If the size reduction of the original vector was significant, then the capacity reduction for the new vector is likely to be significant. We can then swap the original vector with the temporary one to retain its minimized capacity: `std::vector(v).swap(v);` Version C++11 In C++11 we can use the `shrink_to_fit()` member function for a similar effect: `v.shrink_to_fit();` Note: The `shrink_to_fit()` member function is a request and doesn't guarantee to reduce capacity. Section 49.13: Vector size and capacity Vector size is simply the number of elements in the vector: 1. Current vector size is queried by `size()` member function. Convenience `empty()` function returns true if size is 0: `vector v = { 1, 2, 3 }; // size is 3 const vector::size_type size = v.size(); cout << size << endl; // prints 3 cout << boolalpha << v.empty() << endl; // prints false` 2. Default constructed vector starts with a size of 0: `vector v; // size is 0 cout << v.size() << endl; // prints 0` 3. Adding N elements to vector increases size by N (e.g. by `push_back()`, `insert()` or `resize()` functions). 4. Removing N elements from vector decreases size by N (e.g. by `pop_back()`, `erase()` or `clear()` functions). 5. Vector has an implementation-specific upper limit on its size, but you are likely to run out of RAM before reaching it: `vector v;`

`const vector::size_type max_size = v.max_size(); cout << max_size << endl; // prints some large number v.resize( max_size ); // probably won't work v.push_back( 1 ); // definitely won't work` Common mistake: size is not necessarily (or even usually) int: `// !!!bad!!!evil!!! vector`

`v_bad( N, 1 );` // constructs large N size vector for(int i = 0; i < v_bad.size(); ++i) { // size is not supposed to be int! do_something(v_bad[i]); } Vector capacity differs from size. While size is simply how many elements the vector currently has, capacity is for how many elements it allocated/reserved memory for. That is useful, because too frequent (re)allocations of too large sizes can be expensive.

1. Current vector capacity is queried by `capacity()` member function. Capacity is always greater or equal to size: vector `v = { 1, 2, 3 }`; // size is 3, capacity is ≥ 3
`const vector::size_type capacity = v.capacity();` cout « capacity « endl; // prints number ≥ 3
2. You can manually reserve capacity by `reserve( N )` function (it changes vector capacity to N): // !!!bad!!!evil!!! vector `v_bad`; for(int i = 0; i < 10000; ++i) { `v_bad.push_back( i )`; // possibly lot of reallocations } // good vector `v_good`; `v_good.reserve( 10000 );` // good! only one allocation for(int i = 0; i < 10000; ++i) { `v_good.push_back( i )`; // no allocations needed anymore }
3. You can request for the excess capacity to be released by `shrink_to_fit()` (but the implementation doesn't have to obey you). This is useful to conserve used memory: vector `v = { 1, 2, 3, 4, 5 }`; // size is 5, assume capacity is 6 `v.shrink_to_fit();` // capacity is 5 (or possibly still 6) cout « boolalpha « `v.capacity() == v.size()` « endl; // prints likely true (but possibly false) Vector partly manages capacity automatically, when you add elements it may decide to grow. Implementers like to use 2 or 1.5 for the grow factor (golden ratio would be the ideal value - but is impractical due to being rational number). On the other hand vector usually do not automatically shrink. For example: vector `v`; // capacity is possibly (but not guaranteed) to be 0 `v.push_back( 1 );` // capacity is some starter value, likely 1 `v.clear();` // size is 0 but capacity is still same as before!

Chapter 64

Professional Notes: Chapter 50: std::map

To use any of `std::map` or `std::multimap` the header `<map>` should be included. `std::map` and `std::multimap` both keep their elements sorted according to the ascending order of keys. In case of `std::multimap`, no sorting occurs for the values of the same key. The basic difference between `std::map` and `std::multimap` is that the `std::map` one does not allow duplicate values for the same key where `std::multimap` does. Maps are implemented as binary search trees. So `search()`, `insert()`, `erase()` takes $(\log n)$ time in average. For constant time operation use `std::unordered_map`. `size()` and `empty()` functions have (1) time complexity, number of nodes is cached to avoid walking through tree each time these functions are called. Section 50.1: Accessing elements An `std::map` takes (key, value) pairs as input. Consider the following example of `std::map` initialization: `std::map< std::string, int > ranking { std::make_pair("stackoverflow", 2), std::make_pair("docs-beta", 1) };` In an `std::map`, elements can be inserted as follows: `ranking["stackoverflow"]=2; ranking["docs-beta"]=1;` In the above example, if the key `stackoverflow` is already present, its value will be updated to 2. If it isn't already present, a new entry will be created. In an `std::map`, elements can be accessed directly by giving the key as an index: `std::cout << ranking["stackoverflow"] << std::endl;` Note that using the operator `[]` on the map will actually insert a new value with the queried key into the map. This means that you cannot use it on a `const std::map`, even if the key is already stored in the map. To prevent this insertion, check if the element exists (for example by using `find()`) or use `at()` as described below. Version C++11 Elements of a `std::map` can be accessed with `at()`: `std::cout << ranking.at("stackoverflow") << std::endl;` Note that `at()` will throw an `std::out_of_range` exception if the container does not contain the requested element. In both containers `std::map` and `std::multimap`, elements can be accessed using iterators: Version C++11 // Example using `begin()`

```
std::multimap< int, std::string > mmp { std::make_pair(2, "stackoverflow"), std::make_pair(1, "docs-beta"), std::make_pair(2, "stackexchange") }; auto it = mmp.begin(); std::cout << it->first << " : " << it->second << std::endl; // Output: "1 : docs-beta" it++; std::cout << it->first << " : " << it->second << std::endl; // Output: "2 : stackoverflow" it++; std::cout << it->first << " : " << it->second << std::endl; // Output: "2 : stackexchange" // Example using rbegin()
std::map< int, std::string > mp { std::make_pair(2, "stackoverflow"), std::make_pair(1, "docs-beta"), std::make_pair(2, "stackexchange") }; auto it2 = mp.rbegin(); std::cout << it2->first << " : " << it2->second << std::endl; // Output: "2 : stackoverflow" it2++; std::cout << it2->first << " : " << it2->second << std::endl; // Output: "1 : docs-beta"
Section 50.2: Inserting elements
An element can be inserted into a std::map only if its key is not already present in the map. Given for example: std::map< std::string, size_t > fruits_count; A key-value pair is inserted into a std::map through the insert() member function. It requires a pair as an argument: fruits_count.insert({"grapes", 20}); fruits_count.insert(make_pair("orange", 30));
```

`fruits_count.insert(pair<std::string, size_t>("banana", 40)); fruits_count.insert(map<std::string, size_t>::value_type("cherry", 50));` The `insert()` function returns a pair consisting of an iterator and a bool value: If the insertion was successful, the iterator points to the newly inserted element, and the bool value is true. If there was already an element with the same key, the insertion fails. When that happens, the iterator points to the element causing the conflict, and the bool value is false. The following method can be used to combine insertion and searching operation: `auto success = fruits_count.insert({"grapes", 20}); if (!success.second) { // we already have 'grapes' in the map success.first->second += 20; // access the iterator to update the value }` For convenience, the `std::map` container provides the subscript operator to access elements in the map and to insert new ones if they don't exist: `fruits_count["apple"] = 10;` While simpler, it prevents the user from checking if the element already exists. If an element is missing, `std::map::operator[]` implicitly creates it, initializing it with the default constructor before overwriting it with the supplied value.

`insert()` can be used to add several elements at once using a braced list of pairs. This version of `insert()` returns void: `fruits_count.insert({{"apricot", 1}, {"jackfruit", 1}, {"lime", 1}, {"mango", 7}});` `insert()` can also be used to add elements by using iterators denoting the begin and end of value_type values: `std::map< std::string, size_t > fruit_list{ {"lemon", 0}, {"olive", 0}, {"plum", 0}}; fruits_count.insert(fruit_list.begin(), fruit_list.end());` Example: `std::map<std::string, size_t> fruits_count; std::string fruit; while(std::cin >> fruit){ // insert an element with 'fruit' as key and '1' as value // (if the key is already stored in fruits_count, insert does nothing) auto ret = fruits_count.insert({fruit, 1}); if(!ret.second){ // 'fruit' is already in the map ++ret.first->second; // increment the counter } }` Time complexity for an insertion operation is  $O(\log n)$  because `std::map` are implemented as trees. Version C++11 A pair can be constructed explicitly using `make_pair()` and `emplace()`: `std::map< std::string , int > runs; runs.emplace("Babe Ruth", 714); runs.insert(make_pair("Barry Bonds", 762));` If we know where the new element will be inserted, then we can use `emplace_hint()` to specify an iterator hint. If the new element can be inserted just before hint, then the insertion can be done in constant time. Otherwise it behaves in the same way as `emplace()`: `std::map< std::string , int > runs; auto it = runs.emplace("Barry Bonds", 762); // get iterator to the inserted element // the next element will be before "Barry Bonds", so it is inserted before 'it' runs.emplace_hint(it, "Babe Ruth", 714);` Section 50.3: Searching in `std::map` or in `std::multimap` There are several ways to search a key in `std::map` or in `std::multimap`. To get the iterator of the first occurrence of a key, the `find()` function can be used. It returns `end()` if the key does not exist. `std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} }`; `auto it = mmp.find(6); if(it!=mmp.end()) std::cout << it->first << "," << it->second << std::endl; //prints: 6, 5 else`

`std::cout << "Value does not exist!" << std::endl; it = mmp.find(66); if(it!=mmp.end()) std::cout << it->first << "," << it->second << std::endl; else std::cout << "Value does not exist!" << std::endl; // This line would be executed.` Another way to find whether an entry exists in `std::map` or in `std::multimap` is using the `count()` function, which counts how many values are associated with a given key. Since `std::map` associates only one value with each key, its `count()` function can only return 0 (if the key is not present) or 1 (if it is). For `std::multimap`, `count()` can return values greater than 1 since there can be several values associated with the same key. `std::map< int , int > mp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} }`; `if(mp.count(3) > 0) // 3 exists as a key in map std::cout << "The key exists!" << std::endl; // This line would be executed. else std::cout << "The key does not exist!" << std::endl;` If you only care whether some element exists, `find` is strictly better: it documents your intent and, for multimaps, it can stop once the first matching element has been found. In the case of `std::multimap`, there could be several elements having the same key. To get this range, the `equal_range()` function is used which returns `std::pair` having iterator lower bound (inclusive) and upper bound (exclusive) respectively. If the key does not exist, both iterators would point to `end()`. `auto eqr = mmp.equal_range(6); auto st = eqr.first, en = eqr.second; for(auto it = st; it != en;`

```

++it){ std::cout << it->first << ", " << it->second << std::endl; } // prints: 6, 5 // 6, 7
Section 50.4: Initializing a std::map or std::multimap
std::map and std::multimap both can be initialized by providing key-value pairs separated by comma. Key-value pairs could be provided by either {key, value} or can be explicitly created by std::make_pair(key, value). As std::map does not allow duplicate keys and comma operator performs right to left, the pair on right would be overwritten with the pair with same key on the left.
std::multimap < int, std::string > mmp {
std::make_pair(2, "stackoverflow"), std::make_pair(1, "docs-beta"), std::make_pair(2, "stackexchange") }; // 1 docs-beta // 2 stackoverflow // 2 stackexchange
std::map < int, std::string > mp { std::make_pair(2, "stackoverflow"), std::make_pair(1, "docs-beta"), std::make_pair(2, "stackexchange") }; // 1 docs-beta // 2 stackoverflow

```

Both could be initialized with iterator. // From std::map or std::multimap iterator

```

std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7} }; // {1, 2}, {3, 4}, {3, 4}, {6, 5}, {6, 8}, {6, 7}, {8, 9}
auto it = mmp.begin(); std::advance(it,3); //moved cursor on first {6, 5}
std::map< int, int > mp(it, mmp.end()); // {6, 5}, {8, 9}
//From std::pair array
std::pair< int, int > arr[10]; arr[0] = {1, 3}; arr[1] = {1, 5}; arr[2] = {2, 5}; arr[3] = {0, 1};
std::map< int, int > mp(arr,arr+4); // {0, 1}, {1, 3}, {2, 5}
//From std::vector of std::pair
std::vector< std::pair<int, int> > v{ {1, 5}, {5, 1}, {3, 6}, {3, 2} };
std::multimap< int, int > mp(v.begin(), v.end()); // {1, 5}, {3, 6}, {3, 2}, {5, 1}
Section 50.5: Checking number of elements
The container std::map has a member function empty(), which returns true or false, depending on whether the map is empty or not. The member function size() returns the number of element stored in a std::map container:
std::map<std::string , int> rank {{"facebook.com", 1} , {"google.com", 2}, {"youtube.com", 3}};
if(!rank.empty()){ std::cout << "Number of elements in the rank map:" << rank.size() << std::endl; }
else{ std::cout << "The rank map is empty" << std::endl; }
Section 50.6: Types of Maps
Regular Map
A map is an associative container, containing key-value pairs. #include #include std::map<std::string, size_t> fruits_count;
In the above example, std::string is the key type, and size_t is a value. The key acts as an index in the map. Each key must be unique, and must be ordered. If you need mutliple elements with the same key, consider using multimap (explained below)
If your value type does not specify any ordering, or you want to override the default ordering, you may provide one: #include #include

```

```

#include struct StrLess { bool operator()(const std::string& a, const std::string& b) { return strncmp(a.c_str(), b.c_str(), 8)<0; //compare only up to 8 first characters } }
std::map<std::string, size_t, StrLess> fruits_count2;
If StrLess comparator returns false for two keys, they are considered the same even if their actual contents dier.
Multi-Map
Multimap allows multiple key-value pairs with the same key to be stored in the map. Otherwise, its interface and creation is very similar to the regular map. #include #include std::multimap<std::string, size_t> fruits_count;
std::multimap<std::string, size_t, StrLess> fruits_count2;
Hash-Map (Unordered Map)
A hash map stores key-value pairs similar to a regular map. It does not order the elements with respect to the key though. Instead, a hash value for the key is used to quickly access the needed key-value pairs. #include #include std::unordered_map<std::string, size_t> fruits_count;
Unordered maps are usually faster, but the elements are not stored in any predictable order. For example, iterating over all elements in an unordered_map gives the elements in a seemingly random order.
Section 50.7: Deleting elements
Removing all elements:
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.clear(); //empty multimap
Removing element from somewhere with the help of iterator:
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} }; // {1, 2}, {3, 4}, {3, 4}, {6, 5}, {6, 7}, {8, 9}
auto it = mmp.begin(); std::advance(it,3); // moved cursor on first {6, 5}
mmp.erase(it); // {1, 2}, {3, 4}, {3, 4}, {6, 7}, {8, 9}
Removing all elements in a range:
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} }; // {1, 2}, {3, 4}, {3, 4}, {6, 5}, {6, 7}, {8, 9}
auto it = mmp.begin(); auto it2 = it;

```

```

it++; //moved first cursor on first {3, 4}
std::advance(it2,3); //moved second cursor on

```

```

first {6, 5} mmp.erase(it,it2); // {1, 2}, {6, 5}, {6, 7}, {8, 9} Removing all elements having a
provided value as key: std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6,
7} }; // {1, 2}, {3, 4}, {3, 4}, {6, 5}, {6, 7}, {8, 9} mmp.erase(6); // {1, 2}, {3, 4}, {3, 4}, {8,
9} Removing elements that satisfy a predicate pred: std::map<int,int> m; auto it = m.begin();
while (it != m.end()) { if (pred(*it)) it = m.erase(it); else ++it; }

```

Section 50.8: Iterating over `std::map` or `std::multimap` `std::map` or `std::multimap` could be traversed by the following ways:

```

std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} }; //Range based
loop - since C++11 for(const auto &x: mmp) std::cout<< x.first <<":"<< x.second << std::endl;
//Forward iterator for loop: it would loop through first element to last element //it will be a
std::map< int, int >::iterator for (auto it = mmp.begin(); it != mmp.end(); ++it) std::cout<<
it->first <<":"<< it->second << std::endl; //Do something with iterator //Backward iterator for
loop: it would loop through last element to first element //it will be a std::map< int, int
>::reverse_iterator for (auto it = mmp.rbegin(); it != mmp.rend(); ++it) std::cout<< it->first
<< " " << it->second << std::endl; //Do something with iterator

```

While iterating over a `std::map` or a `std::multimap`, the use of `auto` is preferred to avoid useless implicit conversions (see this SO answer for more details). Section 50.9: Creating `std::map` with user-dened types as key In order to be able to use a class as the key in a map, all that is required of the key is that it be copiable and assignable. The ordering within the map is dened by the third argument to the template (and the argument to the constructor, if used). This defaults to `std::less`, which defaults to the `<` operator, but there's no requirement to use the defaults. Just write a comparison operator (preferably as a functional object):

```

struct CmpMyType { bool operator()( MyType const& lhs,
MyType const& rhs ) const {

```

```

// ... } };

```

In C++, the "compare" predicate must be a strict weak ordering. In particular, `compare(X,X)` must return false for any X. i.e. if `CmpMyType()(a, b)` returns true, then `CmpMyType()(b, a)` must return false, and if both return false, the elements are considered equal (members of the same equivalence class). Strict Weak Ordering This is a mathematical term to dene a relationship between two objects. Its denition is: Two objects x and y are equivalent if both `f(x, y)` and `f(y, x)` are false. Note that an object is always (by the irreflexivity invariant) equivalent to itself. In terms of C++ this means if you have two objects of a given type, you should return the following values when compared with the operator `<`.

X a;	X b;	Condition:
Test:	Result	a is equivalent to b:
a < b	false	a is equivalent to b
b < a	false	a is less than b
a < b	true	a is less than b
b < a	false	b is less than a
a < b	false	b is less than a
b < a	true	

How you dene equivalent/less is totally dependent on the type of your object.

Chapter 65

Professional Notes: Chapter 62: Standard Library Algorithms

Section 62.1: `std::next_permutation` `template< class Iterator > bool next_permutation( Iterator first, Iterator last );` `template< class Iterator, class Compare > bool next_permutation( Iterator first, Iterator last, Compare cmpFun );` Effects: Sift the data sequence of the range `[rst, last)` into the next lexicographically higher permutation. If `cmpFun` is provided, the permutation rule is customized. Parameters: `first` - the beginning of the range to be permuted, inclusive `last` - the end of the range to be permuted, exclusive Return Value: Returns true if such permutation exists. Otherwise the range is swapped to the lexicographically smallest permutation and return false. Complexity: $O(n)$, n is the distance from `first` to `last`. Example: `std::vector< int > v { 1, 2, 3 }; do { for( int i = 0; i < v.size(); i += 1 ) { std::cout << v[i]; } std::cout << std::endl; }while( std::next_permutation( v.begin(), v.end() ) );` print all the permutation cases of 1,2,3 in lexicographically-increasing order. output: Section 62.2: `std::for_each` `template<class InputIterator, class Function> Function for_each(InputIterator first, InputIterator last, Function f);` Effects: Applies `f` to the result of dereferencing every iterator in the range `[first, last)` starting from `first` and

proceeding to `last - 1`. Parameters: `first, last` - the range to apply `f` to. `f` - callable object which is applied to the result of dereferencing every iterator in the range `[first, last)`. Return value: `f` (until C++11) and `std::move(f)` (since C++11). Complexity: Applies `f` exactly `last - first` times. Example: Version c++11 `std::vector v { 1, 2, 4, 8, 16 }; std::for_each(v.begin(), v.end(), { std::cout << elem << " "; });` Applies the given function for every element of the vector `v` printing this element to stdout. Section 62.3: `std::accumulate` Defined in header `template<class InputIterator, class T> T accumulate(InputIterator first, InputIterator last, T init);` // (1) `template<class InputIterator, class T, class BinaryOperation> T accumulate(InputIterator first, InputIterator last, T init, BinaryOperation f);` // (2) Effects: `std::accumulate` performs fold operation using `f` function on range `[first, last)` starting with `init` as accumulator value. Effectively it's equivalent of: `T acc = init; for (auto it = first; first != last; ++it) acc = f(acc, it); return acc;` In version (1) `operator+` is used in place of `f`, so `accumulate` over container is equivalent of sum of container elements. Parameters: `first, last` - the range to apply `f` to. `init` - initial value of accumulator. `f` - binary folding function. Return value:

Accumulated value of `f` applications. Complexity: $O(nk)$, where n is the distance from `first` to `last`, $O(k)$ is complexity of `f` function. Example: Simple sum example: `std::vector v { 2, 3, 4 }; auto sum = std::accumulate(v.begin(), v.end(), 1); std::cout << sum << std::endl;` Output: Convert digits to number: Version < c++11 `class Converter { public: int operator()(int a, int d) const { return a * 10 + d; } }; and later const int ds[3] = {1, 2, 3}; int n = std::accumulate(ds, ds + 3, 0, Converter()); std::cout << n << std::endl;` Version c++11 `const std::vector ds = {1, 2, 3}; int n = std::accumulate(ds.begin(), ds.end(), 0, { return a * 10 + d; }); std::cout << n << std::endl;` Output: Section 62.4: `std::inplace_merge` `template <class InputIterator, class T> InputIterator`

find (InputIterator first, InputIterator last, const T& val); Effects Finds the first occurrence of val within the range [first, last) Parameters first => iterator pointing to the beginning of the range last => iterator pointing to the end of the range val => The value to find within the range

Return An iterator that points to the first element within the range that is equal(==) to val, the iterator points to last if val is not found. Example #include #include #include using namespace std; int main(int argc, const char * argv[]) { //create a vector vector<int> intVec {4, 6, 8, 9, 10, 30, 55, 100, 45, 2, 4, 7, 9, 43, 48}; //define iterators vector::iterator itr_9; vector::iterator itr_43; vector::iterator itr_50; //calling find itr_9 = find(intVec.begin(), intVec.end(), 9); //occurs twice itr_43 = find(intVec.begin(), intVec.end(), 43); //occurs once //a value not in the vector itr_50 = find(intVec.begin(), intVec.end(), 50); //does not occur cout << "first occurrence of: " << itr_9 << endl; cout << "only occurrence of: " << itr_43 << endl; // let's prove that itr_9 is pointing to the first occurrence of 9 by looking at the element after 9, which should be 10 not 43 / cout << "element after first 9: " << (*itr_9 + 1) << endl; // to avoid dereferencing intVec.end(), let's look at the element right before the end / cout << "last element: " << *(itr_50 - 1) << endl; return 0; } Output first occurrence of: 9 only occurrence of: 43 element after first 9: 10 last element: 48

Section 62.5: std::min_element template ForwardIterator min_element (ForwardIterator first, ForwardIterator last); template <class ForwardIterator, class Compare> ForwardIterator min_element (ForwardIterator first, ForwardIterator last, Compare comp); Effects Finds the minimum element in a range Parameters first - iterator pointing to the beginning of the range last - iterator pointing to the end of the range comp - a function pointer or function object that takes two arguments and returns true or false indicating whether argument is less than argument 2. This function should not modify inputs Return Iterator to the minimum element in the range Complexity Linear in one less than the number of elements compared. Example #include #include #include //to use make_pair using namespace std; //function compare two pairs bool pairLessThanFunction(const pair<string, int> &p1, const pair<string, int> &p2) { return p1.second < p2.second; } int main(int argc, const char * argv[]) { vector<int> intVec {30, 200, 167, 56, 75, 94, 10, 73, 52, 6, 39, 43}; vector<pair<string, int>> pairVector = {make_pair("y", 25), make_pair("b", 2), make_pair("z", 26), make_pair("e", 5)}; // default using < operator auto minInt = min_element(intVec.begin(), intVec.end()); //Using pairLessThanFunction auto minPairFunction = min_element(pairVector.begin(), pairVector.end(), pairLessThanFunction); //print minimum of intVector cout << "min int from default: " << minInt << endl;

//print minimum of pairVector cout << "min pair from PairLessThanFunction: " << (minPairFunction).second << endl; return 0; } Output min int from default: 6 min pair from PairLessThanFunction: 2 Section 62.6: std::find_if template <class InputIterator, class UnaryPredicate> InputIterator find_if (InputIterator first, InputIterator last, UnaryPredicate pred); Effects Finds the first element in a range for which the predicate function pred returns true. Parameters first => iterator pointing to the beginning of the range last => iterator pointing to the end of the range pred => predicate function (returns true or false) Return An iterator that points to the first element within the range the predicate function pred returns true for. The iterator points to last if val is not found Example #include #include #include using namespace std; // define some functions to use as predicates //Returns true if x is multiple of 10 bool multOf10(int x) { return x % 10 == 0; } //returns true if item greater than passed in parameter class Greater { int _than; public: Greater(int th): _than(th) {} bool operator()(int data) const

{ return data > _than; } }; int main() { vector<int> myvec {2, 5, 6, 10, 56, 7, 48, 89, 850, 7, 456}; //with a lambda function vector::iterator gt10 = find_if(myvec.begin(), myvec.end(), {return x>10;}); // >= C++11 //with a function pointer vector::iterator pow10 = find_if(myvec.begin(), myvec.end(), multOf10); //with functor vector::iterator gt5 = find_if(myvec.begin(), myvec.end(), Greater(5)); //not Found vector::iterator nf = find_if(myvec.begin(), myvec.end(), Greater(1000)); // nf points to myvec.end() //check if pointer points to myvec.end() if(nf != myvec.end()) { cout << "nf points to: " << nf << endl; } else { cout << "item not found" << endl; } cout << "First

item > 10:” « gt10 « endl; cout « “First Item n * 10:” « pow10 « endl; cout « “First Item > 5:” « gt5 « endl; return 0; } Output item not found First item > 10: 56 First Item n * 10: 10 First Item > 5: 6 Section 62.7: Using std::nth_element To Find The Median (Or Other Quantiles) The std::nth_element algorithm takes three iterators: an iterator to the beginning, nth position, and end. Once the function returns, the nth element (by order) will be the nth smallest element. (The function has more elaborate overloads, e.g., some taking comparison functors; see the above link for all the variations.) Note This function is very efficient - it has linear complexity.

For the sake of this example, let’s define the median of a sequence of length n as the element that would be in position n / 2. For example, the median of a sequence of length 5 is the 3rd smallest element, and so is the median of a sequence of length 6. To use this function to find the median, we can use the following. Say we start with std::vector v{5, 1, 2, 3, 4};
std::vector::iterator b = v.begin(); std::vector::iterator e = v.end(); std::vector::iterator med = b;
std::advance(med, v.size() / 2); // This makes the 2nd position hold the median. std::nth_element(b,
med, e);

// The median is now at v[2]. To find the pth quantile, we would change some of the lines above: const std::size_t pos = p * std::distance(b, e); std::advance(nth, pos); and look for the quantile at position pos. Section 62.8: std::count template <class InputIterator, class T> typename iterator_traits::difference_type count (InputIterator first, InputIterator last, const T& val); Effects Counts the number of elements that are equal to val Parameters first => iterator pointing to the beginning of the range last => iterator pointing to the end of the range val => The occurrence of this value in the range will be counted Return The number of elements in the range that are equal(==) to val. Example #include #include #include using namespace std; int main(int argc, const char * argv[]) {

//create vector vector<int> vec{4,6,8,9,10,30,55,100,45,2,4,7,9,43,48}; //count occurrences of 9, 55, and 101 size_t count_9 = count(vec.begin(), vec.end(), 9); //occurs twice size_t count_55 = count(vec.begin(), vec.end(), 55); //occurs once size_t count_101 = count(vec.begin(), vec.end(), 101); //occurs once //print result cout « “There are” « count_9 « " 9s"« endl; cout « "There is " « count_55 « " 55"« endl; cout « "There is " « count_101 « " 101"« endl; //find the first element == 4 in the vector vector::iterator itr_4 = find(vec.begin(), vec.end(), 4); //count its occurrences in the vector starting from the first one size_t count_4 = count(itr_4, vec.end(), itr_4); // should be 2 cout « “There are” « count_4 « " " « itr_4 « endl; return 0; } Output There are 2 9s There is 1 55 There is 0 101 There are 2 4 Section 62.9: std::count_if template <class InputIterator, class UnaryPredicate> typename iterator_traits::difference_type count_if (InputIterator first, InputIterator last, UnaryPredicate red); Effects Counts the number of elements in a range for which a specified predicate function is true Parameters first => iterator pointing to the beginning of the range last => iterator pointing to the end of the range red => predicate function (returns true or false) Return The number of elements within the specified range for which the predicate function returned true. Example #include #include #include using namespace std;

/ Define a few functions to use as predicates //return true if number is odd bool isOdd(int i){ return i%2 == 1; } //functor that returns true if number is greater than the value of the constructor parameter provided class Greater { int _than; public: Greater(int th): _than(th){} bool operator()(int i){ return i > _than; } }; int main(int argc, const char * argv[]) { //create a vector vector<int> myvec = {1,5,8,0,7,6,4,5,2,1,5,0,6,9,7}; //using a lambda function to count even numbers size_t evenCount = count_if(myvec.begin(), myvec.end(), {return i % 2 == 0;}); // >= C++11 //using function pointer to count odd number in the first half of the vector size_t oddCount = count_if(myvec.begin(), myvec.end()- myvec.size()/2, isOdd); //using a functor to count numbers greater than 5 size_t greaterCount = count_if(myvec.begin(), myvec.end(), Greater(5)); cout « “vector size:” « myvec.size() « endl; cout « “even numbers:” « evenCount « " found" « endl; cout « “odd numbers:” « oddCount « " found" « endl; cout « “numbers >


```
5:" « greaterCount « " found"« endl; return 0; } Output vector size: 15 even numbers: 7 found  
odd numbers: 4 found numbers > 5: 6 found
```

Chapter 66

Professional Notes: Chapter 67: Sorting

Section 67.1: Sorting and sequence containers `std::sort`, found in the standard library header `algorithm`, is a standard library algorithm for sorting a range of values, dened by a pair of iterators. `std::sort` takes as the last parameter a functor used to compare two values; this is how it determines the order. Note that `std::sort` is not stable. The comparison function must impose a Strict, Weak Ordering on the elements. A simple less-than (or greater-than) comparison will suce. A container with random-access iterators can be sorted using the `std::sort` algorithm: Version C++11 `#include <vector> #include <algorithm>` `MyVector = {3, 1, 2} //Default comparison of <` `std::sort(MyVector.begin(), MyVector.end());` `std::sort` requires that its iterators are random access iterators. The sequence containers `std::list` and `std::forward_list` (requiring C++11) do not provide random access iterators, so they cannot be used with `std::sort`. However, they do have sort member functions which implement a sorting algorithm that works with their own iterator types. Version C++11 `#include <list> #include <algorithm>` `MyList = {3, 1, 2} //Default comparison of <` `//Whole list only. MyList.sort();` Their member sort functions always sort the entire list, so they cannot sort a sub-range of elements. However, since `list` and `forward_list` have fast splicing operations, you could extract the elements to be sorted from the list, sort them, then stu them back where they were quite eciently like this: `void sort_sublist(std::list& mylist, std::list::const_iterator start, std::list::const_iterator end) { //extract and sort half-open sub range denoted by start and end iterator std::list tmp; tmp.splice(tmp.begin(), list, start, end); tmp.sort(); //re-insert range at the point we extracted it from list.splice(end, tmp); }` Section 67.2: sorting with `std::map` (ascending and descending) This example sorts elements in ascending order of a key using a map. You can use any type, including class,

instead of `std::string`, in the example below. `#include <iostream> #include <map> #include <string> int main() { std::map<double, std::string> sorted_map; // Sort the names of the planets according to their size sorted_map.insert(std::make_pair(0.3829, "Mercury")); sorted_map.insert(std::make_pair(0.9499, "Venus")); sorted_map.insert(std::make_pair(1, "Earth")); sorted_map.insert(std::make_pair(0.532, "Mars")); sorted_map.insert(std::make_pair(10.97, "Jupiter")); sorted_map.insert(std::make_pair(9.14, "Saturn")); sorted_map.insert(std::make_pair(3.981, "Uranus")); sorted_map.insert(std::make_pair(3.865, "Neptune")); for (auto const& entry: sorted_map) { std::cout << entry.second << " (" << entry.first << " of Earth's radius)" << "; } } Output: Mercury (0.3829 of Earth's radius) Mars (0.532 of Earth's radius) Venus (0.9499 of Earth's radius) Earth (1 of Earth's radius) Neptune (3.865 of Earth's radius) Uranus (3.981 of Earth's radius) Saturn (9.14 of Earth's radius) Jupiter (10.97 of Earth's radius) If entries with equal keys are possible, use multimap instead of map (like in the following example). To sort elements in descending manner, declare the map with a proper comparison functor (std::greater<>): #include <iostream> #include <map> #include <string> #include <functional> int main() { std::multimap<int, std::string, std::greater> sorted_map; // Sort the names of animals in descending order of the number of legs sorted_map.insert(std::make_pair(6,`

```

“bug”)); sorted_map.insert(std::make_pair(4, “cat”)); sorted_map.insert(std::make_pair(100,
“centipede”)); sorted_map.insert(std::make_pair(2, “chicken”)); sorted_map.insert(std::make_pair(0,
“fish”)); sorted_map.insert(std::make_pair(4, “horse”)); sorted_map.insert(std::make_pair(8,
“spider”)); for (auto const& entry: sorted_map) { std::cout << entry.second << “ (has ” << entry.first
<< “ legs)” << “;

```

} } Output centipede (has 100 legs) spider (has 8 legs) bug (has 6 legs) cat (has 4 legs) horse (has 4 legs) chicken (has 2 legs) fish (has 0 legs) Section 67.3: Sorting sequence containers by overloaded less operator If no ordering function is passed, `std::sort` will order the elements by calling `operator<` on pairs of elements, which must return a type contextually convertible to `bool` (or just `bool`). Basic types (integers, floats, pointers etc) have already build in comparison operators. We can overload this operator to make the default sort call work on user-defined types. // Include sequence containers #include #include #include // Insert sorting algorithm #include

```

class Base { public: // Constructor that set variable to the value of v Base(int v): variable(v) {
} // Use variable to provide total order operator less //this always represents the left-hand side
of the compare. bool operator<(const Base &b) const { return this->variable < b.variable; } int
variable; }; int main() { std::vector vector; std::deque deque; std::list list; // Create 2 elements
to sort Base a(10); Base b(5); // Insert them into backs of containers vector.push_back(a);
vector.push_back(b);

```

```

deque.push_back(a); deque.push_back(b); list.push_back(a); list.push_back(b); // Now
sort data using operator<(const Base &b) function std::sort(vector.begin(), vector.end()); std::sort(deque.begin(),
deque.end()); // List must be sorted differently due to its design list.sort(); return 0; } Section
67.4: Sorting sequence containers using compare function // Include sequence containers #include #include #include // Insert sorting algorithm #include class Base { public: //
Constructor that set variable to the value of v Base(int v): variable(v) { } int variable; };
bool compare(const Base &a, const Base &b) { return a.variable < b.variable; } int main()
{ std::vector vector; std::deque deque; std::list list; // Create 2 elements to sort Base a(10);
Base b(5); // Insert them into backs of containers vector.push_back(a); vector.push_back(b);
deque.push_back(a); deque.push_back(b); list.push_back(a); list.push_back(b); // Now sort
data using comparing function

```

```

std::sort(vector.begin(), vector.end(), compare); std::sort(deque.begin(), deque.end(), com-
pare); list.sort(compare); return 0; } Section 67.5: Sorting sequence containers using lambda
expressions (C++11) Version C++11 // Include sequence containers #include #include #in-
clude #include #include // Include sorting algorithm #include class Base { public: // Con-
structor that set variable to the value of v Base(int v): variable(v) { } int variable; }; int
main() { // Create 2 elements to sort Base a(10); Base b(5); // We’re using C++11, so
let’s use initializer lists to insert items. std::vector vector = {a, b}; std::deque deque = {a,
b}; std::list list = {a, b}; std::array <Base, 2> array = {a, b}; std::forward_list flist =
{a, b}; // We can sort data using an inline lambda expression std::sort(std::begin(vector),
std::end(vector), { return a.variable < b.variable;}); // We can also pass a lambda object as
the comparator // and reuse the lambda multiple times auto compare = { return a.variable <
b.variable;}; std::sort(std::begin(deque), std::end(deque), compare); std::sort(std::begin(array),
std::end(array), compare); list.sort(compare); flist.sort(compare); return 0; }

```

Section 67.6: Sorting built-in arrays The sort algorithm sorts a sequence defined by two iterators. This is enough to sort a built-in (also known as c-style) array. Version C++11 `int arr1[] = {36, 24, 42, 60, 59};` // sort numbers in ascending order `sort(std::begin(arr1), std::end(arr1));` // sort numbers in descending order `sort(std::begin(arr1), std::end(arr1), std::greater());` Prior to C++11, end of array had to be “calculated” using the size of the array: Version < C++11 // Use a hard-coded number for array size `sort(arr1, arr1 + 5);` // Alternatively, use an expression `const size_t arr1_size = sizeof(arr1) / sizeof(*arr1); sort(arr1, arr1 + arr1_size);` Section 67.7: Sorting sequence containers with specified ordering If the values in a container have cer-

tain operators already overloaded, `std::sort` can be used with specialized functors to sort in either ascending or descending order: Version C++11 `#include <vector> #include <algorithm> #include <functional>` `std::vector v = {5,1,2,4,3}; //sort in ascending order (1,2,3,4,5) std::sort(v.begin(), v.end(), std::less()); //` Or just: `std::sort(v.begin(), v.end()); //sort in descending order (5,4,3,2,1) std::sort(v.begin(), v.end(), std::greater()); //Or just: std::sort(v.rbegin(), v.rend());` Version C++14 In C++14, we don't need to provide the template argument for the comparison function objects and instead let the object deduce based on what it gets passed in: `std::sort(v.begin(), v.end(), std::less<>()); //` ascending order `std::sort(v.begin(), v.end(), std::greater<>()); //` descending order

Chapter 67

STL UNDER THE HOOD

Chapter 68

STL INTERNALS DEEP DIVE

Chapter 69

STL INTERNALS DEEP DIVE (C++98)

To master the STL, you must understand what happens under the hood.

69.0.1 1. The Truth About `std::vector`

`std::vector` is a dynamic array. It guarantees contiguous memory.

- **Layout:** Three pointers: `start`, `finish`, `end_of_storage`.
 - `start`: Points to first element.
 - `finish`: Points to one-past-the-last active element (size).
 - `end_of_storage`: Points to end of allocated buffer (capacity).
- **Growth Strategy:** Geometric growth.
 - When `size() == capacity()`, a new buffer is allocated (usually 2x or 1.5x larger).
 - **Elements are Copied** to the new buffer (C++98 does not have Move semantics).
 - Old buffer is deleted.
 - *Cost*: Amortized $O(1)$ `push_back`, but worst-case $O(N)$.
- **Iterator Invalidation:**
 - **Reallocation**: Invalidates ALL iterators, pointers, and references.
 - **Insertion/Erasure**: Invalidates iterators at and after the point of operation.

69.0.2 2. The `std::deque` Implementation

`std::deque` (Double-Ended Queue) is NOT a contiguous array.

- **Layout:** A “Map” (dynamic array) of pointers to fixed-size “Chunks” (blocks).
 - Iterators are smart pointers that know how to jump between chunks.
- **Performance:**
 - $O(1)$ random access (double dereference).
 - $O(1)$ push/pop at BOTH ends (no full reallocation needed, just add a new chunk).
- **Cache Locality:** Worse than vector, better than list.

69.0.3 3. Why `std::list` is (Almost) Always Wrong

`std::list` is a Doubly Linked List.

- **Layout:** Nodes allocated individually on the heap.

```
— struct Node \{ T val; Node* prev; Node* next; \}
```

- **The Cache Problem:** Nodes are scattered in memory. Traversing a list causes constant Cache Misses.
- **Benchmark:** Iterating a `vector` is orders of magnitude faster than a `list`, even for large types, due to prefetching.
- **Use Case:** Only when you need **Reference Stability** (insertions never invalidate references to other elements) or frequent splicing.

69.0.4 4. Associative Containers (Map/Set)

`std::map`, `std::set`, `std::multimap`, `std::multiset`.

- **Implementation:** Balanced Binary Search Tree (usually **Red-Black Tree**).
- **Node Layout:**

```
struct Node \{ T val; Node* left; Node* right; Node* parent; Color color; \}
```
- **Complexity:** $O(\log N)$ for insert, lookup, delete.
- **Overhead:** 3 pointers + enum per element (heavy memory overhead).

69.0.5 5. Iterator Invalidation Cheat Sheet

Container	Operation	Invalidates
Vector	Capacity Change	ALL
Vector	Insert/Erase	Current & After
Deque	Insert/Erase (ends)	Iterators only (Refs valid!)
Deque	Insert/Erase (middle)	ALL
List	Insert/Erase	Only deleted element
Map/Set	Insert/Erase	Only deleted element

Chapter 70

ERROR HANDLING AND ROBUSTNESS

Chapter 71

ERROR HANDLING & ROBUSTNESS

Chapter 72

ERROR HANDLING & DEBUGGING (C++98)

72.1 1. ASSERTIONS

Use `assert` to catch programming logic errors during development.

```
1 #include <iostream>
2 #include <cassert>
3
4 int divide(int a, int b) {
5     assert(b != 0); // Terminates program if b is 0
6     return a / b;
7 }
8
9 int main() {
10     std::cout << divide(10, 2) << "\n";
11     return 0;
12 }
```

72.2 2. EXCEPTIONS

C++ uses exceptions to handle runtime errors gracefully.

72.2.1 2.1 Basic Try-Catch

```
1 #include <iostream>
2
3 int safe_divide(int a, int b) {
4     if (b == 0) {
5         throw "Division by zero condition!";
6     }
7     return a / b;
8 }
9
10 int main() {
11     try {
12         int result = safe_divide(10, 0);
13         std::cout << result << "\n";
14     } catch (const char* msg) {
15         std::cerr << "Error: " << msg << "\n";
16     }
17 }
```

```

17     return 0;
18 }

```

72.2.2 2.2 Catching Multiple Types

```

1 try {
2     // code...
3 } catch (int e) {
4     std::cerr << "Integer exception: " << e << "\n";
5 } catch (const char* e) {
6     std::cerr << "String exception: " << e << "\n";
7 } catch (...) {
8     std::cerr << "Unknown exception caught!\n";
9 }

```

72.2.3 2.3 Standard Exceptions

Inherit from `std::exception` for custom exceptions.

```

1 #include <iostream>
2 #include <exception>
3 #include <stdexcept> // runtime_error, logic_error
4
5 class MyError : public std::exception {
6 public:
7     virtual const char* what() const throw() {
8         return "My Custom Error";
9     }
10 };
11
12 void test() {
13     throw std::runtime_error("Standard Runtime Error");
14 }
15
16 int main() {
17     try {
18         test();
19     } catch (const std::exception& e) {
20         std::cerr << "Caught: " << e.what() << "\n";
21     }
22     return 0;
23 }

```

c

1. Unit Testing in C++ Test

2. Debugging with

3. Defensive Programming Techniques * **Static Assertions (C++11)**: `static_assert(sizeof(int) =`

72.2.4 2.4 Exception Specifications (C++98)

In C++98, you can specify what a function might throw. (Note: Deprecated in C++11, but valid here).

```

1 // Can throw only int
2 void func() throw(int) {
3     throw 42;
4 }

```

```
5
6 // Cannot throw anything
7 void no_throw() throw() {
8     // logic...
9 }
```

72.2.5 2.5 Stack Unwinding & RAII

When an exception is thrown, the stack is unwound, identifying destructors for all local objects.

```
1 class Resource {
2 public:
3     Resource() { std::cout << "Acquired\n"; }
4     ~Resource() { std::cout << "Released\n"; }
5 };
6
7 void risky() {
8     Resource r; // Destructor guaranteed to run
9     throw 1;
10 }
11
12 int main() {
13     try {
14         risky();
15     } catch (...) {
16         std::cout << "Caught\n";
17     }
18     return 0;
19 }
20 // Output: Acquired -> Released -> Caught
```

72.3 3. DEBUGGING STRATEGIES

72.3.1 3.1 Debug Macros (C++98 Style)

```
1 #include <iostream>
2
3 #ifdef DEBUG
4     #define LOG(x) std::cout << "DEBUG: " << x << "\n"
5 #else
6     #define LOG(x)
7 #endif
8
9 int main() {
10     int x = 10;
11     LOG("x is " << x);
12     return 0;
13 }
```

Chapter 73

Professional Notes: Chapter 72: Exceptions

Section 72.1: Catching exceptions Section 72.2: Rethrow (propagate) exception Section 72.3: Best practice: throw by value, catch by const reference Section 72.4: Custom exception Section 72.5: `std::uncaught_exceptions` Section 72.6: Function Try Block for regular function Section 72.7: Nested exception Section 72.8: Function Try Blocks In constructor Section 72.9: Function Try Blocks In destructor

Chapter 74

Professional Notes: Chapter 72: Exceptions

Section 72.1: Catching exceptions A try/catch block is used to catch exceptions. The code in the try section is the code that may throw an exception, and the code in the catch clause(s) handles the exception. `#include #include #include int main() { std::string str("foo"); try { str.at(10); // access element, may throw std::out_of_range } catch (const std::out_of_range& e) { // what() is inherited from std::exception and contains an explanatory message std::cout << e.what(); } }` Multiple catch clauses may be used to handle multiple exception types. If multiple catch clauses are present, the exception handling mechanism tries to match them in order of their appearance in the code: `std::string str("foo"); try { str.reserve(2); // reserve extra capacity, may throw std::length_error str.at(10); // access element, may throw std::out_of_range } catch (const std::length_error& e) { std::cout << e.what(); } catch (const std::out_of_range& e) { std::cout << e.what(); }` Exception classes which are derived from a common base class can be caught with a single catch clause for the common base class. The above example can replace the two catch clauses for `std::length_error` and `std::out_of_range` with a single clause for `std::exception`: `std::string str("foo"); try { str.reserve(2); // reserve extra capacity, may throw std::length_error str.at(10); // access element, may throw std::out_of_range } catch (const std::exception& e) { std::cout << e.what(); }` Because the catch clauses are tried in order, be sure to write more specific catch clauses first, otherwise your exception handling code might never get called: `try { /* Code throwing exceptions omitted. */ } catch (const std::exception& e) { // Handle all exceptions of type std::exception. / }`

/ This block of code will never execute, because `std::runtime_error` inherits from `std::exception`, and all exceptions of type `std::exception` were already caught by the previous catch clause. / Another possibility is the catch-all handler, which will catch any thrown object: `try { throw 10; } catch (...) { std::cout << "caught an exception"; }` Section 72.2: Rethrow (propagate) exception Sometimes you want to do something with the exception you catch (like write to log or print a warning) and let it bubble up to the upper scope to be handled. To do so, you can rethrow any exception you catch: `try { ... // some code here } catch (const SomeException& e) { std::cout << "caught an exception"; throw; }` Using `throw;` without arguments will re-throw the currently caught exception. Version C++11 To rethrow a managed `std::exception_ptr`, the C++ Standard Library has the `rethrow_exception` function that can be used by including the header in your program. `#include #include #include #include void handle_eptr(std::exception_ptr eptr) // passing by value is ok { try { if (eptr) { std::rethrow_exception(eptr); } } catch (const std::exception& e) { std::cout << "Caught exception " << e.what() << " "; } }` `int main() { std::exception_ptr eptr; try { std::string().at(1); // this generates an std::out_of_range } catch (...) { eptr = std::current_exception(); // capture } handle_eptr(eptr); }` // destructor for `std::out_of_range` called here, when the `eptr` is destructed

Section 72.3: Best practice: throw by value, catch by const reference In general, it is considered good practice to throw by value (rather than by pointer), but catch by (const) reference. try { // throw new std::runtime_error("Error!"); // Don't do this! // This creates an exception object // on the heap and would require you to catch the // pointer and manage the memory yourself. This can // cause memory leaks! throw std::runtime_error("Error!"); } catch (const std::runtime_error& e) { std::cout << e.what() << std::endl; } One reason why catching by reference is a good practice is that it eliminates the need to reconstruct the object when being passed to the catch block (or when propagating through to other catch blocks). Catching by reference also allows the exceptions to be handled polymorphically and avoids object slicing. However, if you are rethrowing an exception (like throw e;, see example below), you can still get object slicing because the throw e; statement makes a copy of the exception as whatever type is declared: #include struct BaseException { virtual const char what() const { return "BaseException"; } }; struct DerivedException : BaseException { // "virtual" keyword is optional here virtual const char* what() const { return "DerivedException"; } }; int main(int argc, char** argv) { try { try { throw DerivedException(); } catch (const BaseException& e) { std::cout << "First catch block:" << e.what() << std::endl; // Output ==> First catch block: DerivedException throw e; // This changes the exception to BaseException // instead of the original DerivedException! } } catch (const BaseException& e) { std::cout << "Second catch block:" << e.what() << std::endl; // Output ==> Second catch block: BaseException } return 0; } If you are sure that you are not going to do anything to change the exception (like add information or modify the message), catching by const reference allows the compiler to make optimizations and can improve performance. But this can still cause object splicing (as seen in the example above). Warning: Beware of throwing unintended exceptions in catch blocks, especially related to allocating extra memory or resources. For example, constructing logic_error, runtime_error or their subclasses might throw bad_alloc

due to memory running out when copying the exception string, I/O streams might throw during logging with respective exception masks set, etc. Section 72.4: Custom exception You shouldn't throw raw values as exceptions, instead use one of the standard exception classes or make your own. Having your own exception class inherited from std::exception is a good way to go about it. Here's a custom exception class which directly inherits from std::exception: #include class Except: virtual public std::exception { protected: int error_number; ///< Error number int error_offset; ///< Error offset std::string error_message; ///< Error message public: /** Constructor (C++ STL string, int, int). * @param msg The error message * @param err_num Error number * @param err_off Error offset */ explicit Except(const std::string& msg, int err_num, int err_off): error_number(err_num), error_offset(err_off), error_message(msg) {} /** Destructor. * Virtual to allow for subclassing. / virtual ~Except() throw () {} / **Returns a pointer to the (constant) error description.** * @return A pointer to a const char. The underlying memory is in possession of the Except object. Callers must * not attempt to free the memory. / virtual const char what() const throw () { return error_message.c_str(); } / Returns error number. @return #error_number */ virtual int getErrorNumber() const throw() { return error_number; } /**Returns error offset. * @return #error_offset */ virtual int getErrorOffset() const throw() {

return error_offset; } }; An example throw catch: try { throw(Except("Couldn't do what you were expecting", -12, -34)); } catch (const Except& e) { std::cout << e.what() << "number:" << e.getErrorNumber() << "offset:" << e.getErrorOffset(); } As you are not only just throwing a dumb error message, also some other values representing what the error exactly was, your error handling becomes much more efficient and meaningful. There's an exception class that let's you handle error messages nicely :std::runtime_error You can inherit from this class too: #include class Except: virtual public std::runtime_error { protected: int error_number; ///< Error number int error_offset; ///< Error offset public: /** Constructor (C++ STL string, int, int). * @param msg The error message * @param err_num Error number * @param err_off Error offset */


```
explicit Except(const std::string& msg, int err_num, int err_off): std::runtime_error(msg) {
    error_number = err_num; error_offset = err_off; } /** Destructor. * Virtual to allow for sub-
    classing. */ virtual ~Except() throw () {} /** Returns error number. * @return #error_number
    */ virtual int getErrorNumber() const throw() { return error_number; } /**Returns error off-
    set.
```

```
    * @return #error_offset */ virtual int getErrorOffset() const throw() { return error_offset; }
}; Note that I haven't overridden the what() function from the base class (std::runtime_error)
i.e we will be using the base class's version of what(). You can override it if you have fur-
ther agenda. Section 72.5: std::uncaught_exceptions Version c++17 C++17 introduces int
std::uncaught_exceptions() (to replace the limited bool std::uncaught_exception()) to know
how many exceptions are currently uncaught. That allows for a class to determine if it is de-
stroyed during a stack unwinding or not. #include #include #include // Apply change on de-
struction: // Rollback in case of exception (failure) // Else Commit (success) class Transaction
{ public: Transaction(const std::string& s) : message(s) {} Transaction(const Transaction&)
= delete; Transaction& operator =(const Transaction&) = delete; void Commit() { std::cout
« message « ": Commit"; } void RollBack() noexcept(true) { std::cout « message « ": Roll-
back"; } // ... ~Transaction() { if (uncaughtExceptionCount == std::uncaught_exceptions())
{ Commit(); // May throw. } else { // current stack unwinding RollBack(); } } private:
std::string message; int uncaughtExceptionCount = std::uncaught_exceptions(); }; class Foo {
public: ~Foo() { try { Transaction transaction("In ~Foo"); // Commit, // even if there is an
uncaught exception //... } catch (const std::exception& e) { std::cerr « "exception/~Foo:" «
e.what() « std::endl;
```

```
    } } }; int main() { try { Transaction transaction("In main"); // RollBack Foo foo; // ~Foo
commit its transaction. //... throw std::runtime_error("Error"); } catch (const std::exception&
e) { std::cerr « "exception/main:" « e.what() « std::endl; } } Output: In ~Foo: Commit In
main: Rollback exception/main:Error Section 72.6: Function Try Block for regular function
void function_with_try_block() try { // try block body } catch (...) { // catch block body
} Which is equivalent to void function_with_try_block() { try { // try block body } catch
(...) { // catch block body } } Note that for constructors and destructors, the behavior is
different as the catch block re-throws an exception anyway (the caught one if there is no other
throw in the catch block body). The function main is allowed to have a function try block like
any other function, but main's function try block will not catch exceptions that occur during
the construction of a non-local static variable or the destruction of any static variable. Instead,
std::terminate is called. Section 72.7: Nested exception Version C++11
```

During exception handling there is a common use case when you catch a generic exception from a low-level function (such as a filesystem error or data transfer error) and throw a more specific high-level exception which indicates that some high-level operation could not be performed (such as being unable to publish a photo on Web). This allows exception handling to react to specific problems with high level operations and also allows, having only error an message, the programmer to find a place in the application where an exception occurred. Downside of this solution is that exception callstack is truncated and original exception is lost. This forces developers to manually include text of original exception into a newly created one. Nested exceptions aim to solve the problem by attaching low-level exception, which describes the cause, to a high level exception, which describes what it means in this particular case. `std::nested_exception` allows to nest exceptions thanks to `std::throw_with_nested`: `#include #include #include #include #include`

```
struct MyException { MyException(const std::string& message) : message(message)
{} std::string message; }; void print_current_exception(int level) { try { throw; } catch (const
std::exception& e) { std::cerr « std::string(level, ' ') « "exception:" « e.what() « "; } catch (const
MyException& e) { std::cerr « std::string(level, ' ') « "MyException:" « e.message « "; } catch
(...) { std::cerr « "Unknown exception"; } } void print_current_exception_with_nested(int
level = 0) { try { throw; } catch (...) { print_current_exception(level); }
```

```
try { throw; } catch (const std::nested_exception& nested) { try { nested.rethrow_nested(); }
catch (...) { print_current_exception_with_nested(level + 1); // recursion } } catch (...) {
//Empty // End recursion } } // sample function that catches an exception and wraps it in a
nested exception void open_file(const std::string& s)
```

```
{ try { std::ifstream file(s); file.exceptions(std::ios_base::failbit); } catch(...) { std::throw_with_nested(My
open" + s)); } } // sample function that catches an exception and wraps it in a nested excep-
tion void run() { try { open_file("nonexistent.file"); } catch(...) { std::throw_with_nested(
std::runtime_error("run() failed") ); } } // runs the sample function above and prints the caught
exception int main() { try { run(); } catch(...) { print_current_exception_with_nested(); }
} Possible output: exception: run() failed MyException: Couldn't open nonexistent.file excep-
tion: basic_ios::clear If you work only with exceptions inherited from std::exception, code can
even be simplified. Section 72.8: Function Try Blocks In constructor The only way to catch
exception in initializer list: struct A : public B { A() try : B(), foo(1), bar(2) { // constructor
body } catch (...) { // exceptions from the initializer list and constructor are caught here //
if no exception is thrown here // then the caught exception is re-thrown. } private: Foo foo;
Bar bar; };
```

Section 72.9: Function Try Blocks In destructor struct A { ~A() noexcept(false) try { //
destructor body } catch (...) { // exceptions of destructor body are caught here // if no
exception is thrown here // then the caught exception is re-thrown. } }; Note that, although
this is possible, one needs to be very careful with throwing from destructor, as if a destructor
called during stack unwinding throws an exception, `std::terminate` is called.

Chapter 75

Chapter 10: Advanced Streams & File I/O

This chapter explores the full power of the C++ `<iostream>`, `<fstream>`, and `<sstream>` libraries, deconstructing how they interact with the filesystem and formatted data.

75.1 10.1 File I/O Foundations

Working with files involves the `std::fstream`, `std::ifstream`, and `std::ofstream` classes.

75.1.1 1. Opening Modes

- `std::ios::in`: Open for reading.
- `std::ios::out`: Open for writing (overwrites existing).
- `std::ios::app`: Append to end of file.
- `std::ios::ate`: Open and seek to end.
- `std::ios::binary`: Binary mode (no CRLF translation).

75.1.2 2. Binary vs Text Mode

In text mode, special characters like `\n` might be translated (e.g., to `\r\n` on Windows). Binary mode ensures that exactly what you write is what appears on disk.

```
1 std::ofstream file("data.bin", std::ios::binary);
2 double d = 3.14;
3 file.write(reinterpret_cast<const char*>(&d), sizeof(d));
```

75.2 10.2 Stream Manipulators

Manipulators allow you to change how data is formatted on the fly.

75.2.1 1. Numeric Formatting

- `std::hex`, `std::oct`, `std::dec`: Change base.
- `std::setprecision(n)`: Set floating point precision (requires `<iomanip>`).
- `std::fixed`, `std::scientific`: Change notation.

75.2.2 2. Padding and Alignment

- `std::setw(n)`: Set width of next field.
- `std::setfill(c)`: Set fill character.
- `std::left`, `std::right`, `std::internal`: Change alignment.

75.3 10.3 String Streams (`stringstream`)

`std::stringstream` allows you to treat a string like a stream, enabling easy conversion between types and strings.

```
1 #include <sstream>
2 std::stringstream ss;
3 ss << "The answer is " << 42;
4 std::string s = ss.str();
```

75.3.1 Professional Notes: Stream Architecture

1. The `ios_base` State Machine

Every stream inherits from `ios_base`, which maintains internal flags for formatting and errors. * **Performance Trap:** Streams are significantly slower than C's `printf/scanf` due to the overhead of object construction and virtual function calls. * **Optimization:** `std::ios::sync_with_stdio(false)`; disables the synchronization between C++ and C streams, making `std::cin` as fast as `scanf`.

2. Stream Buffering (`streambuf`)

The actual data transfer is handled by a buffer object (`rdbuf`). * **Redirection:** You can redirect `cout` to a file by swapping its buffer:

```
1 std::ofstream out("log.txt");
2 std::streambuf *coutbuf = std::cout.rdbuf();
3 std::cout.rdbuf(out.rdbuf()); // cout now writes to log.txt
```

3. Custom Manipulators

You can create your own manipulators by defining functions that take and return a reference to a stream:

```
1 std::ostream& tab(std::ostream& os) {
2     return os << "\t";
3 }
4 std::cout << "Data1" << tab << "Data2" << std::endl;
```

Chapter 76

VOLUME 02 MODERN REVOLUTION C11

Chapter 77

THE MODERN C11 CORE

Chapter 78

THE MODERN C++11 CORE: SYNTAX & TYPE SYSTEM

78.1 1. Type Inference & Safety

78.1.1 1.1 auto: Automatic Type Deduction

C++11 introduced `auto` to let the compiler deduce the type of a variable from its initializer.

```
1 auto i = 42;           // int
2 auto d = 3.14;        // double
3 auto s = "hello";     // const char*
4 auto& ref = i;         // int&
5 const auto* ptr = &i; // const int*
```

Crucial Rules: 1. **Reference Dropping:** `auto` drops top-level references and `const` by default (like template deduction). `cpp` `int x = 0; int& y = x; auto z = y; // z is int`, NOT `int&` 2. **Keeping Qualifiers:** Use `auto&` or `const auto&` to preserve them. `cpp` `const int cx = 10; auto copy = cx; // int (const dropped) const auto& ref = cx; // const int&`

78.1.2 1.2 decltype: Inspecting Types

Unlike `auto`, `decltype` gives the *exact* declared type of an expression, including references and `const`.

```
1 int x = 0;
2 decltype(x) y = 5; // int
3 decltype((x)) z = y; // int& (because (x) is an lvalue expression)
```

Use Case: Trailing Return Types Allows return types to depend on parameter types.

```
1 template<typename T, typename U>
2 auto add(T a, U b) -> decltype(a + b) {
3     return a + b;
4 }
```

78.1.3 1.3 nullptr: The Null Pointer Literal

Replaces `NULL` and `0`. Type-safe and unambiguous.

```
1 void f(int);
2 void f(char*);
3
4 f(0); // Calls f(int) -> Ambiguity resolved!
```

```

5 f(NULL); // Implementation-defined (often f(int))
6 f(nullptr); // Calls f(char*)

```

78.2 2. Initialization Uniformity

78.2.1 2.1 Uniform Initialization (Brace Initialization)

Consistent syntax for initializing everything.

```

1 int x{5}; // Direct initialization
2 int y = {5}; // Copy list initialization
3 std::vector<int> v{1, 2, 3};
4 std::map<int, std::string> m{{1, "a"}, {2, "b"}};

```

Narrowing Prevention:

```

1 int x = 3.14; // Compiles (warning), x becomes 3
2 // int y{3.14}; // ERROR: Narrowing conversion not allowed

```

78.2.2 2.2 std::initializer_list

Allows objects to accept a list of elements (used in constructors of containers).

```

1 #include <initializer_list>
2
3 class MyVector {
4 public:
5     MyVector(std::initializer_list<int> list) {
6         for (int x : list) {
7             // ...
8         }
9     }
10 };

```

78.3 3. Control Flow & Iteration

78.3.1 3.1 Range-Based For Loops

Syntactic sugar for iterating over arrays and containers.

```

1 std::vector<int> v = {1, 2, 3};
2
3 // By Value (Copy)
4 for (int x : v) { /* ... */ }
5
6 // By Reference (Modify)
7 for (auto& x : v) { x *= 2; }
8
9 // By Const Reference (Read-only, avoids copy)
10 for (const auto& x : v) { /* ... */ }

```

Works on anything with `begin()` and `end()` iterators (or arrays).

78.4 4. Class Features

78.4.1 4.1 Explicit Overrides (override, final)

Compiler-checked inheritance safety.

- `override`: Ensures you are actually overriding a base virtual function.
- `final`: Prevents further overriding or inheritance.

```
1 class Base {
2     virtual void foo(int);
3 };
4
5 class Derived : public Base {
6     void foo(int) override;    // OK
7     // void foo(float) override; // Error: signature mismatch
8 };
9
10 class Last final : public Base { // Cannot be inherited from
11     void foo(int) final;        // Cannot be overridden
12 };
```

78.4.2 4.2 Defaulted and Deleted Functions

Control compiler-generated functions.

```
1 class Widget {
2 public:
3     Widget() = default; // Force generation of default constructor
4
5     // Disable copying
6     Widget(const Widget&) = delete;
7     Widget& operator=(const Widget&) = delete;
8 };
```

78.4.3 4.3 Strongly Typed Enums (enum class)

Scoped, strongly typed, and safe.

```
1 enum class Color : char { Red, Green, Blue }; // Underlying type char
2
3 Color c = Color::Red;
4 // int i = c; // Error: No implicit conversion
```

78.4.4 4.4 Delegating Constructors

One constructor calls another.

```
1 class Box {
2     int w, h;
3 public:
4     Box(int width, int height) : w(width), h(height) {}
5     Box() : Box(1, 1) {} // Delegate
6 };
```

78.5 5. constexpr (C++11 Version)

Compile-time constants and functions. In C++11, `constexpr` functions must contain a *single return statement*.

```
1 constexpr int square(int x) {  
2     return x * x;  
3 }  
4  
5 int array[square(5)]; // Valid: array size 25 determined at compile time
```

78.6 6. Static Assert

Compile-time assertions.

```
1 static_assert(sizeof(void*) == 8, "64-bit system required");
```

Chapter 79

MOVE SEMANTICS AND SMART POINTERS

Chapter 80

MOVE SEMANTICS & SMART POINTERS

80.1 1. Rvalue References & Move Semantics

C++11 solves the problem of unnecessary copying with **Move Semantics**.

80.1.1 1.1 Lvalues vs Rvalues

- **Lvalue (Left Value):** Has a name, has an address (identity). Persists beyond the expression.
- **Rvalue (Right Value):** Temporary, no name (or about to die). Cannot take address.

```
1 int x = 10; // x is lvalue, 10 is rvalue
2 int y = x;  // y is lvalue
3 int z = x + y; // (x+y) is rvalue
```

80.1.2 1.2 Rvalue References (T&&)

A reference that binds *only* to rvalues. Represents an object we can “steal” from.

```
1 void f(int& x) { cout << "Lvalue ref"; }
2 void f(int&& x) { cout << "Rvalue ref"; }
3
4 int a = 5;
5 f(a); // Lvalue ref
6 f(5); // Rvalue ref
```

80.1.3 1.3 Move Constructor & Move Assignment

Instead of copying data (slow), we steal pointers (fast).

```
1 class Buffer {
2     int* data;
3     size_t size;
4 public:
5     // Move Constructor
6     Buffer(Buffer&& other) noexcept : data(other.data), size(other.size) {
7         other.data = nullptr; // Nullify source to prevent double free
8         other.size = 0;
9     }
10 }
```

```

11 // Move Assignment
12 Buffer& operator=(Buffer&& other) noexcept {
13     if (this != &other) {
14         delete[] data; // Free own resources
15         data = other.data; // Steal resources
16         size = other.size;
17         other.data = nullptr; // Nullify source
18         other.size = 0;
19     }
20     return *this;
21 }
22 };

```

80.1.4 1.4 std::move

Casts an lvalue to an rvalue, enabling move semantics.

```

1 Buffer b1;
2 Buffer b2 = std::move(b1); // Calls Move Constructor
3 // b1 is now in valid but unspecified state (empty)

```

Warning: Do not use `b1` after moving from it.

80.2 2. Smart Pointers (RAII)

Manual `new/delete` is prone to leaks. C++11 introduces strict ownership models.

80.2.1 2.1 std::unique_ptr

- **Ownership:** Exclusive. Only one pointer owns the object.
- **Copying:** Disabled (`delete`).
- **Moving:** Allowed.
- **Overhead:** Zero (same as raw pointer).

```

1 #include <memory>
2
3 std::unique_ptr<int> p1(new int(5));
4 // std::unique_ptr<int> p2 = p1; // ERROR: Cannot copy
5 std::unique_ptr<int> p3 = std::move(p1); // OK: Ownership transferred

```

80.2.2 2.2 std::shared_ptr

- **Ownership:** Shared (Reference Counted).
- **Destruction:** Object deleted when last `shared_ptr` dies.
- **Overhead:** Control block on heap (ref counts).

```

1 auto sp1 = std::make_shared<int>(10); // Efficient allocation
2 auto sp2 = sp1; // Ref count = 2

```

c

1. RAI:

2. The C++11 Memory Model and Atomics Before C++11, the language had no formal definition of

3. Smart Pointer Gotchas * `std::make_shared` vs. `new: make\_shar`

80.2.3 2.3 `std::weak_ptr`

- **Ownership:** Non-owning observer of `shared_ptr`.
- **Use Case:** Break cyclic references (A->B, B->A).

```

1 std::shared_ptr<int> sp = std::make_shared<int>(42);
2 std::weak_ptr<int> wp = sp;
3
4 if (auto locked = wp.lock()) { // Check if object exists
5     std::cout << *locked;
6 }

```

80.2.4 2.4 `std::make_shared` vs `new`

Always use `std::make_shared`. It allocates the object and control block in a single heap allocation (cache efficient).

80.3 3. Perfect Forwarding

Used in templates to preserve value category (lvalue vs rvalue).

```

1 template<typename T>
2 void wrapper(T&& arg) {
3     // std::forward passes arg as lvalue if given lvalue,
4     // rvalue if given rvalue.
5     func(std::forward<T>(arg));
6 }

```

Chapter 81

Professional Notes: Chapter 33: Smart Pointers

Section 33.1: Unique ownership (`std::unique_ptr`) Section 33.2: Sharing ownership (`std::shared_ptr`)
Section 33.3: Sharing with temporary ownership (`std::weak_ptr`) Section 33.4: Using custom
deleters to create a wrapper to a C interface Section 33.5: Unique ownership without move
semantics (`auto_ptr`) Section 33.6: Casting `std::shared_ptr` pointers Section 33.7: Writing a
smart pointer: `value_ptr` Section 33.8: Getting a `shared_ptr` referring to this

Chapter 82

Professional Notes: Chapter 106: Move Semantics

Section 106.1: Move semantics Section 106.2: Using `std::move` to reduce complexity from $O(n)$ to $O(1)$ Section 106.3: Move constructor Section 106.4: Re-use a moved object Section 106.5: Move assignment Section 106.6: Using move semantics on containers

Chapter 83

Professional Notes: Chapter 33: Smart Pointers

Section 33.1: Unique ownership (`std::unique_ptr`) Version C++11 A `std::unique_ptr` is a class template that manages the lifetime of a dynamically stored object. Unlike for `std::shared_ptr`, the dynamic object is owned by only one instance of a `std::unique_ptr` at any time, // Creates a dynamic int with value of 20 owned by a unique pointer `std::unique_ptr ptr = std::make_unique(20);` (Note: `std::unique_ptr` is available since C++11 and `std::make_unique` since C++14.) Only the variable `ptr` holds a pointer to a dynamically allocated int. When a unique pointer that owns an object goes out of scope, the owned object is deleted, i.e. its destructor is called if the object is of class type, and the memory for that object is released. To use `std::unique_ptr` and `std::make_unique` with array-types, use their array specializations: // Creates a `unique_ptr` to an int with value 59 `std::unique_ptr ptr = std::make_unique(59);` // Creates a `unique_ptr` to an array of 15 ints `std::unique_ptr<int[]> ptr = std::make_unique<int[]>(15);` You can access the `std::unique_ptr` just like a raw pointer, because it overloads those operators. You can transfer ownership of the contents of a smart pointer to another pointer by using `std::move`, which will cause the original smart pointer to point to `nullptr`. // 1. `std::unique_ptr ptr = std::make_unique();` // Change value to 1 `ptr = 1;` // 2. *std::unique_ptr (by moving 'ptr' to 'ptr2', 'ptr' doesn't own the object anymore)* `std::unique_ptr ptr2 = std::move(ptr); int a = ptr2;` // 'a' is 1 `int b = ptr;` // undefined behavior! 'ptr' is 'nullptr' // (because of the move command above) Passing `unique_ptr` to functions as parameter: `void foo(std::unique_ptr ptr) { // Your code goes here } std::unique_ptr ptr = std::make_unique(59); foo(std::move(ptr))` Returning `unique_ptr` from functions. This is the preferred C++11 way of writing factory functions, as it clearly

conveys the ownership semantics of the return: the caller owns the resulting `unique_ptr` and is responsible for it. `std::unique_ptr foo() { std::unique_ptr ptr = std::make_unique(59); return ptr; } std::unique_ptr ptr = foo();` Compare this to: `int foo_cpp03(); int* p = foo_cpp03();` // do I own p? do I have to delete it at some point? // it's not readily apparent what the answer is. Version < C++14 The class template `make_unique` is provided since C++14. It's easy to add it manually to C++11 code: `template<typename T, typename... Args> typename std::enable_if<!std::is_array::value, std::unique_ptr>::type make_unique(Args&&... args) { return std::unique_ptr(new T(std::forward(args)...)); } // Use make_unique for arrays template<typename T> std::enable_if<std::is_array::value, std::unique_ptr>::type make_unique(size_t n) { return std::unique_ptr(new typename std::remove_extent::typen); }` Version C++11 Unlike the dumb smart pointer (`std::auto_ptr`), `unique_ptr` can also be instantiated with vector allocation (not `std::vector`). Earlier examples were for scalar allocations. For example to have a dynamically allocated integer array for 10 elements, you would specify `int[]` as the template type (and not just `int`): `std::unique_ptr<int[]> arr_ptr = std::make_unique<int[]>(10);` Which can be simplified with: `auto arr_ptr = std::make_unique<int[]>(10);` Now, you use `arr_ptr` as if it is

an array: `arr_ptr[2] = 10;` // Modify third element You need not to worry about de-allocation. This template specialized version calls constructors and destructors appropriately. Using vectored version of `unique_ptr` or a vector itself - is a personal choice. In versions prior to C++11, `std::auto_ptr` was available. Unlike `unique_ptr` it is allowed to copy `auto_ptr`s, upon which the source `ptr` will lose the ownership of the contained pointer and the target receives it. Section 33.2: Sharing ownership (`std::shared_ptr`) The class template `std::shared_ptr` denotes a shared pointer that is able to share ownership of an object with

other shared pointers. This contrasts to `std::unique_ptr` which represents exclusive ownership. The sharing behavior is implemented through a technique known as reference counting, where the number of shared pointers that point to the object is stored alongside it. When this count reaches zero, either through the destruction or reassignment of the last `std::shared_ptr` instance, the object is automatically destroyed. // Creation: 'firstShared' is a shared pointer for a new instance of 'Foo' `std::shared_ptr firstShared = std::make_shared(<args>);` To create multiple smart pointers that share the same object, we need to create another `shared_ptr` that aliases the first shared pointer. Here are 2 ways of doing it: `std::shared_ptr secondShared(firstShared);` // 1st way: Copy constructing `std::shared_ptr secondShared; secondShared = firstShared;` // 2nd way: Assigning Either of the above ways makes `secondShared` a shared pointer that shares ownership of our instance of `Foo` with `firstShared`. The smart pointer works just like a raw pointer. This means, you can use `*` to dereference them. The regular `->` operator works as well: `secondShared->test();` // Calls `Foo::test()` Finally, when the last aliased `shared_ptr` goes out of scope, the destructor of our `Foo` instance is called. Warning: Constructing a `shared_ptr` might throw a `bad_alloc` exception when extra data for shared ownership semantics needs to be allocated. If the constructor is passed a regular pointer it assumes to own the object pointed to and calls the deleter if an exception is thrown. This means `shared_ptr(new T(args))` will not leak a `T` object if allocation of `shared_ptr` fails. However, it is advisable to use `make_shared(args)` or `allocate_shared(alloc, args)`, which enable the implementation to optimize the memory allocation. Allocating Arrays(`[]`) using `shared_ptr` Version C++11 Version < C++17 Unfortunately, there is no direct way to allocate Arrays using `make_shared<>`. It is possible to create arrays for `shared_ptr<>` using `new` and `std::default_delete`. For example, to allocate an array of 10 integers, we can write the code as `shared_ptr sh(new int[10], std::default_delete<int[]>());` Specifying `std::default_delete` is mandatory here to make sure that the allocated memory is correctly cleaned up using `delete[]`. If we know the size at compile time, we can do it this way: `template struct shared_array_maker {};` `template<class T, std::size_t N> struct shared_array_maker<T[N]> { std::shared_ptr operator()const{`

```

    auto r = std::make_shared<std::array<T,N>>(); if (!r) return {}; return {r.data(), r}; } };
template auto make_shared_array() -> decltype(shared_array_maker{}()) { return shared_array_maker{}()
} then make_shared_array<int[10]> returns a shared_ptr pointing to 10 ints all default constructed.
Version C++17 With C++17, shared_ptr gained special support for array types. It is no longer necessary to specify the array-deleter explicitly, and the shared pointer can be dereferenced using the [] array index operator: std::shared_ptr<int[]> sh(new int[10]); sh[0] = 42; Shared pointers can point to a sub-object of the object it owns: struct Foo { int x; }; std::shared_ptr p1 = std::make_shared<Foo>(); std::shared_ptr p2(p1, &p1->x); Both p2 and p1 own the object of type Foo, but p2 points to its int member x. This means that if p1 goes out of scope or is reassigned, the underlying Foo object will still be alive, ensuring that p2 does not dangle. Important: A shared_ptr only knows about itself and all other shared_ptr that were created with the alias constructor. It does not know about any other pointers, including all other shared_ptrs created with a reference to the same Foo instance: Foo foo = new Foo; std::shared_ptr shared1(foo); std::shared_ptr shared2(foo); // don't do this shared1.reset(); // this will delete foo, since shared1 // was the only shared_ptr that owned it shared2->test(); // UNDEFINED BEHAVIOR: shared2's foo has been // deleted already!! Ownership Transfer of shared_ptr By default, shared_ptr increments the reference count and doesn't transfer the
```

ownership. However, it can be made to transfer the ownership using `std::move`: `shared_ptr up = make_shared(); // Transferring the ownership shared_ptr up2 = move(up); // At this point, the reference count of up = 0 and the // ownership of the pointer is solely with up2 with reference count = 1` Section 33.3: Sharing with temporary ownership

(`std::weak_ptr`) Instances of `std::weak_ptr` can point to objects owned by instances of `std::shared_ptr` while only becoming temporary owners themselves. This means that weak pointers do not alter the object's reference count and therefore do not prevent an object's deletion if all of the object's shared pointers are reassigned or destroyed. In the following example instances of `std::weak_ptr` are used so that the destruction of a tree object is not inhibited: `#include #include struct TreeNode { std::weak_ptr parent; std::vector< std::shared_ptr > children; }; int main() { // Create a TreeNode to serve as the root/parent. std::shared_ptr root(new TreeNode); // Give the parent 100 child nodes. for (size_t i = 0; i < 100; ++i) { std::shared_ptr child(new TreeNode); root->children.push_back(child); child->parent = root; } // Reset the root shared pointer, destroying the root object, and // subsequently its child nodes. root.reset(); } As child nodes are added to the root node's children, their std::weak_ptr member parent is set to the root node. The member parent is declared as a weak pointer as opposed to a shared pointer such that the root node's reference count is not incremented. When the root node is reset at the end of main(), the root is destroyed. Since the only remaining std::shared_ptr references to the child nodes were contained in the root's collection children, all child nodes are subsequently destroyed as well. Due to control block implementation details, shared_ptr allocated memory may not be released until shared_ptr reference counter and weak_ptr reference counter both reach zero. #include int main() { { std::weak_ptr wk; { // std::make_shared is optimized by allocating only once // while std::shared_ptr(new int(42)) allocates twice. // Drawback of std::make_shared is that control block is tied to our integer std::shared_ptr sh = std::make_shared(42); wk = sh; // sh memory should be released at this point... } // ... but wk is still alive and needs access to control block } // now memory is released (sh and wk)`

} Since `std::weak_ptr` does not keep its referenced object alive, direct data access through a `std::weak_ptr` is not possible. Instead it provides a `lock()` member function that attempts to retrieve a `std::shared_ptr` to the referenced object: `#include #include int main() { { std::weak_ptr wk; std::shared_ptr sp; { std::shared_ptr sh = std::make_shared(42); wk = sh; // calling lock will create a shared_ptr to the object referenced by wk sp = wk.lock(); // sh will be destroyed after this point, but sp is still alive } // sp still keeps the data alive. // At this point we could even call lock() again // to retrieve another shared_ptr to the same data from wk assert(sp == 42); assert(!wk.expired()); // resetting sp will delete the data, // as it is currently the last shared_ptr with ownership sp.reset(); // attempting to lock wk now will return an empty shared_ptr, // as the data has already been deleted sp = wk.lock(); assert(!sp); assert(wk.expired()); } } Section 33.4: Using custom deleters to create a wrapper to a C interface Many C interfaces such as SDL2 have their own deletion functions. This means that you cannot use smart pointers directly: std::unique_ptr a; // won't work, UNSAFE! Instead, you need to define your own deleter. The examples here use the SDL_Surface structure which should be freed using the SDL_FreeSurface() function, but they should be adaptable to many other C interfaces. The deleter must be callable with a pointer argument, and therefore can be e.g. a simple function pointer: std::unique_ptr<SDL_Surface, void()>(SDL_Surface) a(pointer, SDL_FreeSurface); Any other callable object will work, too, for example a class with an operator(): struct SurfaceDeleter { void operator()(SDL_Surface* surf) { SDL_FreeSurface(surf); } };`

`std::unique_ptr<SDL_Surface, SurfaceDeleter> a(pointer, SurfaceDeleter{}); // safe std::unique_ptr<SDL_SurfaceDeleter> b(pointer); // equivalent to the above // as the deleter is value-initialized` This not only provides you with safe, zero overhead (if you use `unique_ptr`) automatic memory management, you also get exception safety. Note that the deleter is part of the type for `unique_ptr`, and the implementation can use the empty base optimization to avoid any change in size for empty custom deleters. So while `std::unique_ptr<SDL_Surface, SurfaceDeleter>`

and `std::unique_ptr<SDL_Surface, void()(SDL_Surface)>` solve the same problem in a similar way, the former type is still only the size of a pointer while the latter type has to hold two pointers: both the `SDL_Surface*` and the function pointer! When having free function custom deleters, it is preferable to wrap the function in an empty type. In cases where reference counting is important, one could use a `shared_ptr` instead of an `unique_ptr`. The `shared_ptr` always stores a deleter, this erases the type of the deleter, which might be useful in APIs. The disadvantages of using `shared_ptr` over `unique_ptr` include a higher memory cost for storing the deleter and a performance cost for maintaining the reference count. `// deleter required at construction time and is part of the type std::unique_ptr<SDL_Surface, void()(SDL_Surface)>`
`a(pointer, SDL_FreeSurface); // deleter is only required at construction time, not part of the type std::shared_ptr b(pointer, SDL_FreeSurface);` Version C++17 With template auto, we can make it even easier to wrap our custom deleters: `template struct FunctionDeleter { template void operator()(T* ptr) { DeleteFn(ptr); } }; template <class T, auto DeleteFn> using unique_ptr_deleter = std::unique_ptr<T, FunctionDeleter>;` With which the above example is simply: `unique_ptr_deleter<SDL_Surface, SDL_FreeSurface> c(pointer);` Here, the purpose of auto is to handle all free functions, whether they return void (e.g. `SDL_FreeSurface`) or not (e.g. `fclose`). Section 33.5: Unique ownership without move semantics (`auto_ptr`) Version < C++11 NOTE: `std::auto_ptr` has been deprecated in C++11 and will be removed in C++17. You should only use this if you are forced to use C++03 or earlier and are willing to be careful. It is recommended to move to `unique_ptr` in combination with `std::move` to replace `std::auto_ptr` behavior.

Before we had `std::unique_ptr`, before we had move semantics, we had `std::auto_ptr`. `std::auto_ptr` provides unique ownership but transfers ownership upon copy. As with all smart pointers, `std::auto_ptr` automatically cleans up resources (see RAII): `{ std::auto_ptr p(new int(42)); std::cout << p; } // p is deleted here, no memory leaked but allows only one owner: std::auto_ptr px = ...; std::auto_ptr py = px; // px is now empty This allows to use std::auto_ptr to keep ownership explicit and unique at the danger of losing ownership unintended: void f(std::auto_ptr) { // assumes ownership of X // deletes it at end of scope }; std::auto_ptr px = ...; f(px); // f acquires ownership of underlying X // px is now empty px->foo(); // NPE! // px.~auto_ptr() does NOT delete The transfer of ownership happened in the “copy” constructor. auto_ptr’s copy constructor and copy assignment operator take their operands by non-const reference so that they could be modied. An example implementation might be: template class auto_ptr { T ptr; public: auto_ptr(auto_ptr& rhs) : ptr(rhs.release()) { } auto_ptr& operator=(auto_ptr& rhs) { reset(rhs.release()); return this; } T release() { T* tmp = ptr; ptr = nullptr; return tmp; } void reset(T* tmp = nullptr) { if (ptr != tmp) { delete ptr; ptr = tmp; } }`

`/* other functions ... / }; This breaks copy semantics, which require that copying an object leaves you with two equivalent versions of it. For any copyable type, T, I should be able to write: T a = ...; T b(a); assert(b == a); But for auto_ptr, this is not the case. As a result, it is not safe to put auto_ptrs in containers. Section 33.6: Casting std::shared_ptr pointers It is not possible to directly use static_cast, const_cast, dynamic_cast and reinterpret_cast on std::shared_ptr to retrieve a pointer sharing ownership with the pointer being passed as argument. Instead, the functions std::static_pointer_cast, std::const_pointer_cast, std::dynamic_pointer_cast and std::reinterpret_pointer_cast should be used: struct Base { virtual ~Base() noexcept { }; }; struct Derived: Base { }; auto derivedPtr(std::make_shared()); auto basePtr(std::static_pointer_cast<DerivedPtr>()); auto constBasePtr(std::const_pointer_cast<BasePtr>()); auto constDerivedPtr(std::dynamic_pointer_cast<ConstBasePtr>()); Note that std::reinterpret_pointer_cast is not available in C++11 and C++14, as it was only proposed by N3920 and adopted into Library Fundamentals TS in February 2014. However, it can be implemented as follows: template <typename To, typename From> inline std::shared_ptr reinterpret_pointer_cast(std::shared_ptr const & ptr) noexcept { return std::shared_ptr(ptr, reinterpret_cast<To> (&ptr.get())); } Sec-`

tion 33.7: Writing a smart pointer: `value_ptr` A `value_ptr` is a smart pointer that behaves like a value. When copied, it copies its contents. When created, it creates its contents. // Like `std::default_delete`: `template struct default_copier { // a copier must handle a null T const* in and return null: T* operator()(T const* tin) const { if (!tin) return nullptr; return new T(tin); } void operator()(void dest, T const* tin) const { if (!tin) return; return new(dest) T(tin); } }; // tag class to handle empty case: struct empty_ptr_t {}; constexpr empty_ptr_t empty_ptr{}; // the value pointer type itself:`

```
template<class T, class Copier=default_copier, class Deleter=std::default_delete, class Base=std::unique_ptr<T, Deleter>> struct value_ptr:Base, private Copier { using copier_type=Copier; // also type-
defs from unique_ptr using Base::Base; value_ptr( T const& t ): Base( std::make_unique(t)
), Copier() {} value_ptr( T && t ): Base( std::make_unique(std::move(t)) ), Copier() {}
// almost-never-empty: value_ptr(): Base( std::make_unique() ), Copier() {} value_ptr(
empty_ptr_t ) {} value_ptr( Base b, Copier c={} ): Base(std::move(b)), Copier(std::move(c))
{} Copier const& get_copier() const { return this; } value_ptr clone() const { return { Base(
get_copier()(this->get()), this->get_deleter() ), get_copier() }; } value_ptr(value_ptr&&)=default;
value_ptr& operator=(value_ptr&&)=default; value_ptr(value_ptr const& o):value_ptr(o.clone())
{} value_ptr& operator=(value_ptr const&o) { if (o && this) { // if we are both non-null,
assign contents: this = o; } else { // otherwise, assign a clone (which could itself be
null): this = o.clone(); } return this; } value_ptr& operator=( T const& t ) { if
(this) { this = t; } else { this = value_ptr(t); }
```

```
return this; } value_ptr& operator=( T && t ) { if (this) { this = std::move(t); }
else { this = value_ptr(std::move(t)); } return this; } T& get() { return this;
} T const& get() const { return **this; } T* get_pointer() { if (!this) return nullptr; re-
turn std::addressof(get()); } T const get_pointer() const { if (!this) return nullptr; return
std::addressof(get()); } // operator-> from unique_ptr }; template<class T, class... Args>
value_ptr make_value_ptr( Args&&... args ) { return {std::make_unique(std::forward(args)...)};
} This particular value_ptr is only empty if you construct it with empty_ptr_t or if you move
from it. It exposes the fact it is a unique_ptr, so explicit operator bool() const works on it.
.get() has been changed to return a reference (as it is almost never empty), and .get_pointer()
returns a pointer instead. This smart pointer can be useful for pImpl cases, where we want
value-semantics but we also don't want to expose the contents of the pImpl outside of the imple-
mentation i.e. With a non-default Copier, it can even handle virtual base classes that know how
to produce instances of their derived and turn them into value-types. Section 33.8: Getting a
shared_ptr referring to this enable_shared_from_this enables you to get a valid shared_ptr in-
stance to this. By deriving your class from the class template enable_shared_from_this, you in-
herit a method shared_from_this that returns a shared_ptr instance to this. Note that the object
must be created as a shared_ptr in rst place: #include class A: public enable_shared_from_this
{ }; A ap1 =new A(); shared_ptr ap2(ap1); // First prepare a shared pointer to the object
and hold it! // Then get a shared pointer to the object from the object itself shared_ptr
ap3 = ap1->shared_from_this(); int c3 =ap3.use_count(); // =2: pointing to the same ob-
ject Note(2) you cannot call enable_shared_from_this inside the constructor. #include //
enable_shared_from_this
```

```
class Widget : public std::enable_shared_from_this< Widget > { public: void DoSome-
thing() { std::shared_ptr< Widget > self = shared_from_this(); someEvent -> Register( self
); } private: }; int main() { auto w = std::make_shared< Widget >(); w -> DoSomething();
} If you use shared_from_this() on an object not owned by a shared_ptr, such as a local
automatic object or a global object, then the behavior is undened. Since C++17 it throws
std::bad_alloc instead. Using shared_from_this() from a constructor is equivalent to using it
on an object not owned by a shared_ptr, because the objects is possessed by the shared_ptr
after the constructor returns.
```

Chapter 84

Professional Notes: Chapter 106: Move Semantics

Section 106.1: Move semantics Move semantics are a way of moving one object to another in C++. For this, we empty the old object and place everything it had in the new object. For this, we must understand what an rvalue reference is. An rvalue reference ($T\&\&$ where T is the object type) is not much different than a normal reference ($T\&$, now called lvalue references). But they act as 2 different types, and so, we can make constructors or functions that take one type or the other, which will be necessary when dealing with move semantics. The reason why we need two different types is to specify two different behaviors. Lvalue reference constructors are related to copying, while rvalue reference constructors are related to moving. To move an object, we will use `std::move(obj)`. This function returns an rvalue reference to the object, so that we can steal the data from that object into a new one. There are several ways of doing this which are discussed below. Important to note is that the use of `std::move` creates just an rvalue reference. In other words the statement `std::move(obj)` does not change the content of `obj`, while `auto obj2 = std::move(obj)` (possibly) does. Section 106.2: Using `std::move` to reduce complexity from $O(n)$ to $O(1)$ C++11 introduced core language and standard library support for moving an object. The idea is that when an object `o` is a temporary and one wants a logical copy, then it's safe to just pilfer `o`'s resources, such as a dynamically allocated buffer, leaving `o` logically empty but still destructible and copyable. The core language support is mainly the rvalue reference type builder `&&`, e.g., `std::string&&` is an rvalue reference to a `std::string`, indicating that that referred to object is a temporary whose resources can just be pilfered (i.e. moved) special support for a move constructor `T( T&& )`, which is supposed to efficiently move resources from the specified other object, instead of actually copying those resources, and special support for a move assignment operator `auto operator=(T&&) -> T&`, which also is supposed to move from the source. The standard library support is mainly the `std::move` function template from the header. This function produces an rvalue reference to the specified object, indicating that it can be moved from, just as if it were a temporary. For a container actual copying is typically of $O(n)$ complexity, where n is the number of items in the container, while moving is $O(1)$, constant time. And for an algorithm that logically copies that container n times, this can reduce the complexity from the usually impractical $O(n)$ to just linear $O(n)$. In his article Containers That Never Change in Dr. Dobbs Journal in September 19 2013, Andrew Koenig presented an interesting example of algorithmic inefficiency when using a style of programming where variables are immutable after initialization. With this style loops are generally expressed using recursion. And for some algorithms such as generating a Collatz sequence, the recursion requires logically copying a container: // Based on an example by Andrew Koenig in his Dr. Dobbs Journal article

// Containers That Never Change September 19, 2013, available at // <url: <http://www.drdobbs.com/cpp/that-never-change/240161543>> // Includes here, e.g. namespace my { template< class Item

```
> using Vector_ = /* E.g. std::vector */; auto concat( Vector_ const& v, int const x ) ->
Vector_ { auto result{ v }; result.push_back( x ); return result; } auto collatz_aux( int const
n, Vector_ const& result ) -> Vector_ { if( n == 1 ) { return result; } auto const new_result
= concat( result, n ); if( n % 2 == 0 ) { return collatz_aux( n/2, new_result ); } else { return
collatz_aux( 3n + 1, new_result ); } } auto collatz( int const n ) -> Vector_ { assert( n !=
0 ); return collatz_aux( n, Vector_() ); } } // namespace my #include using namespace std;
auto main() -> int { for( int const x : my::collatz( 42 ) ) { cout << x << ' '; } cout << '\n'; } Output:
42 21 64 32 16 8 4 2 The number of item copy operations due to copying of the vectors is here
roughly O(n), since it's the sum 1 + 2 + 3 + ... n.
```

In concrete numbers, with g++ and Visual C++ compilers the above invocation of `collatz(42)` resulted in a Collatz sequence of 8 items and 36 item copy operations ($87/2 = 28$, plus some) in vector copy constructor calls. All of these item copy operations can be removed by simply moving vectors whose values are not needed anymore. To do this it's necessary to remove `const` and reference for the vector type arguments, passing the vectors by value. The function returns are already automatically optimized. For the calls where vectors are passed, and not used again further on in the function, just apply `std::move` to move those buers rather than actually copying them: `using std::move; auto concat(Vector_ v, int const x) -> Vector_ { v.push_back(x); // warning: moving a local object in a return statement prevents copy elision [-Wpessimizing-move] // See https://stackoverflow.com/documentation/c%2b%2b/2489/copy-elision // return move(v); return v; } auto collatz_aux(int const n, Vector_ result) -> Vector_ { if(n == 1) { return result; } auto new_result = concat(move(result), n); struct result; // Make absolutely sure no use of result after this. if(n % 2 == 0) { return collatz_aux(n/2, move(new_result)); } else { return collatz_aux(3n + 1, move(new_result)); } } auto collatz(int const n) -> Vector_ { assert(n != 0); return collatz_aux(n, Vector_()); } Here, with g++ and Visual C++ compilers, the number of item copy operations due to vector copy constructor invocations, was exactly 0. The algorithm is necessarily still $O(n)$ in the length of the Collatz sequence produced, but this is a quite dramatic improvement: $O(n)$ $O(n)$. With some language support one could perhaps use moving and still express and enforce the immutability of a variable between its initialization and nal move, after which any use of that variable should be an error. Alas, as of C++14 C++ does not support that. For loop-free code the no use after move can be enforced via a re-declaration of the relevant name as an incomplete struct, as with struct result; above, but this is ugly and not likely to be understood by other programmers; also the diagnostics can be quite misleading.`

Summing up, the C++ language and library support for moving allows drastic improvements in algorithm complexity, but due the support's incompleteness, at the cost of forsaking the code correctness guarantees and code clarity that `const` can provide. For completeness, the instrumented vector class used to measure the number of item copy operations due to copy constructor invocations: `template< class Item > class Copy_tracking_vector { private: static auto n_copy_ops() -> int& { static int value; return value; } vector items_; public: static auto n() -> int { return n_copy_ops(); } void push_back(Item const& o) { items_.push_back(o); } auto begin() const { return items_.begin(); } auto end() const { return items_.end(); } Copy_tracking_vector(){} Copy_tracking_vector(Copy_tracking_vector const& other) : items_(other.items_) { n_copy_ops() += items_.size(); } Copy_tracking_vector(Copy_tracking_vector&& other) : items_(move(other.items_)) {} }; Section 106.3: Move constructor Say we have this code snippet. class A { public: int a; int b; A(const A &other) { this->a = other.a; this->b = other.b; } }; To create a copy constructor, that is, to make a function that copies an object and creates a new one, we normally would choose the syntax shown above, we would have a constructor for A that takes an reference to another object of type A, and we would copy the object manually inside the method. Alternatively, we could have written A(const A &) = default; which automatically copies over all members,`

making use of its copy constructor. To create a move constructor, however, we will be

taking an rvalue reference instead of an lvalue reference, like here. `class Wallet { public: int nrOfDollars; Wallet() = default; //default ctor Wallet(Wallet &&other) { this->nrOfDollars = other.nrOfDollars; other.nrOfDollars = 0; } };` Please notice that we set the old values to zero. The default move constructor (`Wallet(Wallet&&) = default;`) copies the value of `nrOfDollars`, as it is a POD. As move semantics are designed to allow 'stealing' state from the original instance, it is important to consider how the original instance should look like after this stealing. In this case, if we would not change the value to zero we would have doubled the amount of dollars into play. `Wallet a; a.nrOfDollars = 1; Wallet b (std::move(a)); //calling B(B&& other); std::cout << a.nrOfDollars << std::endl; //0 std::cout << b.nrOfDollars << std::endl; //1` Thus we have move constructed an object from an old one. While the above is a simple example, it shows what the move constructor is intended to do. It becomes more useful in more complex cases, such as when resource management is involved. `// Manages operations involving a specified type. // Owns a helper on the heap, and one in its memory (presumably on the stack). // Both helpers are DefaultConstructible, CopyConstructible, and MoveConstructible. template<typename T, template typename HeapHelper, template typename StackHelper> class OperationsManager { using MyType = OperationsManager<T, HeapHelper, StackHelper>; HeapHelper* h_helper; StackHelper s_helper; // ... public: // Default constructor & Rule of Five. OperationsManager() : h_helper(new HeapHelper) {} OperationsManager(const MyType& other) : h_helper(new HeapHelper(other.h_helper)), s_helper(other.s_helper) {} MyType& operator=(MyType copy) { swap(this, copy); return this; } ~OperationsManager() { if (h_helper) { delete h_helper; }`

`} // Move constructor (without swap()). // Takes other's HeapHelper. // Takes other's StackHelper, by forcing the use of StackHelper's move constructor. // Replaces other's HeapHelper* with nullptr, to keep other from deleting our shiny // new helper when it's destroyed. OperationsManager(MyType&& other) noexcept : h_helper(other.h_helper), s_helper(std::move(other.s_helper)) { other.h_helper = nullptr; } // Move constructor (with swap()). // Places our members in the condition we want other's to be in, then switches members // with other. // OperationsManager(MyType&& other) noexcept : h_helper(nullptr) { // swap(this, other); // } // Copy/move helper. friend void swap(MyType& left, MyType& right) noexcept { std::swap(left.h_helper, right.h_helper); std::swap(left.s_helper, right.s_helper); } };` *Section 106.4: Re-use a moved object* You can re-use a moved object: `void consumingFunction(std::vector vec) { // Some operations } int main() { // initialize vec with 1, 2, 3, 4 std::vector vec{1, 2, 3, 4}; // Send the vector by move consumingFunction(std::move(vec)); // Here the vec object is in an indeterminate state. // Since the object is not destroyed, we can assign it a new content. // We will, in this case, assign an empty value to the vector, // making it effectively empty vec = {}; // Since the vector as gained a determinate value, we can use it normally. vec.push_back(42); // Send the vector by move again. consumingFunction(std::move(vec)); }` *Section 106.5: Move assignment* Similarly to how we can assign a value to an object with an lvalue reference, copying it, we can also move the values from an object to another without constructing a new one. We call this move assignment. We move the values from

one object to another existing object. For this, we will have to overload operator =, not so that it takes an lvalue reference, like in copy assignment, but so that it takes an rvalue reference. `class A { int a; A&& operator= (A&& other) { this->a = other.a; other.a = 0; return this; } };` This is the typical syntax to define move assignment. We overload operator = so that we can feed it an rvalue reference and it can assign it to another object. `A a; a.a = 1; A b; b = std::move(a); //calling A& operator= (A&& other) std::cout << a.a << std::endl; //0 std::cout << b.a << std::endl; //1` Thus, we can move assign an object to another one. *Section 106.6: Using move semantics on containers* You can move a container instead of copying it: `void print(const std::vector& vec) { for (auto&& val : vec) { std::cout << val << ","; } std::cout << std::endl; } int main() { // initialize vec1 with 1, 2, 3, 4 and vec2 as an empty vector std::vector vec1{1, 2, 3, 4}; std::vector vec2; // The following line will print 1, 2, 3, 4 print(vec1); // The following`


```
line will print a new line print(vec2); // The vector vec2 is assigned with move assingment.  
// This will “steal” the value of vec1 without copying it. vec2 = std::move(vec1); // Here the  
vec1 object is in an indeterminate state, but still valid. // The object vec1 is not destroyed, //  
but there’s is no guarantees about what it contains. // The following line will print 1, 2, 3, 4  
print(vec2);  
}
```

Chapter 85

FUNCTIONAL PROGRAMMING

Chapter 86

FUNCTIONAL PROGRAMMING IN C++11

86.1 1. Lambda Expressions

Anonymous functions defined inline.

86.1.1 1.1 Basic Syntax

[captures](params) -> return_type \{ body \}

```
1 auto add = [](int a, int b) { return a + b; };  
2 int result = add(2, 3);
```

86.1.2 1.2 Capture Lists

Controls access to outer scope variables.

- []: No capture.
- [x]: Capture x by value (copy).
- [&x]: Capture x by reference.
- [=]: Capture all by value.
- [&]: Capture all by reference.
- [this]: Capture class members.

```
1 int x = 10;  
2 auto addX = [x](int y) { return x + y; }; // x is read-only inside
```

86.1.3 1.3 Mutable Lambdas

By default, value captures are const. mutable allows modification.

```
1 int x = 0;  
2 auto increment = [x]() mutable { return ++x; }; // Modifies local copy
```

86.2 2. `std::function`

A polymorphic wrapper for any callable (function pointer, lambda, functor).

```
1 #include <functional>
2
3 void freeFunc(int) {}
4
5 std::function<void(int)> f;
6 f = freeFunc;
7 f = [](int x) {};
```

Cost: Can incur heap allocation and virtual call overhead.

86.3 3. `std::bind`

Binds arguments to function parameters (Partial Application).

```
1 int add(int a, int b) { return a + b; }
2
3 // Creates a function taking 1 argument (placeholder _1)
4 auto add5 = std::bind(add, 5, std::placeholders::_1);
5 // add5(10) calls add(5, 10)
```

Note: Lambdas largely replace `std::bind` in modern C++ due to readability and optimization.

Chapter 87

Professional Notes: Chapter 52: std::function: To wrap any element that is callable

Section 52.1: Simple usage Section 52.2: std::function used with std::bind Section 52.3: Binding std::function to a different callable types Section 52.4: Storing function arguments in std::tuple Section 52.5: std::function with lambda and std::bind Section 52.6: `function` overhead

Chapter 88

Professional Notes: Chapter 73: Lambdas

Section 73.1: What is a lambda expression? Section 73.2: Specifying the return type Section 73.3: Capture by value Section 73.4: Recursive lambdas Section 73.5: Default capture Section 73.6: Class lambdas and capture of this Section 73.7: Capture by reference Section 73.8: Generic lambdas Section 73.9: Using lambdas for inline parameter pack unpacking Section 73.10: Generalized capture Section 73.11: Conversion to function pointer Section 73.12: Porting lambda functions to C++03 using functors

Chapter 89

Professional Notes: Chapter 52: std::function: To wrap any

element that is callable Section 52.1: Simple usage `#include <functional>` `#include <string>` `std::function<void(int, const std::string&)> myFuncObj; void theFunc(int i, const std::string& s) { std::cout << s << " " << i << std::endl; }` `int main(int argc, char *argv[]) { myFuncObj = theFunc; myFuncObj(10, "hello world"); }` Section 52.2: `std::function` used with `std::bind` Think about a situation where we need to callback a function with arguments. `std::function` used with `std::bind` gives a very powerful design construct as shown below. `class A { public: std::function<void(int, const std::string&)> m_CbFunc = nullptr; void foo() { if (m_CbFunc) { m_CbFunc(100, "event fired"); } } }; class B { public: B() { auto aFunc = std::bind(&B::eventHandler, this, std::placeholders::_1, std::placeholders::_2); anObjA.m_CbFunc = aFunc; } void eventHandler(int i, const std::string& s) { std::cout << s << " " << i << std::endl; } void DoSomethingOnA() { anObjA.foo(); } A anObjA; };`

`int main(int argc, char *argv[]) { B anObjB; anObjB.DoSomethingOnA(); }` Section 52.3: *Binding std::function to a different callable types* / * This example show some ways of using `std::function` to call * a) C-like function * b) class-member function * c) operator() * d) lambda function Function call can be made: * a) with right arguments * b) arguments with different order, types and count */ `#include <functional>` `#include <string>` `#include <iostream>` `using namespace std;` `double foo_fn(int x, float y, double z) { double res = x + y + z; std::cout << "foo_fn called with arguments: " << x << " " << y << " " << z << " result is : " << res << std::endl; return res; }` // structure with member function to call `struct foo_struct { // member function to call double foo_fn(int x, float y, double z) { double res = x + y + z; std::cout << "foo_struct::foo_fn called with arguments: " << x << " " << y << " " << z << " result is : " << res << std::endl; return res; } // this member function has different signature - but it can be used too // please not that argument order is changed too double foo_fn_4(int x, double z, float y, long xx) { double res = x + y + z + xx; std::cout << "foo_struct::foo_fn_4 called with arguments: "`

`<< x << " " << y << " " << z << " " << xx << " result is : " << res << std::endl; return res; }` // overloaded operator() makes whole object to be callable `double operator()(int x, float y, double z) { double res = x + y + z; std::cout << "foo_struct::operator() called with arguments: " << x << " " << y << " " << z << " result is : " << res << std::endl; return res; } }; int main(void) { // typedefs using function_type = std::function<double(int, float, double)>; // foo_struct instance foo_struct fs; // here we will store all binded functions std::vector<function_type> bindings; // var #1 - you can use simple function function_type var1 = foo_fn; bindings.push_back(var1); // var #2 - you can use member function function_type var2 = std::bind(&foo_struct::foo_fn, fs, _1, _2, _3); bindings.push_back(var2); // var #3 - you can use member function with different signature // foo_fn_4 has different count of arguments and types function_type var3 = std::bind(&foo_struct::foo_fn_4, fs, _1, _3, _2, 0l); bindings.push_back(var3); // var #4 -`

you can use object with overloaded operator() function_type var4 = fs; bindings.push_back(var4);
// var #5 - you can use lambda function function_type var5 = { double res = x + y + z;
std::cout << "lambda called with arguments:" << x << "," << y << "," << z << " result is : " << res
<< std::endl; return res; }; bindings.push_back(var5); std::cout << "Test stored functions with
arguments: x = 1, y = 2, z = 3" << std::endl; for (auto f : bindings)

f(1, 2, 3); } Live Output: Test stored functions with arguments: x = 1, y = 2, z = 3
foo_fn called with arguments: 1, 2, 3 result is : 6 foo_struct::foo_fn called with arguments:
1, 2, 3 result is : 6 foo_struct::foo_fn_4 called with arguments: 1, 3, 2, 0 result is : 6
foo_struct::operator() called with arguments: 1, 2, 3 result is : 6 lambda called with arguments:
1, 2, 3 result is : 6 Section 52.4: Storing function arguments in std::tuple Some programs need so
store arguments for future calling of some function. This example shows how to call any function
with arguments stored in std::tuple #include #include #include #include // simple function
to be called double foo_fn(int x, float y, double z) { double res = x + y + z; std::cout << "foo_fn
called. x =" << x << " y =" << y << " z =" << z << " res=" << res; return res; } // helpers for tuple
unrolling template<int ...> struct seq {}; template<int N, int ...S> struct gens : gens<N-1,
N-1, S...> {}; template<int ...S> struct gens<0, S...> { typedef seq<S...> type; }; // in-
vocation helper template<typename FN, typename P, int ...S> double call_fn_internal(const
FN& fn, const P& params, const seq<S...>) { return fn(std::get(params) ...); } // call func-
tion with arguments stored in std::tuple template<typename Ret, typename ...Args> Ret
call_fn(const std::function<Ret(Args...)>& fn, const std::tuple<Args...>& params) { return
call_fn_internal(fn, params, typename gens<sizeof...(Args)>::type()); } int main(void) { //
arguments std::tuple<int, float, double> t = std::make_tuple(1, 5, 10); // function to call

std::function<double(int, float, double)> fn = foo_fn; // invoke a function with stored
arguments call_fn(fn, t); } Live Output: foo_fn called. x = 1 y = 5 z = 10 res=16 Section
52.5: std::function with lambda and std::bind #include #include using std::placeholders::_1;
// to be used in std::bind example int stdf_foobar (int x, std::function<int(int)> moo) { return
x + moo(x); // std::function moo called } int foo (int x) { return 2+x; } int foo_2 (int x, int y)
{ return 9x + y; } int main() { int a = 2; / Function pointers / std::cout << stdf_foobar(a, &foo)
<< std::endl; // 6 (2 + (2+2)) // can also be: stdf_foobar(2, foo) / Lambda expressions //
An unnamed closure from a lambda expression can be * stored in a std::function object: / int
capture_value = 3; std::cout << stdf_foobar(a, *capture_value* -> int { return 7 + capture_value
param; }) << std::endl; // result: 15 == value + (7 * capture_value * value) == 2 + (7 + 3 * 2)
/* std::bind expressions // The result of a std::bind expression can be passed. * For example
by binding parameters to a function pointer call: */
int b = stdf_foobar(a, std::bind(foo_2, _1, 3)); std::cout << b << std::endl; // b == 23 == 2 +
(9*2 + 3) int c = stdf_foobar(a, std::bind(foo_2, 5, _1)); std::cout << c << std::endl; // c ==
49 == 2 + (9*5 + 2) return 0; }

Section 52.6: **function** overhead std::function can cause significant overhead. Because std::function
has [value semantics][1], it must copy or move the given callable into itself. But since it can take
callables of an arbitrary type, it will frequently have to allocate memory dynamically to do this.
Some function implementations have so-called “small object optimization”, where small types
(like function pointers, member pointers, or functors with very little state) will be stored directly
in the function object. But even this only works if the type is noexcept move constructible. Fur-
thermore, the C++ standard does not require that all implementations provide one. Consider
the following: //Header file using MyPredicate = std::function<bool(const MyValue &, const
MyValue &)>; void SortMyContainer(MyContainer &C, const MyPredicate &pred); //Source
file void SortMyContainer(MyContainer &C, const MyPredicate &pred) { std::sort(C.begin(),
C.end(), pred); } A template parameter would be the preferred solution for SortMyContainer,
but let us assume that this is not possible or desirable for whatever reason. SortMyContainer
does not need to store pred beyond its own call. And yet, pred may well allocate memory if
the functor given to it is of some non-trivial size. function allocates memory because it needs

something to copy/move into; function takes ownership of the callable it is given. But `SortMyContainer` does not need to own the callable; it's just referencing it. So using function here is overkill; it may be efficient, but it may not. There is no standard library function type that merely references a callable. So an alternate solution will have to be found, or you can choose to live with the overhead. Also, function has no effective means to control where the memory allocations for the object come from. Yes, it has constructors that take an allocator, but [many implementations do not implement them correctly... or even at all][2]. Version C++17 The function constructors that take an allocator no longer are part of the type. Therefore, there is no way to manage the allocation. Calling a function is also slower than calling the contents directly. Since any function instance could hold any callable, the call through a function must be indirect. The overhead of calling function is on the order of a virtual function call.

Chapter 90

Professional Notes: Chapter 73: Lambdas

Parameter default-capture Details Species how all non-listed variables are captured. Can be = (capture by value) or & (capture by reference). If omitted, non-listed variables are inaccessible within the lambda-body. The default- capture must precede the capture-list. capture-list Species how local variables are made accessible within the lambda-body. Variables without prex are captured by value. Variables prexed with & are captured by reference. Within a class method, this can be used to make all its members accessible by reference. Non-listed variables are inaccessible, unless the list is preceded by a default-capture. argument-list Species the arguments of the lambda function. mutable (optional) Normally variables captured by value are const. Specifying mutable makes them non- const. Changes to those variables are retained between calls. throw-specication (optional) Species the exception throwing behavior of the lambda function. For example: noexcept or throw(std::exception). attributes (optional) Any attributes for the lambda function. For example, if the lambda-body always throws an exception then [[noreturn]] can be used. -> return-type (optional) Species the return type of the lambda function. Required when the return type cannot be determined by the compiler. lambda-body A code block containing the implementation of the lambda function. Section 73.1: What is a lambda expression? A lambda expression provides a concise way to create simple function objects. A lambda expression is a prvalue whose result object is called closure object, which behaves like a function object. The name ‘lambda expression’ originates from lambda calculus, which is a mathematical formalism invented in the 1930s by Alonzo Church to investigate questions about logic and computability. Lambda calculus formed the basis of LISP, a functional programming language. Compared to lambda calculus and LISP, C++ lambda expressions share the properties of being unnamed, and to capture variables from the surrounding context, but they lack the ability to operate on and return functions. A lambda expression is often used as an argument to functions that take a callable object. That can be simpler than creating a named function, which would be only used when passed as the argument. In such cases, lambda expressions are generally preferred because they allow dening the function objects inline. A lambda consists typically of three parts: a capture list [], an optional parameter list () and a body {}, all of which can be empty: // An empty lambda, which does and returns nothing Capture list [] is the capture list. By default, variables of the enclosing scope cannot be accessed by a lambda. Capturing a variable makes it accessible inside the lambda, either as a copy or as a reference. Captured variables become a part of the lambda; in contrast to function arguments, they do not have to be passed when calling the lambda. int a = 0; // Define an integer variable auto f = { return a9; }; // Error: ‘a’ cannot be accessed auto f = a { return a9; }; // OK, ‘a’ is “captured” by value auto f = &a { return a++; }; // OK, ‘a’ is “captured” by reference // Note: It is the responsibility of the programmer // to ensure that a is not destroyed before the

Chapter 91

TEMPLATE METAPROGRAMMING

Chapter 92

TEMPLATE METAPROGRAMMING & POWER FEATURES

92.1 1. Variadic Templates

Templates that accept an arbitrary number of parameters.

```
1 // Base case (recursion terminator)
2 void print() {}
3
4 // Recursive step
5 template<typename T, typename... Args>
6 void print(T first, Args... args) {
7     std::cout << first << " ";
8     print(args...); // Unpack
9 }
10
11 int main() {
12     print(1, "hello", 3.14);
13 }
```

92.1.1 Professional Notes: Metaprogramming Depth

1. Recursive Template Evaluation

Templates are effectively a functional language evaluated at compile time. * **Base Case:** Essential to stop the infinite recursion (as seen in `print()`). * **Arithmetic Metaprogramming:** Computing values at compile time using constant expressions and template specialization.

```
1 template<int N>
2 struct Factorial {
3     static const int value = N * Factorial<N - 1>::value;
4 };
5 template<>
6 struct Factorial<0> {
7     static const int value = 1;
8 };
9 // Factorial<5>::value is computed as 120 by the compiler.
```

2. Advanced Parameter Packs

- **Fold Expressions (C++17):** Binary operators can be applied to all elements of a pack without manual recursion:

```

1 template<typename... Args>
2 auto sum(Args... args) {
3     return (... + args); // Unary left fold
4 }

```

- **Perfect Forwarding:** Use `std::forward<Args>(args)...` when passing packs to another function to preserve lvalue/rvalue properties.

3. SFINAE (Substitution Failure Is Not An Error)

A core principle of C++ templates. If a template argument substitution results in an invalid type or expression, the compiler doesn't throw an error it simply discards that overload. * **std::void_t (C++17):** A powerful helper for creating traits that check for the existence of members or types within a class.

92.1.2 1.1 Parameter Packing (...)

- `typename... Args:` Template parameter pack.
 - `Args... args:` Function parameter pack.
 - `sizeof...(Args):` Number of arguments.
-

92.2 2. Type Traits

Compile-time type inspection and modification.

```

1 #include <type_traits>
2
3 static_assert(std::is_integral<int>::value, "Must be int");
4 static_assert(std::is_pointer<int*>::value, "Must be ptr");
5
6 // Conditional compilation (SFINAE)
7 template<typename T>
8 typename std::enable_if<std::is_integral<T>::value>::type
9 process(T x) {
10     // Only compiles for integers
11 }

```

92.3 3. using Aliases

Replaces typedef, works with templates.

```
1 template<typename T>
2 using Dictionary = std::map<std::string, T>;
3
4 Dictionary<int> scores;
```

92.4 4. std::tuple

Generalization of std::pair to N elements.

```
1 #include <tuple>
2
3 std::tuple<int, double, std::string> t(1, 3.14, "hi");
4 int i = std::get<0>(t);
```

Chapter 93

Professional Notes: Chapter 13: C++ Streams

Section 13.1: String streams `std::ostringstream` is a class whose objects look like an output stream (that is, you can write to them via operator«), but actually store the writing results, and provide them in the form of a stream. Consider the following short code: `#include #include`

```
using namespace std; int main() { ostringstream ss; ss « "the answer to everything is" « 42;
const string result = ss.str(); }
```

The line `ostringstream ss;` creates such an object. This object is rst manipulated like a regular stream: `ss « "the answer to everything is" « 42;` Following that, though, the resulting stream can be obtained like this: `const string result = ss.str();` (the string result will be equal to "the answer to everything is 42"). This is mainly useful when we have a class for which stream serialization has been denied, and for which we want a string form. For example, suppose we have some class `foo` {
// All sort of stuff here. };

`ostream &operator«(ostream &os, const foo &f);` To get the string representation of a `foo` object, `foo f;` we could use `ostringstream ss; ss « f; const string result = ss.str();`

Then `result` contains the string representation of the `foo` object. Section 13.2: Printing collections with `ostream` Basic printing `std::ostream_iterator` allows to print contents of an STL container to any output stream without explicit loops. The second argument of `std::ostream_iterator` constructor sets the delimiter. For example, the following code: `std::vector v = {1,2,3,4}; std::copy(v.begin(), v.end(), std::ostream_iterator(std::cout, " ! "));` will print `1 ! 2 ! 3 ! 4 !`. Implicit type cast `std::ostream_iterator` allows to cast container's content type implicitly. For example, let's tune `std::cout` to print floating-point values with 3 digits after decimal point: `std::cout « std::setprecision(3); std::fixed(std::cout);` and instantiate `std::ostream_iterator` with `float`, while the contained values remain `int`: `std::vector v = {1,2,3,4}; std::copy(v.begin(), v.end(), std::ostream_iterator(std::cout, " ! "));` so the code above yields `1.000 ! 2.000 ! 3.000 ! 4.000 !` despite `std::vector` holds `ints`. Generation and transformation `std::generate`, `std::generate_n` and `std::transform` functions provide a very powerful tool for on-the-y data manipulation. For example, having a vector: `std::vector v = {1,2,3,4,8,16};` we can easily print boolean value of "x is even" statement for each element: `std::boolalpha(std::cout); // print booleans alphabetically std::transform(v.begin(), v.end(), std::ostream_iterator(std::cout, " "), { return (val % 2) == 0; });` or print the squared element: `std::transform(v.begin(), v.end(), std::ostream_iterator(std::cout, " "), { return val * val;`

`});` Printing `N` space-delimited random numbers: `const int N = 10; std::generate_n(std::ostream_iterator(std::cout, " ", N, std::rand));` Arrays As in the section about reading text les, almost all these considerations may be applied to native arrays. For example, let's print squared values from a native array:


```
int v[] = {1,2,3,4,8,16}; std::transform(v, std::end(v), std::ostream_iterator(std::cout," "), {  
return val * val; });
```

Chapter 94

Professional Notes: Chapter 16: Metaprogramming

Section 16.1: Calculating Factorials Section 16.2: Iterating over a parameter pack Section 16.3: Iterating with `std::integer_sequence` Section 16.4: Tag Dispatching Section 16.5: Detect Whether Expression is Valid Section 16.6: If-then-else Section 16.7: Manual distinction of types when given any type T Section 16.8: Calculating power with C++11 (and higher) Section 16.9: Generic Min/Max with variable argument count

Chapter 95

Professional Notes: Chapter 16: Metaprogramming

In C++ Metaprogramming refers to the use of macros or templates to generate code at compile-time. In general, macros are frowned upon in this role and templates are preferred, although they are not as generic. Template metaprogramming often makes use of compile-time computations, whether via templates or constexpr functions, to achieve its goals of generating code, however compile-time computations are not metaprogramming per se. Section 16.1: Calculating Factorials Factorials can be computed at compile-time using template metaprogramming techniques. `#include <template> struct factorial { enum { value = n * factorial<n - 1>::value }; }; template<> struct factorial<0> { enum { value = 1 }; }; int main() { std::cout << factorial<7>::value << std::endl; // prints "5040" }` factorial is a struct, but in template metaprogramming it is treated as a template metafunction. By convention, template metafunctions are evaluated by checking a particular member, either `::type` for metafunctions that result in types, or `::value` for metafunctions that generate values. In the above code, we evaluate the factorial metafunction by instantiating the template with the parameters we want to pass, and using `::value` to get the result of the evaluation. The metafunction itself relies on recursively instantiating the same metafunction with smaller values. The `factorial<0>` specialization represents the terminating condition. Template metaprogramming has most of the restrictions of a functional programming language, so recursion is the primary "looping" construct. Since template metafunctions execute at compile time, their results can be used in contexts that require compile-time values. For example: `int my_array[factorial<5>::value];` Automatic arrays must have a compile-time defined size. And the result of a metafunction is a compile-time constant, so it can be used here. Limitation: Most of the compilers won't allow recursion depth beyond a limit. For example, g++ compiler by default

limits recursion depth to 256 levels. In case of g++, programmer can set recursion depth using `-ftemplate-depth-X` option. Version C++11 Since C++11, the `std::integral_constant` template can be used for this kind of template computation: `#include <template> struct factorial : std::integral_constant<long long, n * factorial<n - 1>::value> {}; template<> struct factorial<0> : std::integral_constant<long long, 1> {}; int main() { std::cout << factorial<7>::value << std::endl; // prints "5040" }` Additionally, constexpr functions become a cleaner alternative. `#include <constexpr> long long factorial(long long n) { return (n == 0) ? 1 : n * factorial(n - 1); } int main() { char test[factorial(3)]; std::cout << factorial(7) << " "; }` The body of `factorial()` is written as a single statement because in C++11 constexpr functions can only use a quite limited subset of the language. Version C++14 Since C++14, many restrictions for constexpr functions have been dropped and they can now be written much more conveniently: `constexpr long long factorial(long long n) { if (n == 0) return 1; else return n * factorial(n - 1); }` Or even: `constexpr long long factorial(int n) { long long result = 1; for (int i = 1; i <= n; ++i) { result = i; } return result; }` Version C++17 Since c++17 one

can use fold expression to calculate factorial: `#include #include template <class T, T N, class I = std::make_integer_sequence<T, N> struct factorial; template <class T, T N, T... Is> struct factorial<T,N,std::index_sequence<T, Is...> { static constexpr T value = (static_cast(1) ... * (Is + 1)); }; int main() { std::cout << factorial<int, 5>::value << std::endl; }` Section 16.2: Iterating over a parameter pack Often, we need to perform an operation over every element in a variadic template parameter pack. There are many ways to do this, and the solutions get easier to read and write with C++17. Suppose we simply want to print every element in a pack. The simplest solution is to recurse: Version C++11 `void print_all(std::ostream& os) { // base case } template <class T, class... Ts> void print_all(std::ostream& os, T const& first, Ts const&... rest) { os << first; print_all(os, rest...); }` We could instead use the expander trick, to perform all the streaming in a single function. This has the advantage of not needing a second overload, but has the disadvantage of less than stellar readability: Version C++11 `template <class... Ts> void print_all(std::ostream& os, Ts const&... args) { using expander = int[]; (void)expander{0, (void(os << args), 0)... }; }` For an explanation of how this works, see T.C's excellent answer. Version C++17 With C++17, we get two powerful new tools in our arsenal for solving this problem. The first is a fold-expression:

```
template <class... Ts> void print_all(std::ostream& os, Ts const&... args) { ((os << args), ...); }
And the second is if constexpr, which allows us to write our original recursive solution in a single function:
template <class T, class... Ts> void print_all(std::ostream& os, T const& first, Ts const&... rest) { os << first; if constexpr (sizeof...(rest) > 0) {
// this line will only be instantiated if there are further // arguments. if rest... is empty,
there will be no call to // print_all(os). print_all(os, rest...); } }
Section 16.3: Iterating with std::integer_sequence
Since C++14, the standard provides the class template template <class T, T... Ints> class integer_sequence;
template <std::size_t... Ints> using index_sequence = std::integer_sequence<std::size_t, Ints...>;
and a generating metafunction for it: template <class T, T N> using make_integer_sequence = std::integer_sequence<T, /* a sequence 0, 1, 2, ..., N-1 />;
template using make_index_sequence = make_integer_sequence<std::size_t, N>;
While this comes standard in C++14, this can be implemented using C++11 tools. We can use this tool to call a function with a std::tuple of arguments (standardized in C++17 as std::apply):
namespace detail { template <class F, class Tuple, std::size_t... Is> decltype(auto) apply_impl(F&& f, Tuple&& tpl, std::index_sequence<Is...>) { return std::forward(f)(std::get<std::forward(Tuple)>(tpl)...); }
} template <class F, class Tuple> decltype(auto) apply(F&& f, Tuple&& tpl) { return detail::apply_impl(std::forward(f), std::forward(tpl), std::make_index_sequence<std::tuple_size<std::decay_t<Tuple>>::value>()); }
// this will print 3 int f(int, char, double);
```

`auto some_args = std::make_tuple(42, 'x', 3.14); int r = apply(f, some_args); // calls f(42, 'x', 3.14)` Section 16.4: Tag Dispatching A simple way of selecting between functions at compile time is to dispatch a function to an overloaded pair of functions that take a tag as one (usually the last) argument. For example, to implement `std::advance()`, we can dispatch on the iterator category: `namespace details { template <class RAlter, class Distance> void advance(RAlter& it, Distance n, std::random_access_iterator_tag) { it += n; } template <class BidirIter, class Distance> void advance(BidirIter& it, Distance n, std::bidirectional_iterator_tag) { if (n > 0) { while (n-- > 0) ++it; } else { while (n++ < 0) --it; } } template <class InputIter, class Distance> void advance(InputIter& it, Distance n, std::input_iterator_tag) { while (n-- > 0) ++it; } } template <class Iter, class Distance> void advance(Iter& it, Distance n) { details::advance(it, n, typename std::iterator_traits::iterator_category{}); }` The `std::XY_iterator_tag` arguments of the overloaded `details::advance` functions are unused function parameters. The actual implementation does not matter (actually it is completely empty). Their only purpose is to allow the compiler to select an overload based on which tag class `details::advance` is called with. In this example, `advance` uses the `iterator_traits::iterator_category` metafunction which returns one of the `iterator_tag` classes, depending on the actual type of `Iter`. A default-constructed object of the `iterator_category::type` then lets the compiler select one of the different overloads of

details::advance. (This function parameter is likely to be completely optimized away, as it is a default-constructed object of an empty struct and never used.) Tag dispatching can give you code that's much easier to read than the equivalents using *SFINAE* and *enable_if*. Note: while C++17's *if constexpr* may simplify the implementation of *advance* in particular, it is not suitable for open implementations unlike tag dispatching. Section 16.5: Detect Whether Expression is Valid It is possible to detect whether an operator or function can be called on a type. To test if a class has an overload of

std::hash, one can do this: `#include // for std::hash #include // for std::false_type and std::true_type #include // for std::declval template<class, class = void> struct has_hash : std::false_type {}; template struct has_hash<T, decltype(std::hash()(std::declval()), void())> : std::true_type {};` Version C++17 Since C++17, *std::void_t* can be used to simplify this type of construct `#include // for std::hash #include // for std::false_type, std::true_type, std::void_t #include // for std::declval template<class, class = std::void_t<>> struct has_hash : std::false_type {}; template struct has_hash<T, std::void_t<decltype(std::hash()(std::declval()))>> : std::true_type {};` where *std::void_t* is defined as: `template< class... > using void_t = void;` For detecting if an operator, such as *operator<* is defined, the syntax is almost the same: `template<class, class = void> struct has_less_than : std::false_type {}; template struct has_less_than<T, decltype(std::declval() < std::declval(), void())> : std::true_type {};` These can be used to use a *std::unordered_map* if *T* has an overload for *std::hash*, but otherwise attempt to use a *std::map*: `template <class K, class V> using hash_invariant_map = std::conditional_t< has_hash::value, std::unordered_map<K, V>, std::map<K, V>;`

Section 16.6: If-then-else Version C++11 The type *std::conditional* in the standard library header can select one type or the other, based on a compile-time boolean value: `template struct ValueOrPointer { typename std::conditional<(sizeof(T) > sizeof(void)), T, T::type vop; };` This struct contains a pointer to *T* if *T* is larger than the size of a pointer, or *T* itself if it is smaller or equal to a pointer's size. Therefore *sizeof(ValueOrPointer)* will always be *<= sizeof(void)*. Section 16.7: Manual distinction of types when given any type *T* When implementing *SFINAE* using *std::enable_if*, it is often useful to have access to helper templates that determines if a given type *T* matches a set of criteria. To help us with that, the standard already provides two types analog to true and false which are *std::true_type* and *std::false_type*. The following example show how to detect if a type *T* is a pointer or not, the *is_pointer* template mimic the behavior of the standard *std::is_pointer* helper: `template struct is_pointer_ : std::false_type {}; template struct is_pointer_<T>: std::true_type {};` `template struct is_pointer: is_pointer_<typename std::remove_cv::type> { }` There are three steps in the above code (sometimes you only need two): 1. The first declaration of *is_pointer_* is the default case, and inherits from *std::false_type*. The default case should always inherit from *std::false_type* since it is analogous to a "false condition". 2. The second declaration specialize the *is_pointer_* template for pointer *T* without caring about what *T* is really. This version inherits from *std::true_type*. 3. The third declaration (the real one) simply remove any unnecessary information from *T* (in this case we remove *const* and *volatile* qualifiers) and then fall backs to one of the two previous declarations. Since *is_pointer* is a class, to access its value you need to either: Use *::value*, e.g. *is_pointer::value* value is a static class member of type *bool* inherited from *std::true_type* or *std::false_type*; Construct an object of this type, e.g. *is_pointer{}* This works because *std::is_pointer* inherits its default constructor from *std::true_type* or *std::false_type* (which have *constexpr* constructors) and both *std::true_type* and *std::false_type* have *constexpr* conversion operators to *bool*.

It is a good habit to provides "helper helper templates" that let you directly access the value: `template constexpr bool is_pointer_v = is_pointer::value;` Version C++17 In C++17 and above, most helper templates already provide a *_v* version, e.g.: `template< class T > constexpr bool is_pointer_v = is_pointer::value;` `template< class T > constexpr bool is_reference_v =`

is_reference::value; Section 16.8: Calculating power with C++11 (and higher) With C++11 and higher calculations at compile time can be much easier. For example calculating the power of a given number at compile time will be following: `template constexpr T calculatePower(T value, unsigned power) { return power == 0 ? 1 : value * calculatePower(value, power-1); }` Keyword `constexpr` is responsible for calculating function in compilation time, then and only then, when all the requirements for this will be met (see more at `constexpr` keyword reference) for example all the arguments must be known at compile time. Note: In C++11 `constexpr` function must compose only from one return statement. Advantages: Comparing this to the standard way of compile time calculation, this method is also useful for runtime calculations. It means, that if the arguments of the function are not known at the compilation time (e.g. `value` and `power` are given as input via user), then function is run in a compilation time, so there's no need to duplicate a code (as we would be forced in older standards of C++). E.g. `void useExample() { constexpr int compileTimeCalculated = calculatePower(3, 3); // computes at compile time, // as both arguments are known at compilation time // and used for a constant expression. int value; std::cin >> value; int runtimeCalculated = calculatePower(value, 3); // runtime calculated, // because value is known only at runtime. }` Version C++17 Another way to calculate power at compile time can make use of fold expression as follows: `#include #include template <class T, T V, T N, class I = std::make_integer_sequence<T, N> struct power; template <class T, T V, T N, T... Is> struct power<T, V, N, std::integer_sequence<T, Is...> { static constexpr T value = (static_cast(1) * ... * (V * static_cast(Is + 1))); }; int main() { std::cout << power<int, 4, 2>::value << std::endl; }` Section 16.9: Generic Min/Max with variable argument count Version > C++11 It's possible to write a generic function (for example `min`) which accepts various numerical types and arbitrary argument count by template meta-programming. This function declares a `min` for two arguments and recursively for more. `template <typename T1, typename T2> auto min(const T1 &a, const T2 &b) -> typename std::common_type<const T1&, const T2&>::type { return a < b ? a : b; } template <typename T1, typename T2, typename... Args> auto min(const T1 &a, const T2 &b, const Args&... args) -> typename std::common_type<const T1&, const T2&, const Args&...>::type { return min(min(a, b), args...); } auto minimum = min(4, 5.8f, 3, 1.8, 3, 1.1, 9);`

Chapter 96

STANDARD LIBRARY EXPANSION

Chapter 97

THE C++11 STANDARD LIBRARY EXPANSION

97.1 1. Unordered Containers

Hash maps/sets. $O(1)$ average access.

- `std::unordered_map`
- `std::unordered_set`
- `std::unordered_multimap`
- `std::unordered_multiset`

Requires a hash function for the key type.

97.2 2. `std::array`

Fixed-size, stack-allocated array. Wrapper around C-style array with STL interface.

```
1 std::array<int, 5> arr = {1, 2, 3, 4, 5};
2 // Bounds checking available: arr.at(10) throws
3 // Knows its size: arr.size()
4 // Doesn't decay to pointer automatically
```

97.3 3. `std::regex`

Regular expression support for pattern matching and text manipulation.

```
1 #include <regex>
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     std::string text = "Contact: user@example.com";
7     std::regex email_regex("(\\w+)@(\\w+)\\.com");
8
9     // 1. Search: Find first occurrence
10    std::smatch matches;
11    if (std::regex_search(text, matches, email_regex)) {
12        std::cout << "Found: " << matches[0] << std::endl;
13        std::cout << "User: " << matches[1] << std::endl;
14    }
```



```

14     }
15
16     // 2. Match: Must match entire string
17     bool is_full_match = std::regex_match(text, email_regex); // false
18
19     // 3. Replace:
20     std::string new_text = std::regex_replace(text, email_regex, "REDACTED");
21
22     return 0;
23 }

```

c

1. Regex Grammar Flavors `std::regex` supports different syntax stan

2. Performance Trap: `std::regex` is Heavy In many implementations (including GCC), `std::regex` is n

3. Capture Groups and `smatch` Capture groups allow you to

97.4 4. `std::chrono`

Type-safe time library.

- **Clocks:** `system_clock`, `steady_clock`.
- **Durations:** `milliseconds`, `seconds`.
- **Time Points:** `time_point`.

```

1 auto start = std::chrono::steady_clock::now();
2 // ...
3 auto end = std::chrono::steady_clock::now();
4 auto diff = end - start;

```

97.5 5. `std::random`

Better random number generation than `rand()`.

```

1 #include <random>
2
3 std::random_device rd;
4 std::mt19937 gen(rd()); // Mersenne Twister
5 std::uniform_int_distribution<> dis(1, 6);
6
7 int roll = dis(gen);

```

Chapter 98

CONCURRENCY

Chapter 99

CONCURRENCY & MULTITHREADING

C++11 brought a standard threading model.

99.1 1. Threads (`std::thread`)

```
1 #include <thread>
2
3 void task() { /*...*/ }
4
5 int main() {
6     std::thread t(task);
7     t.join(); // Wait for finish
8     // t.detach(); // Or let it run freely
9 }
```

99.2 2. Mutexes & Locking

Avoid data races.

- `std::mutex`: Basic lock.
- `std::lock_guard`: RAII wrapper (locks on construction, unlocks on destruction).
- `std::unique_lock`: Flexible RAII wrapper (can unlock manually).

```
1 std::mutex mtx;
2 void safe() {
3     std::lock_guard<std::mutex> lock(mtx);
4     // critical section
5 }
```

99.3 3. Condition Variables

Wait for a condition to be true.

```
1 std::condition_variable cv;
2 std::mutex mtx;
3 bool ready = false;
4
```

```

5 // Waiter
6 std::unique_lock<std::mutex> lk(mtx);
7 cv.wait(lk, []{ return ready; });
8
9 // Notifier
10 {
11     std::lock_guard<std::mutex> lk(mtx);
12     ready = true;
13 }
14 cv.notify_one();

```

99.4 4. Futures & Promises

Asynchronous result retrieval.

- `std::async`: Runs a function asynchronously.
- `std::future`: Holds the result.

```

1 auto f = std::async(std::launch::async, []{ return 42; });
2 int result = f.get(); // Blocks until ready

```

99.5 5. Atomics (`std::atomic`)

Lock-free operations for basic types.

```

1 std::atomic<int> counter(0);
2 counter++; // Thread-safe increment

```

99.5.1 Professional Notes: Concurrency Depth

1. Thread Lifecycle and Exceptions

If a `std::thread` object is destroyed while it is still “joinable” (not joined or detached), `std::terminate()` is called. * **Safety**: Always use a wrapper or ensure `join()/detach()` is called in all exit paths, including exception handlers. * **`std::jthread` (C++20)**: Automatically joins on destruction, solving this problem.

2. Advanced Locking Strategies

- **`std::scoped_lock` (C++17)**: Locks multiple mutexes simultaneously using a deadlock-avoidance algorithm (replaces `std::lock`).
- **`std::shared_mutex` (C++17)**: Allows multiple readers or one writer (Reader-Writer Lock).
- **Lock Strategies**:
 - `std::adopt_lock`: Assume the calling thread already owns the mutex.
 - `std::defer_lock`: Do not lock the mutex on construction.
 - `std::try_to_lock`: Attempt to lock without blocking.

3. Semaphores (C++20)

A semaphore is a synchronization primitive that maintains a counter. * `std::counting_semaphore<N>`: Allows up to N concurrent accesses. * `std::binary_semaphore`: Alias for `counting_semaphore<1>`.

* **Usage**: Useful for limiting access to a pool of resources (e.g., database connections).

4. Thread Local Storage (TLS)

The `thread_local` keyword ensures that each thread has its own unique instance of a variable.

```
1 thread_local int thread_id = 0; // Each thread gets its own copy
```

Chapter 100

Professional Notes: Chapter 80: Threading

Section 80.1: Creating a `std::thread` Section 80.2: Passing a reference to a thread Section 80.3: Using `std::async` instead of `std::thread` Section 80.4: Basic Synchronization Section 80.5: Create a simple thread pool Section 80.6: Ensuring a thread is always joined Section 80.7: Operations on the current thread Section 80.8: Using Condition Variables Section 80.9: Thread operations Section 80.10: Thread-local storage Section 80.11: Reassigning thread objects

Chapter 101

Professional Notes: Chapter 85: Mutexes

Section 85.1: Mutex Types Section 85.2: `std::lock` Section 85.3: `std::unique_lock`, `std::shared_lock`, `std::lock_guard` Section 85.4: Strategies for lock classes: `std::try_to_lock`, `std::adopt_lock`, `std::defer_lock` Section 85.5: `std::mutex` Section 85.6: `std::scoped_lock` (C++ 17)

Chapter 102

Professional Notes: Chapter 55: std::atomics

Section 55.1: atomic types Each instantiation and full specialization of the `std::atomic` template defines an atomic type. If one thread writes to an atomic object while another thread reads from it, the behavior is well-defined (see memory model for details on data races) In addition, accesses to atomic objects may establish inter-thread synchronization and order non-atomic memory accesses as specified by `std::memory_order`. `std::atomic` may be instantiated with any Trivially-Copyable type `T`. `std::atomic` is neither copyable nor movable. The standard library provides specializations of the `std::atomic` template for the following types: 1. One full specialization for the type `bool` and its typedef name is defined that is treated as a non-specialized `std::atomic` except that it has standard layout, trivial default constructor, trivial destructors, and supports aggregate initialization syntax: `Typedef name` Full specialization `std::atomic_bool` `std::atomic` 2) Full specializations and typedefs for integral types, as follows: `Typedef name` Full specialization `std::atomic_char` `std::atomic` `std::atomic_char` `std::atomic` `std::atomic_schar` `std::atomic` `std::atomic_uchar` `std::atomic` `std::atomic_short` `std::atomic` `std::atomic_ushort` `std::atomic` `std::atomic_int` `std::atomic` `std::atomic_uint` `std::atomic` `std::atomic_long` `std::atomic` `std::atomic_ulong` `std::atomic` `std::atomic_llong` `std::atomic` `std::atomic_ullong` `std::atomic` `std::atomic_char16_t` `std::atomic` `std::atomic_char32_t` `std::atomic` `std::atomic_wchar_t` `std::atomic` `std::atomic_int8_t` `std::atomic` `std::atomic_uint8_t` `std::atomic` `std::atomic_int16_t` `std::atomic` `std::atomic_uint16_t` `std::atomic` `std::atomic_int32_t` `std::atomic` `std::atomic_uint32_t` `std::atomic` `std::atomic_int64_t` `std::atomic` `std::atomic_uint64_t` `std::atomic` `std::atomic_int_least8_t` `std::atomic` `std::atomic_uint_least8_t` `std::atomic`

`std::atomic_int_least16_t` `std::atomic` `std::atomic_uint_least16_t` `std::atomic` `std::atomic_int_least32_t` `std::atomic` `std::atomic_uint_least32_t` `std::atomic` `std::atomic_int_least64_t` `std::atomic` `std::atomic_uint_least64_t` `std::atomic` `std::atomic_int_fast8_t` `std::atomic` `std::atomic_uint_fast8_t` `std::atomic` `std::atomic_int_fast16_t` `std::atomic` `std::atomic_uint_fast16_t` `std::atomic` `std::atomic_int_fast32_t` `std::atomic` `std::atomic_uint_fast32_t` `std::atomic` `std::atomic_int_fast64_t` `std::atomic` `std::atomic_uint_fast64_t` `std::atomic` `std::atomic_intptr_t` `std::atomic` `std::atomic_uintptr_t` `std::atomic` `std::atomic_size_t` `std::atomic` `std::atomic_ptrdiff_t` `std::atomic` `std::atomic_intmax_t` `std::atomic` `std::atomic_uintmax_t` `std::atomic` Simple example of using `std::atomic_int` `#include <atomic>` `#include <cout>` `#include <memory_order>` `#include <thread>` `std::atomic_int foo(0); void set_foo(int x) { foo.store(x, std::memory_order_relaxed); // set value atomically } void print_foo() { int x; do { x = foo.load(std::memory_order_relaxed); // get value atomically } while (x==0); std::cout << "foo: " << x << " "; } int main() { std::thread first(print_foo); std::thread second(set_foo, 10); first.join(); //second.join(); return 0; } //output: foo: 10`

Chapter 103

Professional Notes: Chapter 80: Threading

Parameter other Details Takes ownership of other, other doesn't own the thread anymore
func args Function to call in a separate thread Arguments for func Section 80.1: Creating
a std::thread In C++, threads are created using the std::thread class. A thread is a separate
ow of execution; it is analogous to having a helper perform one task while you simultaneously
perform another. When all the code in the thread is executed, it terminates. When creating
a thread, you need to pass something to be executed on it. A few things that you can pass to
a thread: Free functions Member functions Functor objects Lambda expressions Free function
example - executes a function on a separate thread (Live Example): #include #include void
foo(int a) { std::cout << a << " "; } int main() { // Create and execute the thread std::thread
thread(foo, 10); // foo is the function to execute, 10 is the // argument to pass to it // Keep
going; the thread is executed separately // Wait for the thread to finish; we stay here until
it is done thread.join(); return 0; } Member function example - executes a member function
on a separate thread (Live Example): #include #include class Bar { public: void foo(int a) {
std::cout << a << " "; } };

```
int main() { Bar bar; // Create and execute the thread std::thread thread(&Bar::foo, &bar,  
10); // Pass 10 to member function // The member function will be executed in a separate  
thread // Wait for the thread to finish, this is a blocking operation thread.join(); return 0; }  
Functor object example (Live Example): #include #include class Bar { public: void opera-  
tor()(int a) { std::cout << a << " "; } }; int main() { Bar bar; // Create and execute the thread  
std::thread thread(bar, 10); // Pass 10 to functor object // The functor object will be executed  
in a separate thread // Wait for the thread to finish, this is a blocking operation thread.join();  
return 0; } Lambda expression example (Live Example): #include #include int main() { auto  
lambda = { std::cout << a << " "; }; // Create and execute the thread std::thread thread(lambda,  
10); // Pass 10 to the lambda expression // The lambda expression will be executed in a  
separate thread // Wait for the thread to finish, this is a blocking operation thread.join();
```

```
return 0; } Section 80.2: Passing a reference to a thread You cannot pass a reference  
(or const reference) directly to a thread because std::thread will copy/move them. Instead,  
use std::reference_wrapper: void foo(int& b) { b = 10; } int a = 1; std::thread thread{ foo,  
std::ref(a) }; //'a' is now really passed as reference thread.join(); std::cout << a << " "; //Outputs 10  
void bar(const ComplexObject& co) { co.doCalculations(); } ComplexObject object; std::thread  
thread{ bar, std::cref(object) }; //'object' is passed as const& thread.join(); std::cout << ob-  
ject.getResult() << " "; //Outputs the result Section 80.3: Using std::async instead of std::thread  
std::async is also able to make threads. Compared to std::thread it is considered less powerful  
but easier to use when you just want to run a function asynchronously. Asynchronously calling  
a function #include #include unsigned int square(unsigned int i){ return i*i; } int main() {  
auto f = std::async(std::launch::async, square, 8); std::cout << "square currently running"; //do
```

something while square is running `std::cout << "result is" << f.get() << '\n';` //getting the result from square } Common Pitfalls `std::async` returns a `std::future` that holds the return value that will be calculated by the function. When that future gets destroyed it waits until the thread completes, making your code effectively single threaded. This is easily overlooked when you don't need the return value:

`std::async(std::launch::async, square, 5);` //thread already completed at this point, because the returning future got destroyed `std::async` works without a launch policy, so `std::async(square, 5);` compiles. When you do that the system gets to decide if it wants to create a thread or not. The idea was that the system chooses to make a thread unless it is already running more threads than it can run efficiently. Unfortunately implementations commonly just choose not to create a thread in that situation, ever, so you need to override that behavior with `std::launch::async` which forces the system to create a thread. Beware of race conditions. More on `async` on Futures and Promises Section 80.4: Basic Synchronization Thread synchronization can be accomplished using mutexes, among other synchronization primitives. There are several mutex types provided by the standard library, but the simplest is `std::mutex`. To lock a mutex, you construct a lock on it. The simplest lock type is `std::lock_guard`: `std::mutex m; void worker() { std::lock_guard guard(m); // Acquires a lock on the mutex // Synchronized code here } //` the mutex is automatically released when `guard` goes out of scope With `std::lock_guard` the mutex is locked for the whole lifetime of the lock object. In cases where you need to manually control the regions for locking, use `std::unique_lock` instead: `std::mutex m; void worker() { // by default, constructing a unique_lock from a mutex will lock the mutex // by passing the std::defer_lock as a second argument, we // can construct the guard in an unlocked state instead and // manually lock later. std::unique_lock guard(m, std::defer_lock); // the mutex is not locked yet! guard.lock(); // critical section guard.unlock(); // mutex is again released } More Thread synchronization structures Section 80.5: Create a simple thread pool C++11 threading primitives are still relatively low level. They can be used to write a higher level construct, like a thread pool: Version C++14 struct tasks { // the mutex, condition variable and deque form a single // thread-safe triggered queue of tasks: std::mutex m; std::condition_variable v; // note that a packaged_task can store a packaged_task:`

```
std::deque<std::packaged_task<void()>> work; // this holds futures representing the worker
threads being done: std::vector<std::future> finished; // queue( lambda ) will enqueue the
lambda into the tasks for the threads // to use. A future of the type the lambda returns is given
to let you get // the result out. template<class F, class R=std::result_of_t<F&()>> std::future
queue(F&& f) { // wrap the function object into a packaged task, splitting // execution from the
return value: std::packaged_task<R()> p(std::forward(f)); auto r=p.get_future(); // get the
return value before we hand off the task { std::unique_lock l(m); work.emplace_back(std::move(p));
// store the task<R()> as a task<void()> } v.notify_one(); // wake a thread to work on
the task return r; // return the future result of the task } // start N threads in the thread
pool. void start(std::size_t N=1){ for (std::size_t i = 0; i < N; ++i) { // each thread is
a std::async running this->thread_task(): finished.push_back( std::async( std::launch::async,
[this]{ thread_task(); } ) ); } } // abort() cancels all non-started tasks, and tells every working
thread // stop running, and waits for them to finish up. void abort() { cancel_pending();
finish(); } // cancel_pending() merely cancels all non-started tasks: void cancel_pending() {
std::unique_lock l(m); work.clear(); } // finish enques a "stop the thread" message for every
thread, then waits for them: void finish() { { std::unique_lock l(m); for(auto&&unused:finished){
work.push_back({}); } } v.notify_all(); finished.clear(); } ~tasks() { finish(); }
```

```
private: // the work that a worker thread does: void thread_task() { while(true){ //
pop a task off the queue: std::packaged_task<void()> f; { // usual thread-safe queue code:
std::unique_lock l(m); if (work.empty()){ v.wait(l,&[&]{return !work.empty();}); } f = std::move(work.front());
work.pop_front(); } // if the task is invalid, it means we are asked to abort: if (!f.valid()) re-
turn; // otherwise, run the task: f(); } } }; tasks.queue( []{ return "hello world"s; } ) returns
```

a `std::future`, which when the tasks object gets around to running it is populated with hello world. You create threads by running `tasks.start(10)` (which starts 10 threads). The use of `packaged_task<void()>` is merely because there is no type-erased `std::function` equivalent that stores move-only types. Writing a custom one of those would probably be faster than using `packaged_task<void()>`. Live example. Version = C++11 In C++11, replace `result_of_t` with `typename result_of::type`. More on Mutexes. Section 80.6: Ensuring a thread is always joined When the destructor for `std::thread` is invoked, a call to either `join()` or `detach()` must have been made. If a thread has not been joined or detached, then by default `std::terminate` will be called. Using RAII, this is generally simple enough to accomplish: `class thread_joiner { public: thread_joiner(std::thread t) : t_(std::move(t)) { } ~thread_joiner() { if(t_.joinable()) { t_.join(); }`

`} private: std::thread t_; }` This is then used like so: `void perform_work() { // Perform some work } void t() { thread_joiner j{std::thread(perform_work)}; // Do some other calculations while thread is running } // Thread is automatically joined here This also provides exception safety; if we had created our thread normally and the work done in t() performing other calculations had thrown an exception, join() would never have been called on our thread and our process would have been terminated. Section 80.7: Operations on the current thread std::this_thread is a namespace which has functions to do interesting things on the current thread from function it is called from. Function Description get_id Returns the id of the thread sleep_for Sleeps for a specied amount of time sleep_until Sleeps until a specic time yield Reschedule running threads, giving other threads priority Getting the current threads id using std::this_thread::get_id: void foo() { //Print this threads id std::cout << std::this_thread::get_id() << ' '; } std::thread thread{ foo }; thread.join(); //‘threads’ id has now been printed, should be something like 12556 foo(); //The id of the main thread is printed, should be something like 2420 Sleeping for 3 seconds using std::this_thread::sleep_for: void foo() { std::this_thread::sleep_for(std::chrono::seconds(3)); }`

`std::thread thread{ foo }; foo.join(); std::cout << “Waited for 3 seconds!”; Sleeping until 3 hours in the future using std::this_thread::sleep_until: void foo() { std::this_thread::sleep_until(std::chrono::system_clock::now() + std::chrono::hours(3)); } std::thread thread{ foo }; thread.join(); std::cout << “We are now located 3 hours after the thread has been called”; Letting other threads take priority using std::this_thread::yield: void foo(int a) { for (int i = 0; i < a; ++i) std::this_thread::yield(); //Now other threads take priority, because this thread //isn’t doing anything important std::cout << “Hello World!”; } std::thread thread{ foo, 10 }; thread.join(); Section 80.8: Using Condition Variables A condition variable is a primitive used in conjunction with a mutex to orchestrate communication between threads. While it is neither the exclusive or most ecient way to accomplish this, it can be among the simplest to those familiar with the pattern. One waits on a std::condition_variable with a std::unique_lock. This allows the code to safely examine shared state before deciding whether or not to proceed with acquisition. Below is a producer-consumer sketch that uses std::thread, std::condition_variable, std::mutex, and a few others to make things interesting. #include #include #include #include #include #include #include #include int main() { std::condition_variable cond; std::mutex mtx;`

`std::queue<int> intq; bool stopped = false; std::thread producer{& { // Prepare a random number generator. // Our producer will simply push random numbers to intq. // std::default_random_engine gen{}; std::uniform_int_distribution dist{}; std::size_t count = 4006; while(count-->0) { // Always lock before changing // state guarded by a mutex and // condition_variable (a.k.a. “condvar”). std::lock_guard L{mtx}; // Push a random int into the queue intq.push(dist(gen)); // Tell the consumer it has an int cond.notify_one(); } // All done. // Acquire the lock, set the stopped flag, // then inform the consumer. std::lock_guard L{mtx}; std::cout << “Producer is done!” << std::endl; stopped = true; cond.notify_one(); }}; std::thread consumer{& { do{ std::unique_lock L{mtx}; cond.wait(L,& { // Acquire the lock only if // we’ve stopped or the queue // isn’t empty return stopped || ! intq.empty(); }); // We own the mutex here; pop the`

```
queue // until it empties out. while( ! intq.empty()) { const auto val = intq.front(); intq.pop();
std::cout << "Consumer popped:" << val << std::endl; } if(stopped){ // producer has signaled a
stop
```

```
std::cout << "Consumer is done!" << std::endl; break; } }while(true); }); consumer.join();
producer.join(); std::cout << "Example Completed!" << std::endl; return 0; }
```

Section 80.9: Thread operations When you start a thread, it will execute until it is nished. Often, at some point, you need to (possibly - the thread may already be done) wait for the thread to nish, because you want to use the result for example. `int n; std::thread thread{ calculateSomething, std::ref(n) }; //Doing some other stuff //We need 'n' now! //Wait for the thread to finish - if it is not already done thread.join(); //Now 'n' has the result of the calculation done in the separate thread std::cout << n << ' ';` You can also detach the thread, letting it execute freely: `std::thread thread{ doSomething }; //Detaching the thread, we don't need it anymore (for whatever reason) thread.detach(); //The thread will terminate when it is done, or when the main thread returns`

Section 80.10: Thread-local storage Thread-local storage can be created using the `thread_local` keyword. A variable declared with the `thread_local` specifier is said to have thread storage duration. Each thread in a program has its own copy of each thread-local variable. A thread-local variable with function (local) scope will be initialized the rst time control passes through its denition. Such a variable is implicitly static, unless declared extern. A thread-local variable with namespace or class (non-local) scope will be initialized as part of thread startup. Thread-local variables are destroyed upon thread termination. A member of a class can only be thread-local if it is static. There will therefore be one copy of that variable per thread, rather than one copy per (thread, instance) pair.

Example: `void debug_counter() { thread_local int count = 0; Logger::log("This function has been called %d times by this thread", ++count); }`

Section 80.11: Reassigning thread objects We can create empty thread objects and assign work to them later. If we assign a thread object to another active, joinable thread, `std::terminate` will automatically be called before the thread is replaced. `#include void foo() { std::this_thread::sleep_for(std::chrono::seconds(3)); } //create 100 thread objects that do nothing std::thread executors[100]; // Some code // I want to create some threads now for (int i = 0; i < 100; i++) { // If this object doesn't have a thread assigned if (!executors[i].joinable()) executors[i] = std::thread(foo); }`

Chapter 104

Professional Notes: Chapter 85: Mutexes

Section 85.1: Mutex Types C++1x offers a selection of mutex classes: `std::mutex` - offers simple locking functionality. `std::timed_mutex` - offers try_to_lock functionality `std::recursive_mutex` - allows recursive locking by the same thread. `std::shared_mutex`, `std::shared_timed_mutex` - offers shared and unique lock functionality. Section 85.2: `std::lock` `std::lock` uses deadlock avoidance algorithms to lock one or more mutexes. If an exception is thrown during a call to lock multiple objects, `std::lock` unlocks the successfully locked objects before re-throwing the exception. `std::lock(_mutex1, _mutex2)`; Section 85.3: `std::unique_lock`, `std::shared_lock`, `std::lock_guard` Used for the RAII style acquiring of try locks, timed try locks and recursive locks. `std::unique_lock` allows for exclusive ownership of mutexes. `std::shared_lock` allows for shared ownership of mutexes. Several threads can hold `std::shared_lock`s on a `std::shared_mutex`. Available from C++ 14. `std::lock_guard` is a lightweight alternative to `std::unique_lock` and `std::shared_lock`. #include #include #include #include #include #include class PhoneBook { public: std::string getPhoneNo(const std::string & name) { std::shared_lock l(_protect); auto it = _phonebook.find(name); if (it != _phonebook.end()) return (*it).second; return ""; } void addPhoneNo (const std::string & name, const std::string & phone) { std::unique_lock l(_protect); _phonebook[name] = phone; } std::shared_timed_mutex _protect; std::unordered_map<std::string, std::string> _phonebook;

}; Section 85.4: Strategies for lock classes: `std::try_to_lock`, `std::adopt_lock`, `std::defer_lock` When creating a `std::unique_lock`, there are three different locking strategies to choose from: `std::try_to_lock`, `std::defer_lock` and `std::adopt_lock` 1. `std::try_to_lock` allows for trying a lock without blocking: { std::atomic_int temp {0}; std::mutex _mutex; std::thread t([& while(temp!= -1){ std::this_thread::sleep_for(std::chrono::seconds(5)); std::unique_lock lock(_mutex, std::try_to_lock); if(lock.owns_lock()){ //do something temp=0; } } }); while (true) { std::this_thread::sleep_for(std::chrono::seconds(1)); std::unique_lock lock(_mutex, std::try_to_lock); if(lock.owns_lock()){ if (temp < INT_MAX){ ++temp; } std::cout << temp << std::endl; } } } 2. `std::defer_lock` allows for creating a lock structure without acquiring the lock. When locking more than one mutex, there is a window of opportunity for a deadlock if two function callers try to acquire the locks at the same time: { std::unique_lock lock1(_mutex1, std::defer_lock); std::unique_lock lock2(_mutex2, std::defer_lock); lock1.lock() lock2.lock(); // deadlock here std::cout << "Locked! << std::endl; //... } With the following code, whatever happens in the function, the locks are acquired and released in appropriate order: { std::unique_lock lock1(_mutex1, std::defer_lock); std::unique_lock lock2(_mutex2, std::defer_lock); std::lock(lock1,lock2); // no deadlock possible std::cout << "Locked! << std::endl; //... } 3. `std::adopt_lock` does not attempt to lock a second time if the calling thread currently owns the lock. { std::unique_lock lock1(_mutex1, std::adopt_lock); std::unique_lock lock2(_mutex2, std::adopt_lock); std::cout << "Locked! << std::endl; //... } Something to keep in mind is

that `std::adopt_lock` is not a substitute for recursive mutex usage. When the lock goes out of scope the mutex is released. Section 85.5: `std::mutex` `std::mutex` is a simple, non-recursive synchronization structure that is used to protect data which is accessed by multiple threads.

```
std::atomic_int temp{0}; std::mutex __mutex; std::thread t( [&{ while( temp!= -1){ std::this_thread::sleep_for(
std::unique_lock lock( __mutex); temp=0; } }]); while ( true ) { std::this_thread::sleep_for(std::chrono::milliseco
std::unique_lock lock( __mutex, std::try_to_lock); if ( temp < INT_MAX ) temp++; cout «
temp « endl; } } Section 85.6: std::scoped_lock (C++ 17) std::scoped_lock provides RAII style semantics for owning one more mutexes, combined with the lock avoidance algorithms used by std::lock. When std::scoped_lock is destroyed, mutexes are released in the reverse order from which they were acquired. { std::scoped_lock lock{__mutex1,__mutex2}; //do something }
```

Chapter 105

Chapter 16: Concurrency with OpenMP

OpenMP (Open Multi-Processing) is an API that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It is widely used in high-performance computing (HPC) for parallelizing loops and sections of code with simple directives.

105.1 16.1 Getting Started with OpenMP

OpenMP uses compiler directives (`\#pragma omp`) to parallelize code.

105.1.1 1. Parallel Regions

The most basic directive is `\#pragma omp parallel`. It creates a team of threads to execute the following block.

```
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     #pragma omp parallel
6     {
7         int id = omp_get_thread_num();
8         std::cout << "Hello from thread " << id << std::endl;
9     }
10    return 0;
11 }
```

105.1.2 2. Parallelizing Loops

OpenMP excels at parallelizing independent iterations of a loop.

```
1 #pragma omp parallel for
2 for (int i = 0; i < 1000; i++) {
3     results[i] = compute(i);
4 }
```

105.2 16.2 Data Sharing Attributes

- **shared:** Variables are accessible by all threads.

- **private:** Each thread has its own local copy of the variable.
- **reduction:** Combines private copies into a single shared variable (e.g., sum, product).

```
1 double total = 0;
2 #pragma omp parallel for reduction(+:total)
3 for (int i = 0; i < 100; i++) {
4     total += data[i];
5 }
```

c

1. Scheduling Strategies OpenMP provides different ways to distribute loop iterations among threads.

2. False Sharing and Padding **Godhood Warning:** Avoid “False Sharing,” where multiple threads v

3. Thread

Chapter 106

**VOLUME 03 REFINEMENT
GENERIC C14**

Chapter 107

C14 CORE LANGUAGE UPGRADES

Chapter 108

C++14 CORE LANGUAGE UPGRADES

108.1 1. Relaxed constexpr

C++11 `constexpr` functions were extremely limited (single return statement). C++14 removed most restrictions.

108.1.1 1.1 What is Allowed?

- Local variable declarations (not `static/thread_local`).
- Mutation of local objects.
- Control flow statements (`if`, `switch`, loops).
- Multiple return statements.

```
1 // C++11 Style (Recursive, single expression)
2 constexpr int factorial11(int n) {
3     return n <= 1 ? 1 : n * factorial11(n - 1);
4 }
5
6 // C++14 Style (Imperative, readable)
7 constexpr int factorial14(int n) {
8     int result = 1;
9     for (int i = 2; i <= n; ++i) {
10         result *= i;
11     }
12     return result;
13 }
14
15 static_assert(factorial14(5) == 120, "Math check");
```

108.2 2. Binary Literals

Native support for binary representation.

```
1 int b1 = 0b101010; // 42
2 int b2 = 0b1111'0000; // 240 (with separator)
```

108.3 3. Digit Separators

Use single quotes (') to make large numbers readable.

```
1 long long billion = 1'000'000'000;
2 double pi = 3.14159'26535;
3 unsigned int address = 0xDEAD'BEEF;
```

108.4 4. [[deprecated]] Attribute

Mark functions, classes, or variables as deprecated.

```
1 [[deprecated("Use newFunc() instead")]]
2 void oldFunc() {}
3
4 void foo() {
5     oldFunc(); // Compiler warning
6 }
```

Chapter 109

C14 FUNCTIONS AND LAMBDA

Chapter 110

C++14 FUNCTIONS & LAMBIDAS

110.1 1. Generic Lambdas

C++14 allows `auto` in lambda parameters, effectively making them templates.

```
1 auto print = [](auto x) {  
2     std::cout << x << "\n";  
3 };  
4  
5 print(10);           // int  
6 print("hello");      // const char*
```

This is shorthand for a struct with a templated `operator()`.

110.2 2. Generalized Lambda Captures (Init-Capture)

C++14 allows initializing variables inside the lambda capture clause. This is crucial for **move-only** types.

```
1 auto ptr = std::make_unique<int>(10);  
2  
3 // Move ptr into lambda  
4 auto lambda = [p = std::move(ptr)]() {  
5     std::cout << *p << "\n";  
6 };  
7 // ptr is now nullptr
```

110.3 3. Automatic Return Type Deduction

Functions can deduce their return type from the return statement.

```
1 auto add(int a, int b) {  
2     return a + b; // Deduced as int  
3 }  
4  
5 // decltype(auto) preserves references  
6 int& getRef(int& x) { return x; }  
7  
8 decltype(auto) forwardRef(int& x) {  
9     return getRef(x); // Returns int& (auto would return int)  
10 }
```

Chapter 111

C14 TEMPLATES AND METAPROGRAMMING

Chapter 112

C++14 TEMPLATES & METAPROGRAMMING

112.1 1. Variable Templates

Templates for variables, not just functions or classes.

```
1 template<typename T>
2 constexpr T pi = T(3.1415926535897932385);
3
4 int main() {
5     float f = pi<float>;
6     double d = pi<double>;
7 }
```

112.2 2. decltype(auto)

Deduces type exactly as `decltype` would, but without repeating the expression.

- `auto`: Deduces type (decaying references).
- `decltype(auto)`: Deduces type and value category (preserves references).

```
1 int x = 5;
2 int& ref = x;
3
4 auto a = ref;           // int
5 decltype(auto) b = ref; // int&
```

112.3 3. Standard Library Metafunctions

- `std::integer\_sequence`
- `std::index\_sequence`
- `std::make\_index\_sequence`

Useful for unpacking tuples or variadic templates.

```
1 template<typename Tuple, size_t... Is>
2 void print_tuple(const Tuple& t, std::index_sequence<Is...>) {
3     ((std::cout << std::get<Is>(t) << " "), ...);
4 }
```

Chapter 113

C14 STANDARD LIBRARY ENHANCEMENTS

Chapter 114

C++14 STANDARD LIBRARY ENHANCEMENTS

114.1 1. `std::make_unique`

The missing counterpart to `std::make_shared`.

```
1 auto ptr = std::make_unique<int>(42);  
2 // Exception safe, no 'new' keyword.
```

114.2 2. Shared Locks (`std::shared_timed_mutex`)

Reader-writer locking. Multiple readers can hold a shared lock; writers need an exclusive lock.

```
1 #include <shared_mutex>  
2  
3 std::shared_timed_mutex mtx;  
4  
5 void reader() {  
6     std::shared_lock<std::shared_timed_mutex> lock(mtx);  
7     // read  
8 }  
9  
10 void writer() {  
11     std::unique_lock<std::shared_timed_mutex> lock(mtx);  
12     // write  
13 }
```

114.3 3. `std::exchange`

Assigns a new value to an object and returns the old value.

```
1 int x = 10;  
2 int old = std::exchange(x, 20); // x=20, old=10
```

114.4 4. `std::quoted`

Quoted string I/O for streams.

```
1 #include <iomanip>  
2 std::cout << std::quoted("Hello World"); // "Hello World"
```

114.5 5. Tuple Addressing by Type

Access tuple elements by type (if unique).

```
1 std::tuple<int, double> t(1, 3.14);  
2 int i = std::get<int>(t);
```

Chapter 115

VOLUME 04 SIMPLIFICATION MODERNIZATION C17

Chapter 116

C17 CORE LANGUAGE FEATURES

Chapter 117

C++17 CORE LANGUAGE UPGRADES

117.1 1. Structured Bindings

Unpack tuples, pairs, structs, and arrays directly into named variables.

117.1.1 1.1 Unpacking Tuples and Pairs

```
1 #include <tuple>
2 #include <iostream>
3
4 std::tuple<int, double, std::string> get_data() {
5     return {1, 3.14, "hello"};
6 }
7
8 int main() {
9     // Old way (C++11/14)
10    // int i; double d; std::string s;
11    // std::tie(i, d, s) = get_data();
12
13    // C++17 Structured Binding
14    auto [id, val, name] = get_data();
15
16    std::cout << id << ", " << val << ", " << name << "\n";
17 }
```

117.1.2 1.2 Unpacking Structs

```
1 struct Point {
2     int x, y;
3 };
4
5 Point p = {10, 20};
6 auto [px, py] = p; // px = 10, py = 20
```

117.1.3 1.3 Modifiers (const, &)

```
1 std::pair<int, int> p = {1, 2};
2
3 auto& [refX, refY] = p; // References to p.first, p.second
4 refX = 10; // Modifies p
```

```

5
6 const auto& [cRefX, cRefY] = p; // Const references

```

117.2 2. if constexpr

Compile-time branching. Discards the non-taken branch at compile time, allowing instantiation of templates that would otherwise fail.

```

1 #include <type_traits>
2
3 template<typename T>
4 auto get_value(T t) {
5     if constexpr (std::is_pointer_v<T>) {
6         return *t; // Only compiled if T is a pointer
7     } else {
8         return t; // Only compiled if T is NOT a pointer
9     }
10 }
11
12 int main() {
13     int x = 5;
14     int* ptr = &x;
15
16     get_value(x); // Returns int
17     get_value(ptr); // Returns int (dereferenced)
18 }

```

117.3 3. Init-Statements for if and switch

Limit the scope of variables to the if or switch block.

```

1 // Old way
2 {
3     auto it = map.find(key);
4     if (it != map.end()) {
5         // use it
6     }
7 } // it leaks scope or requires extra braces
8
9 // C++17 way
10 if (auto it = map.find(key); it != map.end()) {
11     // use it
12     std::cout << it->second;
13 } // it destroyed here
14
15 // Switch example
16 switch (auto status = get_status(); status) {
17     case OK: break;
18     case ERROR: break;
19 }

```

117.4 4. Inline Variables

Allows defining variables in headers without “multiple definition” errors. Replaces the static member workaround.


```

1 #pragma once
2
3 // C++14: Linker error if included in multiple .cpp files
4 // int global_config = 5;
5
6 // C++17: Safe! The linker merges all definitions.
7 inline int global_config = 5;
8
9 struct MyClass {
10     // Static member initialization in-class!
11     static inline double tolerance = 0.001;
12 };

```

117.5 5. Nested Namespaces

Simplified syntax for deep namespace nesting.

```

1 // Old
2 namespace A {
3     namespace B {
4         namespace C {
5             // ...
6         }
7     }
8 }
9
10 // C++17
11 namespace A::B::C {
12     // ...
13 }

```

117.6 6. Attributes

Standardized attributes to hint compiler behavior.

- `[[nodiscard]]`: Warn if return value is ignored. `cpp` `[[nodiscard]] int calculate\_important\_value();`
- `[[maybe\_unused]]`: Suppress “unused variable” warnings. `cpp` `[[maybe\_unused]] int x = 5;`
- `[[fallthrough]]`: Suppress “implicit fallthrough” warnings in switch cases.

““

Chapter 118

C17 TEMPLATE METAPROGRAMMING

Chapter 119

C++17 TEMPLATE METAPROGRAMMING ENHANCEMENTS

119.1 1. Fold Expressions

Simplifies variadic template unpacking. No more recursion needed for basic operations.

119.1.1 1.1 Unary Folds

```
1 template<typename... Args>
2 auto sum(Args... args) {
3     return (... + args); // Unary left fold: ((arg1 + arg2) + arg3) ...
4 }
5
6 int result = sum(1, 2, 3, 4); // 10
```

119.1.2 1.2 Binary Folds

```
1 template<typename... Args>
2 void print(Args... args) {
3     (std::cout << ... << args) << "\n"; // (cout << arg1) << arg2 ...
4 }
```

119.1.3 1.3 Operators

Supported operators: +, -, *, /, %, ^, &, |, =, <<, >>, +=, -=, etc., ==, !=, <, >, &&, ||, ,, .*, ->.*.

119.2 2. Class Template Argument Deduction (CTAD)

Compiler deduces template arguments for class templates from the constructor.

```
1 #include <vector>
2 #include <tuple>
3 #include <mutex>
4
5 // C++14: Types explicit
6 std::pair<int, double> p1(1, 3.14);
7 std::vector<int> v1 = {1, 2, 3};
8 std::lock_guard<std::mutex> lk(mtx);
```

```

9
10 // C++17: Types deduced
11 std::pair p2(1, 3.14); // pair<int, double>
12 std::vector v2 = {1, 2, 3}; // vector<int>
13 std::lock_guard lk2(mtx); // lock_guard<mutex>

```

119.2.1 2.1 User-Defined Deduction Guides

Sometimes the compiler needs help to deduce the correct type.

```

1 template<typename T>
2 struct Wrapper {
3     T value;
4     Wrapper(T v) : value(v) {}
5 };
6
7 // Deduction guide: "If constructed with const char*, deduce string"
8 Wrapper(const char*) -> Wrapper<std::string>;
9
10 Wrapper w("hello"); // Wrapper<std::string>, not Wrapper<const char*>

```

119.3 3. auto Non-Type Template Parameters

Templates can deduce the type of non-type parameters.

```

1 template<auto Value>
2 void print_value() {
3     std::cout << Value << "\n";
4 }
5
6 int main() {
7     print_value<42>(); // Value is int 42
8     print_value<'c'>(); // Value is char 'c'
9 }

```

119.4 4. std::invoke

Uniform way to invoke any callable (function pointer, functor, lambda, member function pointer).

```

1 #include <functional>
2
3 struct Foo {
4     void bar(int i) { std::cout << "Foo::bar " << i << "\n"; }
5     int data = 10;
6 };
7
8 void free_func(int i) { std::cout << "free_func " << i << "\n"; }
9
10 int main() {
11     Foo f;
12
13     // Call member function
14     std::invoke(&Foo::bar, f, 1);
15
16     // Access member data
17     std::cout << std::invoke(&Foo::data, f) << "\n";
18
19     // Call free function
20     std::invoke(free_func, 2);

```

21 }

Part II

Volume II: C++11 - The Modern Revolution

Chapter 120

C17 VOCABULARY TYPES

Chapter 121

C++17 VOCABULARY TYPES

These types provide standard ways to represent optionality, alternatives, and type-erased values, replacing many custom implementations.

121.1 1. `std::string_view`

A non-owning reference to a string (or substring). **Zero-copy** string operations.

121.1.1 1.1 Efficiency

```
1 // Bad: Copies string (potentially expensive allocation)
2 void print_str(std::string s) {
3     std::cout << s << "\n";
4 }
5
6 // Good: No copy, no allocation
7 void print_view(std::string_view sv) {
8     std::cout << sv << "\n";
9 }
10
11 int main() {
12     const char* cstr = "Hello World";
13     // print_str(cstr); // Creates std::string (allocates!)
14     print_view(cstr);    // No allocation
15
16     std::string s = "Hello World";
17     print_view(s);       // Works with std::string too
18
19     // Substrings are cheap!
20     std::string_view sub = std::string_view(cstr).substr(0, 5);
21     print_view(sub);
22 }
```

121.1.2 1.2 Caveats

- **Non-owning:** Ensure the underlying string outlives the view.
- **Not null-terminated:** Do not pass `.data()` to C APIs unless you are sure.

121.2 2. `std::optional`

Represents a value that may or may not be present. Replaces pointers for nullable values or “magic values” (-1, “”).


```

1 #include <optional>
2
3 std::optional<int> find_even(const std::vector<int>& v) {
4     for (int x : v) {
5         if (x % 2 == 0) return x;
6     }
7     return std::nullopt; // or {}
8 }
9
10 int main() {
11     auto res = find_even({1, 3, 5});
12     if (res) { // or res.has_value()
13         std::cout << *res; // or res.value() (throws if empty)
14     } else {
15         std::cout << "Not found";
16     }
17
18     // Value or default
19     std::cout << res.value_or(0);
20 }

```

121.3 3. std::variant

A type-safe union. Can hold one of several distinct types.

```

1 #include <variant>
2
3 std::variant<int, float, std::string> v;
4
5 v = 10;
6 v = 3.14f;
7 v = "hello";
8
9 // Accessing
10 try {
11     std::string s = std::get<std::string>(v);
12     // int i = std::get<int>(v); // Throws std::bad_variant_access
13 } catch (...) {}
14
15 // std::visit (The Visitor Pattern)
16 std::visit([](auto&& arg) {
17     std::cout << arg << "\n";
18 }, v);

```

121.4 4. std::any

A type-safe container for *single* values of any type. (Like `void*` but safe).

```

1 #include <any>
2
3 std::any a = 1;
4 a = std::string("hello");
5
6 try {
7     std::string s = std::any_cast<std::string>(a);
8 } catch (const std::bad_any_cast& e) {
9     std::cout << e.what();
10 }

```

Chapter 122

Professional Notes: Chapter 51: `std::optional`

Section 51.1: Using optionals to represent the absence of a value Section 51.2: optional as return value Section 51.3: `value_or` Section 51.4: Introduction Section 51.5: Using optionals to represent the failure of a function

Chapter 123

Professional Notes: Chapter 56: `std::variant`

Section 56.1: Create pseudo-method pointers Section 56.2: Basic `std::variant` use Section 56.3:
Constructing a `std::variant`

Chapter 124

Professional Notes: Chapter 51: std::optional

Section 51.1: Using optionals to represent the absence of a value Before C++17, having pointers with a value of nullptr commonly represented the absence of a value. This is a good solution for large objects that have been dynamically allocated and are already managed by pointers. However, this solution does not work well for small or primitive types such as int, which are rarely ever dynamically allocated or managed by pointers. std::optional provides a viable solution to this common problem. In this example, struct Person is defined. It is possible for a person to have a pet, but not necessary. Therefore, the pet member of Person is declared with an std::optional wrapper. #include #include #include struct Animal { std::string name; }; struct Person { std::string name; std::optional pet; }; int main() { Person person; person.name = "John"; if (person.pet) { std::cout << person.name << "'s pet's name is" << person.pet->name << std::endl; } else { std::cout << person.name << " is alone." << std::endl; } } Section 51.2: optional as return value std::optional divide(float a, float b) { if (b!=0.f) return a/b; return {}; } Here we return either the fraction a/b, but if it is not defined (would be infinity) we instead return the empty optional. A more complex case: template<class Range, class Pred> auto find_if(Range&& r, Pred&& p) { using std::begin; using std::end; auto b = begin(r), e = end(r); auto r = std::find_if(b, e , p); using iterator = decltype(r);

if (r==e) return std::optional(); return std::optional(r); } template<class Range, class T> auto find(Range&& r, T const& t) { return find_if(std::forward(r), &t{return x==t;}); } find(some_range, 7) searches the container or range some_range for something equal to the number 7. find_if does it with a predicate. It returns either an empty optional if it was not found, or an optional containing an iterator to the element if it was. This allows you to do: if (find(vec, 7)) { // code } or even if (auto oit = find(vec, 7)) { vec.erase(oit); } without having to mess around with begin/end iterators and tests. Section 51.3: value_or void print_name(std::ostream& os, std::optional const& name) { std::cout << "Name is:" << name.value_or("") << "; } value_or either returns the value stored in the optional, or the argument if there is nothing store there. This lets you take the maybe-null optional and give a default behavior when you actually need a value. By doing it this way, the "default behavior" decision can be pushed back to the point where it is best made and immediately needed, instead of generating some default value deep in the guts of some engine. Section 51.4: Introduction Optionals (also known as Maybe types) are used to represent a type whose contents may or may not be present. They are implemented in C++17 as the std::optional class. For example, an object of type std::optional may contain some value of type int, or it may contain no value. Optionals are commonly used either to represent a value that may not exist or as a return type from a function that can fail to return a meaningful result. Other approaches to optional There are many other approach to solving the problem that std::optional solves, but none of them are quite complete: using a pointer, using a sentinel, or using a pair<bool, T>. Optional vs Pointer

In some cases, we can provide a pointer to an existing object or `nullptr` to indicate failure. But this is limited to those cases where objects already exist - optional, as a value type, can also be used to return new objects without resorting to memory allocation. Optional vs Sentinel A common idiom is to use a special value to indicate that the value is meaningless. This may be 0 or -1 for integral types, or `nullptr` for pointers. However, this reduces the space of valid values (you cannot differentiate between a valid 0 and a meaningless 0) and many types do not have a natural choice for the sentinel value. Optional vs `std::pair<bool, T>` Another common idiom is to provide a pair, where one of the elements is a bool indicating whether or not the value is meaningful. This relies upon the value type being default-constructible in the case of error, which is not possible for some types and possible but undesirable for others. An optional, in the case of error, does not need to construct anything. Section 51.5: Using optionals to represent the failure of a function Before C++17, a function typically represented failure in one of several ways: A null pointer was returned. e.g. Calling a function `Delegate App::get_delegate()` on an App instance that did not have a delegate would return `nullptr`. This is a good solution for objects that have been dynamically allocated or are large and managed by pointers, but isn't a good solution for small objects that are typically stack-allocated and passed by copying. A specific value of the return type was reserved to indicate failure. e.g. Calling a function `shortest_path_distance(Vertex a, Vertex b)` on two vertices that are not connected may return zero to indicate this fact. The value was paired together with a bool to indicate if the returned value was meaningful. e.g. Calling a function `std::pair<int, bool> parse(const std::string &str)` with a string argument that is not an integer would return a pair with an undened int and a bool set to false. In this example, John is given two pets, Fluffy and Furball. The function `Person::pet_with_name()` is then called to retrieve John's pet Whiskers. Since John does not have a pet named Whiskers, the function fails and `std::nullopt` is returned instead.

```
#include <string>
#include <vector>
#include <optional>
#include <string_view>
using namespace std;

struct Animal {
    string name;
};

struct Person {
    string name;
    vector<Animal> pets;
    optional<Animal> pet_with_name(const string &name) {
        for (const Animal &pet : pets) {
```

```
            if (pet.name == name) { return pet; }
        }
        return std::nullopt;
    }
};

int main() {
    Person john;
    john.name = "John";
    Animal fluffy;
    fluffy.name = "Fluffy";
    john.pets.push_back(fluffy);
    Animal furball;
    furball.name = "Furball";
    john.pets.push_back(furball);
    optional<Animal> whiskers = john.pet_with_name("Whiskers");
    if (whiskers) {
        cout << "John has a pet named Whiskers." << endl;
    } else {
        cout << "Whiskers must not belong to John." << endl;
    }
}
```

Chapter 125

Professional Notes: Chapter 56: std::variant

Section 56.1: Create pseudo-method pointers This is an advanced example. You can use variant for light weight type erasure. `template struct pseudo_method { F f; // enable C++17 class type deduction: pseudo_method(F&& fin):f(std::move(fin)) {} // Koenig lookup operator->, as this is a pseudo-method it is appropriate: template // maybe add SFINAE test that LHS is actually a variant. friend decltype(auto) operator->(Variant&& var, pseudo_method const& method) { // var->method returns a lambda that perfect forwards a function call, // behaving like a method pointer basically: return λ ->decltype(auto) { // use visit to get the type of the variant: return std::visit(λ ->decltype(auto) { // decltype(x)(x) is perfect forwarding in a lambda: return method.f(decltype(self)(self), decltype(args)(args)...); }, std::forward(var)); }; } }; this creates a type that overloads operator-> with a Variant on the left hand side. // C++17 class type deduction to find template argument of print here. // a pseudo-method lambda should take self as its first argument, then // the rest of the arguments afterwards, and invoke the action: pseudo_method print = ->decltype(auto) { return decltype(self)(self).print(decltype(args)(args)...); }; Now if we have 2 types each with a print method: struct A { void print(std::ostream& os) const { os << "A"; } }; struct B { void print(std::ostream& os) const { os << "B"; } }; note that they are unrelated types. We can: std::variant<A,B> var = A{};`

`(var->print)(std::cout);` and it will dispatch the call directly to `A::print(std::cout)` for us. If we instead initialized the var with `B{}`, it would dispatch to `B::print(std::cout)`. If we created a new type `C`: `struct C {};` then: `std::variant<A,B,C> var = A{};` `(var->print)(std::cout);` will fail to compile, because there is no `C.print(std::cout)` method. Extending the above would permit free function prints to be detected and used, possibly with use of `if constexpr` within the print pseudo-method. live example currently using `boost::variant` in place of `std::variant`.
Section 56.2: Basic std::variant use This creates a variant (a tagged union) that can store either an int or a string. `std::variant< int, std::string > var;` We can store one of either type in it: `var = "hello";` And we can access the contents via `std::visit`: `// Prints "hello": visit( { std::cout << e << "; } , var );` by passing in a polymorphic lambda or similar function object. If we are certain we know what type it is, we can get it: `auto str = std::get(var);` but this will throw if we get it wrong. `get_if`: `auto* str = std::get_if(&var);` returns `nullptr` if you guess wrong. Variants guarantee no dynamic memory allocation (other than which is allocated by their contained types). Only one of the types in a variant is stored there, and in rare cases (involving exceptions while assigning and no safe way to back out) the variant can become empty. Variants let you store multiple value types in one variable safely and efficiently. They are basically smart, type-safe

unions. Section 56.3: Constructing a std::variant This does not cover allocators. `struct A {};` `struct B { B()=default; B(B const&)=default; B(int){}; };` `struct C { C()=delete; C(int){}; C(C const&)=default; };` `struct D { D( std::initializer_list ) {};` `D(D const&)=default;`

```
D()=default; }; std::variant<A,B> var_ab0; // contains a A() std::variant<A,B> var_ab1 = 7;
// contains a B(7) std::variant<A,B> var_ab2 = var_ab1; // contains a B(7) std::variant<A,B,C>
var_abc0{ std::in_place_type, 7 }; // contains a C(7) std::variant var_c0; // illegal, no default
ctor for C std::variant<A,D> var_ad0( std::in_place_type, {1,3,3,4} ); // contains D{1,3,3,4}
std::variant<A,D> var_ad1( std::in_place_index<0> ); // contains A{} std::variant<A,D>
var_ad2( std::in_place_index<1>, {1,3,3,4} ); // contains D{1,3,3,4}
```

Chapter 126

C17 FILESYSTEM AND IO

Chapter 127

C++17 FILESYSTEM AND IO

127.1 1. std::filesystem

Standardized file system operations. Based on boost::filesystem.

127.1.1 1.1 Paths

```
1 #include <filesystem>
2 namespace fs = std::filesystem;
3
4 fs::path p = "/home/user/data.txt";
5
6 std::cout << p.filename();      // "data.txt"
7 std::cout << p.extension();    // ".txt"
8 std::cout << p.parent_path();  // "/home/user"
```

127.1.2 1.2 Iterating Directories

```
1 for (const auto& entry : fs::directory_iterator("/home/user")) {
2     std::cout << entry.path() << "\n";
3 }
4
5 // Recursive
6 for (const auto& entry : fs::recursive_directory_iterator("/home/user")) {
7     // ...
8 }
```

127.1.3 1.3 Operations

```
1 fs::create_directory("sandbox");
2 fs::copy("a.txt", "b.txt");
3 fs::rename("b.txt", "c.txt");
4 fs::remove("c.txt"); // Returns true if removed
5 bool exists = fs::exists("sandbox");
6 uintmax_t size = fs::file_size("a.txt");
```

127.2 2. Polymorphic Allocators (std::pmr)

Memory resource management that is detached from the type.

```
1 #include <memory_resource>
2 #include <vector>
3
4 char buffer[1024];
5 std::pmr::monotonic_buffer_resource pool(buffer, 1024);
6 std::pmr::vector<int> v(&pool); // Uses stack buffer!
7
8 v.push_back(1);
9 // No heap allocation happens until buffer is exhausted.
```

Chapter 128

C17 PARALLEL ALGORITHMS AND CONCURRENCY

Chapter 129

C++17 PARALLEL ALGORITHMS & CONCURRENCY

129.1 1. Parallel Algorithms (`std::execution`)

Standard algorithms (`sort`, `transform`, `for_each`) now accept an execution policy.

```
1 #include <algorithm>
2 #include <execution>
3 #include <vector>
4
5 std::vector<int> v(1'000'000);
6
7 // Sequential (default)
8 std::sort(std::execution::seq, v.begin(), v.end());
9
10 // Parallel (multi-threaded)
11 std::sort(std::execution::par, v.begin(), v.end());
12
13 // Parallel + Vectorized (SIMD allowed)
14 std::sort(std::execution::par_unseq, v.begin(), v.end());
```

Note: `par_unseq` allows interleaving of instructions, so user code must be vector-safe (no mutexes, no allocations).

129.2 2. `std::scoped_lock`

Multi-lock RAII wrapper. Prevents deadlocks by locking multiple mutexes safely (using a deadlock-avoidance algorithm).

```
1 std::mutex m1, m2;
2
3 void swap_data() {
4     // Locks both m1 and m2 atomically
5     std::scoped_lock lock(m1, m2);
6     // ...
7 }
```

129.3 3. `std::shared_mutex`

Standard reader-writer lock (was `shared_timed_mutex` in C++14).

```
1 #include <shared_mutex>
2
```

```
3 std::shared_mutex smtx;  
4  
5 // Writer  
6 std::unique_lock lock(smtx);  
7  
8 // Reader  
9 std::shared_lock lock(smtx);
```

Chapter 130

C17 STANDARD LIBRARY ADDITIONS

Chapter 131

C++17 STANDARD LIBRARY ADDITIONS

131.1 1. `std::byte`

A distinct type for byte-oriented memory access. Unlike `char`, it is not an arithmetic type (prevents accidental math).

```
1 #include <cstdint>
2
3 std::byte b = std::byte{0xAB};
4 // b += 1; // Error
5 int i = std::to_integer<int>(b);
```

131.2 2. Algorithms

- `std::sample`: Selects `n` random elements from a range.
- `std::clamp`: Clamps a value between `lo` and `hi`.
- `std::gcd`, `std::lcm`: Math utilities.

```
1 int x = std::clamp(10, 0, 5); // 5
2 int g = std::gcd(12, 18);      // 6
```

131.3 3. Mathematical Special Functions

Support for Laguerre polynomials, Bessel functions, elliptic integrals, etc., in `<cmath>`.

131.4 4. Elementary String Conversions (`<charconv>`)

Low-level, allocation-free, locale-independent string conversions. Extremely fast.

```
1 #include <charconv>
2
3 char buffer[10];
4 int value = 42;
5
6 // To chars
7 std::to_chars(buffer, buffer + 10, value);
8
```

```
9 // From chars  
10 std::from_chars(buffer, buffer + 10, value);
```

Chapter 132

**VOLUME 05 GIGANTIC LEAP
C20**

Chapter 133

C20 CONCEPTS

Chapter 134

C++20 CONCEPTS

134.1 1. The Problem with Templates

Before C++20, template errors were notoriously verbose and hard to decipher (“template error explosion”). If you passed a type that didn’t support a required operation (like `+` or `.begin()`), the error would occur deep inside the template implementation, often screens away from the actual call site.

134.2 2. What are Concepts?

Concepts are named sets of requirements for template arguments. They act as predicates that are evaluated at compile time.

134.2.1 2.1 Defining a Concept

```
1 #include <concepts>
2
3 // A concept that checks if T is integral
4 template<typename T>
5 concept Integral = std::is_integral_v<T>;
6
7 // A concept that checks if T is addable
8 template<typename T>
9 concept Addable = requires(T a, T b) {
10     { a + b } -> std::convertible_to<T>;
11 };
```

134.2.2 2.2 Using Concepts (requires clause)

```
1 template<typename T>
2 requires Integral<T>
3 T add(T a, T b) {
4     return a + b;
5 }
6
7 // Shorthand syntax
8 template<Integral T>
9 T subtract(T a, T b) {
10     return a - b;
11 }
12
13 // Terse syntax (C++20 style)
14 auto multiply(Integral auto a, Integral auto b) {
```

```

15     return a * b;
16 }

```

134.3 3. The requires Expression

The core mechanism for defining ad-hoc constraints.

```

1  template<typename T>
2  concept Printable = requires(T x) {
3      std::cout << x; // Expression must be valid
4  };
5
6  template<typename T>
7  concept Container = requires(T c) {
8      typename T::value_type;
9      { c.size() } -> std::same_as<size_t>;
10     { c.begin() };
11     { c.end() };
12 };

```

134.4 4. Standard Concepts (<concepts>)

C++20 provides a rich library of predefined concepts.

- **Core:** `std::same_as`, `std::derived_from`, `std::convertible_to`
- **Arithmetic:** `std::integral`, `std::floating_point`, `std::signed_integral`
- **Object:** `std::movable`, `std::copyable`, `std::regular`
- **Callable:** `std::invocable`, `std::predicate`

134.5 5. Constraint Based Overloading

Concepts allow overloading function templates based on properties of types (replacing SFINAE/`enable_if`).

```

1  void print(std::integral auto x) {
2      std::cout << "Integer: " << x << "\n";
3  }
4
5  void print(std::floating_point auto x) {
6      std::cout << "Float: " << x << "\n";
7  }
8
9  print(5); // Calls integral version
10 print(3.14); // Calls floating_point version

```

Chapter 135

C20 MODULES

Chapter 136

C++20 MODULES

136.1 1. The Death of Headers

Headers (`#include`) are text substitution. They are slow, fragile, and leak macros/symbols. Modules (`import`) are compiled components.

136.1.1 1.1 Problems with Headers

- **Slow Compilation:** A 10,000 line header included in 100 files is parsed 100 times.
- **Macro Leaks:** Macros defined in one header affect all subsequent code.
- **ODR Violations:** Different compiler flags for different TUs can break binary compatibility.

136.2 2. Basic Module Syntax

136.2.1 2.1 Interface Unit (`.cppm` or `.ixx`)

```
1 export module math; // Module declaration
2
3 export int add(int a, int b) { // Exported function
4     return a + b;
5 }
6
7 int internal_helper() { // Not exported (private)
8     return 42;
9 }
```

136.2.2 2.2 Importing a Module

```
1 import math;
2 import <iostream>; // Import header unit (if supported)
3
4 int main() {
5     std::cout << add(1, 2) << "\n";
6     // internal_helper(); // Error: undeclared identifier
7 }
```

136.3 3. Module Partitions

Large modules can be split into partitions.

math__impl.cppm:

```
1 module math:impl; // Partition
2
3 int heavy_computation() { return 100; }
```

math.cppm:

```
1 export module math;
2 import :impl; // Import partition
3
4 export int compute() {
5     return heavy_computation();
6 }
```

136.4 4. The Global Module Fragment

Used to include legacy headers within a module.

```
1 module;
2 #include <vector>
3 #include <cmath>
4
5 export module geometry;
6
7 export double distance(double x, double y) {
8     return std::sqrt(x*x + y*y);
9 }
```

136.5 5. Build System Implications

Modules require a dependency graph to be built *before* compilation (unlike headers). Modern build systems (CMake 3.26+, Build2, XMake) support this.

““

Chapter 137

C20 COROUTINES

Chapter 138

C++20 COROUTINES

138.1 1. Asynchronous Programming

Coroutines are functions that can suspend execution to be resumed later. They enable writing async code that looks synchronous.

138.1.1 1.1 Keywords

- `co_await`: Suspend execution until an awaited operation completes.
- `co_yield`: Suspend execution and return a value (generator).
- `co_return`: Complete execution and return a final value.

A function containing any of these is a coroutine.

138.2 2. Generators (The Python Style)

(Note: `std::generator` arrived in C++23, but the machinery exists in C++20).

```
1 #include <coroutine>
2 #include <iostream>
3
4 struct Generator {
5     struct promise_type;
6     using handle_type = std::coroutine_handle<promise_type>;
7     handle_type h;
8
9     Generator(handle_type h) : h(h) {}
10    ~Generator() { if (h) h.destroy(); }
11
12    struct promise_type {
13        int current_value;
14        Generator get_return_object() { return Generator{handle_type::
15from_promise(*this)}; }
16        std::suspend_always initial_suspend() { return {}; }
17        std::suspend_always final_suspend() noexcept { return {}; }
18        std::suspend_always yield_value(int value) {
19            current_value = value; return {};
20        }
21        void return_void() {}
22        void unhandled_exception() { std::terminate(); }
23    };
24
25    bool next() { h.resume(); return !h.done(); }
```

```
25     int value() const { return h.promise().current_value; }
26 };
27
28 Generator sequence(int start, int end) {
29     for (int i = start; i < end; ++i) {
30         co_yield i; // Suspend here
31     }
32 }
33
34 int main() {
35     auto gen = sequence(0, 5);
36     while (gen.next()) {
37         std::cout << gen.value() << " ";
38     }
39 }
```

138.3 3. Tasks (co_await)

Used for async I/O. (Requires a library like `cppcoro` or custom implementation in C++20).

```
1 Task<int> fetch_data() {
2     auto conn = co_await connect("db.local");
3     auto result = co_await conn.query("SELECT * FROM users");
4     co_return result.size();
5 }
```

138.4 4. Under the Hood

The compiler generates a state machine for the coroutine, allocating the “coroutine frame” (local variables, promise object, instruction pointer) on the heap (usually).

Part III

Volume III: C++14 - Refinement & Generics

Chapter 139

C20 RANGES

Chapter 140

C++20 RANGES

140.1 1. The End of Iterator pairs

Ranges allow operating on a “range” (something with a begin and end) directly, composing algorithms using pipe syntax (|).

```
1 #include <ranges>
2 #include <vector>
3 #include <iostream>
4 #include <algorithm>
5
6 namespace views = std::views;
7 namespace ranges = std::ranges;
8
9 int main() {
10     std::vector<int> nums = {1, 2, 3, 4, 5, 6};
11
12     // Old Way
13     // std::vector<int> temp;
14     // std::copy_if(nums.begin(), nums.end(), std::back_inserter(temp), [](int
15     // i){ return i % 2 == 0; });
16     // for(auto& x : temp) x *= x;
17
18     // C++20 Ranges
19     auto result = nums
20         | views::filter([](int i){ return i % 2 == 0; }) // Keep evens
21         | views::transform([](int i){ return i * i; }); // Square them
22
23     for (int i : result) {
24         std::cout << i << " "; // 4 16 36
25     }
```

140.2 2. Views vs Actions

- **Views:** Lazy. They adapt the iteration logic but don't own data. Computation happens *during* iteration. (e.g., `views::filter`, `views::reverse`).
- **Actions:** Eager. They modify the container immediately. (Not fully standardized in C++20, mostly views).

140.3 3. Projections

Algorithms now accept a “projection” to transform data before comparison.

```
1 struct User { int id; std::string name; };
2 std::vector<User> users = {{2, "Bob"}, {1, "Alice"}};
3
4 // Sort by ID
5 ranges::sort(users, {}, &User::id);
6
7 // Sort by Name
8 ranges::sort(users, {}, &User::name);
```

140.4 4. Dangling Iterators Protection

Ranges algorithms prevent returning iterators to temporary objects that would die immediately.

```
1 auto get_vec() { return std::vector{1, 2, 3}; }
2
3 auto it = ranges::max_element(get_vec()); // Error!
4 // Returns std::ranges::dangling instead of an invalid iterator.
```

Part IV

Volume IV: C++17 - Simplification & Modernization

Chapter 141

C20 CORE LANGUAGE FEATURES

Chapter 142

C++20 CORE LANGUAGE FEATURES

142.1 1. Three-Way Comparison (<=>)

The “Spaceship Operator”. Generates all 6 comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`) automatically.

```
1 #include <compare>
2
3 struct Point {
4     int x, y;
5     auto operator<=>(const Point&) const = default;
6 };
7
8 Point p1{1, 2}, p2{1, 3};
9 // p1 < p2 works!
10 // p1 == p2 works!
```

Returns `strong_ordering`, `weak_ordering`, or `partial_ordering`.

142.2 2. Designated Initializers

C-style struct initialization syntax.

```
1 struct Config {
2     int id;
3     std::string name;
4     bool active;
5 };
6
7 Config c = {
8     .id = 1,
9     .active = true // Order must match declaration! Name is default constructed
10 };
```

142.3 3. `constexpr` (Immediate Functions)

Functions that *must* be executed at compile time.

```
1 constexpr int square(int n) {
2     return n * n;
3 }
```

```
4
5 int x = square(5); // OK: computed at compile time
6 int runtime_val = 10;
7 // int y = square(runtime_val); // Error: argument not constant
```

142.4 4. `constexpr`

Ensures a variable has static initialization (no runtime overhead, no static initialization order fiasco).

```
1 constexpr int g_val = 42; // OK
2 // constexpr int g_rand = rand(); // Error
```

142.5 5. Non-Type Template Parameters (NTP) Enhancements

You can now use floating-point types and structural types (classes with public members) as template parameters.

```
1 template<double Threshold>
2 bool check(double val) { return val > Threshold; }
3
4 struct FixedString { char buf[16]; };
5 template<FixedString S>
6 void print() { std::cout << S.buf; }
```

Part V

Volume V: C++20 - The Gigantic Leap

Chapter 143

C20 STANDARD LIBRARY ADDITIONS

Chapter 144

C++20 STANDARD LIBRARY ADDITIONS

144.1 1. `std::format` (Python-style Formatting)

Type-safe, fast, and readable string formatting. Replaces `printf` and `iostream`.

```
1 #include <format>
2 #include <iostream>
3
4 std::string s = std::format("Hello, {}! The answer is {}.", "World", 42);
5 // "Hello, World! The answer is 42."
6
7 // Positional arguments
8 std::string s2 = std::format("{1} {0}", "World", "Hello");
```

144.2 2. `std::span`

A non-owning view of a contiguous sequence (array, vector). Replaces `pointer + size`.

```
1 #include <span>
2
3 void process(std::span<int> data) {
4     for (int& x : data) x *= 2;
5 }
6
7 int arr[] = {1, 2, 3};
8 std::vector<int> vec = {4, 5, 6};
9
10 process(arr); // Works
11 process(vec); // Works
```

144.3 3. `std::jthread` (Joinable Thread)

Automatically joins on destruction. Supports cooperative interruption (`stop\_token`).

```
1 #include <thread>
2
3 void worker(std::stop_token st) {
4     while (!st.stop_requested()) {
5         // work...
6     }
7 }
8
```

```
9 {  
10     std::jthread t(worker);  
11 } // t requests stop and joins automatically here
```

144.4 4. Concurrency Primitives (<semaphore>, <barrier>, <latch>)

- `std::binary_semaphore`, `std::counting_semaphore`: Lightweight signaling.
- `std::latch`: Single-use countdown barrier.
- `std::barrier`: Reusable barrier for phases of work.

144.5 5. Mathematical Constants (<numbers>)

```
1 #include <numbers>  
2 double pi = std::numbers::pi;  
3 double e = std::numbers::e;
```

144.6 6. Bit Manipulation (<bit>)

- `std::popcount`: Count set bits.
 - `std::bit_ceil`: Next power of 2.
 - `std::endian`: Check system endianness.
-

Chapter 145

VOLUME 06 FUTURE C23 26

Part VI

Volume VI: C++23/26 - The Future

Chapter 146

C23 CORE LANGUAGE

Chapter 147

C++23 CORE LANGUAGE UPGRADES

147.1 1. Deducing this (Explicit Object Parameter)

This is one of the most significant changes in C++23. It allows member functions to deduce the value category (`const`, `&`, `&&`, `&&&`) of the object they are called on, without writing 4+ overloads.

147.1.1 1.1 The Problem (C++20 and older)

To handle lvalues, const lvalues, rvalues, and const rvalues, you needed overloads:

```
1 struct Widget {
2     void process() &;           // lvalue
3     void process() const&;      // const lvalue
4     void process() &&;          // rvalue
5     void process() const&&;     // const rvalue
6 };
```

147.1.2 1.2 The Solution

Pass `this` as an explicit argument.

```
1 struct Widget {
2     // 'self' deduces the type of the object (e.g., Widget&, const Widget&,
3     // Widget&&)
4     template <typename Self>
5     void process(this Self&& self) {
6         // Forward self to another function
7         handle(std::forward<Self>(self));
8     }
9 };
```

147.1.3 1.3 Recursive Lambdas

Deducing `this` allows lambdas to be recursive without `std::function` or hacks.

```
1 auto fib = [](this auto&& self, int n) {
2     if (n <= 1) return n;
3     return self(n - 1) + self(n - 2);
4 };
5
6 int result = fib(10); // 55
```

147.2 2. if consteval

A safer version of if (`std::is\_constant\_evaluated()`).

```

1 consteval int compile_time_algo(int n) { return n * n; }
2 int runtime_algo(int n) { return n * n; }
3
4 constexpr int heavy_math(int n) {
5     if consteval {
6         return compile_time_algo(n); // Executed ONLY at compile time
7     } else {
8         return runtime_algo(n);      // Executed at runtime
9     }
10 }

```

147.3 3. Multidimensional Subscript Operator (operator[])

C++23 finally allows multiple arguments in [].

```

1 struct Matrix {
2     std::vector<double> data;
3     size_t cols;
4
5     double& operator[](size_t r, size_t c) {
6         return data[r * cols + c];
7     }
8 };
9
10 Matrix m;
11 m[1, 2] = 3.14; // Clean syntax!

```

147.4 4. auto(x): Decay Copy

Explicitly create a prvalue copy of an lvalue. Useful in generic code to prevent accidental referencing.

```

1 void process(const auto& x) {
2     auto copy = auto(x); // Explicit decay-copy
3     // ...
4 }

```

147.5 5. Literal Suffixes for size_t

- `uz` or `z` for `size\_t` (unsigned equivalent of `ptrdiff\_t`).
- `z` for `ptrdiff\_t` (signed).

```

1 auto s = 100uz; // std::size_t
2 auto p = 100z;  // std::ptrdiff_t

```

147.6 6. #warning

Standardized preprocessor warning.

```

1 #warning "This feature is experimental"

```


Chapter 148

C23 STD PRINT

Chapter 149

C++23 STD::PRINT & I/O

149.1 1. std::print and std::println

C++ finally gets high-performance, type-safe, and readable console output, replacing `printf` and `iostream`.

149.1.1 1.1 Basic Usage

```
1 #include <print>
2
3 int main() {
4     int id = 42;
5     std::string name = "Alice";
6
7     std::println("User {} has ID {}", name, id);
8     // Output: User Alice has ID 42
9 }
```

149.1.2 1.2 Performance

`std::print` writes directly to the unicode-aware buffer, avoiding stream overhead and allocation. It is often faster than `printf`.

149.1.3 1.3 Formatting

Uses the same format specifications as `std::format` (C++20).

```
1 std::println("Pi: {:.2f}", 3.14159); // Pi: 3.14
2 std::println("Hex: {:#x}", 255);     // Hex: 0xff
```

149.1.4 1.4 Output to Streams

```
1 #include <fstream>
2
3 std::ofstream file("log.txt");
4 std::println(file, "Error code: {}", 500);
```

149.2 2. std::spanstream

A stream wrapper around a fixed buffer (`std::span`), replacing the deprecated `std::strstream`. No allocation.

```
1 #include <spanstream>
2 #include <iostream>
3
4 char buffer[128];
5 std::spanstream ss(buffer);
6
7 ss << "Hello " << 123;
8 std::string_view result = ss.span(); // "Hello 123"
```

Part VII

Volume VII: Advanced Topics & Systems Architecture

Chapter 150

C23 MONADIC OPERATIONS AND EXPECTED

Chapter 151

C++23 MONADIC OPERATIONS & EXPECTED

151.1 1. `std::expected`

The standard way to return values or errors, replacing integer error codes and exceptions in high-performance paths.

151.1.1 1.1 Basics

```
1 #include <expected>
2 #include <string>
3
4 enum class Error { InvalidInput, ConnectionFailed };
5
6 std::expected<int, Error> parse_int(std::string_view input) {
7     if (input.empty()) return std::unexpected(Error::InvalidInput);
8     return std::stoi(std::string(input));
9 }
10
11 int main() {
12     auto result = parse_int("42");
13
14     if (result) {
15         std::cout << "Value: " << *result;
16     } else {
17         std::cout << "Error: " << (int)result.error();
18     }
19 }
```

151.1.2 1.2 Monadic Chain

`and_then`, `transform`, or `_else`.

```
1 auto process = parse_int("42")
2     .and_then([](int i) { return parse_int("10"); }) // Chain potentially
3     .transform([](int i) { return i * 2; })           // Transform value on
4     .or_else([](Error e) { return std::expected<int, Error>(0); }); // Handle
5     error
```

151.2 2. Monadic Operations for `std::optional`

C++23 adds monadic methods to `std::optional` (C++17).

```
1 std::optional<int> get_id();  
2 std::optional<std::string> get_name(int id);  
3  
4 auto name = get_id()  
5     .and_then(get_name)  
6     .value_or("Unknown");
```

Chapter 152

C23 CONTAINERS AND VIEWS

Chapter 153

C++23 CONTAINERS & VIEWS

153.1 1. `std::mdspan`

A non-owning, multidimensional view of a contiguous memory block. Crucial for scientific computing, linear algebra, and image processing.

153.1.1 1.1 Basics

```
1 #include <mdspan>
2 #include <vector>
3
4 int main() {
5     std::vector<int> data(12); // 3x4 matrix
6     std::mdspan m(data.data(), 3, 4);
7
8     m[1, 2] = 42; // Set row 1, col 2
9 }
```

153.1.2 1.2 Layouts

- `std::layout_right` (Row-Major): C/C++ default. Last index varies fastest.
- `std::layout_left` (Column-Major): Fortran/BLAS compatible. First index varies fastest.

153.2 2. `std::flat_map` and `std::flat_set`

Associative containers implemented as sorted vectors.

- **Pros:** contiguous memory, cache-friendly, faster iteration/lookup for small-to-medium datasets.
- **Cons:** $O(N)$ insertion/deletion (vs $O(\log N)$ for `std::map`). Iterators invalidated on insertion.

```
1 #include <flat_map>
2
3 std::flat_map<int, std::string> m;
4 m[1] = "One"; // Insert
```

153.3 3. Ranges Enhancements

- `std::ranges::to`: Convert a range to a container. `cpp` `auto v = some_view | std::ranges::to<std::vector>();`
 - `views::zip`: Iterate multiple ranges in lockstep. `cpp` `std::vector<int> a = {1, 2}, b = {3, 4}; for (auto [x, y] : std::views::zip(a, b)) { // x=1, y=3 ... }`
 - `views::enumerate`: Index + Value. `cpp` `for (auto [idx, val] : std::views::enumerate(data)) { // ... }`
-

Chapter 154

C23 COROUTINES AND STACKTRACE

Chapter 155

C++23 COROUTINES & STACKTRACE

155.1 1. `std::generator`

Standardized generator coroutine. Supports `co_yield` and recursive usage.

155.1.1 1.1 Basic Generator

```
1 #include <generator>
2 #include <ranges>
3
4 std::generator<int> seq(int start) {
5     while (true) {
6         co_yield start++;
7     }
8 }
9
10 int main() {
11     for (int i : seq(0) | std::views::take(5)) {
12         std::println("{} ", i);
13     }
14 }
```

155.1.2 1.2 Recursive Generator

`std::generator` supports `co_yield ranges::elements\of(...)` to yield values from a sub-generator or range efficiently.

155.2 2. `std::stacktrace`

Obtain the call stack at runtime. Useful for logging and debugging.

```
1 #include <stacktrace>
2 #include <print>
3
4 void boom() {
5     auto trace = std::stacktrace::current();
6     std::println("{} ", std::to_string(trace));
7 }
```

Note: Requires compiler/linker flags (e.g., `-lstdc++\libbacktrace` on GCC).

Chapter 156

C23 LIBRARY UTILITIES

Chapter 157

C++23 LIBRARY UTILITIES

157.1 1. `std::unreachable`

Marks code as unreachable. If executed, Undefined Behavior (allows optimization).

```
1 #include <utility>
2
3 enum Color { Red, Green };
4
5 int get_value(Color c) {
6     switch (c) {
7         case Red: return 1;
8         case Green: return 2;
9     }
10     std::unreachable();
11 }
```

157.2 2. `std::to_underlying`

Safely cast an enum class to its underlying integer type.

```
1 enum class Flags : unsigned char { A = 1 };
2 auto val = std::to_underlying(Flags::A); // unsigned char
```

157.3 3. `std::byteswap`

Endianness reversal.

```
1 #include <bit>
2 uint32_t x = 0x12345678;
3 uint32_t y = std::byteswap(x); // 0x78563412
```

157.4 4. `std::stdatomic.h`

C compatibility header for atomics.

157.5 5. `std::move_only_function`

A replacement for `std::function` that works with move-only callables (like lambdas capturing `unique_ptr`).

```
1 #include <functional>
2
3 std::move_only_function<void()> f;
4 auto ptr = std::make_unique<int>(42);
5
6 f = [p = std::move(ptr)]() { /*...*/ }; // OK (std::function would fail)
```

Chapter 158

THE FUTURE C26 PREVIEW

Chapter 159

THE FUTURE - C++26 PREVIEW

As of 2026, the C++26 standard is nearing finalization. Here are the transformative features likely to be included.

159.0.1 13.1 Static Reflection (std::meta)

Reflection allows a program to inspect and modify itself at compile-time. This eliminates the need for external code generators or macros for serialization, ORMs, and enum-to-string conversions.

```
1 #include <meta>
2 #include <iostream>
3 #include <string_view>
4
5 struct Person {
6     std::string name;
7     int age;
8     double salary;
9 };
10
11 // Generic serialization using C++26 Reflection
12 template<typename T>
13 void serialize(const T& obj) {
14     constexpr auto type_info = ^T; // Reflection operator
15
16     template for (constexpr auto member : std::meta::members_of(type_info)) {
17         std::cout << std::meta::name_of(member) << ": "
18             << obj.[member:] << "\n"; // Splicing
19     }
20 }
21
22 int main() {
23     Person p{"Alice", 30, 95000.0};
24     serialize(p);
25     // Output:
26     // name: Alice
27     // age: 30
28     // salary: 95000
29 }
```

159.0.2 13.2 Contracts

Contracts provide a standardized way to specify preconditions, postconditions, and assertions, improving safety and optimizer information.

```
1 // pre: Precondition (Caller must ensure)
```

```

2 // post: Postcondition (Function ensures upon return)
3 // assert: Internal check
4
5 int safe_divide(int a, int b)
6     pre { b != 0 }           // Contract: b must not be zero
7     post(r) { r * b == a }   // Contract: result * divisor equals dividend
8 {
9     return a / b;
10 }
11
12 // Modes:
13 // - enforce: Terminate if violated
14 // - observe: Log/Debug but continue
15 // - ignore: Optimizer hint (assume true)

```

159.0.3 13.3 Senders & Receivers (std::execution)

A unified framework for asynchronous execution, replacing raw threads, futures, and callbacks with a composable pipeline model.

```

1 #include <execution>
2 #include <iostream>
3
4 using namespace std::execution;
5
6 int main() {
7     scheduler auto sch = thread_pool_scheduler{};
8
9     sender auto work = schedule(sch)
10         | then([]{ return 42; })
11         | then([](int i){ return i * 2; })
12         | then([](int i){ std::cout << "Result: " << i << "\n"; });
13
14     // Launch execution
15     std::this_thread::sync_wait(std::move(work));
16
17     return 0;
18 }

```

159.0.4 13.4 Linear Algebra (std::linalg)

Standardized BLAS (Basic Linear Algebra Subprograms) support for high-performance math.

```

1 #include <linalg>
2 #include <mdspan>
3 #include <vector>
4
5 int main() {
6     std::vector<double> A_vec(9), B_vec(3), C_vec(3);
7     // ... fill vectors ...
8
9     std::mdspan A(A_vec.data(), 3, 3);
10    std::mdspan B(B_vec.data(), 3);
11    std::mdspan C(C_vec.data(), 3);
12
13    // Matrix-Vector Multiplication: C = A * B
14    std::linalg::matrix_vector_product(A, B, C);
15
16    return 0;
17 }

```

Chapter 160

VOLUME 07 ADVANCED SYSTEMS

Chapter 161

ADVANCED TEMPLATE METAPROGRAMMING

Chapter 162

ADVANCED TEMPLATE METAPROGRAMMING

162.1 1. The Evolution of TMP

Template Metaprogramming (TMP) is the art of using the compiler to generate code.

- **C++98:** Recursive struct instantiation, `enum` hacks. (Hard).
- **C++11:** `constexpr`, `static_assert`, `using`, Variadic Templates. (Better).
- **C++14:** Variable templates, `auto` return type. (Cleaner).
- **C++17:** `if constexpr`, Fold expressions, `void_t`. (Powerful).
- **C++20:** Concepts (`requires`). (The Holy Grail).

162.2 2. SFINAE (Substitution Failure Is Not An Error)

Before Concepts, SFINAE was the only way to constrain templates.

162.2.1 2.1 `std::enable_if`

```
1 #include <type_traits>
2 #include <iostream>
3
4 // Enable only for integral types
5 template <typename T>
6 typename std::enable_if<std::is_integral<T>::value, void>::type
7 process(T t) {
8     std::cout << "Integral: " << t << "\n";
9 }
10
11 // Enable only for floating point
12 template <typename T>
13 typename std::enable_if<std::is_floating_point<T>::value, void>::type
14 process(T t) {
15     std::cout << "Float: " << t << "\n";
16 }
```

162.2.2 2.2 The void_t Trick (C++17)

Detecting if a type has a member function.

```

1 template <typename T, typename = void>
2 struct has_print : std::false_type {};
3
4 template <typename T>
5 struct has_print<T, std::void_t<decltype(std::declval<T>().print())>> : std::
    true_type {};
6
7 static_assert(has_print<MyClass>::value, "MyClass must have print()");

```

162.3 3. Curiously Recurring Template Pattern (CRTP)

Static polymorphism. The base class knows the derived class type at compile time.

```

1 template <typename Derived>
2 class Base {
3 public:
4     void interface() {
5         // Compile-time dispatch
6         static_cast<Derived*>(this)->implementation();
7     }
8 };
9
10 class Derived : public Base<Derived> {
11 public:
12     void implementation() {
13         std::cout << "Derived impl\n";
14     }
15 };

```

Use Case: Mixins, adding functionality (like equality operators) without virtual overhead.

162.4 4. Policy-Based Design

Designing classes that take “policies” (strategy classes) as template arguments to define behavior.

```

1 template <typename OutputPolicy, typename LanguagePolicy>
2 class HelloWorld : public OutputPolicy, public LanguagePolicy {
3 public:
4     void run() {
5         print(message()); // OutputPolicy::print, LanguagePolicy::message
6     }
7 };

```

162.5 5. Modern TMP with Concepts (C++20)

Replacing SFINAE with readable constraints.

```

1 template<typename T>
2 concept Printable = requires(T t) {
3     { t.print() } -> std::same_as<void>;
4 };
5
6 void process(Printable auto& obj) {
7     obj.print();
8 }

```


Chapter 163

COMPILE TIME PROGRAMMING

Chapter 164

COMPILE-TIME PROGRAMMING

164.1 1. constexpr Evolution

- **C++11:** Single return statement. Very restricted.
- **C++14:** Loops, local variables, multiple returns allowed.
- **C++17:** if constexpr, lambdas can be constexpr.
- **C++20:** Virtual functions, try-catch, dynamic allocation (std::vector, std::string) allowed in constexpr context (transient allocation).

164.2 2. consteval (C++20)

Forces immediate execution. If it can't run at compile time, it's an error.

```
1 consteval int square(int n) { return n * n; }
2
3 int x = square(5); // OK
4 int y = 10;
5 // int z = square(y); // Error: y is not a constant expression
```

164.3 3. Type Traits (<type_traits>)

The building blocks of metaprogramming.

164.3.1 3.1 Queries

- is_integral_v<T>
- is_same_v<T, U>
- is_base_of_v<Base, Derived>

164.3.2 3.2 Transformations

- remove_const_t<T>
- decay_t<T> (Arrays -> pointers, functions -> pointers, remove cv-ref)
- conditional_t<Bool, T, F> (Compile time IF)

164.4 4. std::integral_constant

Wraps a compile-time value as a type.

```
1 using Two = std::integral_constant<int, 2>;
2 using Four = std::integral_constant<int, 4>;
3
4 static_assert(Two::value + Two::value == Four::value);
```

164.5 5. Reflection (C++26 Preview)

Currently, we use libraries like `magic_enum` or macro hacks. C++26 brings static reflection.

```
1 // C++26 Syntax (Proposed)
2 // constexpr auto info = ^MyClass;
3 // for (auto member : info.data_members()) ...
```

Chapter 165

THE CPP MEMORY MODEL

Chapter 166

THE MEMORY MODEL & ATOMICS

166.1 1. The C++ Memory Model

Defined in C++11. It guarantees that if you have a data race, you have Undefined Behavior.

Data Race: Two threads access the same memory location concurrently, at least one is a write, and they are not synchronized (no mutex, no atomics).

166.2 2. Atomic Operations

`std::atomic<T>` ensures individual read/modify/write operations are indivisible.

```
1 std::atomic<int> count = 0;
2 count++; // Thread-safe fetch-add
```

166.3 3. Memory Orderings

This is where C++ becomes “Godhood” level.

- `memory_order_relaxed`: No ordering guarantees. Just atomicity.
- `memory_order_acquire`: (Load) Subsequent reads/writes stay after this load.
- `memory_order_release`: (Store) Prior reads/writes stay before this store.
- `memory_order_acq_rel`: Both.
- `memory_order_seq_cst`: (Default) Sequential Consistency. Global total ordering. Expensive.

166.3.1 3.1 Synchronizes-With

If Thread A stores with `release` and Thread B loads with `acquire`, everything A did before the store is visible to B after the load.

```
1 std::atomic<bool> ready = false;
2 int data = 0;
3
4 void producer() {
5     data = 42;
6     ready.store(true, std::memory_order_release); // "Publish" data
```

```
7 }  
8  
9 void consumer() {  
10     while (!ready.load(std::memory_order_acquire)); // Wait and "Acquire"  
11     assert(data == 42); // Guaranteed to see 42  
12 }
```

166.4 4. Fences

`std::atomic_thread_fence`. Used to enforce ordering without an atomic operation, or to combine with relaxed operations.

Chapter 167

LOCK FREE PROGRAMMING

Chapter 168

LOCK-FREE PROGRAMMING

168.1 1. The Concept

Programming without Mutexes. Guarantees system-wide progress.

- **Lock-Free:** At least one thread always makes progress.
- **Wait-Free:** Every thread makes progress in finite steps.

168.2 2. Compare-And-Swap (CAS)

The primitive of lock-free. `compare\exchange\_weak` VS `compare\exchange\_strong`.

```
1 std::atomic<int> head;
2
3 void push(int new_val) {
4     int old_head = head.load();
5     // Loop until we successfully swap head with new_val
6     while (!head.compare_exchange_weak(old_head, new_val)) {
7         // old_head is updated to current head value automatically
8     }
9 }
```

168.3 3. The ABA Problem

1. Thread 1 reads A.
2. Thread 2 changes A to B, then back to A.
3. Thread 1 CAS(A, new) succeeds, thinking nothing changed.

Solutions: * **Versioned Pointers:** Store `{ptr, count}`. `std::atomic<uint128\_t>` (if supported). * **Hazard Pointers:** Protect pointers currently being read. * **RCU (Read-Copy-Update):** Wait for all readers to finish before reclaiming memory.

168.4 4. Lock-Free Data Structures

- **Lock-Free Stack:** Easy (CAS on head).
- **Lock-Free Queue:** Harder (Head and Tail). Use Michael-Scott Queue algorithm.
- **Lock-Free Hash Map:** Very hard (Split-Ordered Lists).

Chapter 169

ADVANCED CONCURRENCY PATTERNS

Chapter 170

ADVANCED CONCURRENCY PATTERNS

170.1 1. Thread Pools

Spawning threads is expensive (syscall, stack allocation). Thread pools reuse threads.

```
1 // Basic concept
2 class ThreadPool {
3     std::vector<std::thread> workers;
4     std::queue<std::function<void()>> tasks;
5     std::mutex mtx;
6     std::condition_variable cv;
7     // ...
8 };
```

170.2 2. The Actor Model

No shared state. Actors communicate via messages.

- Each Actor has a mailbox (queue) and a thread (or shared thread pool).
- Eliminates locking issues by design.
- Frameworks: CAF (C++ Actor Framework).

170.3 3. Disruptor Pattern

Ring buffer based, high-throughput, low-latency inter-thread messaging. Used in HFT.

- Pre-allocated memory (avoid GC/allocations).
- Single Writer / Multiple Reader or Multiple Writer scenarios.
- Uses memory barriers/fences instead of locks.

170.4 4. Coroutines for Concurrency

Using C++20 Coroutines to write async code that looks sync.

```
1 Task<int> async_algo() {  
2     int a = co_await fetch_a();  
3     int b = co_await fetch_b();  
4     co_return a + b;  
5 }
```

170.5 5. False Sharing

When two threads write to different variables that happen to sit on the same cache line.

Fix: `alignas(64)` (typical cache line size).

```
1 struct alignas(64) PaddedCounter {  
2     std::atomic<int> val;  
3 };
```

Chapter 171

CUSTOM MEMORY ALLOCATORS

Chapter 172

CUSTOM MEMORY ALLOCATORS

172.1 1. Why Custom Allocators?

- **Performance:** `malloc` is general purpose. Specialized allocators are faster.
- **Locality:** Keep related objects close in memory (cache friendly).
- **Fragmentation:** Reduce memory fragmentation.

172.2 2. Linear / Stack Allocator

Moves a pointer forward. $O(1)$ allocation. Deallocation only possible by resetting the whole buffer (LIFO).

```
1 class StackAllocator {
2     char* start;
3     char* current;
4     size_t size;
5 public:
6     void* allocate(size_t n) {
7         if (current + n > start + size) return nullptr;
8         void* ptr = current;
9         current += n;
10        return ptr;
11    }
12    void reset() { current = start; }
13};
```

172.3 3. Pool Allocator

Allocates chunks of fixed size. No fragmentation. $O(1)$ alloc/dealloc (using a free list).

172.4 4. `std::pmr` (Polymorphic Memory Resources)

C++17 feature. Allows changing allocators at runtime without changing the type (e.g., `std::pmr::vector`).

```
1 #include <memory_resource>
2
3 char buffer[1024];
```

```
4 std::pmr::monotonic_buffer_resource pool(buffer, 1024);  
5 std::pmr::vector<int> v(&pool); // Uses stack buffer
```

172.5 5. Alignment

Understanding `alignof`, `alignas`, and `std::max_align_t`.

- Unaligned access can be slow or cause crashes (SIGBUS on ARM).
 - SIMD often requires 16, 32, or 64-byte alignment.
-

Chapter 173

HIGH PERFORMANCE OPTIMIZATION

Chapter 174

HIGH-PERFORMANCE OPTIMIZATION

174.1 1. CPU Caching

Memory is slow. CPU registers are fast. Caches (L1, L2, L3) bridge the gap.

- **Cache Miss:** CPU waits hundreds of cycles for RAM.
- **Data-Oriented Design:** Structure of Arrays (SoA) vs Array of Structures (AoS). SoA is often friendlier to cache and SIMD.

174.2 2. Branch Prediction

CPUs guess which way an `if` will go. If they guess wrong, pipeline flush (expensive).

- **Sorted Data:** Branch prediction loves patterns (TTTTFFFF).
- **Branchless Programming:** Using bitwise ops to avoid branches. `cpp` // Branchy
`if (x > 0) y = 1; else y = 0; // Branchless y = (x > 0);`
- `[[likely]]` / `[[unlikely]]` (C++20).

174.3 3. SIMD (Single Instruction, Multiple Data)

Doing math on 4, 8, or 16 numbers at once.

- **Intrinsics:** `_mm256_add_ps` (AVX). Hard to read.
- **Auto-vectorization:** Compiler does it if code is simple enough.
- **Libraries:** `std::experimental::simd` (C++26?), Highway, Vc.

174.4 4. Link Time Optimization (LTO)

Allows the compiler to inline functions across translation units (object files).

174.5 5. Profile-Guided Optimization (PGO)

1. Compile with instrumentation.
2. Run the program on representative data.
3. Recompile using the profile data. Optimizes hot paths heavily.

c

1. Small Object Optimization (SOO) / Small String Optimization (SSO) M

2. Copy Elision and RVO (Return Value Optimization)

3. Profiling: Measuring before Optimizing Never optimize without data. * **Sampling Profilers (e.g**

Chapter 175

Professional Notes: Chapter 143: Optimization in C++

Section 143.1: Introduction to performance Section 143.2: Empty Base Class Optimization
Section 143.3: Optimizing by executing less code Section 143.4: Using efficient containers Section
143.5: Small Object Optimization

Chapter 176

Professional Notes: Chapter 143: Optimization in C++

Section 143.1: Introduction to performance C and C++ are well known as high-performance languages - largely due to the heavy amount of code customization, allowing a user to specify performance by choice of structure. When optimizing it is important to benchmark relevant code and completely understand how the code will be used. Common optimization mistakes include: Premature optimization: Complex code may perform worse after optimization, wasting time and effort. First priority should be to write correct and maintainable code, rather than optimized code. Optimization for the wrong use case: Adding overhead for the 1% might not be worth the slowdown for the other 99% Micro-optimization: Compilers do this very efficiently and micro-optimization can even hurt the compiler's ability to further optimize the code Typical optimization goals are: To do less work To use more efficient algorithms/structures To make better use of hardware Optimized code can have negative side effects, including: Higher memory usage Complex code -being difficult to read or maintain Compromised API and code design Section 143.2: Empty Base Class Optimization An object cannot occupy less than 1 byte, as then the members of an array of this type would have the same address. Thus `sizeof(T) >= 1` always holds. It's also true that a derived class cannot be smaller than any of its base classes. However, when the base class is empty, its size is not necessarily added to the derived class: `class Base {};` `class Derived : public Base { public: int i; };` In this case, it's not required to allocate a byte for Base within Derived to have a distinct address per type per object. If empty base class optimization is performed (and no padding is required), then `sizeof(Derived) == sizeof(int)`, that is, no additional allocation is done for the empty base. This is possible with multiple base classes as well (in C++, multiple bases cannot have the same type, so no issues arise from that). Note that this can only be performed if the first member of Derived differs in type from any of the base classes. This includes any direct or indirect common bases. If it's the same type as one of the bases (or there's a common base), at least allocating a single byte is required to ensure that no two distinct objects of the same type have the same address.

Section 143.3: Optimizing by executing less code The most straightforward approach to optimizing is by executing less code. This approach usually gives a modest speed-up without changing the time complexity of the code. Even though this approach gives you a clear speedup, this will only give noticeable improvements when the code is called a lot. Removing useless code `void func(const A& a); // Some random function // useless memory allocation + deallocation for the instance auto a1 = std::make_unique(); func(a1.get()); // making use of a stack object prevents auto a2 = A{}; func(&a2);` Version C++14 From C++14, compilers are allowed to optimize this code to remove the allocation and matching deallocation. Doing code only once `std::map<std::string, std::unique_ptr> lookup; // Slow insertion/lookup // Within this function, we will traverse twice through the map lookup an element // and even a third time when it wasn't in` `const A lazyLookupSlow(const std::string& key) { if (lookup.find(key) !=`

```
lookup.cend()) lookup.emplace_back(key, std::make_unique()); return lookup[key].get(); } //
Within this function, we will have the same noticeable effect as the slow variant while going
at double speed as we only traverse once through the code
const A *lazyLookupSlow(const std::string &key) { auto &value = lookup[key]; if (!value) value = std::make_unique(); return
value.get(); }
A similar approach to this optimization can be used to implement a stable version
of unique
std::vector stableUnique(const std::vector &v) { std::vector result; std::set checkU-
nique; for (const auto &s : v) { // As insert returns if the insertion was successful, we can
deduce if the element was already in or not // This prevents an insertion, which will traverse
through the map for every unique element // As a result we can almost gain 50% if v would not
contain any duplicates if (checkUnique.insert(s).second) result.push_back(s); } return result; }
```

Preventing useless reallocating and copying/moving In the previous example, we already prevented lookups in the `std::set`, however the `std::vector` still contains a growing algorithm, in which it will have to realloc its storage. This can be prevented by reserving for the right size.

```
std::vector stableUnique(const std::vector &v) { std::vector result; // By reserving
'result', we can ensure that no copying or moving will be done in the vector // as it will have
capacity for the maximum number of elements we will be inserting // If we make the assump-
tion that no allocation occurs for size zero // and allocating a large block of memory takes the
same time as a small block of memory // this will never slow down the program // Side note:
Compilers can even predict this and remove the checks the growing from the generated code
result.reserve(v.size()); std::set checkUnique; for (const auto &s : v) { // See example above
if (checkUnique.insert(s).second) result.push_back(s); } return result; }
```

Section 143.4: Using efficient containers Optimizing by using the right data structures at the right time can change the time-complexity of the code.

```
// This variant of stableUnique contains a complexity of N log(N)
// N > number of elements in v // log(N) > insert complexity of std::set
std::vector stableUnique(const std::vector &v) { std::vector result; std::set checkUnique; for (const auto &s : v) {
// See Optimizing by executing less code if (checkUnique.insert(s).second) result.push_back(s);
} return result; }
```

By using a container which uses a different implementation for storing its elements (hash container instead of tree), we can transform our implementation to complexity N . As a side effect, we will call the comparison operator for `std::string` less, as it only has to be called when the inserted string should end up in the same bucket.

```
// This variant of
stableUnique contains a complexity of N // N > number of elements in v // 1 > insert com-
plexity of std::unordered_set
std::vector stableUnique(const std::vector &v) { std::vector result;
std::unordered_set checkUnique; for (const auto &s : v) { // See Optimizing by executing less
code if (checkUnique.insert(s).second) result.push_back(s); } return result; }
```

Section 143.5: Small Object Optimization Small object optimization is a technique which is used within low level data structures, for instance the `std::string` (Sometimes referred to as Short/Small String Optimization). It's meant to use stack space as a buffer instead of some allocated memory in case the content is small enough to fit within the reserved space. By adding extra memory overhead and extra calculations, it tries to prevent an expensive heap allocation. The benefits of this technique are dependent on the usage and can even hurt performance if incorrectly used.

Example A very naive way of implementing a string with this optimization would be the following:

```
#include <string>
class string final { constexpr static auto SMALL_BUFFER_SIZE = 16;
bool _isAllocated{false}; //< Remember if we allocated memory
char *_buffer{nullptr}; //< Pointer to the buffer we are using
char _smallBuffer[SMALL_BUFFER_SIZE] = {'\0'}; //< Stack space used for SMALL OBJECT OPTIMIZATION
public: ~string() { if (_isAllocated) delete [] _buffer; }
explicit string(const char *cStyleString) { auto stringSize = std::strlen(cStyleString);
_isAllocated = (stringSize > SMALL_BUFFER_SIZE); if (_isAllocated) _buffer = new char[stringSize];
else _buffer = &_smallBuffer[0]; std::strcpy(_buffer, cStyleString); }
string(string &&rhs) : _isAllocated(rhs._isAllocated), _buffer(rhs._buffer), _smallBuffer(rhs._smallBuffer) //<
Not needed if allocated { if (_isAllocated) { // Prevent double deletion of the memory
rhs._buffer
```

```
= nullptr; } else { // Copy over data std::strcpy(_smallBuffer, rhs._smallBuffer); _buffer =
&_smallBuffer[0]; }
```

```
    } // Other methods, including other constructors, copy constructor, // assignment operators
have been omitted for readability };
```

As you can see in the code above, some extra complexity has been added in order to prevent some new and delete operations. On top of this, the class has a larger memory footprint which might not be used except in a couple of cases. Often it is tried to encode the bool value `_isAllocated`, within the pointer `_buffer` with bit manipulation to reduce the size of a single instance (intel 64 bit: Could reduce size by 8 byte). An optimization which is only possible when its known what the alignment rules of the platform is. When to use? As this optimization adds a lot of complexity, it is not recommended to use this optimization on every single class. It will often be encountered in commonly used, low-level data structures. In common C++11 standard library implementations one can find usages in `std::basic_string<>` and `std::function<>`. As this optimization only prevents memory allocations when the stored data is smaller than the buffer, it will only give benefits if the class is often used with small data. A notable drawback of this optimization is that extra effort is required when moving the buffer, making the move- operation more expensive than when the buffer would not be used. This is especially true when the buffer contains a non-POD type.

Chapter 177

WRITING A C COMPILER BASICS

Chapter 178

WRITING A C++ COMPILER (BASICS)

To understand C++, build a toy compiler.

178.0.1 18.1 Lexical Analysis (Tokenizer)

Converting source code into tokens.

```
1 enum class TokenType { Int, Identifier, Plus, Minus, End };
2
3 struct Token {
4     TokenType type;
5     std::string text;
6 };
7
8 std::vector<Token> tokenize(std::string_view source) {
9     std::vector<Token> tokens;
10    // ... implementation ...
11    return tokens;
12 }
```

178.0.2 18.2 Parsing (Recursive Descent)

Building an Abstract Syntax Tree (AST).

```
1 struct ASTNode { virtual ~ASTNode() = default; };
2 struct BinaryExpr : ASTNode {
3     std::unique_ptr<ASTNode> left, right;
4     char op;
5 };
6
7 // parseExpression() calls parseTerm(), etc.
```

178.0.3 18.3 Semantic Analysis (Types & Scopes)

Before generating code, we must validate it.

Symbol Table:

```
1 struct Symbol { string type; };
2 using Scope = map<string, Symbol>;
3 vector<Scope> scopes; // Stack of scopes
4
5 void enter_scope() { scopes.push_back({}); }
6 void exit_scope() { scopes.pop_back(); }
```

Type Checking: Recursively visit the AST. * `BinaryExpr`: Check `left.type == right.type`.
* `Variable`: Check if exists in symbol table.

Chapter 179

WRITING A GARBAGE COLLECTOR

Chapter 180

WRITING A GARBAGE COLLECTOR

C++ has RAII, but implementing a GC teaches you about the stack and object graph.

180.0.1 29.1 Mark-and-Sweep Basics

1. **Roots:** Pointers on the stack/globals.
2. **Mark:** Traverse object graph from roots, marking reachable objects.
3. **Sweep:** Iterate heap, free unmarked objects.

```
1 struct GCObject {
2     bool marked = false;
3     virtual ~GCObject() = default;
4 };
5
6 class VM {
7     std::vector<GCObject*> heap;
8     std::vector<GCObject*> roots; // Pointers currently on stack
9
10 public:
11     void mark() {
12         for (auto* obj : roots) mark_object(obj);
13     }
14
15     void mark_object(GCObject* obj) {
16         if (!obj || obj->marked) return;
17         obj->marked = true;
18         // ... traverse children ...
19     }
20
21     void sweep() {
22         auto it = std::remove_if(heap.begin(), heap.end(), [](GCObject* obj) {
23             if (!obj->marked) {
24                 delete obj;
25                 return true;
26             }
27             obj->marked = false; // Reset for next cycle
28             return false;
29         });
30         heap.erase(it, heap.end());
31     }
32 };
```

Chapter 181

THE STANDARD LIBRARY FROM SCRATCH

Chapter 182

THE STANDARD LIBRARY FROM SCRATCH

Implementing core STL components to understand their cost.

182.0.1 19.1 Implementing my::vector

Managing raw memory, growth, and construction.

```
1 template<typename T>
2 class Vector {
3     T* data = nullptr;
4     size_t sz = 0;
5     size_t cap = 0;
6
7 public:
8     void push_back(const T& val) {
9         if (sz == cap) {
10             reallocate(cap == 0 ? 1 : cap * 2);
11         }
12         new (data + sz) T(val); // Placement new
13         sz++;
14     }
15
16 private:
17     void reallocate(size_t new_cap) {
18         T* new_data = static_cast<T*>(::operator new(new_cap * sizeof(T)));
19         // Move old elements...
20         // Delete old memory...
21         data = new_data;
22         cap = new_cap;
23     }
24 };
```

182.0.2 19.2 Implementing my::shared_ptr

Understanding the Control Block.

```
1 template<typename T>
2 class SharedPtr {
3     T* ptr;
4     struct ControlBlock {
5         std::atomic<int> ref_count{1};
6     } *cb;
7
8 public:
```

```
9      SharedPtr(T* p) : ptr(p), cb(new ControlBlock()) {}
10
11      SharedPtr(const SharedPtr& other) {
12          ptr = other.ptr;
13          cb = other.cb;
14          if (cb) cb->ref_count++;
15      }
16
17      ~SharedPtr() {
18          if (cb && --cb->ref_count == 0) {
19              delete ptr;
20              delete cb;
21          }
22      }
23  };
```

Chapter 183

Chapter 49: Design Patterns in C++

Design patterns are reusable solutions to common software design problems. In C++, these patterns often leverage strong typing, templates, and the object model to achieve high performance and flexibility.

183.1 49.1 Creational Patterns

183.1.1 1. The Singleton Pattern

Ensures a class has only one instance and provides a global point of access.

```
1 class Singleton {
2 public:
3     static Singleton& getInstance() {
4         static Singleton instance; // Thread-safe in C++11+
5         return instance;
6     }
7 private:
8     Singleton() {} // Private constructor
9     Singleton(const Singleton&) = delete; // No copying
10    void operator=(const Singleton&) = delete;
11};
```

183.1.2 2. Factory Method

Defines an interface for creating an object, but lets subclasses decide which class to instantiate.

183.2 49.2 Structural Patterns

183.2.1 1. Adapter Pattern

Converts the interface of a class into another interface clients expect.

183.2.2 2. Composite Pattern

Composes objects into tree structures to represent part-whole hierarchies.

183.3 49.3 Behavioral Patterns

183.3.1 1. Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

183.3.2 2. Observer Pattern

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.

183.3.3 Professional Notes: Pattern Implementation

1. CRTP (Curiously Recurring Template Pattern)

A powerful C++ idiom for achieving “Static Polymorphism” without the overhead of virtual functions.

```

1 template <typename Derived>
2 class Base {
3 public:
4     void interface() {
5         static_cast<Derived*>(this)->implementation();
6     }
7 };
8
9 class Derived : public Base<Derived> {
10 public:
11     void implementation() {
12         std::cout << "Derived implementation" << std::endl;
13     }
14 };

```

2. Pimpl Idiom (Pointer to Implementation)

A technique to hide the implementation details of a class from its header file, reducing compilation dependencies and improving build times.

```

1 // In Header
2 class Widget {
3     class Impl;
4     std::unique_ptr<Impl> pImpl;
5 public:
6     Widget();
7     ~Widget();
8     void draw();
9 };

```

Chapter 184

Chapter 50: ODR, ADL, and Undefined Behavior

This chapter deconstructs the most subtle and dangerous aspects of the C++ language specification. Understanding these rules is what separates a senior engineer from a novice.

184.1 50.1 The One Definition Rule (ODR)

ODR states that a program shall contain exactly one definition for any variable, function, class type, enumeration type, or template in a given scope.

184.1.1 1. Translation Units

A translation unit (TU) is the result of preprocessing a single source file. * **Variable/Function:** Can be declared in multiple TUs but defined in only one. * **Class/Inline:** Can be defined in multiple TUs as long as the definitions are token-for-token identical.

184.1.2 2. Violations

Violating ODR often leads to **Linker Errors** or, worse, **Undefined Behavior** if the linker chooses one of several conflicting definitions.

184.2 50.2 Argument Dependent Lookup (ADL)

ADL (also known as Koenig Lookup) allows the compiler to find functions in the namespaces of their arguments.

```
1 namespace MyNamespace {  
2     struct MyType {};  
3     void func(MyType) {}  
4 }  
5  
6 int main() {  
7     MyNamespace::MyType obj;  
8     func(obj); // OK: ADL finds func in MyNamespace  
9 }
```

184.2.1 The “std::swap” Idiom

ADL is the reason we write:

```
1 using std::swap;
2 swap(obj1, obj2);
```

This allows the compiler to use a custom `swap` in the object’s namespace if it exists, falling back to `std::swap` otherwise.

184.3 50.3 Undefined Behavior (UB)

UB is code for which the C++ standard imposes no requirements. The compiler is free to assume UB never happens, leading to aggressive (and potentially catastrophic) optimizations.

184.3.1 Common Sources of UB:

- **Dereferencing NULL:** `*ptr` when `ptr == nullptr`.
- **Out-of-bounds access:** `arr[10]` for an array of size 10.
- **Use after free:** Accessing memory after `delete`.
- **Signed integer overflow:** `INT_MAX + 1`.
- **Data Races:** Unsynchronized access from multiple threads.

c

1. Unspecified vs. Implementation-Defined Behavior * **Unspecified:** The standard provides multiple

2. Strict Aliasing Rule Compilers assume that pointers of different types (e.g., `int*` and `float*`)

184.3.2 Professional Notes: Subtle Quirks

1. Unspecified Behavior vs. UB

- **Order of Evaluation:** The order in which function arguments are evaluated is unspecified. `f(a++, a++)` is unspecified (but if it modifies the same variable twice, it’s UB).
- **Static Initialization Order Fiasco:** The order in which globals in different TUs are initialized is unspecified. Use the **Singleton pattern** or **Nifty Counter** idiom to solve this.

2. Deep Recursion and Stack Frames

Every recursive call pushes a new frame onto the stack. * **Stack Depth:** Limited by the OS (e.g., 8MB on Linux). * **Godhood Solution:** Use a manual stack with `std::stack` on the heap to avoid stack overflow for extremely deep traversals.

Chapter 185

Chapter 51: Linkage, Attributes, and C Incompatibilities

This chapter covers the rules for how identifiers are shared across files, how to give hints to the compiler, and the subtle ways C++ differs from its parent language, C.

185.1 51.1 Linkage Specifications

Linkage determines whether a name refers to the same entity across different scopes or translation units.

185.1.1 1. Types of Linkage

- **External Linkage:** The name can be referred to from other translation units (e.g., non-static global variables, non-inline functions).
- **Internal Linkage:** The name is only visible within its own translation unit (e.g., `static` globals, variables in anonymous namespaces).
- **No Linkage:** The name is local to its scope (e.g., local variables).

185.1.2 2. `extern "C"`

Tells the C++ compiler to use C-style linkage (no name mangling). This is essential for calling C functions from C++ or vice versa.

185.2 51.2 C++ Attributes (`[[...]]`)

Attributes provide a standardized way to provide extra information to the compiler to improve optimization or warnings.

185.2.1 Common Attributes:

- `[[nodiscard]]`: Warns if the return value of a function is ignored.
- `[[maybe_unused]]`: Suppresses warnings for unused variables.
- `[[deprecated("reason")]]`: Marks an entity as obsolete.

- `[[fallthrough]]`: Signals intentional fallthrough in a switch statement.
 - `[[likely]]` / `[[unlikely]]` (C++20): Hints to the optimizer about branch probability.
-

185.3 51.3 C Incompatibilities

While C++ is mostly a superset of C, there are several “breaking” differences.

185.3.1 1. Implicit Conversions

- C: Allows implicit conversion from `void*` to any other pointer type.
- C++: Requires an explicit cast.

185.3.2 2. Struct Definitions

- C: Requires the `struct` keyword every time you refer to the type (unless `typedef`’d).
- C++: The struct name becomes a type name automatically.

185.3.3 3. Functions with No Arguments

- C: `int func()` means a function taking an *unspecified* number of arguments.
- C++: `int func()` means a function taking *no* arguments (equivalent to `int func(void)`).

c

1. Static vs. Dynamic Linking * **Static:** Object code is copied into the executable at build time. Lea
 #### 2. Linker Symbols and Mangling Use tools like `nm` or `objdump`

Chapter 186

Chapter 52: Build Systems and Tooling

Mastering the C++ ecosystem requires knowledge of how to manage large-scale projects and automate the compilation of millions of lines of code.

186.1 52.1 The Build Process (Architectural View)

A build system automates the invocation of the compiler, assembler, and linker.

186.1.1 1. Makefile (The Foundation)

`make` uses a dependency graph to determine which files need recompilation. It only rebuilds files whose source has changed.

```
1 # Simple Makefile
2 app: main.o utils.o
3     g++ -o app main.o utils.o
4
5 main.o: main.cpp
6     g++ -c main.cpp
7
8 utils.o: utils.cpp
9     g++ -c utils.cpp
```

186.1.2 2. CMake (The Modern Standard)

CMake is a “Meta-build” system. It generates Makefiles, Ninja files, or Visual Studio solutions from a high-level `CMakeLists.txt`.

```
1 cmake_minimum_required(VERSION 3.10)
2 project(MyProject)
3 add_executable(app main.cpp utils.cpp)
```

186.2 52.2 Linker Errors (Common Traps)

Linker errors happen after successful compilation when the linker cannot resolve symbols.

186.2.1 1. undefined reference to 'X'

The compiler saw a declaration of `x`, but the linker couldn't find its definition. * **Cause:** Missing source file in build, missing library in link command, or signature mismatch (e.g., `const` mismatch in parameters).

186.2.2 2. multiple definition of 'X'

Violates the One Definition Rule (ODR). * **Cause:** Defining a non-inline function in a header file included by multiple TUs. * **Fix:** Add `inline` or move definition to a `.cpp` file.

1. Sanitizers and Analyzers Modern toolchains include powerful debugging tools: * **AddressSanitizer**
 ##### 2. Precompiled Headers (PCH) Compiler
 ##### 3. Compilation Databases The `compile\commands.json`

Chapter 187

**VOLUME 08 SPECIALIZED
MASTERY**

Part VIII

Volume VIII: Specialized Domains & Expert Mastery

Chapter 188

DISTRIBUTED C

Chapter 189

DISTRIBUTED C++

Moving beyond a single process: Networking, RPC, and Consensus.

189.0.1 16.1 Serialization (Binary Protocols)

Efficiently packing data for network transmission.

```
1 #include <vector>
2 #include <cstring>
3 #include <string>
4
5 // Simple Binary Serializer
6 class Buffer {
7     std::vector<uint8_t> data;
8 public:
9     template<typename T>
10    void write(const T& val) {
11        static_assert(std::is_trivially_copyable_v<T>);
12        const uint8_t* ptr = reinterpret_cast<const uint8_t*>(&val);
13        data.insert(data.end(), ptr, ptr + sizeof(T));
14    }
15
16    void write_string(const std::string& s) {
17        write<uint32_t>(s.size());
18        const uint8_t* ptr = reinterpret_cast<const uint8_t*>(s.data());
19        data.insert(data.end(), ptr, ptr + s.size());
20    }
21
22    const uint8_t* begin() const { return data.data(); }
23    size_t size() const { return data.size(); }
24 };
```

189.0.2 16.2 RPC (Remote Procedure Call) Concept

Calling a function on another machine.

Stub Interface:

```
1 // User Code
2 // auto result = service.Add(5, 3);
3
4 // Generated Stub
5 int Add(int a, int b) {
6     Buffer buf;
7     buf.write(101); // Function ID for 'Add'
8     buf.write(a);
9     buf.write(b);
10    return network.send_and_wait(buf); // Blocks
}
```

```
11 }
```

189.0.3 16.3 Consensus (Raft Basics)

Distributed systems need to agree on state.

Raft State Machine:

```
1 enum class State { Follower, Candidate, Leader };
2
3 struct Node {
4     State state = State::Follower;
5     int current_term = 0;
6     int voted_for = -1;
7
8     void on_timeout() {
9         if (state == State::Follower) {
10             state = State::Candidate;
11             current_term++;
12             voted_for = my_id;
13             request_votes();
14         }
15     }
16 };
```

Chapter 190

NETWORKING FROM SCRATCH

Chapter 191

NETWORKING FROM SCRATCH

Understanding `asio` requires understanding BSD Sockets.

191.0.1 28.1 Berkeley Sockets API

The foundation of the Internet.

```
1 #include <sys/socket.h>
2 #include <netinet/in.h>
3 #include <unistd.h>
4
5 int main() {
6     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
7
8     sockaddr_in address;
9     address.sin_family = AF_INET;
10    address.sin_addr.s_addr = INADDR_ANY;
11    address.sin_port = htons(8080);
12
13    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
14    listen(server_fd, 3);
15
16    int new_socket = accept(server_fd, nullptr, nullptr);
17    char buffer[1024] = {0};
18    read(new_socket, buffer, 1024);
19
20    // Send HTTP response
21    const char* hello = "HTTP/1.1 200 OK\nContent-Type: text/plain\n\nHello!";
22    write(new_socket, hello, strlen(hello));
23
24    close(new_socket);
25    close(server_fd);
26    return 0;
27 }
```

191.0.2 28.2 Non-Blocking I/O & Epoll (Linux)

How Nginx/Node.js handle 10k connections.

```
1 // 1. Create epoll instance
2 int epoll_fd = epoll_create1(0);
3
4 // 2. Add server socket
5 epoll_event event;
6 event.events = EPOLLIN; // Read available
7 event.data.fd = server_fd;
8 epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &event);
```

```
9
10 // 3. Event Loop
11 while (true) {
12     epoll_event events[10];
13     int event_count = epoll_wait(epoll_fd, events, 10, -1);
14     for (int i = 0; i < event_count; i++) {
15         if (events[i].data.fd == server_fd) {
16             // Accept new connection...
17         } else {
18             // Read data...
19         }
20     }
21 }
```

Chapter 192

C IN THE CLOUD

Chapter 193

C++ IN THE CLOUD

Modern C++ is a first-class citizen in Cloud Native architectures.

193.0.1 20.1 Microservices with C++

Using frameworks like **Drogon** or **Userver** (Yandex) for high-throughput services.

Example: Simple HTTP Endpoint (Drogon style)

```
1 // Controller
2 void Handler::get(const HttpRequestPtr& req, std::function<void (const
  HttpResponsePtr &)> &&callback) {
3     auto resp = HttpResponse::newHttpResponse();
4     resp->setBody("Hello from High-Performance Microservice!");
5     callback(resp);
6 }
```

193.0.2 20.2 Serverless C++ (AWS Lambda)

Using the AWS Lambda C++ Runtime to run native binaries. * **Cold Start:** < 5ms (vs 100ms+ for Java/Node). * **Cost:** Lower duration due to speed.

```
1 #include <aws/lambda-runtime/runtime.h>
2
3 aws::lambda_runtime::invocation_response handler(aws::lambda_runtime::
  invocation_request const& req) {
4     return aws::lambda_runtime::invocation_response::success("Processed!", "
  application/json");
5 }
6
7 int main() {
8     aws::lambda_runtime::run_handler(handler);
9     return 0;
10 }
```

Chapter 194

CROSS-PLATFORM DEVELOPMENT

Chapter 195

CROSS-PLATFORM DEVELOPMENT

Write once, run everywhere (Desktop, Web, Mobile).

195.0.1 21.1 WebAssembly (Wasm) with Emscripten

Compiling C++ to run in the browser.

```
1 emcc main.cpp -o index.html -s WASM=1
```

```
1 #include <emscripten/emscripten.h>
2
3 extern "C" {
4     EMSCRIPTEN_KEEPALIVE
5     int add(int a, int b) {
6         return a + b; // callable from JavaScript
7     }
8 }
```

195.0.2 21.2 Mobile C++ (Android NDK & JNI)

Integrating C++ with Java/Kotlin.

```
1 #include <jni.h>
2
3 extern "C" JNIEXPORT jstring JNICALL
4 Java_com_example_myapp_MainActivity_stringFromJNI(JNIEnv* env, jobject /* this
5     */) {
6     return env->NewStringUTF("Hello from C++");
7 }
```

Chapter 196

GUI DEVELOPMENT WITH C

Chapter 197

GUI DEVELOPMENT WITH C++

Building desktop applications and tools.

197.0.1 22.1 Qt Framework (Retained Mode)

Qt uses a unique Signal/Slot mechanism (via MOC - Meta-Object Compiler).

```
1 // MainWindow.h
2 class MainWindow : public QMainWindow {
3     Q_OBJECT // Macro for MOC
4 public:
5     MainWindow(QWidget *parent = nullptr);
6 public slots:
7     void handleButton(); // Slot
8 };
9
10 // MainWindow.cpp
11 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
12     QPushButton *button = new QPushButton("Click me", this);
13     connect(button, &QPushButton::clicked, this, &MainWindow::handleButton);
14 }
```

197.0.2 22.2 Dear ImGui (Immediate Mode)

Ideal for game engines and internal tools. Re-renders UI every frame.

```
1 // Main Loop
2 void Render() {
3     ImGui::Begin("Debug Tools");
4     static float col[3] = { 0.0f, 0.0f, 0.0f };
5     ImGui::ColorEdit3("Background Color", col);
6     if (ImGui::Button("Reset")) {
7         col[0] = col[1] = col[2] = 0.0f;
8     }
9     ImGui::End();
10 }
```


Chapter 198

SCIENTIFIC COMPUTING GPU

Chapter 199

SCIENTIFIC COMPUTING & GPU

C++ is the language of high-performance math.

199.0.1 23.1 Eigen (Linear Algebra)

Template-heavy library that avoids temporaries using Expression Templates.

```
1 #include <Eigen/Dense>
2 using Eigen::MatrixXd;
3
4 void solve_system() {
5     MatrixXd A(3, 3);
6     A << 1, 2, 3,
7         4, 5, 6,
8         7, 8, 10;
9
10    Eigen::VectorXd b(3);
11    b << 3, 3, 4;
12
13    Eigen::VectorXd x = A.colPivHouseholderQr().solve(b);
14 }
```

199.0.2 23.2 CUDA (GPU Programming)

Running C++ directly on NVIDIA GPUs.

```
1 // Kernel (runs on GPU)
2 __global__ void vectorAdd(float* A, float* B, float* C, int N) {
3     int i = blockDim.x * blockIdx.x + threadIdx.x;
4     if (i < N) C[i] = A[i] + B[i];
5 }
6
7 // Host (runs on CPU)
8 void launch_kernel(float* d_A, float* d_B, float* d_C, int N) {
9     int threadsPerBlock = 256;
10    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
11    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
12 }
```

Chapter 200

INTEROPERABILITY

Chapter 201

INTEROPERABILITY

C++ is the dark matter of the software universe: it binds everything together.

201.0.1 35.1 Python Bindings (pybind11)

Bridging the gap between Python's ease of use and C++'s raw power. * **The Problem:** Python objects are ref-counted (PyObject*). C++ has RAIL. Who owns the pointer? * **Return Value Policies:** * `py::return_value_policy::copy`: Python gets a copy (Safe). * `py::return_value_policy::reference`: Python references C++ memory (Dangerous if C++ deletes it). * `py::return_value_policy::take_ownership`: Python takes over delete.

```
1 #include <pybind11/pybind11.h>
2 namespace py = pybind11;
3
4 struct Pet {
5     std::string name;
6     Pet(const std::string &name) : name(name) { }
7 };
8
9 PYBIND11_MODULE(example, m) {
10     py::class_<Pet>(m, "Pet")
11         .def(py::init<const std::string &>())
12         .def_readwrite("name", &Pet::name);
13 }
```

201.0.2 35.2 Java Native Interface (JNI)

The bridge to the Enterprise (and Android). * **Cost:** Crossing the JVM boundary is expensive (pointer chasing, pinning objects). * **Pitfall:** `JNIEnv*` is thread-local. Do not share it between threads. * **Local References:** JNI creates local refs for every object returned. If you don't `DeleteLocalRef` in a loop, the JVM OOMs.

```
1 extern "C" JNIEXPORT jstring JNICALL
2 Java_com_example_MyClass_nativeMethod(JNIEnv *env, jobject thiz) {
3     return env->NewStringUTF("Hello from C++");
4 }
```

201.0.3 35.3 Stable C ABI

To speak to Rust, C#, or Go, use the lingua franca: C. * **extern "C":** Disables C++ name mangling (e.g., `_Z3foov` becomes `foo`). * **Struct Layout:** Use `StandardLayoutType` structs (no virtuals, all public).

Chapter 202

SECURITY ENGINEERING

Chapter 203

SECURITY ENGINEERING

203.0.1 36.1 Fuzzing (libFuzzer / AFL++)

Unit tests test what you *know*. Fuzzing tests what you *don't*. * **Coverage-Guided:** The fuzzer instruments binaries to see which inputs explore new code paths. * **Sanitizers:** Always fuzz with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled.

203.0.2 36.2 Cryptography & Timing Attacks

NEVER write your own crypto. Use libsodium or BoringSSL. * **The Trap:** `memcmp(hash1, hash2, 32)` exits early if the first byte differs. * **The Attack:** Attacker measures time. If it takes longer, they guessed the first byte right. * **The Fix:** Constant-time comparison. `cpp // libsodium`
`'s constant time check if (crypto\_verify\_32(hash1, hash2) != 0) \{ /* Error */ \}`

203.0.3 36.3 Side-Channel Mitigations (Spectre)

Modern CPUs execute instructions speculatively. * **Scenario:** `if (x < array\_len) \{ val = array[x]; \}` * **Attack:** CPU predicts true, loads `array[x]` (where `x` is out of bounds secret). Even if checks fail, `array[x]` is now in L1 cache. * **Mitigation:** `LFENCE` (Load Fence) or `std::clamp` indices to 0 on failure (masking).

Chapter 204

SPECIALIZED DOMAINS

Chapter 205

SPECIALIZED DOMAINS

205.0.1 37.1 Game Development (Data-Oriented Design)

OOP is cache-poison. Games use **Entity-Component-Systems (ECS)**. * **AoS (Array of Structs)**: [Pos, Vel], [Pos, Vel] -> Bad stride. * **SoA (Structure of Arrays)**: [Pos, Pos], [Vel, Vel] -> SIMD friendly.

205.0.2 37.2 Embedded Systems

- **Memory-Mapped I/O**: Casting integer addresses to pointers.
- `volatile`: Tells compiler “Hardware changes this value, do not optimize reads”.
- **Freestanding Implementation**: No OS, no heap (`malloc` is a myth).

205.0.3 37.3 High-Frequency Trading (HFT)

- **Kernel Bypass**: Solarflare `OpenOnload` maps NIC ring buffers directly to userspace, skipping the Linux Kernel (save ~2-3 microseconds).
- **Warm-up**: Pinging order gateways to ensure the TCP congestion window is open and CPU is in C0 state (max turbo).

205.0.4 37.4 Automotive (MISRA C++ / AUTOSAR)

- **No Dynamic Memory**: All containers have fixed capacity (`etl::vector` or `boost::static_vector`).
- **No Exceptions**: `try-catch` adds binary size and non-deterministic unwind paths. Use `std::expected` or error codes.
- **Static Analysis**: Code must pass MISRA-2008 rules (e.g., “Rule 5-0-15: Array indexing shall be checked”).

```
1 // Stack-only pattern (Automotive safe)
2 template <typename T, size_t N>
3 class FixedVector {
4     T data[N];
5     size_t count = 0;
6 public:
7     void push_back(const T& val) {
8         if (count < N) data[count++] = val;
9     }
10 };
```

205.1 FINAL COMPREHENSIVE CHECKLIST

205.1.1 C++98/03 Foundation

- Variables and basic types
- Operators and control flow
- Functions and overloading
- Arrays and pointers
- Classes and objects
- Constructors and destructors
- Inheritance
- Virtual functions and polymorphism
- STL containers (vector, list, map, set)
- Algorithms
- Strings and I/O

205.1.2 C++11 Major Features

- Auto type deduction
- `nullptr` and `nullptr_t`
- Uniform initialization `{}`
- Range-based for loops
- Smart pointers (`unique_ptr`, `shared_ptr`)
- Move semantics
- Rvalue references
- Lambda functions
- Variadic templates
- `std::array` and `std::tuple`
- `std::unordered_map` and `std::unordered_set`

205.1.3 C++14 Improvements

- Generic lambdas
- Return type deduction
- `std::make_unique`
- Digit separators (1'000'000)
- `decltype(auto)`

205.1.4 C++17 Modern Features

- Structured bindings
- `std::optional`
- `std::variant`
- `std::any`
- `if constexpr`
- Fold expressions
- Filesystem library
- Parallel algorithms
- `std::string_view`

205.1.5 C++20 Revolutionary

- Concepts and constraints
- Ranges
- Coroutines
- Modules
- Spaceship operator `<=>`
- Designated initializers
- Requires expressions

205.1.6 C++23 Latest

- Deducing `this`
- `std::expected`
- Literal classes in `constexpr`

205.1.7 Advanced Concepts

- Template metaprogramming
- CRTP (Curiously Recurring Template Pattern)
- SFINAE (Substitution Failure Is Not An Error)
- Type traits
- Memory management (stack vs heap)
- Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`)
- Move semantics and forwarding
- Perfect forwarding with `std::forward`

205.1.8 Concurrency

- Threading basics
- Mutexes and locks
- Condition variables
- Atomic operations
- Memory ordering
- Lock-free programming

205.1.9 Performance & Optimization

- Memory profiling
- Cache optimization
- SIMD and vectorization
- Compiler flags (-O2, -O3)
- Profiling tools (perf, valgrind)

205.1.10 STL Mastery

- All container types
- All algorithms
- Iterators
- Custom comparators
- Ranges (C++20)

205.1.11 Design Patterns

- Singleton
- Factory
- Observer
- Strategy
- CRTP

205.1.12 Professional Development

- CMake build system
- Testing frameworks (Google Test)
- Debugging (gdb)
- Profiling
- Code organization

- RAII pattern
 - Error handling
 - Memory leak detection
-

205.2 Key Insights for C++ Mastery

1. **RAII** = Guaranteed resource cleanup
 2. **Smart Pointers** = No manual memory management
 3. **Move Semantics** = Zero-copy optimization
 4. **Const Correctness** = Prevents bugs, enables optimizations
 5. **Templates** = Compile-time computation
 6. **Concepts** (C++20) = Type-safe constraints
 7. **Ranges** (C++20) = Composable algorithms
 8. **Coroutines** (C++20) = Async programming made easy
 9. **Atomic Operations** = Safe concurrent access
 10. **Performance** = Measure, profile, then optimize
-

205.3 Learning Path

1. **Week 1-2:** C++98 basics (variables, control flow, functions)
 2. **Week 3-4:** Classes and OOP (constructors, inheritance, polymorphism)
 3. **Week 5:** STL containers and algorithms
 4. **Week 6-7:** C++11 (smart pointers, lambdas, move semantics)
 5. **Week 8:** C++14 and C++17 features
 6. **Week 9:** C++20 features (concepts, ranges)
 7. **Week 10+:** Advanced topics (metaprogramming, concurrency, optimization)
-

205.4 Resources

205.4.1 Official Documentation

- cppreference.com - C++ standard library reference
- en.cppreference.com - Excellent resource
- isocpp.org - C++ standards committee

205.4.2 Books

- “A Tour of C++” by Bjarne Stroustrup
- “Effective Modern C++” by Scott Meyers
- “C++ Concurrency in Action” by Anthony Williams

205.4.3 Practice

- LeetCode.com
- HackerRank.com
- Codeforces.com
- ProjectEuler.net

You are now equipped to master C++ from absolute zero to expert level!

Last Updated: December 2025 C++ Versions Covered: C++98 through C++23

Chapter 206

ABA PROBLEM MEMORY RECLAMATION

Chapter 207

ABA PROBLEM & MEMORY RECLAMATION

In the rarefied air of lock-free programming, the **ABA Problem** is the dragon that guards the gate. Conquering it requires understanding the very fabric of memory lifecycles.

207.0.1 1. The ABA Problem Explained

The Compare-And-Swap (CAS) primitive (`std::atomic::compare_exchange_weak`) checks if a value is *equal* to an expected value. It does **not** check if it is the *same* object.

The Scenario: 1. Thread 1 reads pointer A from a lock-free stack top. 2. Thread 1 is preempted. 3. Thread 2 pops A, frees it, then pushes B, then pushes a *new* object allocated at address A (recycled memory). 4. Thread 1 wakes up, performs CAS. The address is still A. CAS succeeds. 5. **Catastrophe:** Thread 1 has popped the *new* A, but its local logic assumes it's the *old* A (e.g., pointing to B as the next node). The stack is now corrupted.

207.0.2 2. Solution I: Tagged Pointers (Version Counters)

Pack a version counter into the unused bits of a pointer (usually top 16 bits on 64-bit systems). * **Mechanism:** Every modification increments the counter. `Ptr(A, v1) != Ptr(A, v2)`. * **Limitation:** Reduces addressable memory space; requires platform-specific bit manipulation.

```
1 // Example of a 64-bit tagged pointer
2 struct TaggedPtr {
3     uint64_t data; // 48 bits pointer, 16 bits tag
4
5     TaggedPtr(void* ptr, uint16_t tag) {
6         data = (reinterpret_cast<uint64_t>(ptr) & 0x0000FFFFFFFFFFFF) | (
7             static_cast<uint64_t>(tag) << 48);
8     }
9
10    void* get_ptr() const { return reinterpret_cast<void*>(data & 0
11        x0000FFFFFFFFFFFF); }
12    uint16_t get_tag() const { return static_cast<uint16_t>(data >> 48); }
```

207.0.3 3. Solution II: Hazard Pointers (The Gold Standard)

A **Hazard Pointer (HP)** is a thread-local signal saying “I am reading this object, do not delete it.”

The Protocol: 1. **Reader:** publish the pointer P to a thread-local HP slot. 2. **Reader:** Verify P is still in the data structure. If not, retry. 3. **Writer (Deleter):** Unlink P from the

structure. 4. **Writer:** Check all other threads' HPs. * If `p` is found in any HP, add `p` to a "Retire List" (do not `delete` yet). * If `p` is not found, `delete` immediately. 5. **Cleanup:** Periodically scan the Retire List and free objects no longer protected by HPs.

- **Pros:** Wait-free readers, deterministic memory bound.
- **Cons:** Heavy memory barrier usage (Store-Load fence needed after publishing HP).

207.0.4 4. Solution III: Epoch-Based Reclamation (EBR)

Used by `malloc` implementations and databases (like Silo). * **Concept:** A Global Epoch counter (`E`) and per-thread Local Epochs (`e_t`). * **Operation:** 1. Global Epoch `E` increments periodically. 2. Threads update `e_t = E` when entering a critical section. 3. Objects retired in Epoch `E` can be safely deleted when all threads have reached Epoch `E+1` or higher. * **Pros:** Extremely fast (just checking integers). * **Cons:** One stalled thread prevents *all* memory reclamation (OOM risk).

Chapter 208

TEMPLATE METAPROGRAMMING PATTERNS

Chapter 209

TEMPLATE METAPROGRAMMING PATTERNS

Moving computation from runtime to compile-time saves cycles and enables zero-cost abstractions.

209.0.1 1. Expression Templates (Lazy Evaluation)

Avoid temporary objects in math operations. **Naive:** `Vector sum = A + B + C;` creates `tmp = A+B`, then `sum = tmp+C`. Allocations! **Expression Template:** `A + B` returns a lightweight `Sum<Vec, Vec>` object.

```
1 template <typename L, typename R>
2 struct Sum {
3     const L& l; const R& r;
4     auto operator[](size_t i) const { return l[i] + r[i]; }
5 };
6
7 template <typename L, typename R>
8 auto operator+(const L& l, const R& r) {
9     return Sum<L, R>{l, r};
10 }
11
12 // Usage
13 // Vector result = A + B + C;
14 // Becomes: result[i] = A[i] + B[i] + C[i] in a single loop!
```

209.0.2 2. Type Erasure (The `std::any` Pattern)

Polymorphism without inheritance. * **Technique:** Hold a `void*` or a `unique_ptr<Base>`, where `Base` is an abstract class inside a templated wrapper. * **Example:** `std::function`, `std::any`.

209.0.3 3. The Detection Idiom (`void_t`)

Check if a type has a member function or typedef at compile time.

```
1 template <typename, typename = std::void_t<>>
2 struct has_serialize : std::false_type {};
3
4 template <typename T>
5 struct has_serialize<T, std::void_t<decltype(std::declval<T>().serialize())>> :
6     std::true_type {};
```

```
6
7 static_assert(has_serialize<MyClass>::value, "MyClass must implement serialize
  ("));
```

Note: In C++20, simply use Concepts.

209.0.4 4. Policy-Based Design

Compose behavior via template arguments.

```
1 template <typename T, typename CheckingPolicy, typename ThreadingPolicy>
2 class SmartPtr : public CheckingPolicy, public ThreadingPolicy {
3     T* ptr;
4     // ...
5 };
6 // User chooses: SmartPtr<int, NoCheck, MultiThreaded>
```

Chapter 210

HIGH-PERFORMANCE DATA STRUCTURES

Chapter 211

HIGH-PERFORMANCE DATA STRUCTURES

When `std::unordered_map` is too slow, we descend into the hardware.

211.0.1 1. The Disruptor (Ring Buffer on Steroids)

A lock-free ring buffer designed for high-throughput messaging (LMAX Trading). * **Key Concept:** Pre-allocated memory, sequence numbers, and “barriers”. * **False Sharing Prevention:** Padding sequence counters to 64 bytes (cache line). * **Batching:** Consumers process up to the known “published” sequence.

211.0.2 2. Swiss Table (Open Addressing + Metadata)

Used in `absl::flat_hash_map`. * **Structure:** Arrays of control bytes (metadata) and data slots. * **Control Byte:** 7 bits of hash + 1 bit for empty/deleted. * **SIMD Probing:** Load 16 control bytes into a vector register (SSE/AVX). Compare all 16 tags in parallel to find the slot. * **Result:** Drastically fewer cache misses than chaining.

211.0.3 3. Burst Tries / Judy Arrays

Cache-efficient digital trees (tries) for integer keys. * **Idea:** Nodes dynamically change type based on population (Linear list -> Bitmap -> Sub-trie).

211.0.4 4. Slot Map

O(1) insertion, deletion, and access with stable “handles” (indices) instead of pointers. * **Generational Indices:** Handle = [Index | Generation]. Prevents “Dangling Reference” equivalent (accessing a slot that was re-used for a new object).



Chapter 212

REAL-TIME AUDIO SIGNAL PROCESSING

Chapter 213

REAL-TIME AUDIO & SIGNAL PROCESSING

The Golden Rule: In the audio callback, thou shalt not: 1. **Block** (No Mutexes). 2. **Allocate** (No `malloc/new`). 3. **Perform I/O** (No file reads, no `printf`).

213.0.1 1. Lock-Free IPC (SPSC Queue)

Communication between the UI Thread (writes parameters) and Audio Thread (reads parameters) must be wait-free. * **Structure:** Single-Producer Single-Consumer Circular Buffer. * **Atomic Indices:** `head` (write) and `tail` (read) are `std::atomic<size_t>`.

213.0.2 2. SIMD for DSP

Digital Signal Processing (Filters, FFT) is purely math-bound. * **Auto-vectorization:** Help the compiler (restrict pointers, fixed loop bounds). * **Intrinsics:** Manually using `<immintrin.h>` for AVX-512 processing of 16 samples per cycle. * **Biquad Filter:** The workhorse of EQ.

```
cpp // Processing 4 stereo channels (8 floats) at once with AVX    \_ \_m256 samples = \_mm256_load_ps(input_ptr);    \_ \_m256 result = \_mm256_add_ps(\_mm256_mul_ps(samples, b0), ...);
```

213.0.3 3. Double Buffering

For spectral analysis (FFT) which requires blocks (e.g., 1024 samples), while audio comes in small chunks (e.g., 64 samples). * **Technique:** Fill Buffer A. When full, signal worker thread to process A, swap to filling Buffer B.



Chapter 214

ROBOTICS ROS2 DEVELOPMENT

Chapter 215

ROBOTICS & ROS2 DEVELOPMENT

Robotics is where Soft Real-Time (Navigation) meets Hard Real-Time (Motor Control).

215.0.1 1. ROS2 Architecture & DDS

Robot Operating System 2 (ROS2) runs on top of DDS (Data Distribution Service). * **Nodes:** Independent processes. * **Topics:** Pub/Sub channels. * **Services:** RPC-style calls.

215.0.2 2. Zero-Copy Transport (Iceoryx)

Standard ROS2 serialization is slow for large data (LiDAR point clouds, 4K video). * **Solution:** Shared Memory. * **Mechanism:** 1. Publisher requests a memory chunk from shared segment. 2. Publisher writes data directly. 3. Publisher sends the *offset* (pointer) to Subscriber. 4. Subscriber reads directly. **Zero copies.**

215.0.3 3. Real-Time Executors

Standard ROS2 executors can suffer from priority inversion. * **Callback-group-level Executor:** Prioritize “Safety Stop” topic callbacks over “Camera Logging” callbacks.

215.0.4 4. Custom Allocators (`std::pmr`)

In the real-time control loop (1kHz+), heap fragmentation is fatal. * **Pattern:** Use `std::pmr::monotonic_buffer_resource` on the stack for message generation.



Chapter 216

MACHINE LEARNING INFRASTRUCTURE

Chapter 217

MACHINE LEARNING INFRASTRUCTURE

Deep Learning frameworks (PyTorch, TensorFlow) are C++ engines wrapped in Python.

217.0.1 1. Tensor Memory Layout

A Tensor is a block of memory + a “View”. * **Strides:** The number of elements to skip to reach the next index in a dimension. * $\text{Element}(i, j) = \text{data}[i * \text{stride_i} + j * \text{stride_j}]$ * **Contiguity:** Transposing a tensor (`A.T`) usually just modifies strides, touching **zero** data.

217.0.2 2. Broadcasting Implementation

How to add `[32, 1]` vector to `[32, 100]` matrix? * **Virtual Expansion:** Set stride for the dimension of size 1 to 0. Accessing that dimension repeatedly reads the same value.

217.0.3 3. Automatic Differentiation (Autograd)

- **Computational Graph:** Directed Acyclic Graph (DAG) of operations.
- **Reverse Mode (Backprop):**
 1. Forward pass: Compute $y = f(x)$, store intermediate values (tape).
 2. Backward pass: Compute dL/dx using stored values and chain rule.
- **Implementation:** Node class with `virtual Tensor backward(Tensor grad)`.

217.0.4 4. Operator Fusion

Optimizing `ReLU(Add(MatMul(A, B), C))` into a single kernel launch to minimize VRAM bandwidth.

Chapter 218

DATABASE INTERNALS LSM TREES

Chapter 219

DATABASE INTERNALS (LSM TREES)

How RocksDB and LevelDB achieve millions of writes per second.

219.0.1 1. The Write Problem

Random writes to disk (B-Tree update) are slow (IOPS bottleneck). Sequential writes are fast.

219.0.2 2. LSM Tree (Log-Structured Merge Tree)

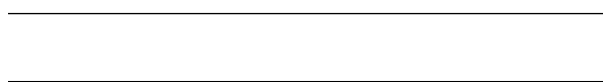
- **MemTable:** In-memory sorted structure (SkipList or Red-Black Tree).
 - Writes go here first (fast RAM access).
 - WAL (Write Ahead Log) on disk for durability.
- **Immutable MemTable:** When MemTable is full, it becomes immutable and is flushed to disk.
- **SSTable (Sorted String Table):** The flushed file on disk. Key-Value pairs sorted by Key.
- **Compaction:** Background process merges multiple SSTables, discarding overwritten/deleted keys (Leveled Compaction).

219.0.3 3. Bloom Filters

To read a key, we might have to check *all* SSTables. Slow! * **Optimization:** Each SSTable has a Bloom Filter. * **Check:** If Bloom says “No”, key is definitely not in this file. Skip it.

219.0.4 4. Memory Mapped I/O (mmap)

Mapping the SSTable file directly into virtual address space. The OS manages paging. * **Benefits:** Zero-copy from disk cache to user space. * **Risks:** SIGBUS if file is truncated; lack of control over eviction.



Chapter 220

THE ULTIMATE ALGORITHM REFERENCE

Chapter 221

THE ULTIMATE ALGORITHM REFERENCE

Stop writing loops. Use the STL.

221.0.1 27.1 Non-Modifying Sequence Operations

- `std::all_of(begin, end, pred)`: True if all match.
- `std::any_of(begin, end, pred)`: True if any match.
- `std::none_of(begin, end, pred)`: True if none match.
- `std::for_each(begin, end, func)`: Apply function to all.
- `std::count(begin, end, val)`: Count occurrences.
- `std::mismatch(b1, e1, b2)`: Find first difference.
- `std::find(begin, end, val)`: Linear search.

221.0.2 27.2 Modifying Sequence Operations

- `std::copy(b, e, out)`: Copy range.
- `std::transform(b, e, out, op)`: Map function over range.
- `std::generate(b, e, gen)`: Fill with generator.
- `std::remove_if(b, e, pred)`: **Erase-Remove Idiom** step 1. Move valid elements to front.
- `std::replace(b, e, old, new)`: Replace values.
- `std::unique(b, e)`: Remove consecutive duplicates.

221.0.3 27.3 Partitioning

- `std::partition(b, e, pred)`: Reorder so true predicates come first. $O(N)$.
- `std::stable_partition`: Preserves relative order.

221.0.4 27.4 Sorting

- `std::sort(b, e)`: IntroSort (Quick + Heap + Insertion). $O(N \log N)$.
- `std::partial_sort(b, mid, e)`: Top K elements sorted.
- `std::nth_element(b, nth, e)`: Element at `nth` is what it would be if sorted. $O(N)$.

221.0.5 27.5 Binary Search (On Sorted Ranges)

- `std::lower_bound(b, e, val)`: First element $\geq$ `val`.
- `std::upper_bound(b, e, val)`: First element $>$ `val`.
- `std::binary_search(b, e, val)`: True/False existence.

221.0.6 27.6 Numeric Operations ()

- `std::iota(b, e, start)`: Fill with 0, 1, 2...
 - `std::accumulate(b, e, init)`: Sum (fold left).
 - `std::reduce(b, e)`: Parallelizable sum (C++17).
 - `std::inner_product`: Dot product.
-
-

Chapter 222

CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK

Chapter 223

CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK

This capstone project integrates C++20/23 features into a realistic high-frequency trading (HFT) component. It demonstrates Modules, Concepts, Ranges, Coroutines, and modern error handling.

223.0.1 Project Structure

```
1 order_book/  
2   src/  
3     types.cppm      (Module: Common types)  
4     order.cppm      (Module: Order definition)  
5     book.cppm       (Module: OrderBook logic)  
6     main.cpp        (Entry point)  
7   CMakeLists.txt  
8   README.md
```

223.0.2 1. Types Module (types.cppm)

```
1 export module types;  
2  
3 import <stdint>;  
4 import <compare>;  
5  
6 export namespace hft {  
7     using Price = int64_t;  
8     using Quantity = uint32_t;  
9     using OrderId = uint64_t;  
10  
11     enum class Side : uint8_t { Buy, Sell };  
12 }
```

223.0.3 2. Order Module (order.cppm)

```
1 export module order;  
2  
3 import types;  
4 import <format>;  
5 import <string>;
```

```

6
7 export namespace hft {
8     struct Order {
9         OrderId id;
10        Side side;
11        Price price;
12        Quantity quantity;
13
14        // C++20 Spaceship for easy comparison
15        auto operator<=>(const Order&) const = default;
16
17        // C++23 Deducing This for generic accessors (example)
18        template<typename Self>
19        auto&& get_price(this Self&& self) {
20            return std::forward<Self>(self).price;
21        }
22    };
23 }
24
25 // C++20 Formatter specialization
26 template<>
27 struct std::formatter<hft::Order> {
28     constexpr auto parse(format_parse_context& ctx) { return ctx.begin(); }
29
30     auto format(const hft::Order& o, format_context& ctx) const {
31         return std::format_to(ctx.out(), "[ID:{}] {} @ {}",
32             o.id, (o.side == hft::Side::Buy ? "BUY" : "SELL"), o.price);
33     }
34 };

```

223.0.4 3. Order Book Module (book.cppm)

```

1 export module book;
2
3 import types;
4 import order;
5 import <vector>;
6 import <map>;
7 import <ranges>;
8 import <algorithm>;
9 import <expected>;
10 import <print>;
11 import <coroutine>;
12
13 export namespace hft {
14
15     // C++20 Concept for Order Container
16     template<typename T>
17     concept OrderContainer = requires(T c) {
18         c.push_back(std::declval<Order>());
19         c.size();
20     };
21
22     class OrderBook {
23     private:
24         // Use std::flat_map (C++23) for cache locality if available,
25         // else std::map. Simulated here as vector for simplicity + ranges
26         std::vector<Order> bids;
27         std::vector<Order> asks;
28
29     public:
30         // C++23 std::expected for error handling

```

```

31     std::expected<void, std::string> add_order(Order o) {
32         if (o.quantity == 0) return std::unexpected("Invalid quantity");
33
34         auto& side_vec = (o.side == Side::Buy) ? bids : asks;
35         side_vec.push_back(o);
36
37         // Keep sorted (simplified)
38         std::ranges::sort(side_vec, {}, &Order::price);
39         if (o.side == Side::Buy) std::ranges::reverse(side_vec);
40
41         return {};
42     }
43
44     // C++20 Coroutine Generator to stream top orders
45     // Note: Requires <generator> (C++23) or custom implementation
46     // Here we simulate a simple generator pattern or use ranges
47     auto top_levels(Side side, int depth) const {
48         const auto& vec = (side == Side::Buy) ? bids : asks;
49         return vec | std::views::take(depth);
50     }
51
52     void print_book() const {
53         std::println("--- Order Book ---");
54         std::println("ASKS:");
55         for (const auto& o : asks | std::views::reverse) std::println(" {}
56         ", o);
57         std::println("BIDS:");
58         for (const auto& o : bids) std::println(" {}", o);
59         std::println("-----");
60     }
61 }

```

223.0.5 4. Main Application (main.cpp)

```

1  import book;
2  import order;
3  import types;
4  import <print>;
5
6  int main() {
7      hft::OrderBook book;
8
9      book.add_order({1, hft::Side::Buy, 100, 10});
10     book.add_order({2, hft::Side::Buy, 99, 5});
11     book.add_order({3, hft::Side::Sell, 101, 20});
12     book.add_order({4, hft::Side::Sell, 102, 15});
13
14     book.print_book();
15
16     // Demonstrate Error Handling
17     if (auto res = book.add_order({5, hft::Side::Buy, 100, 0}); !res) {
18         std::println(stderr, "Error adding order: {}", res.error());
19     }
20
21     return 0;
22 }

```

Chapter 224

APPENDICES

Chapter 225

APPENDICES

Chapter 226

Appendix A: C++ Keywords & Operators Reference

226.0.1 Essential Keywords (Non-Exhaustive)

- **alignas** / **alignof**: Memory alignment queries and specifications.
- **asm**: Inline assembly block.
- **auto**: Type deduction (C++11).
- **const** / **volatile**: cv-qualifiers for type safety and hardware access.
- **constexpr** / **consteval** / **constinit**: Compile-time constant specifications.
- **decltype**: Inspect declared type of an entity.
- **explicit**: Prevent implicit conversions in constructors.
- **export**: Module interface export (C++20).
- **friend**: Allow access to private members.
- **inline**: Suggest inlining to compiler; allow definition in header.
- **mutable**: Allow modification of member in const object.
- **noexcept**: Specifier for functions that don't throw.
- **nullptr**: Null pointer literal (C++11).
- **operator**: Overload operators.
- **requires**: Constraint clause for Concepts (C++20).
- **static_assert**: Compile-time assertion.
- **template**: Define generic classes/functions.
- **thread_local**: Storage duration specifier.
- **typeid**: Runtime type identification (RTTI).
- **typename**: Declare a type parameter in templates.
- **virtual**: Declare virtual function for polymorphism.

226.0.2 Special Operators

- `::` Scope resolution
- `->*` Pointer to member selection
- `<=>` Three-way comparison (Spaceship) (C++20)
- `co\_await`, `co\_yield`, `co\_return` Coroutine operators (C++20)



Chapter 227

Appendix B: Common Acronyms

- **ABI**: Application Binary Interface.
- **API**: Application Programming Interface.
- **COW**: Copy On Write.
- **CRTP**: Curiously Recurring Template Pattern.
- **CTAD**: Class Template Argument Deduction.
- **UB**: Undefined Behavior (Avoid at all costs!).
- **IB**: Implementation-defined Behavior.
- **IIFE**: Immediately Invoked Function Expression (often with lambdas).
- **Nrvo** / **RVO**: (Named) Return Value Optimization.
- **ODR**: One Definition Rule.
- **PIMPL**: Pointer to Implementation (Opaque Pointer).
- **RAII**: Resource Acquisition Is Initialization.
- **RTTI**: Run-Time Type Information.
- **SFINAE**: Substitution Failure Is Not An Error.
- **SOO** / **SSO**: Small Object/String Optimization.
- **STL**: Standard Template Library.
- **TMP**: Template Metaprogramming.
- **TU**: Translation Unit.

Chapter 228

Appendix C: Recommended Tooling

228.0.1 Compilers

- **GCC (GNU Compiler Collection)**: Standard on Linux.
- **Clang/LLVM**: Excellent error messages, widely used on macOS/Linux.
- **MSVC (Microsoft Visual C++)**: Standard on Windows.

228.0.2 Build Systems

- **CMake**: The industry standard meta-build system.
- **Meson**: Modern, fast, Python-based.
- **Bazel**: Google’s build system, good for monorepos.

228.0.3 Package Managers

- **Conan**: Decentralized package manager for C/C++.
- **vcpkg**: Microsoft’s C++ library manager.

228.0.4 Static Analysis & Sanitizers

- **AddressSanitizer (ASan)**: Detects memory errors (buffer overflows, use-after-free).
- **UndefinedBehaviorSanitizer (UBSan)**: Detects undefined behavior.
- **ThreadSanitizer (TSan)**: Detects data races.
- **Clang-Tidy**: Linter and static analysis tool.
- **Cppcheck**: Static analysis tool.

Chapter 229

Appendix D: Common C++ Traps & Pitfalls

229.0.1 I. General & Syntax Traps

1. Most Vexing Parse

- *Issue:* `MyClass obj();` declares a function returning `MyClass`, not a default-constructed object.
- *Fix:* Use brace initialization: `MyClass obj{\{\}};`.

2. The “dangling else” Problem

- *Issue:* Nested `if-else` without braces can associate `else` with the wrong `if`.
- *Fix:* Always use braces `\{\}` for control structures.

3. Integer Division

- *Issue:* `1/2` results in `0` (integer), not `0.5`.
- *Fix:* Cast one operand to float/double: `1.0/2` or `static\_cast<double>(1)/2`.

4. Loop Variable Type Mismatch

- *Issue:* `for (unsigned i = v.size() - 1; i >= 0; --i)` causes an infinite loop because `unsigned` is never negative.
- *Fix:* Use `int` (and cast size) or standard iterators/ranges.

5. Shadowing Variables

- *Issue:* Declaring a local variable with the same name as a member or outer variable hides the outer one.
- *Fix:* Enable compiler warnings (`-Wshadow`) and use `this->member` if necessary.

229.0.2 II. Pointers, References & Memory

6. Object Slicing

- *Issue:* Assigning a `Derived` object to a `Base` value slices off the derived part.
- *Fix:* Use pointers `Base*` or references `Base\&` for polymorphism.

7. Dangling References

- *Issue*: Returning a reference to a local stack variable.
- *Fix*: Return by value or use smart pointers/dynamic allocation.

8. Iterator Invalidation

- *Issue*: Adding elements to a `std::vector` may reallocate memory, invalidating all pointers/iterators to elements.
- *Fix*: Don't cache iterators across mutating operations; use `reserve()` if possible.

9. `delete` VS `delete[]`

- *Issue*: Mismatching `new` with `delete[]` or `new[]` with `delete` causes undefined behavior.
- *Fix*: Use `std::vector` or `std::unique_ptr` instead of manual management.

10. Use-After-Move

- *Issue*: Accessing an object after `std::move()` (except for reassignment/destruction).
- *Fix*: Treat moved-from objects as empty; do not read their state.

229.0.3 III. Classes & OOP

11. Virtual Destructor Missing

- *Issue*: Deleting a derived class via a base pointer when the base destructor is not `virtual` leaks derived resources.
- *Fix*: Always mark base class destructors `virtual` (or `protected` if non-polymorphic).

12. Calling Virtual Functions in Constructor/Destructor

- *Issue*: Calls the *base* class version, not the derived one, because the derived part isn't initialized/is already destroyed.
- *Fix*: Use two-phase initialization or factory methods.

13. Copy Constructor/Assignment Missing

- *Issue*: Classes managing raw pointers will default to shallow copy (double free error).
- *Fix*: Follow the **Rule of Three/Five/Zero**.

14. Initialization Order

- *Issue*: Members are initialized in *declaration order*, not initializer list order.
- *Fix*: Keep initializer list order identical to member declaration order to avoid warnings.

229.0.4 IV. Concurrency

15. Data Races

- *Issue:* Multiple threads accessing shared memory without synchronization (at least one writer).
- *Fix:* Use `std::mutex`, `std::atomic`, or `std::shared_mutex`.

16. Deadlocks

- *Issue:* Two threads waiting on each other's locks.
- *Fix:* Acquire locks in a consistent global order; use `std::scoped_lock` (C++17) to lock multiple mutexes safely.

17. False Sharing

- *Issue:* Independent atomic variables on the same cache line degrade performance due to cache coherency protocols.
- *Fix:* Use `alignas(hardware_destructive_interference_size)` to pad variables.

229.0.5 V. Modern C++ & Macros

18. `std::vector<bool>` Weirdness

- *Issue:* It's a template specialization (bitfield), not a vector of bools. Returns a proxy object, not `bool&`.
- *Fix:* Use `std::deque<bool>` or `std::vector<char>` if you need real references.

19. Auto Type Deduction

- *Issue:* `auto` drops references and `const`.
- *Fix:* Use `auto&` or `const auto&` explicitly when needed.

20. Macro Side Effects

- *Issue:* `#define MAX(a,b) ((a) > (b) ? (a) : (b))` evaluates arguments twice. `MAX(x++, y)` increments `x` twice.
- *Fix:* Use `inline` functions or templates instead of macros.

21. Static Initialization Order Fiasco

- *Issue:* Global objects in different files have undefined initialization order.
- *Fix:* Use the “Construct On First Use” idiom (Meyers Singleton).

Chapter 230

Appendix E: C++ Interview Cheat Sheet

230.0.1 Core Concepts

1. **Virtual Functions:** Enable runtime polymorphism via `vtable/vptr`. Destructors must be `virtual` in base classes.
2. **Smart Pointers:**
 - `unique_ptr`: Exclusive ownership, no overhead.
 - `shared_ptr`: Shared ownership, ref-counted (atomic), control block overhead.
 - `weak_ptr`: Non-owning reference to `shared_ptr` (breaks cycles).
3. **Move Semantics:** Transfers resources (pointers) instead of deep copying. Enabled by rvalue references (`&&`) and `std::move`.
4. **RAII:** Resource Acquisition Is Initialization. Constructor acquires, destructor releases. Core to C++ safety.
5. **Cast Types:**
 - `static_cast`: Compile-time safe conversions.
 - `dynamic_cast`: Runtime checked downcasting (requires RTTI).
 - `reinterpret_cast`: Bitwise reinterpretation (unsafe).
 - `const_cast`: Remove/add constness.

230.0.2 Modern C++ (C++11/14/17/20)

1. **Lambdas:** Anonymous function objects. Capture [=], [&], or move-only [`x = std::move(y)`].
2. **Auto:** Type deduction. Always initialize.
3. **Structured Bindings (C++17):** `auto [x, y] = pair;`
4. **Concepts (C++20):** Constrain templates for better errors/readability.
5. **Coroutines (C++20):** Functions that can suspend/resume.

230.0.3 System Design Questions

1. **Vector vs List:** Vector (contiguous, cache-friendly) is almost always better than List (node-based, cache misses) unless aggressive splicing is needed.
2. **Map vs Unordered Map:** Map (BST, $O(\log n)$, sorted) vs Unordered Map (Hash Table, $O(1)$ avg, unsorted).
3. **Handling 1M connections:** Use non-blocking I/O (epoll/kqueue) or `io_uring`, not one thread per connection.
4. **Memory Layout:** Stack (local vars) vs Heap (dynamic) vs Data (globals) vs Text (code).

230.0.4 Quick Coding

- **Implement Singleton:** Use static local variable (Thread-safe in C++11+).
- **Implement String Class:** Handle deep copy, move semantics, and destructor.
- **Reverse Linked List:** Classic pointer manipulation.

Chapter 231

Appendix F: The C++ Standard Evolution Matrix

231.0.1 1. Versioned Changelog

C++98 (ISO/IEC 14882:1998) - *The Foundation*

Released: 1998 * **Core:** Templates, Exceptions, Namespaces, `bool` type, cast operators (`static_cast`, etc.), `mutable`, `explicit`. * **STL:** Containers (`vector`, `list`, `map`, `set`, `deque`), Algorithms (`sort`, `find`, `transform`), Iterators, Strings (`std::string`), I/O Streams (`iostream`). * **Memory:** `std::auto_ptr` (Deprecated in C++11).

C++03 (ISO/IEC 14882:2003) - *The Bug Fix*

Released: 2003 * **Focus:** Defect Report (DR) fixes for C++98 to ensure consistency across compilers. * **Features:** Value initialization `T()`, fixes to `std::vector` contiguous memory guarantee.

C++11 (ISO/IEC 14882:2011) - *The Modern Revolution*

Released: September 2011 * **Language:** `auto`, `nullptr`, Range-based `for`, Lambda expressions, Rvalue references (`&&`) & Move semantics, Variadic templates, `constexpr` (limited), `decltype`, Uniform initialization `{ }`, `static_assert`, `override`, `final`, `enum class`. * **Concurrency:** `std::thread`, `std::mutex`, `std::atomic`, `std::future`, `std::async`. * **Library:** Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`), `std::array`, `std::tuple`, `std::unordered_map/set`, `std::regex`, `std::chrono`.

C++14 (ISO/IEC 14882:2014) - *The Refinement*

Released: December 2014 * **Language:** Generic lambdas (auto params), Relaxed `constexpr` (loops/variables allowed), Binary literals (`0b1010`), Digit separators (`1'000`), Variable templates, Return type deduction. * **Library:** `std::make_unique`, `std::shared_timed_mutex`, `std::integer_sequence`, `std::exchange`, `std::quoted`.

C++17 (ISO/IEC 14882:2017) - *The Major Update*

Released: December 2017 * **Language:** Structured bindings `auto [x,y] = p;`, `if constexpr`, Fold expressions (`... + args`), Class Template Argument Deduction (CTAD), Inline variables, `__has_include`. * **Library:** `std::filesystem`, `std::optional`, `std::variant`, `std::any`, `std::string_view`, Parallel Algorithms (`std::execution::par`), `std::invoke`, `std::byte`, `std::pmr` (Polymorphic Memory Resources).

C++20 (ISO/IEC 14882:2020) - *The Gigantic Leap*

Released: December 2020 * **Language:** Concepts (Constraints), Modules (`import/export`), Coroutines (`co\_await`), Three-way comparison (`<=>`), Designated initializers `\{.x=1\}`, `constexpr` (Immediate functions), `constexpr`, Range-based `for` with `init`. * **Library:** Ranges (`std::ranges`), `std::span`, `std::format`, `std::jthread`, `std::stop\_token`, `std::barrier`, `std::latch`, `std::semaphore`, `std::bit\_cast`, `std::source\_location`, Calendars & Timezones.

C++23 (ISO/IEC 14882:2023) - *The Completion*

Released: October 2023 * **Language:** Deducing `this` (Explicit object parameter), `if constexpr`, Multidimensional subscript `m[1,2]`, Static `operator()`, `auto(x)` decay copy. * **Library:** `std::print`, `std::println`, `std::expected` (Error handling), `std::mdspan`, `std::flat\_map`, `std::flat\_set`, `std::generator` (Synchronous coroutines), `std::stacktrace`, `std::stdatomic.h`.

231.0.2 2. Feature Matrix

Feature	C++98	C++11	C++14	C++17	C++20	C++23
Memory Variables	<code>auto_ptr</code>	<code>unique_ptr</code>	<code>make_unique</code>	<code>pmr</code>	<code>shared_ptr</code>	<code>atomic</code>
Loops	Type req.	<code>auto</code>	Var templates	Structured Bindings	<code>constexpr</code>	<code>constexpr</code>
Templates	<code>for(;;)</code>	Range-for	-	-	Range-for init	Concepts
Lambdas	Basic	Variadic	Variable	Fold Expressions	Concepts	Deduction Guides
Concurrency	-	Basic	Generic	<code>constexpr</code>	Template	Relaxed Ordering
String	-	<code>thread</code>	<code>shared_lock</code>	Parallel Algos	<code>jthread</code> , Latches	String Views
Metaprog.	<code>string</code>	<code>to_string</code>	<code>quoted</code>	<code>string_view</code>	<code>format</code>	String Literals
Modules	Traits	<code>static_assert</code>	<code>integer_seq</code>	<code>if constexpr</code>	<code>constexpr</code>	if constexpr
Coroutines	-	-	-	-	Modules	std::atomic
	-	-	-	-	Async	g++

231.0.3 3. Timeline & Release Accuracy

Standard	ISO Publication	Codename	Compiler Flag (GCC/Clang)
C++98	1998-09	C++98	<code>-std=c++98</code>
C++03	2003-10	C++03	<code>-std=c++03</code>
C++11	2011-09	C++0x	<code>-std=c++11</code>
C++14	2014-12	C++1y	<code>-std=c++14</code>
C++17	2017-12	C++1z	<code>-std=c++17</code>
C++20	2020-12	C++2a	<code>-std=c++20</code>
C++23	2023-10	C++2b	<code>-std=c++23</code>
C++26	<i>Expected 2026</i>	C++2c	<code>-std=c++26</code> / <code>-std=c++2c</code>

Chapter 232

Appendix G: C++ Standard Library Headers Reference

232.0.1 Concepts & Utilities

- `<concepts>` (C++20): Fundamental concepts library.
- `<coroutine>` (C++20): Coroutine support library.
- `<functional>`: Function objects, binder, and reference wrappers.
- `<memory>`: Smart pointers and allocators.
- `<tuple>`: Tuple library.
- `<type_traits>`: Compile-time type information.
- `<utility>`: Utility components (`std::pair`, `std::move`).

232.0.2 Containers

- `<array>` (C++11): Fixed-size array class.
- `<deque>`: Double-ended queue.
- `<list>`: Doubly-linked list.
- `<map>`: Associative containers (Red-Black Tree).
- `<queue>`: Queue adapter.
- `<set>`: Associative containers (Red-Black Tree).
- `<stack>`: Stack adapter.
- `<unordered_map>` (C++11): Hash map.
- `<vector>`: Dynamic array.
- `<span>` (C++20): Non-owning view of contiguous memory.

232.0.3 Algorithms & Iterators

- `<algorithm>`: Algorithms that operate on ranges.
- `<execution>` (C++17): Parallel algorithms.
- `<iterator>`: Iterator primitives.
- `<numeric>`: Numeric operations (`accumulate`, `reduce`).
- `<ranges>` (C++20): Range primitives and views.

232.0.4 Concurrency

- `<atomic>` (C++11): Atomic operations.
- `<barrier>` (C++20): Barriers.
- `<condition_variable>` (C++11): Condition variables.
- `<future>` (C++11): Futures and promises.
- `<latch>` (C++20): Latches.
- `<mutex>` (C++11): Mutual exclusion primitives.
- `<semaphore>` (C++20): Semaphores.
- `<shared_mutex>` (C++14): Shared mutexes.
- `<thread>` (C++11): Thread class.

232.0.5 Input/Output

- `<filesystem>` (C++17): File system operations.
- `<format>` (C++20): Formatting library.
- `<fstream>`: File stream classes.
- `<iostream>`: Standard I/O stream objects.
- `<print>` (C++23): Print functions.
- `<sstream>`: String stream classes.

232.0.6 Numerics & Math

- `<bit>` (C++20): Bit manipulation.
 - `<complex>`: Complex number arithmetic.
 - `<random>` (C++11): Random number generation.
 - `<ratio>` (C++11): Compile-time rational arithmetic.
 - `<valarray>`: Class for representing and manipulating arrays of values.
 - `<numbers>` (C++20): Mathematical constants.
-

Chapter 233

Appendix H: Professional C++ Idioms

233.0.1 1. RAII (Resource Acquisition Is Initialization)

- **Concept:** Bind resource lifecycle to object lifecycle. Constructor acquires, destructor releases.
- **Use Case:** Memory, file handles, mutex locks, sockets.
- **Example:** `std::lock_guard`, `std::unique_ptr`.

233.0.2 2. Pimpl (Pointer to Implementation)

- **Concept:** Hide private members in a separate class/struct, accessed via a pointer.
- **Benefit:** ABI stability, reduced compilation times (header dependency changes don't trigger rebuilds of clients).
- **Pattern:**

```
cpp      class Widget \{          struct Impl;          std::unique_ptr<Impl> pImpl
;      public:          Widget();          ~Widget(); // Defined in .cpp where Impl is visible
          \};
```

233.0.3 3. Copy-and-Swap

- **Concept:** Implement assignment operator in terms of copy constructor and swap.
- **Benefit:** Strong Exception Safety guarantee; removes code duplication.
- **Pattern:**

```
cpp      T& operator=(T other) \{ // Pass by value (copy)          swap(*this, other
);          return *this;          \}
```

233.0.4 4. NVI (Non-Virtual Interface)

- **Concept:** Public interface is non-virtual; virtual functions are private/protected.
- **Benefit:** Separation of interface (pre/post-conditions) from implementation.
- **Pattern:**

```
cpp      class Base \{      public:          void doWork() \{          // Pre-condition
      logic          doWorkImpl();          // Post-condition logic          \}      private
:          virtual void doWorkImpl() = 0;          \};
```

233.0.5 5. Erase-Remove Idiom

- **Concept:** Standard way to remove elements from a `std::vector` (before C++20 `std::erase`).
- **Pattern:** `v.erase(std::remove(v.begin(), v.end(), value), v.end());`

233.0.6 6. SFINAE (Substitution Failure Is Not An Error)

- **Concept:** Remove functions from overload resolution set if types don't match constraints.
- **Modern Replacement:** C++20 Concepts (`requires`).

233.0.7 7. CRTP (Curiously Recurring Template Pattern)

- **Concept:** Class `Derived` inherits from `Base<Derived>`.
- **Use Case:** Static polymorphism (compile-time), adding functionality (mixins) without vtable overhead.
- **Example:** `std::enable_shared_from_this`.
